

Debugging with GDB

The GNU Source-Level Debugger

Tenth Edition, for GDB version 18.0.50.20260503-git

(GDB)

Richard Stallman, Roland Pesch, Stan Shebs, et al.

(Send bugs and comments on GDB to <https://www.gnu.org/software/gdb/bugs/>.)

Debugging with GDB
T_EXinfo 2017-08-23.19

Published by the Free Software Foundation
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301, USA
ISBN 978-0-9831592-3-0

Copyright © 1988-2026 Free Software Foundation, Inc.

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with the Invariant Sections being “Free Software” and “Free Software Needs Free Documentation”, with the Front-Cover Texts being “A GNU Manual,” and with the Back-Cover Texts as in (a) below.

(a) The FSF’s Back-Cover Text is: “You are free to copy and modify this GNU Manual. Buying copies from GNU Press supports the FSF in developing GNU and promoting software freedom.”

Table of Contents

Summary of GDB	1
Free Software	1
Free Software Needs Free Documentation	1
Contributors to GDB	3
1 A Sample GDB Session	7
2 Getting In and Out of GDB	11
2.1 Invoking GDB	11
2.1.1 Choosing Files	12
2.1.2 Choosing Modes	13
2.1.3 What GDB Does During Startup	17
2.1.4 Initialization Files	18
2.1.4.1 Home directory early initialization files	18
2.1.4.2 System wide initialization files	19
2.1.4.3 Home directory initialization file	19
2.1.4.4 Local directory initialization file	20
2.2 Quitting GDB	20
2.3 Shell Commands	20
2.4 Logging Output	22
3 GDB Commands	23
3.1 Command Syntax	23
3.2 Command Settings	23
3.3 Command Completion	24
3.4 Filenames As Command Arguments	27
3.5 Command options	28
3.6 Getting Help	29
4 Running Programs Under GDB	33
4.1 Compiling for Debugging	33
4.2 Starting your Program	34
4.3 Your Program's Arguments	38
4.4 Your Program's Environment	38
4.5 Your Program's Working Directory	40
4.6 Your Program's Input and Output	41
4.7 Debugging an Already-running Process	41
4.8 Killing the Child Process	42
4.9 Debugging Multiple Inferiors Connections and Programs	43
4.9.1 Inferior-Specific Breakpoints	47
4.10 Debugging Programs with Multiple Threads	48

4.11	Debugging Forks	54
4.12	Setting a <i>Bookmark</i> to Return to Later	56
4.12.1	A Non-obvious Benefit of Using Checkpoints	58
5	Stopping and Continuing	59
5.1	Breakpoints, Watchpoints, and Catchpoints	59
5.1.1	Setting Breakpoints	60
5.1.2	Setting Watchpoints	68
5.1.3	Setting Catchpoints	71
5.1.4	Deleting Breakpoints	75
5.1.5	Disabling Breakpoints	76
5.1.6	Break Conditions	78
5.1.7	Breakpoint Command Lists	79
5.1.8	Dynamic Printf	81
5.1.9	How to save breakpoints to a file	82
5.1.10	Static Probe Points	83
5.1.11	“Cannot insert breakpoints”	84
5.1.12	“Breakpoint address adjusted...”	85
5.2	Continuing and Stepping	85
5.3	Skipping Over Functions and Files	89
5.4	Signals	92
5.5	Stopping and Starting Multi-thread Programs	95
5.5.1	All-Stop Mode	96
5.5.2	Non-Stop Mode	97
5.5.3	Background Execution	98
5.5.4	Thread-Specific Breakpoints	99
5.5.5	Interrupted System Calls	100
5.5.6	Observer Mode	100
6	Running programs backward	103
7	Recording Inferior’s Execution and Replaying It	105
8	Examining the Stack	115
8.1	Stack Frames	115
8.2	Backtraces	116
8.3	Selecting a Frame	119
8.4	Information About a Frame	121
8.5	Applying a Command to Several Frames	122
8.6	Management of Frame Filters	124

9	Examining Source Files	127
9.1	Printing Source Lines	127
9.2	Location Specifications	128
9.2.1	Linespec Locations	129
9.2.2	Explicit Locations	131
9.2.3	Address Locations	132
9.3	Editing Source Files	132
9.3.1	Choosing your Editor	133
9.4	Searching Source Files	133
9.5	Specifying Source Directories	133
9.6	Source and Machine Code	137
9.7	Disable Reading Source Code	142
10	Examining Data	143
10.1	Expressions	147
10.2	Ambiguous Expressions	148
10.3	Program Variables	149
10.4	Artificial Arrays	152
10.5	Output Formats	152
10.6	Examining Memory	154
10.7	Memory Tagging	157
10.8	Automatic Display	158
10.9	Print Settings	159
10.10	Pretty Printing	170
10.10.1	Pretty-Printer Introduction	170
10.10.2	Pretty-Printer Example	170
10.10.3	Pretty-Printer Commands	171
10.11	Value History	172
10.12	Convenience Variables	173
10.13	Convenience Functions	177
10.14	Registers	181
10.15	Floating Point Hardware	183
10.16	Vector Unit	183
10.17	Operating System Auxiliary Information	183
10.18	Memory Region Attributes	186
10.18.1	Attributes	186
10.18.1.1	Memory Access Mode	187
10.18.1.2	Memory Access Size	187
10.18.1.3	Data Cache	187
10.18.2	Memory Access Checking	187
10.19	Copy Between Memory and a File	188
10.20	How to Produce a Core File from Your Program	188
10.21	Character Sets	190
10.22	Caching Data of Targets	192
10.23	Search Memory	193
10.24	Value Sizes	195

11	Debugging Optimized Code	197
11.1	Inline Functions	197
11.2	Tail Call Frames	198
12	C Preprocessor Macros	201
13	Tracepoints	205
13.1	Commands to Set Tracepoints	205
13.1.1	Create and Delete Tracepoints	206
13.1.2	Enable and Disable Tracepoints	208
13.1.3	Tracepoint Passcounts	208
13.1.4	Tracepoint Conditions	209
13.1.5	Trace State Variables	209
13.1.6	Tracepoint Action Lists	210
13.1.7	Listing Tracepoints	212
13.1.8	Listing Static Tracepoint Markers	213
13.1.9	Starting and Stopping Trace Experiments	214
13.1.10	Tracepoint Restrictions	216
13.2	Using the Collected Data	217
13.2.1	<code>tfind n</code>	217
13.2.2	<code>tdump</code>	219
13.2.3	<code>save tracepoints filename</code>	220
13.3	Convenience Variables for Tracepoints	220
13.4	Using Trace Files	221
14	Debugging Programs That Use Overlays ..	223
14.1	How Overlays Work	223
14.2	Overlay Commands	224
14.3	Automatic Overlay Debugging	226
14.4	Overlay Sample Program	227
15	Using GDB with Different Languages	229
15.1	Switching Between Source Languages	229
15.1.1	List of Filename Extensions and Languages	229
15.1.2	Setting the Working Language	230
15.1.3	Having GDB Infer the Source Language	230
15.2	Displaying the Language	231
15.3	Type and Range Checking	231
15.3.1	An Overview of Type Checking	231
15.3.2	An Overview of Range Checking	232
15.4	Supported Languages	233
15.4.1	C and C++	233
15.4.1.1	C and C++ Operators	233
15.4.1.2	C and C++ Constants	235
15.4.1.3	C++ Expressions	236
15.4.1.4	C and C++ Defaults	237

15.4.1.5	C and C++ Type and Range Checks.....	237
15.4.1.6	GDB and C	237
15.4.1.7	GDB Features for C++.....	237
15.4.1.8	Decimal Floating Point format.....	239
15.4.2	D	240
15.4.3	Go	240
15.4.4	Objective-C	240
15.4.4.1	Method Names in Commands.....	241
15.4.4.2	The Print Command With Objective-C	241
15.4.5	OpenCL C	241
15.4.5.1	OpenCL C Datatypes.....	242
15.4.5.2	OpenCL C Expressions	242
15.4.5.3	OpenCL C Operators.....	242
15.4.6	Fortran	242
15.4.6.1	Fortran Types.....	242
15.4.6.2	Fortran Operators and Expressions	243
15.4.6.3	Fortran Intrinsics	243
15.4.6.4	Special Fortran Commands	244
15.4.7	Pascal.....	245
15.4.8	Rust	245
15.4.9	Modula-2	246
15.4.9.1	Operators	246
15.4.9.2	Built-in Functions and Procedures	247
15.4.9.3	Constants	248
15.4.9.4	Modula-2 Types.....	249
15.4.9.5	Modula-2 Defaults.....	250
15.4.9.6	Deviations from Standard Modula-2	251
15.4.9.7	Modula-2 Type and Range Checks.....	251
15.4.9.8	The Scope Operators :: and	251
15.4.9.9	GDB and Modula-2	252
15.4.10	Ada.....	252
15.4.10.1	Introduction.....	252
15.4.10.2	Omissions from Ada	252
15.4.10.3	Additions to Ada.....	254
15.4.10.4	Overloading support for Ada	255
15.4.10.5	Stopping at the Very Beginning.....	256
15.4.10.6	Ada Exceptions	256
15.4.10.7	Extensions for Ada Tasks.....	256
15.4.10.8	Tasking Support when Debugging Core Files.....	260
15.4.10.9	Tasking Support when using the Ravenscar Profile..	261
15.4.10.10	Ada Source Character Set	261
15.4.10.11	Known Peculiarities of Ada Mode	262
15.5	Unsupported Languages.....	263

16 Examining the Symbol Table..... 265

17	Altering Execution	279
17.1	Assignment to Variables	279
17.2	Continuing at a Different Address	280
17.3	Giving your Program a Signal	281
17.4	Returning from a Function	282
17.5	Calling Program Functions	283
17.5.1	Calling functions with no debug info	286
17.6	Patching Programs	287
17.7	Compiling and injecting code in GDB	287
17.7.1	Compilation options for the <code>compile</code> command	289
17.7.2	Caveats when using the <code>compile</code> command	289
17.7.3	Compiler search for the <code>compile</code> command	291
18	GDB Files	293
18.1	Commands to Specify Files	293
18.2	File Caching	302
18.3	Debugging Information in Separate Files	303
18.4	Debugging information in a special section	306
18.5	Index Files Speed Up GDB	307
18.5.1	Automatic symbol index cache	308
18.5.2	Building the index with GNU <code>gold</code>	309
18.6	Extensions to <code>‘.debug_names’</code>	309
18.7	Errors Reading Symbol Files	310
18.8	GDB Data Files	312
19	Specifying a Debugging Target	313
19.1	Active Targets	313
19.2	Commands for Managing Targets	313
19.3	Choosing Target Byte Order	316
20	Debugging Remote Programs	317
20.1	Connecting to a Remote Target	317
20.1.1	Types of Remote Connections	317
20.1.2	Host and Target Files	318
20.1.3	Remote Connection Commands	319
20.2	Sending files to a remote system	322
20.3	Using the <code>gdbserver</code> Program	322
20.3.1	Running <code>gdbserver</code>	323
20.3.1.1	Attaching to a Running Program	323
20.3.1.2	TCP port allocation lifecycle of <code>gdbserver</code>	324
20.3.1.3	Other Command-Line Arguments for <code>gdbserver</code> ...	324
20.3.2	Connecting to <code>gdbserver</code>	325
20.3.3	Monitor Commands for <code>gdbserver</code>	326
20.3.4	Tracepoints support in <code>gdbserver</code>	327
20.4	Remote Configuration	328
20.5	Implementing a Remote Stub	335
20.5.1	What the Stub Can Do for You	336

20.5.2	What You Must Do for the Stub.....	336
20.5.3	Putting it All Together	338
21	Configuration-Specific Information.....	339
21.1	Native	339
21.1.1	BSD libkvm Interface	339
21.1.2	Process Information	339
21.1.3	Features for Debugging DJGPP Programs.....	341
21.1.4	Features for Debugging MS Windows PE Executables...	344
21.1.4.1	Support for DLLs without Debugging Symbols	345
21.1.4.2	DLL Name Prefixes.....	346
21.1.4.3	Working with Minimal Symbols.....	346
21.1.5	Commands Specific to GNU Hurd Systems	347
21.1.6	Darwin.....	349
21.1.7	FreeBSD	350
21.2	Embedded Operating Systems	350
21.3	Embedded Processors	350
21.3.1	Synopsys ARC	350
21.3.2	ARM	351
21.3.3	BPF	352
21.3.4	M68k	352
21.3.5	MicroBlaze	352
21.3.6	MIPS Embedded.....	353
21.3.7	OpenRISC 1000.....	353
21.3.8	PowerPC Embedded.....	354
21.3.9	Atmel AVR	355
21.3.10	CRIS	355
21.3.11	Renesas Super-H.....	356
21.4	Architectures.....	356
21.4.1	AArch64	356
21.4.1.1	AArch64 Scalable Vector Extension	356
21.4.1.2	AArch64 Scalable Matrix Extension.....	357
21.4.1.3	AArch64 Scalable Matrix Extension 2	359
21.4.1.4	AArch64 Pointer Authentication.....	359
21.4.1.5	AArch64 Memory Tagging Extension	360
21.4.1.6	AArch64 Guarded Control Stack.....	360
21.4.1.7	AArch64 Floating Point Mode Register	361
21.4.2	x86	361
21.4.2.1	x87 registers.....	361
21.4.2.2	Intel Control-Flow Enforcement Technology.	361
21.4.3	Alpha.....	362
21.4.4	MIPS.....	362
21.4.5	HPPA.....	364
21.4.6	PowerPC.....	364
21.4.7	Sparc64	364
21.4.7.1	ADI Support	364
21.4.8	S12Z	365
21.4.9	AMD GPU.....	365

21.4.9.1	AMD GPU Architectures	365
21.4.9.2	AMD GPU Device Driver and AMD ROCm Runtime ..	365
21.4.9.3	AMD GPU Wavefronts	366
21.4.9.4	AMD GPU Code Objects	366
21.4.9.5	AMD GPU Entity Target Identifiers and Convenience Variables	366
21.4.9.6	AMD GPU Signals	367
21.4.9.7	AMD GPU Memory Violation Reporting	367
21.4.9.8	AMD GPU Logging	368
21.4.9.9	AMD GPU Restrictions	368
21.4.10	RISC-V	369
22	Controlling GDB	371
22.1	Prompt	371
22.2	Command Editing	371
22.3	Command History	372
22.4	Screen Size	374
22.5	Output Styling	375
22.6	Numbers	379
22.7	Configuring the Current ABI	380
22.8	Automatically loading associated files	381
22.8.1	Automatically loading init file in the current directory ..	383
22.8.2	Automatically loading thread debugging library	383
22.8.3	Security restriction for auto-loading	384
22.8.4	Displaying files tried for auto-load	385
22.9	Optional Warnings and Messages	386
22.10	Optional Messages about Internal Happenings	387
22.11	Other Miscellaneous Settings	393
23	Extending GDB	397
23.1	Canned Sequences of Commands	397
23.1.1	User-defined Commands	397
23.1.2	User-defined Command Hooks	400
23.1.3	Command Files	401
23.1.4	Commands for Controlled Output	403
23.1.5	Controlling auto-loading native GDB scripts	405
23.2	Command Aliases	405
23.2.1	Default Arguments	406
23.3	Extending GDB using Python	407
23.3.1	Python Commands	408
23.3.2	Python API	409
23.3.2.1	Basic Python	410
23.3.2.2	Threading in GDB	415
23.3.2.3	Exception Handling	416
23.3.2.4	Values From Inferior	417
23.3.2.5	Types In Python	425
23.3.2.6	Pretty Printing API	430

23.3.2.7	Selecting Pretty-Printers.....	432
23.3.2.8	Writing a Pretty-Printer.....	433
23.3.2.9	Type Printing API.....	435
23.3.2.10	Filtering Frames.....	436
23.3.2.11	Decorating Frames.....	438
23.3.2.12	Writing a Frame Filter.....	441
23.3.2.13	Unwinding Frames in Python.....	445
23.3.2.14	Xmethods In Python.....	450
23.3.2.15	Xmethod API.....	451
23.3.2.16	Writing an Xmethod.....	452
23.3.2.17	Inferiors In Python.....	455
23.3.2.18	Events In Python.....	458
23.3.2.19	Threads In Python.....	465
23.3.2.20	Recordings In Python.....	467
23.3.2.21	CLI Commands In Python.....	471
23.3.2.22	GDB/MI Commands In Python.....	474
23.3.2.23	GDB/MI Notifications In Python.....	477
23.3.2.24	Parameters In Python.....	477
23.3.2.25	Writing new convenience functions.....	483
23.3.2.26	Program Spaces In Python.....	484
23.3.2.27	Objfiles In Python.....	487
23.3.2.28	Accessing inferior stack frames from Python.....	490
23.3.2.29	Accessing blocks from Python.....	493
23.3.2.30	Python representation of Symbols.....	495
23.3.2.31	Symbol table representation in Python.....	500
23.3.2.32	Manipulating line tables using Python.....	502
23.3.2.33	Manipulating breakpoints using Python.....	503
23.3.2.34	Finish Breakpoints.....	507
23.3.2.35	Python representation of lazy strings.....	508
23.3.2.36	Python representation of colors.....	509
23.3.2.37	Python representation of styles.....	510
23.3.2.38	Python representation of architectures.....	512
23.3.2.39	Registers In Python.....	513
23.3.2.40	Connections In Python.....	514
23.3.2.41	Implementing new TUI windows.....	516
23.3.2.42	Instruction Disassembly In Python.....	517
23.3.2.43	Missing Debug Info In Python.....	526
23.3.2.44	Missing Objfiles In Python.....	528
23.3.2.45	Core Files In Python.....	531
23.3.3	Python Auto-loading.....	532
23.3.4	Python modules.....	533
23.3.4.1	gdb.printing.....	533
23.3.4.2	gdb.types.....	533
23.3.4.3	gdb.prompt.....	535
23.3.4.4	gdb.ptwrite.....	536
23.4	Extending GDB using Guile.....	538
23.4.1	Guile Introduction.....	538
23.4.2	Guile Commands.....	538

23.4.3	Guile API	539
23.4.3.1	Basic Guile	539
23.4.3.2	Guile Configuration	541
23.4.3.3	GDB Scheme Data Types	541
23.4.3.4	Guile Exception Handling	543
23.4.3.5	Values From Inferior In Guile	545
23.4.3.6	Arithmetic In Guile	550
23.4.3.7	Types In Guile	551
23.4.3.8	Guile Pretty Printing API	556
23.4.3.9	Selecting Guile Pretty-Printers	558
23.4.3.10	Writing a Guile Pretty-Printer	559
23.4.3.11	Commands In Guile	561
23.4.3.12	Parameters In Guile	565
23.4.3.13	Program Spaces In Guile	567
23.4.3.14	Objfiles In Guile	568
23.4.3.15	Accessing inferior stack frames from Guile	569
23.4.3.16	Accessing blocks from Guile	572
23.4.3.17	Guile representation of Symbols	574
23.4.3.18	Symbol table representation in Guile	577
23.4.3.19	Manipulating breakpoints using Guile	578
23.4.3.20	Guile representation of lazy strings	582
23.4.3.21	Guile representation of architectures	583
23.4.3.22	Disassembly In Guile	585
23.4.3.23	I/O Ports in Guile	586
23.4.3.24	Memory Ports in Guile	586
23.4.3.25	Iterators In Guile	587
23.4.3.26	Colors In Guile	589
23.4.4	Guile Auto-loading	590
23.4.5	Guile Modules	590
23.4.5.1	Guile Printing Module	591
23.4.5.2	Guile Types Module	591
23.5	Auto-loading extensions	592
23.5.1	The <i>objfile-gdb.ext</i> file	592
23.5.2	The <i>.debug_gdb_scripts</i> section	593
23.5.2.1	Script File Entries	593
23.5.2.2	Script Text Entries	594
23.5.3	Which flavor to choose?	595
23.6	Multiple Extension Languages	595
23.6.1	Python comes first	595

24 Command Interpreters 597

25 GDB Text User Interface 599

25.1	TUI Overview	599
25.2	TUI Key Bindings	600
25.3	TUI Single Key Mode	601
25.4	TUI Mouse Support	602

25.5	TUI-specific Commands	602
25.6	TUI Configuration Variables	605
26	Using GDB under GNU Emacs	607
27	The GDB/MI Interface	609
	Function and Purpose	609
	Notation and Terminology	609
27.3	GDB/MI General Design	609
27.3.1	Context management	610
27.3.1.1	Threads and Frames	610
27.3.1.2	Language	611
27.3.2	Asynchronous command execution and non-stop mode ..	611
27.3.3	Thread groups	612
27.4	GDB/MI Command Syntax	612
27.4.1	GDB/MI Input Syntax	613
27.4.2	GDB/MI Output Syntax	613
27.5	GDB/MI Compatibility with CLI	615
27.6	GDB/MI Development and Front Ends	615
27.7	GDB/MI Output Records	617
27.7.1	GDB/MI Result Records	617
27.7.2	GDB/MI Stream Records	617
27.7.3	GDB/MI Async Records	618
27.7.4	GDB/MI Breakpoint Information	622
27.7.5	GDB/MI Frame Information	625
27.7.6	GDB/MI Thread Information	625
27.7.7	GDB/MI Ada Exception Information	626
27.8	Simple Examples of GDB/MI Interaction	626
27.9	GDB/MI Command Description Format	627
27.10	GDB/MI Breakpoint Commands	627
27.11	GDB/MI Catchpoint Commands	638
27.11.1	Shared Library GDB/MI Catchpoints	638
27.11.2	Ada Exception GDB/MI Catchpoints	639
27.11.3	C++ Exception GDB/MI Catchpoints	640
27.12	GDB/MI Program Context	642
27.13	GDB/MI Thread Commands	645
27.14	GDB/MI Ada Tasking Commands	646
27.15	GDB/MI Program Execution	647
27.16	GDB/MI Stack Manipulation Commands	654
27.17	GDB/MI Variable Objects	660
27.18	GDB/MI Data Manipulation	671
27.19	GDB/MI Tracepoint Commands	681
27.20	GDB/MI Symbol Query Commands	686
27.21	GDB/MI File Commands	694
27.22	GDB/MI Target Manipulation Commands	698
27.23	GDB/MI File Transfer Commands	702
27.24	Ada Exceptions GDB/MI Commands	703

27.25	GDB/MI Support Commands	704
27.26	Miscellaneous GDB/MI Commands	706
28	GDB Annotations	715
28.1	What is an Annotation?	715
28.2	The Server Prefix	716
28.3	Annotation for GDB Input	716
28.4	Errors	717
28.5	Invalidation Notices	717
28.6	Running the Program	717
28.7	Displaying Source	718
29	Debugger Adapter Protocol	719
30	JIT Compilation Interface	721
30.1	JIT Declarations	721
30.2	Registering Code	722
30.3	Unregistering Code	722
30.4	Custom Debug Info	722
30.4.1	Using JIT Debug Info Readers	723
30.4.2	Writing JIT Debug Info Readers	723
31	In-Process Agent	725
31.1	In-Process Agent Protocol	725
31.1.1	IPA Protocol Objects	726
31.1.2	IPA Protocol Commands	727
32	Reporting Bugs in GDB	729
32.1	Have You Found a Bug?	729
32.2	How to Report Bugs	729
33	Command Line Editing	733
33.1	Introduction to Line Editing	733
33.2	Readline Interaction	733
33.2.1	Readline Bare Essentials	733
33.2.2	Readline Movement Commands	734
33.2.3	Readline Killing Commands	734
33.2.4	Readline Arguments	735
33.2.5	Searching for Commands in the History	735
33.3	Readline Init File	736
33.3.1	Readline Init File Syntax	736
33.3.2	Conditional Init Constructs	745
33.3.3	Sample Init File	746
33.4	Bindable Readline Commands	749
33.4.1	Commands For Moving	749
33.4.2	Commands For Manipulating The History	750

33.4.3	Commands For Changing Text	751
33.4.4	Killing And Yanking	753
33.4.5	Specifying Numeric Arguments	754
33.4.6	Letting Readline Type For You	754
33.4.7	Keyboard Macros	755
33.4.8	Some Miscellaneous Commands	755
33.5	Readline vi Mode	757
34	Using History Interactively	759
34.1	History Expansion	759
34.1.1	Event Designators	759
34.1.2	Word Designators	760
34.1.3	Modifiers	760
Appendix A	In Memoriam	763
Appendix B	Formatting Documentation	765
Appendix C	Installing GDB	767
C.1	Requirements for Building GDB	767
C.2	Invoking the GDB <code>configure</code> Script	769
C.3	Compiling GDB in Another Directory	770
C.4	Specifying Names for Hosts and Targets	771
C.5	<code>configure</code> Options	771
C.6	System-wide configuration and settings	775
C.6.1	Installed System-wide Configuration Scripts	776
Appendix D	Maintenance Commands	777
Appendix E	GDB Remote Serial Protocol	795
E.1	Overview	795
E.2	Standard Replies	796
E.3	Packets	797
E.4	Stop Reply Packets	807
E.5	General Query Packets	811
E.6	Architecture-Specific Protocol Details	839
E.6.1	ARM-specific Protocol Details	839
E.6.1.1	ARM Breakpoint Kinds	840
E.6.1.2	ARM Memory Tag Types	840
E.6.2	MIPS-specific Protocol Details	840
E.6.2.1	MIPS Register Packet Format	840
E.6.2.2	MIPS Breakpoint Kinds	840
E.7	Tracepoint Packets	840
E.7.1	Relocate instruction reply packet	847
E.8	Host I/O Packets	847
E.9	Interrupts	849

E.10	Notification Packets	850
E.11	Remote Protocol Support for Non-Stop Mode	852
E.12	Packet Acknowledgment	853
E.13	Examples	853
E.14	File-I/O Remote Protocol Extension	854
E.14.1	File-I/O Overview	854
E.14.2	Protocol Basics	854
E.14.3	The F Request Packet	855
E.14.4	The F Reply Packet	855
E.14.5	The ‘ Ctrl-C ’ Message	856
E.14.6	Console I/O	856
E.14.7	List of Supported Calls	856
open		856
close		858
read		858
write		858
lseek		859
rename		859
unlink		860
stat/fstat		860
gettimeofday		861
isatty		861
system		861
E.14.8	Protocol-specific Representation of Datatypes	862
Integral Datatypes		862
Pointer Values		862
Memory Transfer		862
struct stat		863
struct timeval		863
E.14.9	Constants	864
Open Flags		864
mode_t Values		864
Errno Values		864
Lseek Flags		865
Limits		865
E.14.10	File-I/O Examples	865
E.15	Library List Format	865
E.16	Library List Format for SVR4 Targets	866
E.17	Memory Map Format	867
E.18	Thread List Format	868
E.19	Traceframe Info Format	869
E.20	Branch Trace Format	869
E.21	Branch Trace Configuration Format	870

Appendix F	The GDB Agent	
	Expression Mechanism	871
F.1	General Bytecode Design	871
F.2	Bytecode Descriptions	873
F.3	Using Agent Expressions	878
F.4	Varying Target Capabilities	879
F.5	Rationale	879
Appendix G	Target Descriptions	883
G.1	Retrieving Descriptions	883
G.2	Target Description Format	883
G.2.1	Inclusion	884
G.2.2	Architecture	884
G.2.3	OS ABI	885
G.2.4	Compatible Architecture	885
G.2.5	Features	885
G.2.6	Types	885
G.2.7	Registers	886
G.3	Predefined Target Types	887
G.4	Enum Target Types	888
G.5	Standard Target Features	889
G.5.1	AArch64 Features	889
G.5.1.1	AArch64 core registers feature	889
G.5.1.2	AArch64 floating-point registers feature	889
G.5.1.3	AArch64 SVE registers feature	890
G.5.1.4	AArch64 Pointer Authentication registers feature	891
G.5.1.5	AArch64 TLS registers feature	891
G.5.1.6	AArch64 MTE registers feature	892
G.5.1.7	AArch64 SME registers feature	892
G.5.1.8	AArch64 SME2 registers feature	892
G.5.1.9	AArch64 GCS registers feature	893
G.5.1.10	AArch64 FPMR feature	893
G.5.2	Alpha Features	894
G.5.3	ARC Features	894
G.5.4	ARM Features	895
G.5.4.1	Core register set for non-M-profile	895
G.5.4.2	Core register set for M-profile	895
G.5.4.3	FPA registers feature (obsolete)	896
G.5.4.4	M-profile Vector Extension (MVE)	896
G.5.4.5	XScale iwMMXt feature	896
G.5.4.6	Vector Floating-Point (VFP) feature	897
G.5.4.7	NEON architecture feature	897
G.5.4.8	M-profile Pointer Authentication and Branch Target Identification feature	897
G.5.4.9	M-profile system registers feature	897
G.5.4.10	M-profile Security Extensions feature	898
G.5.4.11	TLS registers feature	898

G.5.5	i386 Features	898
G.5.6	LoongArch Features	899
G.5.7	MicroBlaze Features	900
G.5.8	MIPS Features	900
G.5.9	M68K Features	900
G.5.10	NDS32 Features	900
G.5.11	Openrisc 1000 Features	901
G.5.12	PowerPC Features	901
G.5.13	RISC-V Features	902
G.5.14	RX Features	903
G.5.15	S/390 and System z Features	903
G.5.16	Sparc Features	903
G.5.17	TMS320C6x Features	904
 Appendix H Operating System Information ..		905
H.1	Process list	905
 Appendix I Trace File Format		907
 Appendix J .gdb_index section format		909
 Appendix K Download debugging resources with Debuginfod		913
K.1	Debuginfod Settings	913
 Appendix L Manual pages		915
 Appendix M GNU GENERAL PUBLIC LICENSE		925
 Appendix N GNU Free Documentation License ..		937
 Concept Index		945
 Command, Variable, and Function Index		961

Summary of GDB

The purpose of a debugger such as GDB is to allow you to see what is going on “inside” another program while it executes—or what another program was doing at the moment it crashed.

GDB can do four main kinds of things (plus other things in support of these) to help you catch bugs in the act:

- Start your program, specifying anything that might affect its behavior.
- Make your program stop on specified conditions.
- Examine what has happened, when your program has stopped.
- Change things in your program, so you can experiment with correcting the effects of one bug and go on to learn about another.

You can use GDB to debug programs written in C and C++. For more information, see Section 15.4 [Supported Languages], page 233. For more information, see Section 15.4.1 [C and C++], page 233.

Support for D is partial. For information on D, see Section 15.4.2 [D], page 240.

Support for Modula-2 is partial. For information on Modula-2, see Section 15.4.9 [Modula-2], page 246.

Support for OpenCL C is partial. For information on OpenCL C, see Section 15.4.5 [OpenCL C], page 241.

Debugging Pascal programs which use sets, subranges, file variables, or nested functions does not currently work. GDB does not support entering expressions, printing values, or similar features using Pascal syntax.

GDB can be used to debug programs written in Fortran, although it may be necessary to refer to some variables with a trailing underscore.

GDB can be used to debug programs written in Objective-C, using either the Apple/NeXT or the GNU Objective-C runtime.

Free Software

GDB is *free software*, protected by the GNU General Public License (GPL). The GPL gives you the freedom to copy or adapt a licensed program—but every person getting a copy also gets with it the freedom to modify that copy (which means that they must get access to the source code), and the freedom to distribute further copies. Typical software companies use copyrights to limit your freedoms; the Free Software Foundation uses the GPL to preserve these freedoms.

Fundamentally, the General Public License is a license which says that you have these freedoms and that you cannot take these freedoms away from anyone else.

Free Software Needs Free Documentation

The biggest deficiency in the free software community today is not in the software—it is the lack of good free documentation that we can include with the free software. Many of our most important programs do not come with free reference manuals and free introductory

texts. Documentation is an essential part of any software package; when an important free software package does not come with a free manual and a free tutorial, that is a major gap. We have many such gaps today.

Consider Perl, for instance. The tutorial manuals that people normally use are non-free. How did this come about? Because the authors of those manuals published them with restrictive terms—no copying, no modification, source files not available—which exclude them from the free software world.

That wasn't the first time this sort of thing happened, and it was far from the last. Many times we have heard a GNU user eagerly describe a manual that he is writing, his intended contribution to the community, only to learn that he had ruined everything by signing a publication contract to make it non-free.

Free documentation, like free software, is a matter of freedom, not price. The problem with the non-free manual is not that publishers charge a price for printed copies—that in itself is fine. (The Free Software Foundation sells printed copies of manuals, too.) The problem is the restrictions on the use of the manual. Free manuals are available in source code form, and give you permission to copy and modify. Non-free manuals do not allow this.

The criteria of freedom for a free manual are roughly the same as for free software. Redistribution (including the normal kinds of commercial redistribution) must be permitted, so that the manual can accompany every copy of the program, both on-line and on paper.

Permission for modification of the technical content is crucial too. When people modify the software, adding or changing features, if they are conscientious they will change the manual too—so they can provide accurate and clear documentation for the modified program. A manual that leaves you no choice but to write a new manual to document a changed version of the program is not really available to our community.

Some kinds of limits on the way modification is handled are acceptable. For example, requirements to preserve the original author's copyright notice, the distribution terms, or the list of authors, are ok. It is also no problem to require modified versions to include notice that they were modified. Even entire sections that may not be deleted or changed are acceptable, as long as they deal with nontechnical topics (like this one). These kinds of restrictions are acceptable because they don't obstruct the community's normal use of the manual.

However, it must be possible to modify all the *technical* content of the manual, and then distribute the result in all the usual media, through all the usual channels. Otherwise, the restrictions obstruct the use of the manual, it is not free, and we need another manual to replace it.

Please spread the word about this issue. Our community continues to lose manuals to proprietary publishing. If we spread the word that free software needs free reference manuals and free tutorials, perhaps the next person who wants to contribute by writing documentation will realize, before it is too late, that only free manuals contribute to the free software community.

If you are writing documentation, please insist on publishing it under the GNU Free Documentation License or another free documentation license. Remember that this decision requires your approval—you don't have to let the publisher decide. Some commercial publishers will use a free license if you insist, but they will not propose the option; it is up

to you to raise the issue and say firmly that this is what you want. If the publisher you are dealing with refuses, please try other publishers. If you're not sure whether a proposed license is free, write to licensing@gnu.org.

You can encourage commercial publishers to sell more free, copylefted manuals and tutorials by buying them, and particularly by buying copies from the publishers that paid for their writing or for major improvements. Meanwhile, try to avoid buying non-free documentation at all. Check the distribution terms of a manual before you buy it, and insist that whoever seeks your business must respect your freedom. Check the history of the book, and try to reward the publishers that have paid or pay the authors to work on it.

The Free Software Foundation maintains a list of free documentation published by other publishers, at <http://www.fsf.org/doc/other-free-books.html>.

Contributors to GDB

Richard Stallman was the original author of GDB, and of many other GNU programs. Many others have contributed to its development. This section attempts to credit major contributors. One of the virtues of free software is that everyone is free to contribute to it; with regret, we cannot actually acknowledge everyone here. The file **ChangeLog** in the GDB distribution approximates a blow-by-blow account.

Changes much prior to version 2.0 are lost in the mists of time.

Plea: Additions to this section are particularly welcome. If you or your friends (or enemies, to be evenhanded) have been unfairly omitted from this list, we would like to add your names!

So that they may not regard their many labors as thankless, we particularly thank those who shepherded GDB through major releases: Andrew Cagney (releases 6.3, 6.2, 6.1, 6.0, 5.3, 5.2, 5.1 and 5.0); Jim Blandy (release 4.18); Jason Molenda (release 4.17); Stan Shebs (release 4.14); Fred Fish (releases 4.16, 4.15, 4.13, 4.12, 4.11, 4.10, and 4.9); Stu Grossman and John Gilmore (releases 4.8, 4.7, 4.6, 4.5, and 4.4); John Gilmore (releases 4.3, 4.2, 4.1, 4.0, and 3.9); Jim Kingdon (releases 3.5, 3.4, and 3.3); and Randy Smith (releases 3.2, 3.1, and 3.0).

Richard Stallman, assisted at various times by Peter TerMaat, Chris Hanson, and Richard Mlynarik, handled releases through 2.8.

Michael Tiemann is the author of most of the GNU C++ support in GDB, with significant additional contributions from Per Bothner and Daniel Berlin. James Clark wrote the GNU C++ demangler. Early work on C++ was by Peter TerMaat (who also did much general update work leading to release 3.0).

GDB uses the BFD subroutine library to examine multiple object-file formats; BFD was a joint project of David V. Henkel-Wallace, Rich Pixley, Steve Chamberlain, and John Gilmore.

David Johnson wrote the original COFF support; Pace Willison did the original support for encapsulated COFF.

Brent Benson of Harris Computer Systems contributed DWARF 2 support.

Adam de Boor and Bradley Davis contributed the ISI Optimum V support. Per Bothner, Noboyuki Hikichi, and Alessandro Forin contributed MIPS support. Jean-Daniel Fekete contributed Sun 386i support. Chris Hanson improved the HP9000 support. Noboyuki

Hikichi and Tomoyuki Hasei contributed Sony/News OS 3 support. David Johnson contributed Encore Umax support. Jyrki Kuoppala contributed Altos 3068 support. Jeff Law contributed HP PA and SOM support. Keith Packard contributed NS32K support. Doug Rabson contributed Acorn Risc Machine support. Bob Rusk contributed Harris Nighthawk CX-UX support. Chris Smith contributed Convex support (and Fortran debugging). Jonathan Stone contributed Pyramid support. Michael Tiemann contributed SPARC support. Tim Tucker contributed support for the Gould NP1 and Gould Powernode. Pace Willison contributed Intel 386 support. Jay Vosburgh contributed Symmetry support. Marko Mlinar contributed OpenRISC 1000 support.

Andreas Schwab contributed M68K GNU/Linux support.

Rich Schaefer and Peter Schauer helped with support of SunOS shared libraries.

Jay Fenlason and Roland McGrath ensured that GDB and GAS agree about several machine instruction sets.

Patrick Duval, Ted Goldstein, Vikram Koka and Glenn Engel helped develop remote debugging. Intel Corporation, Wind River Systems, AMD, and ARM contributed remote debugging modules for the i960, VxWorks, A29K UDI, and RDI targets, respectively.

Brian Fox is the author of the readline libraries providing command-line editing and command history.

Andrew Beers of SUNY Buffalo wrote the language-switching code, the Modula-2 support, and contributed the Languages chapter of this manual.

Fred Fish wrote most of the support for Unix System Vr4. He also enhanced the command-completion support to cover C++ overloaded symbols.

Hitachi America (now Renesas America), Ltd. sponsored the support for H8/300, H8/500, and Super-H processors.

NEC sponsored the support for the v850, Vr4xxx, and Vr5xxx processors.

Mitsubishi (now Renesas) sponsored the support for D10V, D30V, and M32R/D processors.

Toshiba sponsored the support for the TX39 Mips processor.

Matsushita sponsored the support for the MN10200 and MN10300 processors.

Fujitsu sponsored the support for SPARClite and FR30 processors.

Kung Hsu, Jeff Law, and Rick Sladkey added support for hardware watchpoints.

Michael Snyder added support for tracepoints.

Stu Grossman wrote gdbserver.

Jim Kingdon, Peter Schauer, Ian Taylor, and Stu Grossman made nearly innumerable bug fixes and cleanups throughout GDB.

The following people at the Hewlett-Packard Company contributed support for the PA-RISC 2.0 architecture, HP-UX 10.20, 10.30, and 11.0 (narrow mode), HP's implementation of kernel threads, HP's aC++ compiler, and the Text User Interface (nee Terminal User Interface): Ben Krepp, Richard Title, John Bishop, Susan Macchia, Kathy Mann, Satish Pai, India Paul, Steve Rehrauer, and Elena Zannoni. Kim Haase provided HP-specific information in this manual.

DJ Delorie ported GDB to MS-DOS, for the DJGPP project. Robert Hoehne made significant contributions to the DJGPP port.

Cygnus Solutions has sponsored GDB maintenance and much of its development since 1991. Cygnus engineers who have worked on GDB fulltime include Mark Alexander, Jim Blandy, Per Bothner, Kevin Buettner, Edith Epstein, Chris Faylor, Fred Fish, Martin Hunt, Jim Ingham, John Gilmore, Stu Grossman, Kung Hsu, Jim Kingdon, John Metzler, Fernando Nasser, Geoffrey Noer, Dawn Perchik, Rich Pixley, Zdenek Radouch, Keith Seitz, Stan Shebs, David Taylor, and Elena Zannoni. In addition, Dave Brolley, Ian Carmichael, Steve Chamberlain, Nick Clifton, JT Conklin, Stan Cox, DJ Delorie, Ulrich Drepper, Frank Eigler, Doug Evans, Sean Fagan, David Henkel-Wallace, Richard Henderson, Jeff Holcomb, Jeff Law, Jim Lemke, Tom Lord, Bob Manson, Michael Meissner, Jason Merrill, Catherine Moore, Drew Moseley, Ken Raeburn, Gavin Romig-Koch, Rob Savoye, Jamie Smith, Mike Stump, Ian Taylor, Angela Thomas, Michael Tiemann, Tom Tromey, Ron Unrau, Jim Wilson, and David Zuhn have made contributions both large and small.

Andrew Cagney, Fernando Nasser, and Elena Zannoni, while working for Cygnus Solutions, implemented the original GDB/MI interface.

Jim Blandy added support for preprocessor macros, while working for Red Hat.

Andrew Cagney designed GDB's architecture vector. Many people including Andrew Cagney, Stephane Carrez, Randolph Chung, Nick Duffek, Richard Henderson, Mark Kettenis, Grace Sainsbury, Kei Sakamoto, Yoshinori Sato, Michael Snyder, Andreas Schwab, Jason Thorpe, Corinna Vinschen, Ulrich Weigand, and Elena Zannoni, helped with the migration of old architectures to this new framework.

Andrew Cagney completely re-designed and re-implemented GDB's unwinder framework, this consisting of a fresh new design featuring frame IDs, independent frame sniffers, and the sentinel frame. Mark Kettenis implemented the DWARF 2 unwinder, Jeff Johnston the libunwind unwinder, and Andrew Cagney the dummy, sentinel, tramp, and trad unwinders. The architecture-specific changes, each involving a complete rewrite of the architecture's frame code, were carried out by Jim Blandy, Joel Brobecker, Kevin Buettner, Andrew Cagney, Stephane Carrez, Randolph Chung, Orjan Friberg, Richard Henderson, Daniel Jacobowitz, Jeff Johnston, Mark Kettenis, Theodore A. Roth, Kei Sakamoto, Yoshinori Sato, Michael Snyder, Corinna Vinschen, and Ulrich Weigand.

Christian Zankel, Ross Morley, Bob Wilson, and Maxim Grigoriev from Tensilica, Inc. contributed support for Xtensa processors. Others who have worked on the Xtensa port of GDB in the past include Steve Tjiang, John Newlin, and Scott Foehner.

Michael Eager and staff of Xilinx, Inc., contributed support for the Xilinx MicroBlaze architecture.

Initial support for the FreeBSD/mips target and native configuration was developed by SRI International and the University of Cambridge Computer Laboratory under DARPA/AFRL contract FA8750-10-C-0237 ("CTSRD"), as part of the DARPA CRASH research programme.

Initial support for the FreeBSD/riscv target and native configuration was developed by SRI International and the University of Cambridge Computer Laboratory (Department of Computer Science and Technology) under DARPA contract HR0011-18-C-0016 ("ECATS"), as part of the DARPA SSITH research programme.

The original port to the OpenRISC 1000 is believed to be due to Alessandro Forin and Per Bothner. More recent ports have been the work of Jeremy Bennett, Franck Jullien, Stefan Wallentowitz and Stafford Horne.

Weimin Pan, David Faust and Jose E. Marchesi contributed support for the Linux kernel BPF virtual architecture. This work was sponsored by Oracle.

1 A Sample GDB Session

You can use this manual at your leisure to read all about GDB. However, a handful of commands are enough to get started using the debugger. This chapter illustrates those commands.

In this sample session, we emphasize user input like this: **input**, to make it easier to pick out from the surrounding output.

One of the preliminary versions of GNU **m4** (a generic macro processor) exhibits the following bug: sometimes, when we change its quote strings from the default, the commands used to capture one macro definition within another stop working. In the following short **m4** session, we define a macro **foo** which expands to **0000**; we then use the **m4** built-in **defn** to define **bar** as the same thing. However, when we change the open quote string to **<QUOTE>** and the close quote string to **<UNQUOTE>**, the same procedure fails to define a new synonym **baz**:

```
$ cd gnu/m4
$ ./m4
define(foo,0000)

foo
0000
define(bar,defn('foo'))

bar
0000
changequote(<QUOTE>,<UNQUOTE>)

define(baz,defn(<QUOTE>foo<UNQUOTE>))
baz
Ctrl-d
m4: End of input: 0: fatal error: EOF in string
```

Let us use GDB to try to see what is going on.

```
$ gdb m4
GDB is free software and you are welcome to distribute copies
of it under certain conditions; type "show copying" to see
the conditions.
There is absolutely no warranty for GDB; type "show warranty"
for details.

GDB 18.0.50.20260503-git, Copyright 1999 Free Software Foundation, Inc...
(gdb)
```

GDB reads only enough symbol data to know where to find the rest when needed; as a result, the first prompt comes up very quickly. We now tell GDB to use a narrower display width than usual, so that examples fit in this manual.

```
(gdb) set width 70
```

We need to see how the **m4** built-in **changequote** works. Having looked at the source, we know the relevant subroutine is **m4_changequote**, so we set a breakpoint there with the GDB **break** command.

```
(gdb) break m4.changequote
Breakpoint 1 at 0x62f4: file builtin.c, line 879.
```

Using the `run` command, we start `m4` running under GDB control; as long as control does not reach the `m4_changequote` subroutine, the program runs as usual:

```
(gdb) run
Starting program: /work/Editorial/gdb/gnu/m4/m4
define(foo,0000)

foo
0000
```

To trigger the breakpoint, we call `changequote`. GDB suspends execution of `m4`, displaying information about the context where it stops.

```
changequote(<QUOTE>,<UNQUOTE>)

Breakpoint 1, m4_changequote (argc=3, argv=0x33c70)
  at builtin.c:879
879      if (bad_argc(TOKEN_DATA_TEXT(argv[0]),argc,1,3))
```

Now we use the command `n` (`next`) to advance execution to the next line of the current function.

```
(gdb) n
882      set_quotes((argc >= 2) ? TOKEN_DATA_TEXT(argv[1])\
: nil,
```

`set_quotes` looks like a promising subroutine. We can go into it by using the command `s` (`step`) instead of `next`. `step` goes to the next line to be executed in *any* subroutine, so it steps into `set_quotes`.

```
(gdb) s
set_quotes (lq=0x34c78 "<QUOTE>", rq=0x34c88 "<UNQUOTE>")
  at input.c:530
530      if (lquote != def_lquote)
```

The display that shows the subroutine where `m4` is now suspended (and its arguments) is called a stack frame display. It shows a summary of the stack. We can use the `backtrace` command (which can also be spelled `bt`), to see where we are in the stack as a whole: the `backtrace` command displays a stack frame for each active subroutine.

```
(gdb) bt
#0  set_quotes (lq=0x34c78 "<QUOTE>", rq=0x34c88 "<UNQUOTE>")
  at input.c:530
#1  0x6344 in m4_changequote (argc=3, argv=0x33c70)
  at builtin.c:882
#2  0x8174 in expand_macro (sym=0x33320) at macro.c:242
#3  0x7a88 in expand_token (obs=0x0, t=209696, td=0xf7ffa30)
  at macro.c:71
#4  0x79dc in expand_input () at macro.c:40
#5  0x2930 in main (argc=0, argv=0xf7ffb20) at m4.c:195
```

We step through a few more lines to see what happens. The first two times, we can use `'s'`; the next two times we use `n` to avoid falling into the `xstrdup` subroutine.

```
(gdb) s
0x3b5c 532      if (rquote != def_rquote)
(gdb) s
0x3b80 535      lquote = (lq == nil || *lq == '\0') ? \
def_lquote : xstrdup(lq);
(gdb) n
536      rquote = (rq == nil || *rq == '\0') ? def_rquote\
: xstrdup(rq);
(gdb) n
```

```
538         len_lquote = strlen(rquote);
```

The last line displayed looks a little odd; we can examine the variables `lquote` and `rquote` to see if they are in fact the new left and right quotes we specified. We use the command `p` (`print`) to see their values.

```
(gdb) p lquote
$1 = 0x35d40 "<QUOTE>"
(gdb) p rquote
$2 = 0x35d50 "<UNQUOTE>"
```

`lquote` and `rquote` are indeed the new left and right quotes. To look at some context, we can display ten lines of source surrounding the current line with the `l` (`list`) command.

```
(gdb) l
533         xfree(rquote);
534
535         lquote = (lq == nil || *lq == '\0') ? def_lquote\
: xstrdup (lq);
536         rquote = (rq == nil || *rq == '\0') ? def_rquote\
: xstrdup (rq);
537
538         len_lquote = strlen(rquote);
539         len_rquote = strlen(lquote);
540     }
541
542     void
```

Let us step past the two lines that set `len_lquote` and `len_rquote`, and then examine the values of those variables.

```
(gdb) n
539         len_rquote = strlen(lquote);
(gdb) n
540     }
(gdb) p len_lquote
$3 = 9
(gdb) p len_rquote
$4 = 7
```

That certainly looks wrong, assuming `len_lquote` and `len_rquote` are meant to be the lengths of `lquote` and `rquote` respectively. We can set them to better values using the `p` command, since it can print the value of any expression—and that expression can include subroutine calls and assignments.

```
(gdb) p len_lquote==strlen(lquote)
$5 = 7
(gdb) p len_rquote==strlen(rquote)
$6 = 9
```

Is that enough to fix the problem of using the new quotes with the `m4` built-in `defn`? We can allow `m4` to continue executing with the `c` (`continue`) command, and then try the example that caused trouble initially:

```
(gdb) c
Continuing.

define(baz,defn(<QUOTE>foo<UNQUOTE>))

baz
0000
```

Success! The new quotes now work just as well as the default ones. The problem seems to have been just the two typos defining the wrong lengths. We allow `m4` exit by giving it an EOF as input:

```
Ctrl-d
Program exited normally.
```

The message ‘`Program exited normally.`’ is from GDB; it indicates `m4` has finished executing. We can end our GDB session with the GDB `quit` command.

```
(gdb) quit
```

2 Getting In and Out of GDB

This chapter discusses how to start GDB, and how to get out of it. The essentials are:

- type `'gdb'` to start GDB.
- type `quit`, `exit` or `Ctrl-d` to exit.

2.1 Invoking GDB

Invoke GDB by running the program `gdb`. Once started, GDB reads commands from the terminal until you tell it to exit.

You can also run `gdb` with a variety of arguments and options, to specify more of your debugging environment at the outset.

The command-line options described here are designed to cover a variety of situations; in some environments, some of these options may effectively be unavailable.

The most usual way to start GDB is with one argument, specifying an executable program:

```
gdb program
```

You can also start with both an executable program and a core file specified:

```
gdb program core
```

You can, instead, specify a process ID as a second argument or use option `-p`, if you want to debug a running process:

```
gdb program 1234
gdb -p 1234
```

would attach GDB to process 1234. With option `-p` you can omit the *program* filename.

Taking advantage of the second command-line argument requires a fairly complete operating system; when you use GDB as a remote debugger attached to a bare board, there may not be any notion of “process”, and there is often no way to get a core dump. GDB will warn you if it is unable to attach or to read core dumps.

You can optionally have `gdb` pass any arguments after the executable file to the inferior using `--args` or `--no-escape-args`. These options stop option processing.

```
gdb --args gcc -O2 -c foo.c
```

This will cause `gdb` to debug `gcc`, and to set `gcc`’s command-line arguments (see Section 4.3 [Arguments], page 38) to `-O2 -c foo.c`. For the differences between `--args` and `--no-escape-args`, see [`--args` and `--no-escape-args`], page 15.

You can run `gdb` without printing the front material, which describes GDB’s non-warranty, by specifying `--silent` (or `-q/--quiet`):

```
gdb --silent
```

You can further control how GDB starts up by using command-line options. GDB itself can remind you of the options available.

Type

```
gdb -help
```

to display all available options and briefly describe their use (`'gdb -h'` is a shorter equivalent).

All options and command line arguments you give are processed in sequential order. The order makes a difference when the `'-x'` option is used.

2.1.1 Choosing Files

When GDB starts, it reads any arguments other than options as specifying an executable file and core file (or process ID). This is the same as if the arguments were specified by the ‘-se’ and ‘-c’ (or ‘-p’) options respectively. (GDB reads the first argument that does not have an associated option flag as equivalent to the ‘-se’ option followed by that argument; and the second argument that does not have an associated option flag, if any, as equivalent to the ‘-c’/‘-p’ option followed by that argument.) If the second argument begins with a decimal digit, GDB will first attempt to attach to it as a process, and if that fails, attempt to open it as a corefile. If you have a corefile whose name begins with a digit, you can prevent GDB from treating it as a pid by prefixing it with ./, e.g. ./12345.

If GDB has not been configured to include core file support, such as for most embedded targets, then it will complain about a second argument and ignore it.

For the ‘-s’, ‘-e’, and ‘-se’ options, and their long form equivalents, the method used to search the file system for the symbol and/or executable file is the same as that used by the `file` command. See Section 18.1 [file], page 293.

Many options have both long and short forms; both are shown in the following list. GDB also recognizes the long forms if you truncate them, so long as enough of the option is present to be unambiguous. (If you prefer, you can flag option arguments with ‘--’ rather than ‘-’, though we illustrate the more usual convention.)

```
-symbols file
-s file      Read symbol table from file file.

-exec file
-e file      Use file file as the executable file to execute when appropriate, and for examining
              pure data in conjunction with a core dump.

-se file     Read symbol table from file file and use it as the executable file.

-core file
-c file      Use file file as a core dump to examine.

-pid number
-p number    Connect to process ID number, as with the attach command.

-command file
-x file      Execute commands from file file. The contents of this file is evaluated exactly
              as the source command would. See Section 23.1.3 [Command files], page 401.

-eval-command command
-ex command
              Execute a single GDB command.

              This option may be used multiple times to call multiple commands. It may also
              be interleaved with ‘-command’ as required.
              gdb -ex 'target sim' -ex 'load' \
                -x setbreakpoints -ex 'run' a.out

-init-command file
-ix file     Execute commands from file file before loading the inferior (but after loading
              gdbinit files). See Section 2.1.3 [Startup], page 17.
```

- `-init-eval-command command`
- `-iex command`
Execute a single GDB command before loading the inferior (but after loading gdbinit files). See Section 2.1.3 [Startup], page 17.
- `-early-init-command file`
- `-eix file` Execute commands from *file* very early in the initialization process, before any output is produced. See Section 2.1.3 [Startup], page 17.
- `-early-init-eval-command command`
- `-eiex command`
Execute a single GDB command very early in the initialization process, before any output is produced.
- `-directory directory`
- `-d directory`
Add *directory* to the path to search for source and script files.
- `-r`
- `-readnow` Read each symbol file's entire symbol table immediately, rather than the default, which is to read it incrementally as it is needed. This makes startup slower, but makes future operations faster.
- `--readnever`
Do not read each symbol file's symbolic debug information. This makes startup faster but at the expense of not being able to perform symbolic debugging. DWARF unwind information is also not read, meaning backtraces may become incomplete or inaccurate. One use of this is when a user simply wants to do the following sequence: attach, dump core, detach. Loading the debugging information in this case is an unnecessary cause of delay.

2.1.2 Choosing Modes

You can run GDB in various alternative modes—for example, in batch mode or quiet mode.

- `-nx`
- `-n` Do not execute commands found in any initialization files (see Section 2.1.4 [Initialization Files], page 18).
- `-nh` Do not execute commands found in any home directory initialization file (see Section 2.1.4 [Home directory initialization file], page 18). The system wide and current directory initialization files are still loaded.
- `-quiet`
- `-silent`
- `-q` “Quiet”. Do not print the introductory and copyright messages. These messages are also suppressed in batch mode.

This can also be enabled using `set startup-quietly on`. The default is off. Use `show startup-quietly` to see the current setting. Place `set startup-quietly on` into your early initialization file (see Section 2.1.4 [Initialization Files], page 18) to have future GDB sessions startup quietly.

-batch Run in batch mode. Exit with status 0 after processing all the command files specified with ‘-x’ (and all commands from initialization files, if not inhibited with ‘-n’). Exit with nonzero status if an error occurs in executing the GDB commands in the command files. Batch mode also disables pagination, sets unlimited terminal width and height see Section 22.4 [Screen Size], page 374, and acts as if *set confirm off* were in effect (see Section 22.9 [Messages/Warnings], page 386).

Batch mode may be useful for running GDB as a filter, for example to download and run a program on another computer; in order to make this more useful, the message

Program exited normally.

(which is ordinarily issued whenever a program running under GDB control terminates) is not issued when running in batch mode.

-batch-silent

Run in batch mode exactly like ‘-batch’, but totally silently. All GDB output to `stdout` is prevented (`stderr` is unaffected). This is much quieter than ‘-silent’ and would be useless for an interactive session.

This is particularly useful when using targets that give ‘Loading section’ messages, for example.

Note that targets that give their output via GDB, as opposed to writing directly to `stdout`, will also be made silent.

-return-child-result

The return code from GDB will be the return code from the child process (the process being debugged), with the following exceptions:

- GDB exits abnormally. E.g., due to an incorrect argument or an internal error. In this case the exit code is the same as it would have been without ‘-return-child-result’.
- The user quits with an explicit value. E.g., ‘quit 1’.
- The child process never runs, or is not allowed to terminate, in which case the exit code will be -1.

This option is useful in conjunction with ‘-batch’ or ‘-batch-silent’, when GDB is being used as a remote program loader or simulator interface.

-nowindows

-nw “No windows”. If GDB comes with a graphical user interface (GUI) built in, then this option tells GDB to only use the command-line interface. If no GUI is available, this option has no effect.

-windows

-w If GDB includes a GUI, then this option requires it to be used if possible.

-cd *directory*

Run GDB using *directory* as its working directory, instead of the current directory.

-data-directory *directory*

-D *directory*

Run GDB using *directory* as its data directory. The data directory is where GDB searches for its auxiliary files. See Section 18.8 [Data Files], page 312.

-binary-output

Instructs GDB to set the translation mode of its `stdout/stderr` to binary. MS-Windows only. Useful for running the GDB testsuite. By default, GDB opens `stdout/stderr` in text mode, and translates ‘\n’ (LF, a single Line Feed) into ‘\r\n’ (CR-LF, a Carriage Return-Line Feed combination). If this option is set, no translation is done.

-fullname

-f GNU Emacs sets this option when it runs GDB as a subprocess. It tells GDB to output the full file name and line number in a standard, recognizable fashion each time a stack frame is displayed (which includes each time your program stops). This recognizable format looks like two ‘\032’ characters, followed by the file name, line number and character position separated by colons, and a newline. The Emacs-to-GDB interface program uses the two ‘\032’ characters as a signal to display the source code for the frame.

-annotate *level*

This option sets the *annotation level* inside GDB. Its effect is identical to using ‘`set annotate level`’ (see Chapter 28 [Annotations], page 715). The *annotation level* controls how much information GDB prints together with its prompt, values of expressions, source lines, and other types of output. Level 0 is the normal, level 1 is for use when GDB is run as a subprocess of GNU Emacs, level 3 is the maximum annotation suitable for programs that control GDB, and level 2 has been deprecated.

The annotation mechanism has largely been superseded by GDB/MI (see Chapter 27 [GDB/MI], page 609).

--args

Change interpretation of command line so that arguments following the executable file are passed as command line arguments to the inferior. This option stops option processing.

Arguments supplied using **--args** will have backslashes applied to escape any special shell characters. This ensures that when the inferior starts it is passed arguments exactly as GDB receives them.

For example, consider the following command run under a shell:

```
$ gdb --args ls *.c
```

In this case the shell will expand `*.c` at the time GDB is invoked, not at the time that the inferior is invoked. As a result, if an additional `.c` file is created after GDB is started, but before the inferior is started, then the inferior will not show the file in its output; the list of matching files was resolved at the time GDB was started.

If you quote the `*` character used on the GDB command line argument then this will prevent the shell that starts GDB from expanding the `*.c` pattern, however, this quoting will also be passed to the shell that GDB invokes in order to start

the inferior (see [set startup-with-shell], page 36), and this will prevent the `*.c` pattern being expanded at this point either:

```
$ gdb --args ls '*.c'
(gdb) show args
```

Argument list to give program being debugged when it is started is `"*.log"`.

If this quoting behaviour does not meet your needs, then you could use `--no-escape-args` instead, which is described below.

`--no-escape-args`

Change interpretation of command line so that arguments following the executable file are passed as command line arguments to the inferior. This option stops option processing.

Unlike `--args`, arguments after the executable name will not have any escaping applied to them. As a result, any special shell characters that are not expanded by the shell that invokes GDB will be expanded by the shell that GDB uses to start the inferior.

```
$ gdb --no-escape-args ls '*.c'
(gdb) show args
```

Argument list to give program being debugged when it is started is `"*.log"`.

`-baud bps`

`-b bps` Set the line speed (baud rate or bits per second) of any serial interface used by GDB for remote debugging.

`-l timeout`

Set the timeout (in seconds) of any communication used by GDB for remote debugging.

`-tty device`

`-t device` Run using *device* for your program's standard input and output.

`-tui`

Activate the *Text User Interface* when starting. The Text User Interface manages several text windows on the terminal, showing source, assembly, registers and GDB command outputs (see Chapter 25 [GDB Text User Interface], page 599). Do not use this option if you run GDB from Emacs (see Chapter 26 [Using GDB under GNU Emacs], page 607).

`-interpreter interp`

Use the interpreter *interp* for interface with the controlling program or device. This option is meant to be set by programs which communicate with GDB using it as a back end. See Chapter 24 [Command Interpreters], page 597.

`'--interpreter=mi'` (or `'--interpreter=mi3'`) causes GDB to use the GDB/MI *interface* version 3 (see Chapter 27 [The GDB/MI Interface], page 609) included since GDB version 9.1. GDB/MI version 2 (*mi2*), included in GDB 6.0 and version 1 (*mi1*), included in GDB 5.3, are also available. Earlier GDB/MI interfaces are no longer supported.

`-write`

Open the executable and core files for both reading and writing. This is equivalent to the `'set write on'` command inside GDB (see Section 17.6 [Patching], page 287).

-statistics

This option causes GDB to print statistics about time and memory usage after it completes each command and returns to the prompt.

-version This option causes GDB to print its version number and no-warranty blurb, and exit.

-configuration

This option causes GDB to print details about its build-time configuration parameters, and then exit. These details can be important when reporting GDB bugs (see Chapter 32 [GDB Bugs], page 729).

2.1.3 What GDB Does During Startup

Here's the description of what GDB does during session startup:

1. Performs minimal setup required to initialize basic internal state.
2. Reads commands from the early initialization file (if any) in your home directory. Only a restricted set of commands can be placed into an early initialization file, see Section 2.1.4 [Initialization Files], page 18, for details.
3. Executes commands and command files specified by the `'-eiex'` and `'-eix'` command line options in their specified order. Only a restricted set of commands can be used with `'-eiex'` and `'eix'`, see Section 2.1.4 [Initialization Files], page 18, for details.
4. Sets up the command interpreter as specified by the command line (see Section 2.1.2 [Mode Options], page 13).
5. Reads the system wide initialization file and the files from the system wide initialization directory, see [System Wide Init Files], page 19.
6. Reads the initialization file (if any) in your home directory and executes all the commands in that file, see [Home Directory Init File], page 19.
7. Executes commands and command files specified by the `'-iex'` and `'-ix'` options in their specified order. Usually you should use the `'-ex'` and `'-x'` options instead, but this way you can apply settings before GDB init files get executed and before inferior gets loaded.
8. Processes command line options and operands.
9. Reads and executes the commands from the initialization file (if any) in the current working directory as long as `'set auto-load local-gdbinit'` is set to `'on'` (see Section 22.8.1 [Init File in the Current Directory], page 383). This is only done if the current directory is different from your home directory. Thus, you can have more than one init file, one generic in your home directory, and another, specific to the program you are debugging, in the directory where you invoke GDB. See [Init File in the Current Directory during Startup], page 20.
10. If the command line specified a program to debug, or a process to attach to, or a core file, GDB loads any auto-loaded scripts provided for the program or for its loaded shared libraries. See Section 22.8 [Auto-loading], page 381.

If you wish to disable the auto-loading during startup, you must do something like the following:

```
$ gdb -iex "set auto-load python-scripts off" myprogram
```

Option `'-ex'` does not work because the auto-loading is then turned off too late.

11. Executes commands and command files specified by the ‘-ex’ and ‘-x’ options in their specified order. See Section 23.1.3 [Command Files], page 401, for more details about GDB command files.
12. Reads the command history recorded in the *history file*. See Section 22.3 [Command History], page 372, for more details about the command history and the files where GDB records it.

2.1.4 Initialization Files

During startup (see Section 2.1.3 [Startup], page 17) GDB will execute commands from several initialization files. These initialization files use the same syntax as *command files* (see Section 23.1.3 [Command Files], page 401) and are processed by GDB in the same way.

To display the list of initialization files loaded by GDB at startup, in the order they will be loaded, you can use `gdb --help`.

The *early initialization* file is loaded very early in GDB’s initialization process, before the interpreter (see Chapter 24 [Interpreters], page 597) has been initialized, and before the default target (see Chapter 19 [Targets], page 313) is initialized. Only **set** or **source** commands should be placed into an early initialization file, and the only **set** commands that can be used are those that control how GDB starts up.

Commands that can be placed into an early initialization file will be documented as such throughout this manual. Any command that is not documented as being suitable for an early initialization file should instead be placed into a general initialization file. Command files passed to `--early-init-command` or `-eix` are also early initialization files, with the same command restrictions. Only commands that can appear in an early initialization file should be passed to `--early-init-eval-command` or `-eiex`.

In contrast, the *general initialization* files are processed later, after GDB has finished its own internal initialization process, any valid command can be used in these files.

Throughout the rest of this document the term *initialization file* refers to one of the general initialization files, not the early initialization file. Any discussion of the early initialization file will specifically mention that it is the early initialization file being discussed.

As the system wide and home directory initialization files are processed before most command line options, changes to settings (e.g. ‘**set complaints**’) can affect subsequent processing of command line options and operands.

The following sections describe where GDB looks for the early initialization and initialization files, and the order that the files are searched for.

2.1.4.1 Home directory early initialization files

GDB initially looks for an early initialization file in the users home directory¹. There are a number of locations that GDB will search in the home directory, these locations are searched in order and GDB will load the first file that it finds, and subsequent locations will not be checked.

On non-macOS hosts the locations searched are:

- The file `gdb/gdbearlyinit` within the directory pointed to by the environment variable `XD_CONFIG_HOME`, if it is defined.

¹ On DOS/Windows systems, the home directory is the one pointed to by the `HOME` environment variable.

- The file `.config/gdb/gdbearlyinit` within the directory pointed to by the environment variable `HOME`, if it is defined.
- The file `.gdbearlyinit` within the directory pointed to by the environment variable `HOME`, if it is defined.

By contrast, on macOS hosts the locations searched are:

- The file `Library/Preferences/gdb/gdbearlyinit` within the directory pointed to by the environment variable `HOME`, if it is defined.
- The file `.gdbearlyinit` within the directory pointed to by the environment variable `HOME`, if it is defined.

It is possible to prevent the home directory early initialization file from being loaded using the `-nx` or `-nh` command line options, see Section 2.1.2 [Choosing Modes], page 13.

2.1.4.2 System wide initialization files

There are two locations that are searched for system wide initialization files. Both of these locations are always checked:

`system.gdbinit`

This is a single system-wide initialization file. Its location is specified with the `--with-system-gdbinit` configure option (see Section C.6 [System-wide configuration], page 775). It is loaded first when GDB starts, before command line options have been processed.

`system.gdbinit.d`

This is the system-wide initialization directory. Its location is specified with the `--with-system-gdbinit-dir` configure option (see Section C.6 [System-wide configuration], page 775). Files in this directory are loaded in alphabetical order immediately after `system.gdbinit` (if enabled) when GDB starts, before command line options have been processed. Files need to have a recognized scripting language extension (`.py/.scm`) or be named with a `.gdb` extension to be interpreted as regular GDB commands. GDB will not recurse into any subdirectories of this directory.

It is possible to prevent the system wide initialization files from being loaded using the `-nx` command line option, see Section 2.1.2 [Choosing Modes], page 13.

2.1.4.3 Home directory initialization file

After loading the system wide initialization files GDB will look for an initialization file in the users home directory². There are a number of locations that GDB will search in the home directory, these locations are searched in order and GDB will load the first file that it finds, and subsequent locations will not be checked.

On non-Apple hosts the locations searched are:

```
$XDG_CONFIG_HOME/gdb/gdbinit
$HOME/.config/gdb/gdbinit
$HOME/.gdbinit
```

While on Apple hosts the locations searched are:

² On DOS/Windows systems, the home directory is the one pointed to by the `HOME` environment variable.

```
$HOME/Library/Preferences/gdb/gdbinit
$HOME/.gdbinit
```

It is possible to prevent the home directory initialization file from being loaded using the ‘-nx’ or ‘-nh’ command line options, see Section 2.1.2 [Choosing Modes], page 13.

The DJGPP port of GDB uses the name `gdb.ini` instead of `.gdbinit` or `gdbinit`, due to the limitations of file names imposed by DOS filesystems. The Windows port of GDB uses the standard name, but if it finds a `gdb.ini` file in your home directory, it warns you about that and suggests to rename the file to the standard name.

2.1.4.4 Local directory initialization file

GDB will check the current directory for a file called `.gdbinit`. It is loaded last, after command line options other than ‘-x’ and ‘-ex’ have been processed. The command line options ‘-x’ and ‘-ex’ are processed last, after `.gdbinit` has been loaded, see Section 2.1.1 [Choosing Files], page 12.

If the file in the current directory was already loaded as the home directory initialization file then it will not be loaded a second time.

It is possible to prevent the local directory initialization file from being loaded using the ‘-nx’ command line option, see Section 2.1.2 [Choosing Modes], page 13.

2.2 Quitting GDB

```
quit [expression]
exit [expression]
```

q To exit GDB, use the `quit` command (abbreviated `q`), the `exit` command, or type an end-of-file character (usually `Ctrl-d`). If you do not supply *expression*, GDB will terminate normally; otherwise it will terminate using the result of *expression* as the error code.

An interrupt (often `Ctrl-c`) does not exit from GDB, but rather terminates the action of any GDB command that is in progress and returns to GDB command level. It is safe to type the interrupt character at any time because GDB does not allow it to take effect until a time when it is safe.

If you have been using GDB to control an attached process or device, you can release it with the `detach` command (see Section 4.7 [Debugging an Already-running Process], page 41).

2.3 Shell Commands

If you need to execute occasional shell commands during your debugging session, there is no need to leave or suspend GDB; you can just use the `shell` command.

```
shell command-string
!command-string
```

Invoke a shell to execute *command-string*. Note that no space is needed between `!` and *command-string*. On GNU and Unix systems, the environment variable `SHELL`, if it exists, determines which shell to run. Otherwise GDB uses the default shell (`/bin/sh` on GNU and Unix systems, `cmd.exe` on MS-Windows, `COMMAND.COM` on MS-DOS, etc.).

You may also invoke shell commands from expressions, using the `$_shell` convenience function. See [\$_shell convenience function], page 179.

The utility `make` is often needed in development environments. You do not have to use the `shell` command for this purpose in GDB:

`make make-args`

Execute the `make` program with the specified arguments. This is equivalent to ‘`shell make make-args`’.

`pipe [command] | shell_command`

`| [command] | shell_command`

`pipe -d delim command delim shell_command`

`| -d delim command delim shell_command`

Executes *command* and sends its output to *shell_command*. Note that no space is needed around `|`. If no *command* is provided, the last command executed is repeated.

In case the *command* contains a `|`, the option `-d delim` can be used to specify an alternate delimiter string *delim* that separates the *command* from the *shell_command*.

Example:

```
(gdb) p var
$1 = {
  black = 144,
  red = 233,
  green = 377,
  blue = 610,
  white = 987
}
(gdb) pipe p var|wc
      7      19      80
(gdb) lp var|wc -l
7
(gdb) p /x var
$4 = {
  black = 0x90,
  red = 0xe9,
  green = 0x179,
  blue = 0x262,
  white = 0x3db
}
(gdb) ||grep red
red => 0xe9,
(gdb) | -d ! echo this contains a | char\n ! sed -e 's/|/PIPE/'
this contains a PIPE char
(gdb) | -d xxx echo this contains a | char!\n xxx sed -e 's/|/PIPE/'
this contains a PIPE char!
(gdb)
```

The convenience variables `$_shell_exitcode` and `$_shell_exitcode` can be used to examine the exit status of the last shell command launched by `shell`, `make`, `pipe` and `|`. See Section 10.12 [Convenience Variables], page 173.

2.4 Logging Output

You may want to save the output of GDB commands to a file. There are several commands to control GDB's logging.

set logging enabled [on|off]
Enable or disable logging.

set logging file *file*
Change the name of the current logfile. The default logfile is `gdb.txt`.

set logging overwrite [on|off]
By default, GDB will append to the logfile. Set **overwrite** if you want **set logging enabled on** to overwrite the logfile instead.

set logging redirect [on|off]
By default, GDB output will go to both the terminal and the logfile. Set **redirect** if you want output to go only to the log file.

set logging debugredirect [on|off]
By default, GDB debug output will go to both the terminal and the logfile. Set **debugredirect** if you want debug output to go only to the log file.

show logging
Show the current values of the logging settings.

You can also redirect the output of a GDB command to a shell command. See [pipe], page 21.

3 GDB Commands

You can abbreviate a GDB command to the first few letters of the command name, if that abbreviation is unambiguous; and you can repeat certain GDB commands by typing just **RET**. You can also use the **TAB** key to get GDB to fill out the rest of a word in a command (or to show you the alternatives available, if there is more than one possibility).

3.1 Command Syntax

A GDB command is a single line of input. There is no limit on how long it can be. It starts with a command name, which is followed by arguments whose meaning depends on the command name. For example, the command **step** accepts an argument which is the number of times to step, as in ‘**step 5**’. You can also use the **step** command with no arguments. Some commands do not allow any arguments.

GDB command names may always be truncated if that abbreviation is unambiguous. Other possible command abbreviations are listed in the documentation for individual commands. In some cases, even ambiguous abbreviations are allowed; for example, **s** is specially defined as equivalent to **step** even though there are other commands whose names start with **s**. You can test abbreviations by using them as arguments to the **help** command.

A blank line as input to GDB (typing just **RET**) means to repeat the previous command. Certain commands (for example, **run**) will not repeat this way; these are commands whose unintentional repetition might cause trouble and which you are unlikely to want to repeat. User-defined commands can disable this feature; see Section 23.1.1 [Define], page 397.

The **list** and **x** commands, when you repeat them with **RET**, construct new arguments rather than repeating exactly as typed. This permits easy scanning of source or memory.

GDB can also use **RET** in another way: to partition lengthy output, in a way similar to the common utility **more** (see Section 22.4 [Screen Size], page 374). Since it is easy to press one **RET** too many in this situation, GDB disables command repetition after any command that generates this sort of display.

Any text from a **#** to the end of the line is a comment; it does nothing. This is useful mainly in command files (see Section 23.1.3 [Command Files], page 401).

The **Ctrl-o** binding is useful for repeating a complex sequence of commands. This command accepts the current line, like **RET**, and then fetches the next line relative to the current line from the history for editing.

3.2 Command Settings

Many commands change their behavior according to command-specific variables or settings. These settings can be changed with the **set** subcommands. For example, the **print** command (see Chapter 10 [Examining Data], page 143) prints arrays differently depending on settings changeable with the commands **set print elements NUMBER-OF-ELEMENTS** and **set print array-indexes**, among others.

You can change these settings to your preference in the **gdbinit** files loaded at GDB startup. See Section 2.1.3 [Startup], page 17.

The settings can also be changed interactively during the debugging session. For example, to change the limit of array elements to print, you can do the following:

```
(gdb) set print elements 10
```

```
(gdb) print some_array
$1 = {0, 10, 20, 30, 40, 50, 60, 70, 80, 90...}
```

The above `set print elements 10` command changes the number of elements to print from the default of 200 to 10. If you only intend this limit of 10 to be used for printing `some_array`, then you must restore the limit back to 200, with `set print elements 200`.

Some commands allow overriding settings with command options. For example, the `print` command supports a number of options that allow overriding relevant global print settings as set by `set print` subcommands. See [print options], page 143. The example above could be rewritten as:

```
(gdb) print -elements 10 -- some_array
$1 = {0, 10, 20, 30, 40, 50, 60, 70, 80, 90...}
```

Alternatively, you can use the `with` command to change a setting temporarily, for the duration of a command invocation.

```
with setting [value] [-- command]
w setting [value] [-- command]
```

Temporarily set *setting* to *value* for the duration of *command*.

setting is any setting you can change with the `set` subcommands. *value* is the value to assign to *setting* while running *command*.

If no *command* is provided, the last command executed is repeated.

If a *command* is provided, it must be preceded by a double dash (`--`) separator. This is required because some settings accept free-form arguments, such as expressions or filenames.

For example, the command

```
(gdb) with print array on -- print some_array
```

is equivalent to the following 3 commands:

```
(gdb) set print array on
(gdb) print some_array
(gdb) set print array off
```

The `with` command is particularly useful when you want to override a setting while running user-defined commands, or commands defined in Python or Guile. See Chapter 23 [Extending GDB], page 397.

```
(gdb) with print pretty on -- my_complex_command
```

To change several settings for the same command, you can nest `with` commands. For example, `with language ada -- with print elements 10` temporarily changes the language to Ada and sets a limit of 10 elements to print for arrays and strings.

3.3 Command Completion

GDB can fill in the rest of a word in a command for you, if there is only one possibility; it can also show you what the valid possibilities are for the next word in a command, at any time. This works for GDB commands, GDB subcommands, command options, and the names of symbols in your program.

Press the `TAB` key whenever you want GDB to fill out the rest of a word. If there is only one possibility, GDB fills in the word, and waits for you to finish the command (or press `RET` to enter it). For example, if you type

```
(gdb) info breTAB
```

GDB fills in the rest of the word ‘**breakpoints**’, since that is the only **info** subcommand beginning with ‘**bre**’:

```
(gdb) info breakpoints
```

You can either press **RET** at this point, to run the **info breakpoints** command, or backspace and enter something else, if ‘**breakpoints**’ does not look like the command you expected. (If you were sure you wanted **info breakpoints** in the first place, you might as well just type **RET** immediately after ‘**info bre**’, to exploit command abbreviations rather than command completion).

If there is more than one possibility for the next word when you press **TAB**, GDB sounds a bell. You can either supply more characters and try again, or just press **TAB** a second time; GDB displays all the possible completions for that word. For example, you might want to set a breakpoint on a subroutine whose name begins with ‘**make_**’, but when you type **b make_TAB** GDB just sounds the bell. Typing **TAB** again displays all the function names in your program that begin with those characters, for example:

```
(gdb) b make_TAB
```

GDB sounds bell; press **TAB** again, to see:

```
make_a_section_from_file    make_envron
make_abs_section           make_function_type
make_blockvector           make_pointer_type
make_cleanup               make_reference_type
make_command               make_symbol_completion_list
(gdb) b make_
```

After displaying the available possibilities, GDB copies your partial input (‘**b make_**’ in the example) so you can finish the command.

If the command you are trying to complete expects either a keyword or a number to follow, then ‘**NUMBER**’ will be shown among the available completions, for example:

```
(gdb) print -elements TABTAB
NUMBER    unlimited
(gdb) print -elements
```

Here, the option expects a number (e.g., 100), not literal **NUMBER**. Such metasyntactical arguments are always presented in uppercase.

If you just want to see the list of alternatives in the first place, you can press **M-?** rather than pressing **TAB** twice. **M-?** means **META ?**. You can type this either by holding down a key designated as the **META** shift on your keyboard (if there is one) while typing **?**, or as **ESC** followed by **?**.

If the number of possible completions is large, GDB will print as much of the list as it has collected, as well as a message indicating that the list may be truncated.

```
(gdb) b mTABTAB
main
<... the rest of the possible completions ...>
*** List may be truncated, max-completions reached. ***
(gdb) b m
```

This behavior can be controlled with the following commands:

```
set max-completions limit
set max-completions unlimited
```

Set the maximum number of completion candidates. GDB will stop looking for more completions once it collects this many candidates. This is useful when

completing on things like function names as collecting all the possible candidates can be time consuming. The default value is 200. A value of zero disables tab-completion. Note that setting either no limit or a very large limit can make completion slow.

show max-completions

Show the maximum number of candidates that GDB will collect and show during completion.

Sometimes the string you need, while logically a “word”, may contain parentheses or other characters that GDB normally excludes from its notion of a word. To permit word completion to work in this situation, you may enclose words in `'` (single quote marks) in GDB commands.

A likely situation where you might need this is in typing an expression that involves a C++ symbol name with template parameters. This is because when completing expressions, GDB treats the `<` character as word delimiter, assuming that it’s the less-than comparison operator (see Section 15.4.1.1 [C and C++ Operators], page 233).

For example, when you want to call a C++ template function interactively using the `print` or `call` commands, you may need to distinguish whether you mean the version of `name` that was specialized for `int`, `name<int>()`, or the version that was specialized for `float`, `name<float>()`. To use the word-completion facilities in this situation, type a single quote `'` at the beginning of the function name. This alerts GDB that it may need to consider more information than usual when you press `TAB` or `M-?` to request word completion:

```
(gdb) p 'func<M-?
func<int>()    func<float>()
(gdb) p 'func<
```

When setting breakpoints however (see Section 9.2 [Location Specifications], page 128), you don’t usually need to type a quote before the function name, because GDB understands that you want to set a breakpoint on a function:

```
(gdb) b func<M-?
func<int>()    func<float>()
(gdb) b func<
```

This is true even in the case of typing the name of C++ overloaded functions (multiple definitions of the same function, distinguished by argument type). For example, when you want to set a breakpoint you don’t need to distinguish whether you mean the version of `name` that takes an `int` parameter, `name(int)`, or the version that takes a `float` parameter, `name(float)`.

```
(gdb) b bubble(M-?
bubble(int)    bubble(double)
(gdb) b bubble(douM-?
bubble(double)
```

See [quoting names], page 265, for a description of other scenarios that require quoting.

For more information about overloaded functions, see Section 15.4.1.3 [C++ Expressions], page 236. You can use the command `set overload-resolution off` to disable overload resolution; see Section 15.4.1.7 [GDB Features for C++], page 237.

When completing in an expression which looks up a field in a structure, GDB also tries¹ to limit completions to the field names available in the type of the left-hand-side:

```
(gdb) p gdb_stdout.M-?
magic          to_fputs          to_rewind
to_data        to_isatty         to_write
to_delete      to_put            to_write_async_safe
to_flush       to_read
```

This is because the `gdb_stdout` is a variable of the type `struct ui_file` that is defined in GDB sources as follows:

```
struct ui_file
{
    int *magic;
    ui_file_flush_ftype *to_flush;
    ui_file_write_ftype *to_write;
    ui_file_write_async_safe_ftype *to_write_async_safe;
    ui_file_fputs_ftype *to_fputs;
    ui_file_read_ftype *to_read;
    ui_file_delete_ftype *to_delete;
    ui_file_isatty_ftype *to_isatty;
    ui_file_rewind_ftype *to_rewind;
    ui_file_put_ftype *to_put;
    void *to_data;
}
```

3.4 Filenames As Command Arguments

When passing filenames (or directory names) as arguments to a command, if the filename argument does not include any whitespace, double quotes, or single quotes, then for all commands the filename can be written as a simple string, for example:

```
(gdb) file /path/to/some/file
```

If the filename does include whitespace, double quotes, or single quotes, then GDB has two approaches for how these filenames should be formatted; which format to use depends on which command is being used.

Most GDB commands don't require, or support, quoting and escaping. These commands treat any text after the command name, that is not a command option (see Section 3.5 [Command Options], page 28), as the filename, even if the filename contains whitespace or quote characters. In the following example the user is adding `/path/that contains/two spaces/` to the auto-load safe-path (see [add-auto-load-safe-path], page 384):

```
(gdb) add-auto-load-safe-path /path/that contains/two spaces/
```

A small number of commands require that filenames containing whitespace or quote characters are either quoted, or have the special characters escaped with a backslash. Commands that support this style are marked as such in the manual, any command not marked as accepting quoting and escaping of its filename argument, does not accept this filename argument style.

For example, to load the file `/path/with spaces/to/a file` with the `file` command (see Section 18.1 [Commands to Specify Files], page 293), you can escape the whitespace characters with a backslash:

```
(gdb) file /path/with\ spaces/to/a\ file
```

¹ The completer can be confused by certain kinds of invalid expressions. Also, it only examines the static type of the expression, not the dynamic type.

Alternatively the entire filename can be wrapped in either single or double quotes, in which case no backslashes are needed, for example:

```
(gdb) symbol-file "/path/with spaces/to/a file"
(gdb) exec-file '/path/with spaces/to/a file'
```

It is possible to include a quote character within a quoted filename by escaping it with a backslash, for example, within a filename surrounded by double quotes, a double quote character should be escaped with a backslash, but a single quote character should not be escaped. Within a single quoted string a single quote character needs to be escaped, but a double quote character does not.

A literal backslash character can also be included by escaping it with a backslash.

3.5 Command options

Some commands accept options starting with a leading dash. For example, `print -pretty`. Similarly to command names, you can abbreviate a GDB option to the first few letters of the option name, if that abbreviation is unambiguous, and you can also use the TAB key to get GDB to fill out the rest of a word in an option (or to show you the alternatives available, if there is more than one possibility).

Some commands take raw input as argument. For example, the `print` command processes arbitrary expressions in any of the languages supported by GDB. With such commands, because raw input may start with a leading dash that would be confused with an option or any of its abbreviations, e.g. `print -p` (short for `print -pretty` or printing negative `p?`), if you specify any command option, then you must use a double-dash (`--`) delimiter to indicate the end of options.

Some options are described as accepting an argument which can be either `on` or `off`. These are known as *boolean options*. Similarly to boolean settings commands—`on` and `off` are the typical values, but any of `1`, `yes` and `enable` can also be used as “true” value, and any of `0`, `no` and `disable` can also be used as “false” value. You can also omit a “true” value, as it is implied by default.

For example, these are equivalent:

```
(gdb) print -object on -pretty off -element unlimited -- *myptr
(gdb) p -o -p 0 -e u -- *myptr
```

You can discover the set of options some command accepts by completing on `-` after the command name. For example:

```
(gdb) print -TABTAB
-address          -max-depth        -object           -static-members
-array            -memory-tag-violations -pretty           -symbol
-array-indexes    -nibbles           -raw-values       -union
-elements         -null-stop         -repeats          -vtbl
```

Completion will in some cases guide you with a suggestion of what kind of argument an option expects. For example:

```
(gdb) print -elements TABTAB
NUMBER          unlimited
```

Here, the option expects a number (e.g., 100), not literal `NUMBER`. Such metasyntactical arguments are always presented in uppercase.

(For more on using the `print` command, see Chapter 10 [Examining Data], page 143.)

3.6 Getting Help

You can always ask GDB itself for information on its commands, using the command **help**.

help

h You can use **help** (abbreviated **h**) with no arguments to display a short list of named classes of commands:

```
(gdb) help
List of classes of commands:

aliases -- User-defined aliases of other commands
breakpoints -- Making program stop at certain points
data -- Examining data
files -- Specifying and examining files
internals -- Maintenance commands
obscure -- Obscure features
running -- Running the program
stack -- Examining the stack
status -- Status inquiries
support -- Support facilities
tracepoints -- Tracing of program execution without
               stopping the program
user-defined -- User-defined commands

Type "help" followed by a class name for a list of
commands in that class.
Type "help" followed by command name for full
documentation.
Command name abbreviations are allowed if unambiguous.
(gdb)
```

help class

Using one of the general help classes as an argument, you can get a list of the individual commands in that class. If a command has aliases, the aliases are given after the command name, separated by commas. If an alias has default arguments, the full definition of the alias is given after the first line. For example, here is the help display for the class **status**:

```
(gdb) help status
Status inquiries.

List of commands:

info, inf, i -- Generic command for showing things
               about the program being debugged
info address, iamain -- Describe where symbol SYM is stored.
               alias iamain = info address main
info all-registers -- List of all registers and their contents,
               for selected stack frame.
...
show, info set -- Generic command for showing things
               about the debugger

Type "help" followed by command name for full
documentation.
Command name abbreviations are allowed if unambiguous.
(gdb)
```

help *command*

With a command name as **help** argument, GDB displays a short paragraph on how to use that command. If that command has one or more aliases, GDB will display a first line with the command name and all its aliases separated by commas. This first line will be followed by the full definition of all aliases having default arguments. When asking the help for an alias, the documentation for the aliased command is shown.

A user-defined alias can optionally be documented using the **document** command (see Section 23.1.1 [Define], page 397). GDB then considers this alias as different from the aliased command: this alias is not listed in the aliased command help output, and asking help for this alias will show the documentation provided for the alias instead of the documentation of the aliased command.

apropos [-v] *regex*

The **apropos** command searches through all of the GDB commands and aliases, and their documentation, for the regular expression specified in *args*. It prints out all matches found. The optional flag **-v**, which stands for **'verbose'**, indicates to output the full documentation of the matching commands and highlight the parts of the documentation matching *regex*. For example:

```
apropos alias
```

results in:

```
alias -- Define a new command that is an alias of an existing command
aliases -- User-defined aliases of other commands
```

while

```
apropos -v cut.*thread apply
```

results in the below output, where **'cut for 'thread apply'** is highlighted if styling is enabled.

```
taas -- Apply a command to all threads (ignoring errors
and empty output).
Usage: taas COMMAND
shortcut for 'thread apply all -s COMMAND'

tfaas -- Apply a command to all frames of all threads
(ignoring errors and empty output).
Usage: tfaas COMMAND
shortcut for 'thread apply all -s frame apply all -s COMMAND'
```

complete *args*

The **complete *args*** command lists all the possible completions for the beginning of a command. Use *args* to specify the beginning of the command you want completed. For example:

```
complete i
```

results in:

```
if
ignore
info
inspect
```

This is intended for use by GNU Emacs.

In addition to **help**, you can use the GDB commands **info** and **show** to inquire about the state of your program, or the state of GDB itself. Each command supports many topics of inquiry; this manual introduces each of them in the appropriate context. The listings under **info** and under **show** in the Command, Variable, and Function Index point to all the sub-commands. See [Command and Variable Index], page 961.

- info** This command (abbreviated **i**) is for describing the state of your program. For example, you can show the arguments passed to a function with **info args**, list the registers currently in use with **info registers**, or list the breakpoints you have set with **info breakpoints**. You can get a complete list of the **info** sub-commands with **help info**.
- set** You can assign the result of an expression to an environment variable with **set**. For example, you can set the GDB prompt to a \$-sign with **set prompt \$**.
- show** In contrast to **info**, **show** is for describing the state of GDB itself. You can change most of the things you can **show**, by using the related command **set**; for example, you can control what number system is used for displays with **set radix**, or simply inquire which is currently in use with **show radix**.
- To display all the settable parameters and their current values, you can use **show** with no arguments; you may also use **info set**. Both commands produce the same display.

Here are several miscellaneous **show** subcommands, all of which are exceptional in lacking corresponding **set** commands:

show version

Show what version of GDB is running. You should include this information in GDB bug-reports. If multiple versions of GDB are in use at your site, you may need to determine which version of GDB you are running; as GDB evolves, new commands are introduced, and old ones may wither away. Also, many system vendors ship variant versions of GDB, and there are variant versions of GDB in GNU/Linux distributions as well. The version number is the same as the one announced when you start GDB.

show copying

info copying

Display information about permission for copying GDB.

show warranty

info warranty

Display the GNU “NO WARRANTY” statement, or a warranty, if your version of GDB comes with one.

show configuration

Display detailed information about the way GDB was configured when it was built. This displays the optional arguments passed to the **configure** script and also configuration parameters detected automatically by **configure**. When reporting a GDB bug (see Chapter 32 [GDB Bugs], page 729), it is important to include this information in your report.

This command also displays the version number of the Readline library that GDB is using. After the version number will appear either `'(system)'` indicating GDB is using the shared system Readline library, or `'(internal)'` indicating GDB is using a statically linked in version of the Readline library.

4 Running Programs Under GDB

When you run a program under GDB, you must first generate debugging information when you compile it.

You may start GDB with its arguments, if any, in an environment of your choice. If you are doing native debugging, you may redirect your program's input and output, debug an already running process, or kill a child process.

4.1 Compiling for Debugging

In order to debug a program effectively, you need to generate debugging information when you compile it. This debugging information is stored in the object file; it describes the data type of each variable or function and the correspondence between source line numbers and addresses in the executable code.

To request debugging information, specify the `-g` option when you run the compiler.

Programs that are to be shipped to your customers are compiled with optimizations, using the `-O` compiler option. However, some compilers are unable to handle the `-g` and `-O` options together. Using those compilers, you cannot generate optimized executables containing debugging information.

GCC, the GNU C/C++ compiler, supports `-g` with or without `-O`, making it possible to debug optimized code. We recommend that you *always* use `-g` whenever you compile a program. You may think your program is correct, but there is no sense in pushing your luck. For more information, see Chapter 11 [Optimized Code], page 197.

Older versions of the GNU C compiler permitted a variant option `-gg` for debugging information. GDB no longer supports this format; if your GNU C compiler has this option, do not use it.

GDB knows about preprocessor macros and can show you their expansion (see Chapter 12 [Macros], page 201). Most compilers do not include information about preprocessor macros in the debugging information if you specify the `-g` flag alone. Version 3.1 and later of GCC, the GNU C compiler, provides macro information if you are using the DWARF debugging format, and specify the option `-g3`.

See Section “Options for Debugging Your Program or GCC” in *Using the GNU Compiler Collection (GCC)*, for more information on GCC options affecting debug information.

You will have the best debugging experience if you use the latest version of the DWARF debugging format that your compiler supports. DWARF is currently the most expressive and best supported debugging format in GDB.

4.2 Starting your Program

run

r Use the **run** command to start your program under GDB. You must first specify the program name with an argument to GDB (see Chapter 2 [Getting In and Out of GDB], page 11), or by using the **file** or **exec-file** command (see Section 18.1 [Commands to Specify Files], page 293).

If you are running your program in an execution environment that supports processes, **run** creates an inferior process and makes that process run your program. In some environments without processes, **run** jumps to the start of your program. Other targets, like ‘**remote**’, are always running. If you get an error message like this one:

```
The "remote" target does not support "run".
Try "help target" or "continue".
```

then use **continue** to run your program. You may need **load** first (see [load], page 315).

The execution of a program is affected by certain information it receives from its superior. GDB provides ways to specify this information, which you must do *before* starting your program. (You can change it after starting your program, but such changes only affect your program the next time you start it.) This information may be divided into four categories:

The arguments.

Specify the arguments to give your program as the arguments of the **run** command. If a shell is available on your target, the shell is used to pass the arguments, so that you may use normal conventions (such as wildcard expansion or variable substitution) in describing the arguments. In Unix systems, you can control which shell is used with the **SHELL** environment variable. If you do not define **SHELL**, GDB uses the default shell (**/bin/sh**). You can disable use of any shell with the **set startup-with-shell** command (see below for details).

The environment.

Your program normally inherits its environment from GDB, but you can use the GDB commands **set environment** and **unset environment** to change parts of the environment that affect your program. See Section 4.4 [Your Program’s Environment], page 38.

The working directory.

You can set your program’s working directory with the command **set cwd**. If you do not set any working directory with this command, your program will inherit GDB’s working directory if native debugging, or the remote server’s working directory if remote debugging. See Section 4.5 [Your Program’s Working Directory], page 40.

The standard input and output.

Your program normally uses the same device for standard input and standard output as GDB is using. You can redirect input and output in the **run** command line, or you can use the **tty** command to set a different device for your program. See Section 4.6 [Your Program’s Input and Output], page 41.

Warning: While input and output redirection work, you cannot use pipes to pass the output of the program you are debugging to another program; if you attempt this, GDB is likely to wind up debugging the wrong program.

When you issue the **run** command, your program begins to execute immediately. See Chapter 5 [Stopping and Continuing], page 59, for discussion of how to arrange for your program to stop. Once your program has stopped, you may call functions in your program, using the **print** or **call** commands. See Chapter 10 [Examining Data], page 143.

If the modification time of your symbol file has changed since the last time GDB read its symbols, GDB discards its symbol table, and reads it again. When it does this, GDB tries to retain your current breakpoints.

start The name of the main procedure can vary from language to language. With C or C++, the main procedure name is always **main**, but other languages such as Ada do not require a specific name for their main procedure. The debugger provides a convenient way to start the execution of the program and to stop at the beginning of the main procedure, depending on the language used.

The ‘**start**’ command does the equivalent of setting a temporary breakpoint at the beginning of the main procedure and then invoking the ‘**run**’ command.

Some programs contain an *elaboration* phase where some startup code is executed before the main procedure is called. This depends on the languages used to write your program. In C++, for instance, constructors for static and global objects are executed before **main** is called. It is therefore possible that the debugger stops before reaching the main procedure. However, the temporary breakpoint will remain to halt execution.

Specify the arguments to give to your program as arguments to the ‘**start**’ command. These arguments will be given verbatim to the underlying ‘**run**’ command. Note that the same arguments will be reused if no argument is provided during subsequent calls to ‘**start**’ or ‘**run**’.

It is sometimes necessary to debug the program during elaboration. In these cases, using the **start** command would stop the execution of your program too late, as the program would have already completed the elaboration phase. Under these circumstances, either insert breakpoints in your elaboration code before running your program or use the **starti** command.

starti The ‘**starti**’ command does the equivalent of setting a temporary breakpoint at the first instruction of a program’s execution and then invoking the ‘**run**’ command. For programs containing an elaboration phase, the **starti** command will stop execution at the start of the elaboration phase.

set exec-wrapper wrapper
show exec-wrapper
unset exec-wrapper

When ‘**exec-wrapper**’ is set, the specified wrapper is used to launch programs for debugging. GDB starts your program with a shell command of the form **exec wrapper program**. Quoting is added to *program* and its arguments, but not to *wrapper*, so you should add quotes if appropriate for your shell. The wrapper runs until it executes your program, and then GDB takes control.

You can use any program that eventually calls `execve` with its arguments as a wrapper. Several standard Unix utilities do this, e.g. `env` and `nohup`. Any Unix shell script ending with `exec "$@"` will also work.

For example, you can use `env` to pass an environment variable to the debugged program, without setting the variable in your shell's environment:

```
(gdb) set exec-wrapper env 'LD_PRELOAD=libtest.so'
(gdb) run
```

This command is available when debugging locally on most targets, excluding DJGPP, Cygwin, and MS Windows.

```
set startup-with-shell
set startup-with-shell on
set startup-with-shell off
show startup-with-shell
```

On Unix systems, by default, if a shell is available on your target, GDB) uses it to start your program. Arguments of the `run` command are passed to the shell, which does variable substitution, expands wildcard characters and performs redirection of I/O. In some circumstances, it may be useful to disable such use of a shell, for example, when debugging the shell itself or diagnosing startup failures such as:

```
(gdb) run
Starting program: ./a.out
During startup program terminated with signal SIGSEGV, Segmentation fault.
```

which indicates the shell or the wrapper specified with `'exec-wrapper'` crashed, not your program. Most often, this is caused by something odd in your shell's non-interactive mode initialization file—such as `.cshrc` for C-shell, `$.zshenv` for the Z shell, or the file specified in the `BASH_ENV` environment variable for BASH.

```
set auto-connect-native-target
set auto-connect-native-target on
set auto-connect-native-target off
show auto-connect-native-target
```

By default, if the current inferior is not connected to any target yet (e.g., with `target remote`), the `run` command starts your program as a native process under GDB, on your local machine. If you're sure you don't want to debug programs on your local machine, you can tell GDB to not connect to the native target automatically with the `set auto-connect-native-target off` command.

If `on`, which is the default, and if the current inferior is not connected to a target already, the `run` command automatically connects to the native target, if one is available.

If `off`, and if the current inferior is not connected to a target already, the `run` command fails with an error:

```
(gdb) run
Don't know how to run. Try "help target".
```

If the current inferior is already connected to a target, GDB always uses it with the `run` command.

In any case, you can explicitly connect to the native target with the `target native` command. For example,

```
(gdb) set auto-connect-native-target off
(gdb) run
Don't know how to run. Try "help target".
(gdb) target native
(gdb) run
Starting program: ./a.out
[Inferior 1 (process 10421) exited normally]
```

In case you connected explicitly to the `native` target, GDB remains connected even if all inferiors exit, ready for the next `run` command. Use the `disconnect` command to disconnect.

Examples of other commands that likewise respect the `auto-connect-native-target` setting: `attach`, `info proc`, `info os`.

`set disable-randomization`

`set disable-randomization on`

This option (enabled by default in GDB) will turn off the native randomization of the virtual address space of the started program. This option is useful for multiple debugging sessions to make the execution better reproducible and memory addresses reusable across debugging sessions.

This feature is implemented only on certain targets, including GNU/Linux. On GNU/Linux you can get the same behavior using

```
(gdb) set exec-wrapper setarch 'uname -m' -R
```

`set disable-randomization off`

Leave the behavior of the started executable unchanged. Some bugs rear their ugly heads only when the program is loaded at certain addresses. If your bug disappears when you run the program under GDB, that might be because GDB by default disables the address randomization on platforms, such as GNU/Linux, which do that for stand-alone programs. Use `set disable-randomization off` to try to reproduce such elusive bugs.

On targets where it is available, virtual address space randomization protects the programs against certain kinds of security attacks. In these cases the attacker needs to know the exact location of a concrete executable code. Randomizing its location makes it impossible to inject jumps misusing a code at its expected addresses.

Prelinking shared libraries provides a startup performance advantage but it makes addresses in these libraries predictable for privileged processes by having just unprivileged access at the target system. Reading the shared library binary gives enough information for assembling the malicious code misusing it. Still even a prelinked shared library can get loaded at a new random address just requiring the regular relocation process during the startup. Shared libraries not already prelinked are always loaded at a randomly chosen address.

Position independent executables (PIE) contain position independent code similar to the shared libraries and therefore such executables get loaded at a randomly chosen address upon startup. PIE executables always load even already

prelinked shared libraries at a random address. You can build such executable using `gcc -fPIE -pie`.

Heap (malloc storage), stack and custom mmap areas are always placed randomly (as long as the randomization is enabled).

show disable-randomization

Show the current setting of the explicit disable of the native randomization of the virtual address space of the started program.

4.3 Your Program's Arguments

The arguments to your program can be specified by the arguments of the **run** command. They are passed to a shell, which expands wildcard characters and performs redirection of I/O, and thence to your program. Your **SHELL** environment variable (if it exists) specifies what shell GDB uses. If you do not define **SHELL**, GDB uses the default shell (`/bin/sh` on Unix).

On non-Unix systems, the program is usually invoked directly by GDB, which emulates I/O redirection via the appropriate system calls, and the wildcard characters are expanded by the startup code of the program, not by the shell.

run with no arguments uses the same arguments used by the previous **run**, or those set by the **set args** command.

set args Specify the arguments to be used the next time your program is run. If **set args** has no arguments, **run** executes your program with no arguments. Once you have run your program with arguments, using **set args** before the next **run** is the only way to run it again without arguments.

It is possible to set arguments containing a newline character. This can be done by enclosing an argument within a quotes. For example, start by entering:

```
(gdb) set args "ab
```

and then enter the newline, GDB gives a secondary prompt `>` and allows you to continue entering the arguments:

```
(gdb) set args "ab
>cd"
(gdb) show args
Argument list to give program being debugged when it is started is "ab
cd".
```

On Unix systems both single or double quotes can be used around arguments containing a newline. On MS-Windows, only double quotes can be used.

show args Show the arguments to give your program when it is started.

4.4 Your Program's Environment

The *environment* consists of a set of environment variables and their values. Environment variables conventionally record such things as your user name, your home directory, your terminal type, and your search path for programs to run. Usually you set up environment variables with the shell and they are inherited by all the other programs you run. When

debugging, it can be useful to try running your program with a modified environment without having to start GDB over again.

path *directory*

Add *directory* to the front of the **PATH** environment variable (the search path for executables) that will be passed to your program. The value of **PATH** used by GDB does not change. You may specify several directory names, separated by whitespace or by a system-dependent separator character (':' on Unix, ';' on MS-DOS and MS-Windows). If *directory* is already in the path, it is moved to the front, so it is searched sooner.

You can use the string '\$cwd' to refer to whatever is the current working directory at the time GDB searches the path. If you use '.' instead, it refers to the directory where you executed the **path** command. GDB replaces '.' in the *directory* argument (with the current path) before adding *directory* to the search path.

show paths

Display the list of search paths for executables (the **PATH** environment variable).

show environment [*varname*]

Print the value of environment variable *varname* to be given to your program when it starts. If you do not supply *varname*, print the names and values of all environment variables to be given to your program. You can abbreviate **environment** as **env**.

set environment *varname* [=*value*]

Set environment variable *varname* to *value*. The value changes for your program (and the shell GDB uses to launch it), not for GDB itself. The *value* may be any string; the values of environment variables are just strings, and any interpretation is supplied by your program itself. The *value* parameter is optional; if it is eliminated, the variable is set to a null value.

For example, this command:

```
set env USER = foo
```

tells the debugged program, when subsequently run, that its user is named 'foo'. (The spaces around '=' are used for clarity here; they are not actually required.)

Note that on Unix systems, GDB runs your program via a shell, which also inherits the environment set with **set environment**. If necessary, you can avoid that by using the 'env' program as a wrapper instead of using **set environment**. See [set exec-wrapper], page 35, for an example doing just that.

Environment variables that are set by the user are also transmitted to **gdbserver** to be used when starting the remote inferior. see [QEnvironmentHexEncoded], page 813.

unset environment *varname*

Remove variable *varname* from the environment to be passed to your program. This is different from '**set env *varname* =**'; **unset environment** removes the variable from the environment, rather than assigning it an empty value.

Environment variables that are unset by the user are also unset on `gdbserver` when starting the remote inferior. see [QEnvironmentUnset], page 814.

Warning: On Unix systems, GDB runs your program using the shell indicated by your `SHELL` environment variable if it exists (or `/bin/sh` if not). If your `SHELL` variable names a shell that runs an initialization file when started non-interactively—such as `.cshrc` for C-shell, `$.zshenv` for the Z shell, or the file specified in the `BASH_ENV` environment variable for BASH—any variables you set in that file affect your program. You may wish to move setting of environment variables to files that are only run when you sign on, such as `.login` or `.profile`.

4.5 Your Program’s Working Directory

Each time you start your program with `run`, the inferior will be initialized with the current working directory specified by the `set cwd` command. If no directory has been specified by this command, then the inferior will inherit GDB’s current working directory as its working directory if native debugging, or it will inherit the remote server’s current working directory if remote debugging.

`set cwd [directory]`

Set the inferior’s working directory to *directory*, which will be `glob`-expanded in order to resolve tildes (`~`). If no argument has been specified, the command clears the setting and resets it to an empty state. This setting has no effect on GDB’s working directory, and it only takes effect the next time you start the inferior. The `~` in *directory* is a short for the *home directory*, usually pointed to by the `HOME` environment variable. On MS-Windows, if `HOME` is not defined, GDB uses the concatenation of `HOMEDRIVE` and `HOMEPATH` as fallback.

You can also change GDB’s current working directory by using the `cd` command. See [cd command], page 40.

`show cwd` Show the inferior’s working directory. If no directory has been specified by `set cwd`, then the default inferior’s working directory is the same as GDB’s working directory.

`cd [directory]`

Set the GDB working directory to *directory*. If not given, *directory* uses `'~'`.

The GDB working directory serves as a default for the commands that specify files for GDB to operate on. See Section 18.1 [Commands to Specify Files], page 293. See [set cwd command], page 40.

`pwd` Print the GDB working directory.

It is generally impossible to find the current working directory of the process being debugged (since a program can change its directory during its run). If you work on a system where GDB supports the `info proc` command (see Section 21.1.2 [Process Information], page 339), you can use the `info proc` command to find out the current working directory of the debuggee.

4.6 Your Program's Input and Output

By default, the program you run under GDB does input and output to the same terminal that GDB uses. GDB switches the terminal to its own terminal modes to interact with you, but it records the terminal modes your program was using and switches back to them when you continue running your program.

info terminal

Displays information recorded by GDB about the terminal modes your program is using.

You can redirect your program's input and/or output using shell redirection with the **run** command. For example,

```
run > outfile
```

starts your program, diverting its output to the file **outfile**.

Another way to specify where your program should do input and output is with the **tty** command. This command accepts a file name as argument, and causes this file to be the default for future **run** commands. It also resets the controlling terminal for the child process, for future **run** commands. For example,

```
tty /dev/ttyb
```

directs that processes started with subsequent **run** commands default to do input and output on the terminal **/dev/ttyb** and have that as their controlling terminal.

An explicit redirection in **run** overrides the **tty** command's effect on the input/output device, but not its effect on the controlling terminal.

When you use the **tty** command or redirect input in the **run** command, only the input *for your program* is affected. The input for GDB still comes from your terminal. **tty** is an alias for **set inferior-tty**.

You can use the **show inferior-tty** command to tell GDB to display the name of the terminal that will be used for future runs of your program.

set inferior-tty [tty]

Set the tty for the program being debugged to *tty*. Omitting *tty* restores the default behavior, which is to use the same terminal as GDB.

show inferior-tty

Show the current tty for the program being debugged.

4.7 Debugging an Already-running Process

attach process-id

This command attaches to a running process—one that was started outside GDB. (**info files** shows your active targets.) The command takes as argument a process ID. The usual way to find out the *process-id* of a Unix process is with the **ps** utility, or with the '**jobs -l**' shell command.

attach does not repeat if you press RET a second time after executing the command.

To use **attach**, your program must be running in an environment which supports processes; for example, **attach** does not work for programs on bare-board targets that lack an operating system. You must also have permission to send the process a signal.

When you use **attach**, the debugger finds the program running in the process first by looking in the current working directory, then (if the program is not found) by using the source file search path (see Section 9.5 [Specifying Source Directories], page 133). You can also use the **file** command to load the program. See Section 18.1 [Commands to Specify Files], page 293.

If the debugger can determine that the executable file running in the process it is attaching to does not match the current **exec-file** loaded by GDB, the option **exec-file-mismatch** specifies how to handle the mismatch. GDB tries to compare the files by comparing their build IDs (see [build ID], page 303), if available.

set exec-file-mismatch ‘ask|warn|off’

Whether to detect mismatch between the current executable file loaded by GDB and the executable file used to start the process. If ‘**ask**’, the default, display a warning and ask the user whether to load the process executable file; if ‘**warn**’, just display a warning; if ‘**off**’, don’t attempt to detect a mismatch. If the user confirms loading the process executable file, then its symbols will be loaded as well.

show exec-file-mismatch

Show the current value of **exec-file-mismatch**.

The first thing GDB does after arranging to debug the specified process is to stop it. You can examine and modify an attached process with all the GDB commands that are ordinarily available when you start processes with **run**. You can insert breakpoints; you can step and continue; you can modify storage. If you would rather the process continue running, you may use the **continue** command after attaching GDB to the process.

detach When you have finished debugging the attached process, you can use the **detach** command to release it from GDB control. Detaching the process continues its execution. After the **detach** command, that process and GDB become completely independent once more, and you are ready to **attach** another process or start one with **run**. **detach** does not repeat if you press RET again after executing the command.

If you exit GDB while you have an attached process, you detach that process. If you use the **run** command, you kill that process. By default, GDB asks for confirmation if you try to do either of these things; you can control whether or not you need to confirm by using the **set confirm** command (see Section 22.9 [Optional Warnings and Messages], page 386).

4.8 Killing the Child Process

kill Kill the child process in which your program is running under GDB.

This command is useful if you wish to debug a core dump instead of a running process. GDB ignores any core dump file while your program is running.

On some operating systems, a program cannot be executed outside GDB while you have breakpoints set on it inside GDB. You can use the **kill** command in this situation to permit running your program outside the debugger.

The `kill` command is also useful if you wish to recompile and relink your program, since on many systems it is impossible to modify an executable file while it is running in a process. In this case, when you next type `run`, GDB notices that the file has changed, and reads the symbol table again (while trying to preserve your current breakpoint settings).

4.9 Debugging Multiple Inferiors Connections and Programs

GDB lets you run and debug multiple programs in a single session. In addition, GDB on some systems may let you run several programs simultaneously (otherwise you have to exit from one before starting another). On some systems GDB may even let you debug several programs simultaneously on different remote systems. In the most general case, you can have multiple threads of execution in each of multiple processes, launched from multiple executables, running on different machines.

GDB represents the state of each program execution with an object called an *inferior*. An inferior typically corresponds to a process, but is more general and applies also to targets that do not have processes. Inferiors may be created before a process runs, and may be retained after a process exits. Inferiors have unique identifiers that are different from process ids. Usually each inferior will also have its own distinct address space, although some embedded targets may have several inferiors running in different parts of a single address space. Each inferior may in turn have multiple threads running in it.

The commands `info inferiors` and `info connections`, which will be introduced below, accept a space-separated *ID list* as their argument specifying one or more elements on which to operate. A list element can be either a single non-negative number, like ‘5’, or an ascending range of such numbers, like ‘5-7’. A list can consist of any combination of such elements, even duplicates or overlapping ranges are valid. E.g. ‘1 4-6 5 4-4’ or ‘1 2 4-7’.

To find out what inferiors exist at any moment, use `info inferiors`:

`info inferiors`

Print a list of all inferiors currently being managed by GDB. By default all inferiors are printed, but the ID list *id...* can be used to limit the display to just the requested inferiors.

GDB displays for each inferior (in this order):

1. the inferior number assigned by GDB
2. the target system’s inferior identifier
3. the target connection the inferior is bound to, including the unique connection number assigned by GDB, and the protocol used by the connection.
4. the name of the executable the inferior is running.

If an inferior is currently debugging a core file, then an additional line will be printed after the line describing the inferior, this additional line gives the file name of the core file being debugged.

An asterisk ‘*’ preceding the GDB inferior number indicates the current inferior.

For example,

```
(gdb) info inferiors
  Num  Description      Connection      Executable
*  1    process 3401     1 (native)     goodbye
  2    process 2307     2 (extended0remote host:10000) hello
```

Adding an additional inferior that is debugging a core file causes the additional core file line to be displayed, for example

```
(gdb) info inferiors
      Num  Description      Connection      Executable
* 1      process 3401      1 (native)      goodbye
  2      process 2307      2 (extended-remote host:10000) hello
  3      process 1578      3 (core)        broken
      core file core.1578
```

To get information about the current inferior, use `inferior`:

`inferior` Shows information about the current inferior.

For example,

```
(gdb) inferior
[Current inferior is 1 [process 3401] (helloworld)]
```

To find out what open target connections exist at any moment, use `info connections`:

`info connections`

Print a list of all open target connections currently being managed by GDB. By default all connections are printed, but the ID list *id...* can be used to limit the display to just the requested connections.

GDB displays for each connection (in this order):

1. the connection number assigned by GDB.
2. the protocol used by the connection.
3. a textual description of the protocol used by the connection.

An asterisk ‘*’ preceding the connection number indicates the connection of the current inferior.

For example,

```
(gdb) info connections
      Num  What      Description
* 1      extended-remote host:10000 Extended remote serial target in gdb-specific protocol
  2      native      Native process
  3      core        Local core dump file
```

To switch focus between inferiors, use the `inferior` command:

`inferior infno`

Make inferior number *infno* the current inferior. The argument *infno* is the inferior number assigned by GDB, as shown in the first field of the ‘`info inferiors`’ display.

The debugger convenience variable ‘`$_inferior`’ contains the number of the current inferior. You may find this useful in writing breakpoint conditional expressions, command scripts, and so forth. See Section 10.12 [Convenience Variables], page 173, for general information on convenience variables.

You can get multiple executables into a debugging session via the `add-inferior` and `clone-inferior` commands. On some systems GDB can add inferiors to the debug session automatically by following calls to `fork` and `exec`. To remove inferiors from the debugging session use the `remove-inferiors` command.

add-inferior [**-copies** *n*] [**-exec** *executable*] [**-no-connection**]

Adds *n* inferiors to be run using *executable* as the executable; *n* defaults to 1. If no executable is specified, the inferiors begins empty, with no program. You can still assign or change the program assigned to the inferior at any time by using the **file** command with the executable name as its argument.

By default, the new inferior begins connected to the same target connection as the current inferior. For example, if the current inferior was connected to **gdbserver** with **target remote**, then the new inferior will be connected to the same **gdbserver** instance. The **-no-connection** option starts the new inferior with no connection yet. You can then for example use the **target remote** command to connect to some other **gdbserver** instance, use **run** to spawn a local program, etc.

Not all connections can be shared between inferiors. For example, the **target core** target is unique for each inferior. That is, multiple inferiors can use **target core** at the same time, but each **target core** is different. If you try to **add-inferior**, and the current inferior is **target core**, then GDB will give a warning and create the new inferior without a connection, like this

```
(gdb) file test1
Reading symbols from test1...
(gdb) target core core.test1.433190
[New LWP 433190]
Core was generated by './test1'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0 0x0000000000401111 in foo () at test1.c:6
6   return *global_ptr;
(gdb) add-inferior
[New inferior 2]
warning: can't share connection 1 (core) between inferiors
Added inferior 2
```

Another target that cannot be shared is **target remote** (see Section 20.1 [Types of Remote Connections], page 317). A remote target doesn't allow new inferiors to be started, as such creating a new, non-running, inferior that shares a remote connection doesn't make sense. As with the core target above, if you try to **add-inferior**, and the current inferior is **target remote**, then GDB will give a warning and create the new inferior without a connection.

clone-inferior [**-copies** *n*] [**-no-connection**] [*infno*]

Adds *n* inferiors ready to execute the same program as inferior *infno*; *n* defaults to 1, and *infno* defaults to the number of the current inferior. This command copies the values of the *args*, *inferior-tty* and *cwd* properties from the current inferior to the new one. It also propagates changes the user made to environment variables using the **set environment** and **unset environment** commands. This is a convenient command when you want to run another instance of the inferior you are debugging.

```
(gdb) info inferiors
      Num  Description      Connection  Executable
* 1      process 29964      1 (native)  helloworld
(gdb) clone-inferior
Added inferior 2.
1 inferiors added.
```

```
(gdb) info inferiors
      Num  Description      Connection  Executable
*  1      process 29964      1 (native) helloworld
   2      <null>            1 (native) helloworld
```

You can now simply switch focus to inferior 2 and run it.

Like `add-inferior`, `clone-inferior` shares the connection with the inferior *infno*. If the `-no-connection` option is given, then the new inferior will be created without a connection. If the connection of inferior *infno* can't be shared, then GDB will give a warning, and the new inferior will be created without a connection.

`remove-inferiors infno...`

Removes the inferior or inferiors *infno...*. It is not possible to remove an inferior that is running with this command. For those, use the `kill` or `detach` command first.

To quit debugging one of the running inferiors that is not the current inferior, you can either detach from it by using the `detach inferior` command (allowing it to run independently), or kill it using the `kill inferiors` command:

`detach inferior infno...`

Detach from the inferior or inferiors identified by GDB inferior number(s) *infno...*. Note that the inferior's entry still stays on the list of inferiors shown by `info inferiors`, but its Description will show '`<null>`'.

`kill inferiors infno...`

Kill the inferior or inferiors identified by GDB inferior number(s) *infno...*. Note that the inferior's entry still stays on the list of inferiors shown by `info inferiors`, but its Description will show '`<null>`'.

After the successful completion of a command such as `detach`, `detach inferiors`, `kill` or `kill inferiors`, or after a normal process exit, the inferior is still valid and listed with `info inferiors`, ready to be restarted.

To be notified when inferiors are started or exit under GDB's control use `set print inferior-events`:

```
set print inferior-events
set print inferior-events on
set print inferior-events off
```

The `set print inferior-events` command allows you to enable or disable printing of messages when GDB notices that new inferiors have started or that inferiors have exited or have been detached. By default, these messages will be printed.

`show print inferior-events`

Show whether messages will be printed when GDB detects that inferiors have started, exited or have been detached.

Many commands will work the same with multiple programs as with a single program: e.g., `print myglobal` will simply display the value of `myglobal` in the current inferior.

Occasionally, when debugging GDB itself, it may be useful to get more info about the relationship of inferiors, programs, address spaces in a debug session. You can do that with the `maint info program-spaces` command.

`maint info program-spaces`

Print a list of all program spaces currently being managed by GDB.

GDB displays for each program space (in this order):

1. the program space number assigned by GDB
2. the name of the executable loaded into the program space, with e.g., the `file` command.

An asterisk ‘*’ preceding the GDB program space number indicates the current program space.

In addition, below each program space line, GDB prints extra information that isn’t suitable to display in tabular form. For example, the list of inferiors bound to the program space.

```
(gdb) maint info program-spaces
  Id  Executable
* 1   hello
  2   goodbye
    Bound inferiors: ID 1 (process 21561)
```

Here we can see that no inferior is running the program `hello`, while `process 21561` is running the program `goodbye`. On some targets, it is possible that multiple inferiors are bound to the same program space. The most common example is that of debugging both the parent and child processes of a `vfork` call. For example,

```
(gdb) maint info program-spaces
  Id  Executable
* 1   vfork-test
    Bound inferiors: ID 2 (process 18050), ID 1 (process 18045)
```

Here, both inferior 2 and inferior 1 are running in the same program space as a result of inferior 1 having executed a `vfork` call.

4.9.1 Inferior-Specific Breakpoints

When debugging multiple inferiors, you can choose whether to set breakpoints for all inferiors, or for a particular inferior.

`break locspec inferior inferior-id`

`break locspec inferior inferior-id if ...`

locspec specifies a code location or locations in your program. See Section 9.2 [Location Specifications], page 128, for details.

Use the qualifier ‘`inferior inferior-id`’ with a breakpoint command to specify that you only want GDB to stop when a particular inferior reaches this breakpoint. The *inferior-id* specifier is one of the inferior identifiers assigned by GDB, shown in the first column of the ‘`info inferiors`’ output.

If you do not specify ‘`inferior inferior-id`’ when you set a breakpoint, the breakpoint applies to *all* inferiors of your program.

You can use the `inferior` qualifier on conditional breakpoints as well; in this case, place ‘`inferior inferior-id`’ before or after the breakpoint condition, like this:

```
(gdb) break frik.c:13 inferior 2 if bartab > lim
```

Inferior-specific breakpoints are automatically deleted when the corresponding inferior is removed from GDB. For example:

```
(gdb) remove-inferiors 2
Inferior-specific breakpoint 3 deleted - inferior 2 has been removed.
```

A breakpoint can’t be both inferior-specific and thread-specific (see Section 5.5.4 [Thread-Specific Breakpoints], page 99), or task-specific (see Section 15.4.10.7 [Ada Tasks], page 256); using more than one of the `inferior`, `thread`, or `task` keywords when creating a breakpoint will give an error.

4.10 Debugging Programs with Multiple Threads

In some operating systems, such as GNU/Linux and Solaris, a single program may have more than one *thread* of execution. The precise semantics of threads differ from one operating system to another, but in general the threads of a single program are akin to multiple processes—except that they share one address space (that is, they can all examine and modify the same variables). On the other hand, each thread has its own registers and execution stack, and perhaps private memory.

GDB provides these facilities for debugging multi-thread programs:

- automatic notification of new threads
- ‘`thread thread-id`’, a command to switch among threads
- ‘`info threads`’, a command to inquire about existing threads
- ‘`thread apply [thread-id-list | all] args`’, a command to apply a command to a list of threads
- thread-specific breakpoints
- ‘`set print thread-events`’, which controls printing of messages on thread start and exit.
- ‘`set libthread-db-search-path path`’, which lets the user specify which `libthread_db` to use if the default choice isn’t compatible with the program.

The GDB thread debugging facility allows you to observe all threads while your program runs—but whenever GDB takes control, one thread in particular is always the focus of debugging. This thread is called the *current thread*. Debugging commands show program information from the perspective of the current thread.

Whenever GDB detects a new thread in your program, it displays the target system’s identification for the thread with a message in the form ‘`[New systag]`’, where *systag* is a thread identifier whose form varies depending on the particular system. For example, on GNU/Linux, you might see

```
[New Thread 0x41e02940 (LWP 25582)]
```

when GDB notices a new thread. In contrast, on other systems, the *systag* is simply something like ‘`process 368`’, with no further qualifier.

For debugging purposes, GDB associates its own thread number —always a single integer—with each thread of an inferior. This number is unique between all threads of an inferior, but not unique between threads of different inferiors.

You can refer to a given thread in an inferior using the qualified *inferior-num.thread-num* syntax, also known as *qualified thread ID*, with *inferior-num* being the inferior number and *thread-num* being the thread number of the given inferior. For example, thread 2.3 refers to thread number 3 of inferior 2. If you omit *inferior-num* (e.g., **thread 3**), then GDB infers you're referring to a thread of the current inferior.

Until you create a second inferior, GDB does not show the *inferior-num* part of thread IDs, even though you can always use the full *inferior-num.thread-num* form to refer to threads of inferior 1, the initial inferior.

Some commands accept a space-separated *thread ID list* as argument. A list element can be:

1. A thread ID as shown in the first field of the **'info threads'** display, with or without an inferior qualifier. E.g., **'2.1'** or **'1'**.
2. A range of thread numbers, again with or without an inferior qualifier, as in *inf.thr1-thr2* or *thr1-thr2*. E.g., **'1.2-4'** or **'2-4'**.
3. All threads of an inferior, specified with a star wildcard, with or without an inferior qualifier, as in *inf.** (e.g., **'1.*'**) or *****. The former refers to all threads of the given inferior, and the latter form without an inferior qualifier refers to all threads of the current inferior.

For example, if the current inferior is 1, and inferior 7 has one thread with ID 7.1, the thread list **'1 2-3 4.5 6.7-9 7.*'** includes threads 1 to 3 of inferior 1, thread 5 of inferior 4, threads 7 to 9 of inferior 6 and all threads of inferior 7. That is, in expanded qualified form, the same as **'1.1 1.2 1.3 4.5 6.7 6.8 6.9 7.1'**.

In addition to a *per-inferior* number, each thread is also assigned a unique *global* number, also known as *global thread ID*, a single integer. Unlike the thread number component of the thread ID, no two threads have the same global ID, even when you're debugging multiple inferiors.

From GDB's perspective, a process always has at least one thread. In other words, GDB assigns a thread number to the program's "main thread" even if the program is not multi-threaded.

The debugger convenience variables **'\$_thread'** and **'\$_gthread'** contain, respectively, the per-inferior thread number and the global thread number of the current thread. You may find this useful in writing breakpoint conditional expressions, command scripts, and so forth. The convenience variable **'\$_inferior_thread_count'** contains the number of live threads in the current inferior. See Section 10.12 [Convenience Variables], page 173, for general information on convenience variables.

When running in non-stop mode (see Section 5.5.2 [Non-Stop Mode], page 97), where new threads can be created, and existing threads exit, at any time, **'\$_inferior_thread_count'** could return a different value each time it is evaluated.

If GDB detects the program is multi-threaded, it augments the usual message about stopping at a breakpoint with the ID and name of the thread that hit the breakpoint.

```
Thread 2 "client" hit Breakpoint 1, send_message () at client.c:68
```

Likewise when the program receives a signal:

```
Thread 1 "main" received signal SIGINT, Interrupt.
```

info threads [-gid] [-stopped] [-running] [*thread-id-list*]

Display information about one or more threads. With no arguments displays information about all threads. You can specify the list of threads that you want to display using the thread ID list syntax (see [thread ID lists], page 49).

GDB displays for each thread (in this order):

1. the per-inferior thread number assigned by GDB
2. the global thread number assigned by GDB, if the ‘-gid’ option was specified
3. the target system’s thread identifier (*systag*)
4. the thread’s name, if one is known. A thread can either be named by the user (see **thread name**, below), or, in some cases, by the program itself.
5. the current stack frame summary for that thread

An asterisk ‘*’ to the left of the GDB thread number indicates the current thread.

For example,

```
(gdb) info threads
  Id  Target Id          Frame
* 1   process 35 thread 13  main (argc=1, argv=0x7ffffff8)
  2   process 35 thread 23  0x34e5 in sigpause ()
  3   process 35 thread 27  0x34e5 in sigpause ()
      at threadtest.c:68
```

If you’re debugging multiple inferiors, GDB displays thread IDs using the qualified *inferior-num.thread-num* format. Otherwise, only *thread-num* is shown.

If you specify the ‘-stopped’ option, GDB filters the output of the command to print the stopped threads only. Similarly, if you specify the ‘-running’ option, GDB filters the output to print the running threads only. These options can be helpful to reduce the output list if there is a large number of threads. If you specify both options, GDB prints both stopped and running threads.

If you specify the ‘-gid’ option, GDB displays a column indicating each thread’s global thread ID:

```
(gdb) info threads
  Id  GId  Target Id          Frame
 1.1  1    process 35 thread 13  main (argc=1, argv=0x7ffffff8)
 1.2  3    process 35 thread 23  0x34e5 in sigpause ()
 1.3  4    process 35 thread 27  0x34e5 in sigpause ()
* 2.1  2    process 65 thread 1   main (argc=1, argv=0x7ffffff8)
```

On Solaris, you can display more information about user threads with a Solaris-specific command:

maint info sol-threads

Display info on Solaris user threads.

thread *thread-id*

Make thread ID *thread-id* the current thread. The command argument *thread-id* is the GDB thread ID, as shown in the first field of the ‘**info threads**’ display, with or without an inferior qualifier (e.g., ‘2.1’ or ‘1’).

GDB responds by displaying the system identifier of the thread you selected, and its current stack frame summary:

```
(gdb) thread 2
[Switching to thread 2 (Thread 0xb7fdab70 (LWP 12747))]
#0  some_function (ignore=0x0) at example.c:8
8      printf ("hello\n");
```

As with the ‘[New ...]’ message, the form of the text after ‘Switching to’ depends on your system’s conventions for identifying threads.

thread apply [*thread-id-list* | *all* [-*ascending*]] [*flag*]... *command*

The **thread apply** command allows you to apply the named *command* to one or more threads. Specify the threads that you want affected using the thread ID list syntax (see [thread ID lists], page 49), or specify *all* to apply to all threads. To apply a command to all threads in descending order, type **thread apply all command**. To apply a command to all threads in ascending order, type **thread apply all -ascending command**.

The *flag* arguments control what output to produce and how to handle errors raised when applying *command* to a thread. *flag* must start with a - directly followed by one letter in *qcs*. If several flags are provided, they must be given individually, such as **-c -q**.

By default, GDB displays some thread information before the output produced by *command*, and an error raised during the execution of a *command* will abort **thread apply**. The following flags can be used to fine-tune this behavior:

- c** The flag **-c**, which stands for ‘*continue*’, causes any errors in *command* to be displayed, and the execution of **thread apply** then continues.
- s** The flag **-s**, which stands for ‘*silent*’, causes any errors or empty output produced by a *command* to be silently ignored. That is, the execution continues, but the thread information and errors are not printed.
- q** The flag **-q** (‘*quiet*’) disables printing the thread information.

Flags **-c** and **-s** cannot be used together.

taas [*option*]... *command*

Shortcut for **thread apply all -s [option]... command**. Applies *command* on all threads, ignoring errors and empty output.

The **taas** command accepts the same options as the **thread apply all** command. See [thread apply all], page 51.

tfaas [*option*]... *command*

Shortcut for **thread apply all -s -- frame apply all -s [option]... command**. Applies *command* on all frames of all threads, ignoring errors and empty output. Note that the flag **-s** is specified twice: The first **-s** ensures that **thread apply** only shows the thread information of the threads for which **frame apply** produces some output. The second **-s** is needed to ensure that **frame apply** shows the frame information of a frame only if the *command* successfully produced some output.

It can for example be used to print a local variable or a function argument without knowing the thread or frame where this variable or argument is, using:

```
(gdb) tfaas p some_local_var_i_do_not_remember_where_it_is
```

The **tfaas** command accepts the same options as the **frame apply** command. See Section 8.5 [frame apply], page 122.

thread name [*name*]

This command assigns a name to the current thread. If no argument is given, any existing user-specified name is removed. The thread name appears in the ‘**info threads**’ display.

On some systems, such as GNU/Linux, GDB is able to determine the name of the thread as given by the OS. On these systems, a name specified with ‘**thread name**’ will override the system-give name, and removing the user-specified name will cause GDB to once again display the system-specified name.

thread find [*regex*]

Search for and display thread ids whose name or *systag* matches the supplied regular expression.

As well as being the complement to the ‘**thread name**’ command, this command also allows you to identify a thread by its target *systag*. For instance, on GNU/Linux, the target *systag* is the LWP id.

```
(gdb) thread find 26688
Thread 4 has target id 'Thread 0x41e02940 (LWP 26688)'
(gdb) info thread 4
  Id   Target Id           Frame
  4    Thread 0x41e02940 (LWP 26688) 0x00000031ca6cd372 in select ()
```

set print thread-events

set print thread-events on

set print thread-events off

The **set print thread-events** command allows you to enable or disable printing of messages when GDB notices that new threads have started or that threads have exited. By default, these messages will be printed if detection of these events is supported by the target. Note that these messages cannot be disabled on all targets.

show print thread-events

Show whether messages will be printed when GDB detects that threads have started and exited.

See Section 5.5 [Stopping and Starting Multi-thread Programs], page 95, for more information about how GDB behaves when you stop and start programs with multiple threads.

See Section 5.1.2 [Setting Watchpoints], page 68, for information about watchpoints in programs with multiple threads.

set libthread-db-search-path [*path*]

If this variable is set, *path* is a colon-separated list of directories GDB will use to search for **libthread_db**. If you omit *path*, ‘**libthread-db-search-path**’ will be reset to its default value (**\$sdir:\$pdir** on GNU/Linux and Solaris systems). Internally, the default value comes from the **LIBTHREAD_DB_SEARCH_PATH** macro.

On GNU/Linux and Solaris systems, GDB uses a “helper” `libthread_db` library to obtain information about threads in the inferior process. GDB will use ‘`libthread-db-search-path`’ to find `libthread_db`. GDB also consults first if inferior specific thread debugging library loading is enabled by ‘`set auto-load libthread-db`’ (see Section 22.8.2 [`libthread_db.so.1` file], page 383).

A special entry ‘`$sdir`’ for ‘`libthread-db-search-path`’ refers to the default system directories that are normally searched for loading shared libraries. The ‘`$sdir`’ entry is the only kind not needing to be enabled by ‘`set auto-load libthread-db`’ (see Section 22.8.2 [`libthread_db.so.1` file], page 383).

A special entry ‘`$pdir`’ for ‘`libthread-db-search-path`’ refers to the directory from which `libpthread` was loaded in the inferior process.

For any `libthread_db` library GDB finds in above directories, GDB attempts to initialize it with the current inferior process. If this initialization fails (which could happen because of a version mismatch between `libthread_db` and `libpthread`), GDB will unload `libthread_db`, and continue with the next directory. If none of `libthread_db` libraries initialize successfully, GDB will issue a warning and thread debugging will be disabled.

Setting `libthread-db-search-path` is currently implemented only on some platforms.

show libthread-db-search-path

Display current `libthread_db` search path.

set debug libthread-db

show debug libthread-db

Turns on or off display of `libthread_db`-related events. Use 1 to enable, 0 to disable.

set debug threads [on|off]

show debug threads

When ‘on’ GDB will print additional messages when threads are created and deleted.

For some debugging targets, GDB has support for accessing variables that reside in Thread Local Storage (TLS). TLS variables are similar to global variables, except that each thread has its own copy of the variable. While often used in multi-threaded programs, TLS variables can also be used in programs without threads. The C library variable `errno` is, perhaps, the most prominent example of a TLS variable that is frequently used in non-threaded programs. For targets where GDB does not have good TLS support, printing or changing the value of `errno` might not be directly possible.

GNU/Linux and FreeBSD targets have support for accessing TLS variables. On GNU/Linux, the helper library, `libthread_db`, is used to help resolve the addresses of TLS variables. Some FreeBSD and some GNU/Linux targets also have GDB-internal TLS resolution code. GNU/Linux targets will attempt to use the TLS address lookup functionality provided by `libthread_db`, but will fall back to using its internal TLS support when `libthread_db` is not available. This can happen in cross-debugging scenarios or when debugging programs that are linked in such a way that `libthread_db` support

is unavailable – this includes statically linked programs, linking against GLIBC versions earlier than 2.34, but not with `libpthread`, and use of other (non-GLIBC) C libraries.

```
maint set force-internal-tls-address-lookup
maint show force-internal-tls-address-lookup
```

Turns on or off forced use of GDB-internal TLS address lookup code. Use **on** to enable and **off** to disable. The default for this setting is **off**.

When disabled, GDB will attempt to use a helper `libthread_db` library if possible, but will fall back to use of its own internal TLS address lookup mechanisms if necessary.

When enabled, GDB will only use GDB's internal TLS address lookup mechanisms, if they exist.

This command is only available for GNU/Linux targets. Its primary use is for testing – certain tests in the GDB test suite use this command to force use of internal TLS address lookup.

4.11 Debugging Forks

On most systems, GDB has no special support for debugging programs which create additional processes using the `fork` function. When a program forks, GDB will continue to debug the parent process and the child process will run unimpeded. If you have set a breakpoint in any code which the child then executes, the child will get a `SIGTRAP` signal which (unless it catches the signal) will cause it to terminate.

However, if you want to debug the child process there is a workaround which isn't too painful. Put a call to `sleep` in the code which the child process executes after the fork. It may be useful to sleep only if a certain environment variable is set, or a certain file exists, so that the delay need not occur when you don't want to run GDB on the child. While the child is sleeping, use the `ps` program to get its process ID. Then tell GDB (a new invocation of GDB if you are also debugging the parent process) to attach to the child process (see Section 4.7 [Attach], page 41). From that point on you can debug the child process just like any other process which you attached to.

On some systems, GDB provides support for debugging programs that create additional processes using the `fork` or `vfork` functions. On GNU/Linux platforms, this feature is supported with kernel version 2.5.46 and later.

The fork debugging commands are supported in native mode and when connected to `gdbserver` in either **target remote** mode or **target extended-remote** mode.

By default, when a program forks, GDB will continue to debug the parent process and the child process will run unimpeded.

If you want to follow the child process instead of the parent process, use the command `set follow-fork-mode`.

```
set follow-fork-mode mode
```

Set the debugger response to a program call of `fork` or `vfork`. A call to `fork` or `vfork` creates a new process. The *mode* argument can be:

parent The original process is debugged after a fork. The child process runs unimpeded. This is the default.

child The new process is debugged after a fork. The parent process runs unimpeded.

show follow-fork-mode

Display the current debugger response to a **fork** or **vfork** call.

On Linux, if you want to debug both the parent and child processes, use the command **set detach-on-fork**.

set detach-on-fork mode

Tells gdb whether to detach one of the processes after a fork, or retain debugger control over them both.

on The child process (or parent process, depending on the value of **follow-fork-mode**) will be detached and allowed to run independently. This is the default.

off Both processes will be held under the control of GDB. One process (child or parent, depending on the value of **follow-fork-mode**) is debugged as usual, while the other is held suspended.

show detach-on-fork

Show whether detach-on-fork mode is on/off.

If you choose to set ‘**detach-on-fork**’ mode off, then GDB will retain control of all forked processes (including nested forks). You can list the forked processes under the control of GDB by using the **info inferiors** command, and switch from one fork to another by using the **inferior** command (see Section 4.9 [Debugging Multiple Inferiors Connections and Programs], page 43).

To quit debugging one of the forked processes, you can either detach from it by using the **detach inferiors** command (allowing it to run independently), or kill it using the **kill inferiors** command. See Section 4.9 [Debugging Multiple Inferiors Connections and Programs], page 43.

If you ask to debug a child process and a **vfork** is followed by an **exec**, GDB executes the new target up to the first breakpoint in the new target. If you have a breakpoint set on **main** in your original program, the breakpoint will also be set on the child process’s **main**.

On some systems, when a child process is spawned by **vfork**, you cannot debug the child or parent until an **exec** call completes.

If you issue a **run** command to GDB after an **exec** call executes, the new target restarts. To restart the parent process, use the **file** command with the parent executable name as its argument. By default, after an **exec** call executes, GDB discards the symbols of the previous executable image. You can change this behavior with the **set follow-exec-mode** command.

set follow-exec-mode mode

Set debugger response to a program call of **exec**. An **exec** call replaces the program image of a process.

follow-exec-mode can be:

new GDB creates a new inferior and rebinds the process to this new inferior. The program the process was running before the **exec** call can be restarted afterwards by restarting the original inferior.

For example:

```
(gdb) info inferiors
(gdb) info inferior
  Id  Description  Executable
* 1   <null>      prog1
(gdb) run
process 12020 is executing new program: prog2
Program exited normally.
(gdb) info inferiors
  Id  Description  Executable
  1   <null>      prog1
* 2   <null>      prog2
```

same GDB keeps the process bound to the same inferior. The new executable image replaces the previous executable loaded in the inferior. Restarting the inferior after the `exec` call, with e.g., the `run` command, restarts the executable the process was running after the `exec` call. This is the default mode.

For example:

```
(gdb) info inferiors
  Id  Description  Executable
* 1   <null>      prog1
(gdb) run
process 12020 is executing new program: prog2
Program exited normally.
(gdb) info inferiors
  Id  Description  Executable
* 1   <null>      prog2
```

`follow-exec-mode` is supported in native mode and `target extended-remote` mode.

You can use the `catch` command to make GDB stop whenever a `fork`, `vfork`, or `exec` call is made. See Section 5.1.3 [Setting Catchpoints], page 71.

4.12 Setting a *Bookmark* to Return to Later

On certain operating systems¹, GDB is able to save a *snapshot* of a program's state, called a *checkpoint*, and come back to it later.

Returning to a checkpoint effectively undoes everything that has happened in the program since the `checkpoint` was saved. This includes changes in memory, registers, and even (within some limits) system state. Effectively, it is like going back in time to the moment when the checkpoint was saved.

Thus, if you're stepping through a program and you think you're getting close to the point where things go wrong, you can save a checkpoint. Then, if you accidentally go too far and miss the critical statement, instead of having to restart your program from the beginning, you can just go back to the checkpoint and start again from there.

This can be especially useful if it takes a lot of time or steps to reach the point where you think the bug occurs.

To use the `checkpoint/restart` method of debugging:

¹ Currently, only GNU/Linux.

checkpoint

Save a snapshot of the debugged program's current execution state. The **checkpoint** command takes no arguments, but each checkpoint is assigned a small integer id, similar to a breakpoint id.

info checkpoints

List the checkpoints that have been saved in the current debugging session. For each checkpoint, the following information will be listed:

Checkpoint ID
Active state indicator
Process ID
Code Address
Source line, or label

Checkpoint IDs will be displayed as either a non-negative integer or in the form *i.n*, where *i* is the inferior number, a positive integer, as shown by the command **info inferiors**, and *n*, a non-negative integer, is the checkpoint number for that inferior. The single non-negative integer form is used when there is only one inferior. The *i.n* form is used when there are multiple inferiors.

The active state indicator is a single letter, either 'y' or 'n', indicating yes or no. Only one checkpoint per inferior may be active at once. The active checkpoint in the current inferior is also shown by a '*' at the start of the line. Checkpoints whose active state is 'n' can be switched to using the **restart** command or deleted using the **delete checkpoint** command.

restart *checkpoint-id*

Restore the program state that was saved as checkpoint number *checkpoint-id*. All program variables, registers, stack frames etc. will be returned to the values that they had when the checkpoint was saved. In essence, gdb will "wind back the clock" to the point in time when the checkpoint was saved. The checkpoint number *checkpoint-id* is specified in the same form as that output by the **info checkpoints** command.

Note that breakpoints, GDB variables, command history etc. are not affected by restoring a checkpoint. In general, a checkpoint only restores things that reside in the program being debugged, not in the debugger.

delete checkpoint *checkpoint-id*

Delete the previously-saved checkpoint identified by *checkpoint-id*.

Returning to a previously saved checkpoint will restore the user state of the program being debugged, plus a significant subset of the system (OS) state, including file pointers. It won't "un-write" data from a file, but it will rewind the file pointer to the previous location, so that the previously written data can be overwritten. For files opened in read mode, the pointer will also be restored so that the previously read data can be read again.

Of course, characters that have been sent to a printer (or other external device) cannot be "snatched back", and characters received from eg. a serial device can be removed from internal program buffers, but they cannot be "pushed back" into the serial pipeline, ready to be received again. Similarly, the actual contents of files that have been changed cannot be restored (at this time).

However, within those constraints, you actually can “rewind” your program to a previously saved point in time, and begin debugging it again — and you can change the course of events so as to debug a different execution path this time.

Finally, there is one bit of internal program state that will be different when you return to a checkpoint — the program’s process id. Each checkpoint will have a unique process id (or *pid*), and each will be different from the program’s original *pid*. If your program has saved a local copy of its process id, this could potentially pose a problem.

4.12.1 A Non-obvious Benefit of Using Checkpoints

On some systems such as GNU/Linux, address space randomization is performed on new processes for security reasons. This makes it difficult or impossible to set a breakpoint, or watchpoint, on an absolute address if you have to restart the program, since the absolute location of a symbol will change from one execution to the next.

A checkpoint, however, is an *identical* copy of a process. Therefore if you create a checkpoint at (eg.) the start of main, and simply return to that checkpoint instead of restarting the process, you can avoid the effects of address randomization and your symbols will all stay in the same place.

5 Stopping and Continuing

The principal purposes of using a debugger are so that you can stop your program before it terminates; or so that, if your program runs into trouble, you can investigate and find out why.

Inside GDB, your program may stop for any of several reasons, such as a signal, a breakpoint, or reaching a new line after a GDB command such as **step**. You may then examine and change variables, set new breakpoints or remove old ones, and then continue execution. Usually, the messages shown by GDB provide ample explanation of the status of your program—but you can also explicitly request this information at any time.

info program

Display information about the status of your program: whether it is running or not, what process it is, and why it stopped.

5.1 Breakpoints, Watchpoints, and Catchpoints

A *breakpoint* makes your program stop whenever a certain point in the program is reached. For each breakpoint, you can add conditions to control in finer detail whether your program stops. You can set breakpoints with the **break** command and its variants (see Section 5.1.1 [Setting Breakpoints], page 60), to specify the place where your program should stop by line number, function name or exact address in the program.

On some systems, you can set breakpoints in shared libraries before the executable is run.

A *watchpoint* is a special breakpoint that stops your program when the value of an expression changes. The expression may be a value of a variable, or it could involve values of one or more variables combined by operators, such as ‘**a + b**’. This is sometimes called *data breakpoints*. You must use a different command to set watchpoints (see Section 5.1.2 [Setting Watchpoints], page 68), but aside from that, you can manage a watchpoint like any other breakpoint: you enable, disable, and delete both breakpoints and watchpoints using the same commands.

You can arrange to have values from your program displayed automatically whenever GDB stops at a breakpoint. See Section 10.8 [Automatic Display], page 158.

A *catchpoint* is another special breakpoint that stops your program when a certain kind of event occurs, such as the throwing of a C++ exception or the loading of a library. As with watchpoints, you use a different command to set a catchpoint (see Section 5.1.3 [Setting Catchpoints], page 71), but aside from that, you can manage a catchpoint like any other breakpoint. (To stop when your program receives a signal, use the **handle** command; see Section 5.4 [Signals], page 92.)

GDB assigns a number to each breakpoint, watchpoint, or catchpoint when you create it; these numbers are successive integers starting with one. In many of the commands for controlling various features of breakpoints you use the breakpoint number to say which breakpoint you want to change. Each breakpoint may be *enabled* or *disabled*; if disabled, it has no effect on your program until you enable it again.

Some GDB commands accept a space-separated list of breakpoints on which to operate. A list element can be either a single breakpoint number, like ‘5’, or a range of such numbers,

like ‘5-7’. When a breakpoint list is given to a command, all breakpoints in that list are operated on.

5.1.1 Setting Breakpoints

Breakpoints are set with the **break** command (abbreviated **b**). The debugger convenience variable ‘\$bpnum’ records the number of the breakpoint you’ve set most recently:

```
(gdb) b main
Breakpoint 1 at 0x11c6: file zeoes.c, line 24.
(gdb) p $bpnum
$1 = 1
```

A breakpoint may be mapped to multiple code locations for example with inlined functions, Ada generics, C++ templates or overloaded function names. GDB then indicates the number of code locations in the breakpoint command output:

```
(gdb) b some_func
Breakpoint 2 at 0x1179: some_func. (3 locations)
(gdb) p $bpnum
$2 = 2
(gdb)
```

When your program stops on a breakpoint, the convenience variables ‘\$_hit_bpnum’ and ‘\$_hit_locno’ are respectively set to the number of the encountered breakpoint and the number of the breakpoint’s code location:

```
Thread 1 "zeoes" hit Breakpoint 2.1, some_func () at zeoes.c:8
8  printf("some func\n");
(gdb) p $_hit_bpnum
$5 = 2
(gdb) p $_hit_locno
$6 = 1
(gdb)
```

Note that ‘\$_hit_bpnum’ and ‘\$bpnum’ are not equivalent: ‘\$_hit_bpnum’ is set to the breakpoint number **last hit**, while ‘\$bpnum’ is set to the breakpoint number **last set**.

If the encountered breakpoint has only one code location, ‘\$_hit_locno’ is set to 1:

```
Breakpoint 1, main (argc=1, argv=0x7fffffffe018) at zeoes.c:24
24  if (argc > 1)
(gdb) p $_hit_bpnum
$3 = 1
(gdb) p $_hit_locno
$4 = 1
(gdb)
```

The ‘\$_hit_bpnum’ and ‘\$_hit_locno’ variables can typically be used in a breakpoint command list. (see Section 5.1.7 [Breakpoint Command Lists], page 79). For example, as part of the breakpoint command list, you can disable completely the encountered breakpoint using *disable \$_hit_bpnum* or disable the specific encountered breakpoint location using *disable \$_hit_bpnum.\$_hit_locno*. If a breakpoint has only one location, ‘\$_hit_locno’ is set to 1 and the commands *disable \$_hit_bpnum* and *disable \$_hit_bpnum.\$_hit_locno* both disable the breakpoint.

You can also define aliases to easily disable the last hit location or last hit breakpoint:

```
(gdb) alias lld = disable $_hit_bpnum.$_hit_locno
(gdb) alias lbd = disable $_hit_bpnum
```

break *locspec*

Set a breakpoint at all the code locations in your program that result from resolving the given *locspec*. *locspec* can specify a function name, a line number, an address of an instruction, and more. See Section 9.2 [Location Specifications], page 128, for the various forms of *locspec*. The breakpoint will stop your program just before it executes the instruction at the address of any of the breakpoint's code locations.

When using source languages that permit overloading of symbols, such as C++, a function name may refer to more than one symbol, and thus more than one place to break. See Section 10.2 [Ambiguous Expressions], page 148, for a discussion of that situation.

It is also possible to insert a breakpoint that will stop the program only if a specific thread (see Section 5.5.4 [Thread-Specific Breakpoints], page 99), specific inferior (see Section 4.9.1 [Inferior-Specific Breakpoints], page 47), or a specific task (see Section 15.4.10.7 [Ada Tasks], page 256) hits that breakpoint.

break

When called without any arguments, **break** sets a breakpoint at the next instruction to be executed in the selected stack frame (see Chapter 8 [Examining the Stack], page 115). In any selected frame but the innermost, this makes your program stop as soon as control returns to that frame. This is similar to the effect of a **finish** command in the frame inside the selected frame—except that **finish** does not leave an active breakpoint. If you use **break** without an argument in the innermost frame, GDB stops the next time it reaches the current location; this may be useful inside loops.

GDB normally ignores breakpoints when it resumes execution, until at least one instruction has been executed. If it did not do this, you would be unable to proceed past a breakpoint without first disabling the breakpoint. This rule applies whether or not the breakpoint already existed when your program stopped.

break ... if *cond*

Set a breakpoint with condition *cond*; evaluate the expression *cond* each time the breakpoint is reached, and stop only if the value is nonzero—that is, if *cond* evaluates as true. ‘...’ stands for one of the possible arguments described above (or no argument) specifying where to break. See Section 5.1.6 [Break Conditions], page 78, for more information on breakpoint conditions.

The breakpoint may be mapped to multiple locations. If the breakpoint condition *cond* is invalid at some but not all of the locations, the locations for which the condition is invalid are disabled. For example, GDB reports below that two of the three locations are disabled.

```
(gdb) break func if a == 10
warning: failed to validate condition at location 0x11ce, disabling:
No symbol "a" in current context.
warning: failed to validate condition at location 0x11b6, disabling:
No symbol "a" in current context.
Breakpoint 1 at 0x11b6: func. (3 locations)
```

Locations that are disabled because of the condition are denoted by an uppercase N in the output of the **info breakpoints** command:

```
(gdb) info breakpoints
```

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	y	<MULTIPLE>	
	stop only if a == 10				
1.1			N*	0x00000000000011b6 in ...	
1.2			y	0x00000000000011c2 in ...	
1.3			N*	0x00000000000011ce in ...	

(*): Breakpoint condition is invalid at this location.

If the breakpoint condition *cond* is invalid in the context of *all* the locations of the breakpoint, GDB refuses to define the breakpoint. For example, if variable *foo* is an undefined variable:

```
(gdb) break func if foo
No symbol "foo" in current context.
```

break ... -force-condition if *cond*

There may be cases where the condition *cond* is invalid at all the current locations, but the user knows that it will be valid at a future location; for example, because of a library load. In such cases, by using the **-force-condition** keyword before 'if', GDB can be forced to define the breakpoint with the given condition expression instead of refusing it.

```
(gdb) break func -force-condition if foo
warning: failed to validate condition at location 1, disabling:
No symbol "foo" in current context.
warning: failed to validate condition at location 2, disabling:
No symbol "foo" in current context.
warning: failed to validate condition at location 3, disabling:
No symbol "foo" in current context.
Breakpoint 1 at 0x1158: test.c:18. (3 locations)
```

This causes all the present locations where the breakpoint would otherwise be inserted, to be disabled, as seen in the example above. However, if there exist locations at which the condition is valid, the **-force-condition** keyword has no effect.

tbreak *args*

Set a breakpoint enabled only for one stop. The *args* are the same as for the **break** command, and the breakpoint is set in the same way, but the breakpoint is automatically deleted after the first time your program stops there. See Section 5.1.5 [Disabling Breakpoints], page 76.

hbreak *args*

Set a hardware-assisted breakpoint. The *args* are the same as for the **break** command and the breakpoint is set in the same way, but the breakpoint requires hardware support and some target hardware may not have this support. The main purpose of this is EPROM/ROM code debugging, so you can set a breakpoint at an instruction without changing the instruction. This can be used with the new trap-generation provided by SPARClite DSU and most x86-based targets. These targets will generate traps when a program accesses some data or instruction address that is assigned to the debug registers. However the hardware breakpoint registers can take a limited number of breakpoints. For example, on the DSU, only two data breakpoints can be set at a time, and GDB will reject this command if more than two are used. Delete or disable unused hardware breakpoints before setting new ones (see Section 5.1.5 [Dis-

abling Breakpoints], page 76). See Section 5.1.6 [Break Conditions], page 78. For remote targets, you can restrict the number of hardware breakpoints GDB will use, see [set remote hardware-breakpoint-limit], page 329.

thbreak args

Set a hardware-assisted breakpoint enabled only for one stop. The *args* are the same as for the **hbreak** command and the breakpoint is set in the same way. However, like the **tbreak** command, the breakpoint is automatically deleted after the first time your program stops there. Also, like the **hbreak** command, the breakpoint requires hardware support and some target hardware may not have this support. See Section 5.1.5 [Disabling Breakpoints], page 76. See also Section 5.1.6 [Break Conditions], page 78.

rbreak regex

Set breakpoints on all functions matching the regular expression *regex*. This command sets an unconditional breakpoint on all matches, printing a list of all breakpoints it set. Once these breakpoints are set, they are treated just like the breakpoints set with the **break** command. You can delete them, disable them, or make them conditional the same way as any other breakpoint.

In programs using different languages, GDB chooses the syntax to print the list of all breakpoints it sets according to the ‘**set language**’ value: using ‘**set language auto**’ (see Section 15.1.3 [Set Language Automatically], page 230) means to use the language of the breakpoint’s function, other values mean to use the manually specified language (see Section 15.1.2 [Set Language Manually], page 230).

The syntax of the regular expression is the standard one used with tools like **grep**. Note that this is different from the syntax used by shells, so for instance **foo*** matches all functions that include an **fo** followed by zero or more **os**. There is an implicit **.*** leading and trailing the regular expression you supply, so to match only functions that begin with **foo**, use **^foo**.

When debugging C++ programs, **rbreak** is useful for setting breakpoints on overloaded functions that are not members of any special classes.

The **rbreak** command can be used to set breakpoints in **all** the functions in a program, like this:

```
(gdb) rbreak .
```

rbreak file:regex

If **rbreak** is called with a filename qualification, it limits the search for functions matching the given regular expression to the specified *file*. This can be used, for example, to set breakpoints on every function in a given file:

```
(gdb) rbreak file.c:.
```

The colon separating the filename qualifier from the regex may optionally be surrounded by spaces.

info breakpoints [list...]

info break [list...]

Print a table of all breakpoints, watchpoints, tracepoints, and catchpoints set and not deleted. Optional argument *n* means print information only about the

specified breakpoint(s) (or watchpoint(s) or tracepoint(s) or catchpoint(s)). For each breakpoint, following columns are printed:

Breakpoint Numbers

Type Breakpoint, watchpoint, tracepoint, or catchpoint.

Disposition

Whether the breakpoint is marked to be disabled or deleted when hit.

Enabled or Disabled

Enabled breakpoints are marked with ‘y’. ‘n’ marks breakpoints that are not enabled.

Address Where the breakpoint is in your program, as a memory address. For a pending breakpoint whose address is not yet known, this field will contain ‘<PENDING>’. Such breakpoint won’t fire until a shared library that has the symbol or line referred by breakpoint is loaded. See below for details. A breakpoint with several locations will have ‘<MULTIPLE>’ in this field—see below for details.

What Where the breakpoint is in the source for your program, as a file and line number. For a pending breakpoint, the original string passed to the breakpoint command will be listed as it cannot be resolved until the appropriate shared library is loaded in the future.

If a breakpoint is conditional, there are two evaluation modes: “host” and “target”. If mode is “host”, breakpoint condition evaluation is done by GDB on the host’s side. If it is “target”, then the condition is evaluated by the target. The **info break** command shows the condition on the line following the affected breakpoint, together with its condition evaluation mode in between parentheses. Breakpoint commands, if any, are listed after that. A pending breakpoint is allowed to have a condition specified for it. The condition is not parsed for validity until a shared library is loaded that allows the pending breakpoint to resolve to a valid location.

info break with a breakpoint number *n* as argument lists only that breakpoint. The convenience variable `$_` and the default examining-address for the **x** command are set to the address of the last breakpoint listed (see Section 10.6 [Examining Memory], page 154).

info break displays a count of the number of times the breakpoint has been hit. This is especially useful in conjunction with the **ignore** command. You can ignore a large number of breakpoint hits, look at the breakpoint info to see how many times the breakpoint was hit, and then run again, ignoring one less than that number. This will get you quickly to the last hit of that breakpoint.

For a breakpoints with an enable count (xref) greater than 1, **info break** also displays that count.

GDB allows you to set any number of breakpoints at the same place in your program. There is nothing silly or meaningless about this. When the breakpoints are conditional, this is even useful (see Section 5.1.6 [Break Conditions], page 78).

It is possible that a single logical breakpoint is set at several code locations in your program. See Section 9.2 [Location Specifications], page 128, for examples.

A breakpoint with multiple code locations is displayed in the breakpoint table using several rows—one header row, followed by one row for each code location. The header row has ‘<MULTIPLE>’ in the address column. Each code location row contains the actual address, source file, source line and function of its code location. The number column for a code location is of the form *breakpoint-number.location-number*.

For example:

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	y	<MULTIPLE>	
	stop only if i==1				
	breakpoint already hit 1 time				
1.1			y	0x080486a2 in void foo<int>() at t.cc:8	
1.2			y	0x080486ca in void foo<double>() at t.cc:8	

You cannot delete the individual locations from a breakpoint. However, each location can be individually enabled or disabled by passing *breakpoint-number.location-number* as argument to the **enable** and **disable** commands. It’s also possible to **enable** and **disable** a range of *location-number* locations using a *breakpoint-number* and two *location-numbers*, in increasing order, separated by a hyphen, like *breakpoint-number.location-number1-location-number2*, in which case GDB acts on all the locations in the range (inclusive). Disabling or enabling the parent breakpoint (see Section 5.1.5 [Disabling], page 76) affects all of the locations that belong to that breakpoint.

Locations that are enabled while their parent breakpoint is disabled won’t trigger a break, and are denoted by **y-** in the **Enb** column. For example:

```
(gdb) info breakpoints
```

Num	Type	Disp	Enb	Address	What
1	breakpoint	keep	n	<MULTIPLE>	
1.1			y-	0x00000000000011b6 in ...	
1.2			y-	0x00000000000011c2 in ...	
1.3			n	0x00000000000011ce in ...	

It’s quite common to have a breakpoint inside a shared library. Shared libraries can be loaded and unloaded explicitly, and possibly repeatedly, as the program is executed. To support this use case, GDB updates breakpoint locations whenever any shared library is loaded or unloaded. Typically, you would set a breakpoint in a shared library at the beginning of your debugging session, when the library is not loaded, and when the symbols from the library are not available. When you try to set breakpoint, GDB will ask you if you want to set a so called *pending breakpoint*—breakpoint whose address is not yet resolved.

After the program is run, whenever a new shared library is loaded, GDB reevaluates all the breakpoints. When a newly loaded shared library contains the symbol or line referred to by some pending breakpoint, that breakpoint is resolved and becomes an ordinary breakpoint. When a library is unloaded, all breakpoints that refer to its symbols or source lines become pending again.

This logic works for breakpoints with multiple locations, too. For example, if you have a breakpoint in a C++ template function, and a newly loaded shared library has an instantiation of that template, a new location is added to the list of locations for the breakpoint.

Except for having unresolved address, pending breakpoints do not differ from regular breakpoints. You can set conditions or commands, enable and disable them and perform other breakpoint operations.

GDB provides some additional commands for controlling what happens when the ‘**break**’ command cannot resolve the location spec to any code location in your program (see Section 9.2 [Location Specifications], page 128):

set breakpoint pending auto

This is the default behavior. When GDB cannot resolve the location spec, it queries you whether a pending breakpoint should be created.

set breakpoint pending on

This indicates that when GDB cannot resolve the location spec, it should create a pending breakpoint without confirmation.

set breakpoint pending off

This indicates that pending breakpoints are not to be created. If GDB cannot resolve the location spec, it aborts the breakpoint creation with an error. This setting does not affect any pending breakpoints previously created.

show breakpoint pending

Show the current behavior setting for creating pending breakpoints.

The settings above only affect the **break** command and its variants. Once a breakpoint is set, it will be automatically updated as shared libraries are loaded and unloaded.

For some targets, GDB can automatically decide if hardware or software breakpoints should be used, depending on whether the breakpoint address is read-only or read-write. This applies to breakpoints set with the **break** command as well as to internal breakpoints set by commands like **next** and **finish**. For breakpoints set with **hbreak**, GDB will always use hardware breakpoints.

You can control this automatic behavior with the following commands:

set breakpoint auto-hw on

This is the default behavior. When GDB sets a breakpoint, it will try to use the target memory map to decide if software or hardware breakpoint must be used.

set breakpoint auto-hw off

This indicates GDB should not automatically select breakpoint type. If the target provides a memory map, GDB will warn when trying to set software breakpoint at a read-only address.

GDB normally implements breakpoints by replacing the program code at the breakpoint address with a special instruction, which, when executed, given control to the debugger. By default, the program code is so modified only when the program is resumed. As soon as the program stops, GDB restores the original instructions. This behavior guards against leaving breakpoints inserted in the target should gdb abruptly disconnect. However, with slow remote targets, inserting and removing breakpoint can reduce the performance. This behavior can be controlled with the following commands::

set breakpoint always-inserted off

All breakpoints, including newly added by the user, are inserted in the target only when the target is resumed. All breakpoints are removed from the target when it stops. This is the default mode.

set breakpoint always-inserted on

Causes all breakpoints to be inserted in the target at all times. If the user adds a new breakpoint, or changes an existing breakpoint, the breakpoints in the target are updated immediately. A breakpoint is removed from the target only when breakpoint itself is deleted.

GDB handles conditional breakpoints by evaluating these conditions when a breakpoint breaks. If the condition is true, then the process being debugged stops, otherwise the process is resumed.

If the target supports evaluating conditions on its end, GDB may download the breakpoint, together with its conditions, to it.

This feature can be controlled via the following commands:

set breakpoint condition-evaluation host

This option commands GDB to evaluate the breakpoint conditions on the host's side. Unconditional breakpoints are sent to the target which in turn receives the triggers and reports them back to GDB for condition evaluation. This is the standard evaluation mode.

set breakpoint condition-evaluation target

This option commands GDB to download breakpoint conditions to the target at the moment of their insertion. The target is responsible for evaluating the conditional expression and reporting breakpoint stop events back to GDB whenever the condition is true. Due to limitations of target-side evaluation, some conditions cannot be evaluated there, e.g., conditions that depend on local data that is only known to the host. Examples include conditional expressions involving convenience variables, complex types that cannot be handled by the agent expression parser and expressions that are too long to be sent over to the target, specially when the target is a remote system. In these cases, the conditions will be evaluated by GDB.

set breakpoint condition-evaluation auto

This is the default mode. If the target supports evaluating breakpoint conditions on its end, GDB will download breakpoint conditions to the target (limitations mentioned previously apply). If the target does not support breakpoint condition evaluation, then GDB will fallback to evaluating all these conditions on the host's side.

GDB itself sometimes sets breakpoints in your program for special purposes, such as proper handling of `longjmp` (in C programs). These internal breakpoints are assigned negative numbers, starting with `-1`; `'info breakpoints'` does not display them. You can see these breakpoints with the GDB maintenance command `'maint info breakpoints'` (see [maint info breakpoints], page 777).

5.1.2 Setting Watchpoints

You can use a watchpoint to stop execution whenever the value of an expression changes, without having to predict a particular place where this may happen. (This is sometimes called a *data breakpoint*.) The expression may be as simple as the value of a single variable, or as complex as many variables combined by operators. Examples include:

- A reference to the value of a single variable.
- An address cast to an appropriate data type. For example, `*(int *)0x12345678` will watch a 4-byte region at the specified address (assuming an `int` occupies 4 bytes).
- An arbitrarily complex expression, such as `a*b + c/d`. The expression can use any operators valid in the program's native language (see Chapter 15 [Languages], page 229).

You can set a watchpoint on an expression even if the expression can not be evaluated yet. For instance, you can set a watchpoint on `*global_ptr` before `global_ptr` is initialized. GDB will stop when your program sets `global_ptr` and the expression produces a valid value. If the expression becomes valid in some other way than changing a variable (e.g. if the memory pointed to by `*global_ptr` becomes readable as the result of a `malloc` call), GDB may not stop until the next time the expression changes.

Depending on your system, watchpoints may be implemented in software or hardware. GDB does software watchpointing by single-stepping your program and testing the variable's value each time, which is hundreds of times slower than normal execution. (But this may still be worth it, to catch errors where you have no clue what part of your program is the culprit.)

On some systems, such as most PowerPC or x86-based targets, GDB includes support for hardware watchpoints, which do not slow down the running of your program.

watch [-l|-location] *expr* [*thread thread-id*] [*mask maskvalue*] [*task task-id*]

Set a watchpoint for an expression. GDB will break when the expression *expr* is written into by the program and its value changes. The simplest (and the most popular) use of this command is to watch the value of a single variable:

```
(gdb) watch foo
```

If the command includes a [*thread thread-id*] argument, GDB breaks only when the thread identified by *thread-id* changes the value of *expr*. If any other threads change the value of *expr*, GDB will not break. Note that watchpoints restricted to a single thread in this way only work with Hardware Watchpoints.

Similarly, if the *task* argument is given, then the watchpoint will be specific to the indicated Ada task (see Section 15.4.10.7 [Ada Tasks], page 256).

Ordinarily a watchpoint respects the scope of variables in *expr* (see below). The *-location* argument tells GDB to instead watch the memory referred to by *expr*. In this case, GDB will evaluate *expr*, take the address of the result, and watch the memory at that address. The type of the result is used to determine the size of the watched memory. If the expression's result does not have an address, then GDB will print an error.

The [*mask maskvalue*] argument allows creation of masked watchpoints, if the current architecture supports this feature (e.g., PowerPC Embedded architecture, see Section 21.3.8 [PowerPC Embedded], page 354.) A *masked watchpoint* specifies a mask in addition to an address to watch. The mask specifies that

some bits of an address (the bits which are reset in the mask) should be ignored when matching the address accessed by the inferior against the watchpoint address. Thus, a masked watchpoint watches many addresses simultaneously—those addresses whose unmasked bits are identical to the unmasked bits in the watchpoint address. The `mask` argument implies `-location`. Examples:

```
(gdb) watch foo mask 0xffff00ff
(gdb) watch *0xdeadbeef mask 0xffffffff00
```

rwatch [-1|-location] *expr* [thread *thread-id*] [mask *maskvalue*]

Set a watchpoint that will break when the value of *expr* is read by the program.

awatch [-1|-location] *expr* [thread *thread-id*] [mask *maskvalue*]

Set a watchpoint that will break when *expr* is either read from or written into by the program.

info watchpoints [*list...*]

This command prints a list of watchpoints, using the same format as **info break** (see Section 5.1.1 [Set Breaks], page 60).

If you watch for a change in a numerically entered address you need to dereference it, as the address itself is just a constant number which will never change. GDB refuses to create a watchpoint that watches a never-changing value:

```
(gdb) watch 0x600850
Cannot watch constant value 0x600850.
(gdb) watch *(int *) 0x600850
Watchpoint 1: *(int *) 6293584
```

GDB sets a *hardware watchpoint* if possible. Hardware watchpoints execute very quickly, and the debugger reports a change in value at the exact instruction where the change occurs. If GDB cannot set a hardware watchpoint, it sets a software watchpoint, which executes more slowly and reports the change in value at the next *statement*, not the instruction, after the change occurs.

You can force GDB to use only software watchpoints with the **set can-use-hw-watchpoints 0** command. With this variable set to zero, GDB will never try to use hardware watchpoints, even if the underlying system supports them. (Note that hardware-assisted watchpoints that were set *before* setting **can-use-hw-watchpoints** to zero will still use the hardware mechanism of watching expression values.)

set can-use-hw-watchpoints

Set whether or not to use hardware watchpoints.

show can-use-hw-watchpoints

Show the current mode of using hardware watchpoints.

For remote targets, you can restrict the number of hardware watchpoints GDB will use, see [set remote hardware-breakpoint-limit], page 329.

When you issue the **watch** command, GDB reports

```
Hardware watchpoint num: expr
```

if it was able to set a hardware watchpoint.

Currently, the **awatch** and **rwatch** commands can only set hardware watchpoints, because accesses to data that don't change the value of the watched expression cannot be

detected without examining every instruction as it is being executed, and GDB does not do that currently. If GDB finds that it is unable to set a hardware breakpoint with the `awatch` or `rwatch` command, it will print a message like this:

```
Expression cannot be implemented with read/access watchpoint.
```

Sometimes, GDB cannot set a hardware watchpoint because the data type of the watched expression is wider than what a hardware watchpoint on the target machine can handle. For example, some systems can only watch regions that are up to 4 bytes wide; on such systems you cannot set hardware watchpoints for an expression that yields a double-precision floating-point number (which is typically 8 bytes wide). As a work-around, it might be possible to break the large region into a series of smaller ones and watch them with separate watchpoints.

If you set too many hardware watchpoints, GDB might be unable to insert all of them when you resume the execution of your program. Since the precise number of active watchpoints is unknown until such time as the program is about to be resumed, GDB might not be able to warn you about this when you set the watchpoints, and the warning will be printed only when the program is resumed:

```
Hardware watchpoint num: Could not insert watchpoint
```

If this happens, delete or disable some of the watchpoints.

Watching complex expressions that reference many variables can also exhaust the resources available for hardware-assisted watchpoints. That's because GDB needs to watch every variable in the expression with separately allocated resources.

If you call a function interactively using `print` or `call`, any watchpoints you have set will be inactive until GDB reaches another kind of breakpoint or the call completes.

GDB automatically deletes watchpoints that watch local (automatic) variables, or expressions that involve such variables, when they go out of scope, that is, when the execution leaves the block in which these variables were defined. In particular, when the program being debugged terminates, *all* local variables go out of scope, and so only watchpoints that watch global variables remain set. If you rerun the program, you will need to set all such watchpoints again. One way of doing that would be to set a code breakpoint at the entry to the `main` function and when it breaks, set all the watchpoints.

In multi-threaded programs, watchpoints will detect changes to the watched expression from every thread.

Warning: In multi-threaded programs, software watchpoints have only limited usefulness. If GDB creates a software watchpoint, it can only watch the value of an expression *in a single thread*. If you are confident that the expression can only change due to the current thread's activity (and if you are also confident that no other thread can become current), then you can use software watchpoints as usual. However, GDB may not notice when a non-current thread's activity changes the expression. (Hardware watchpoints, in contrast, watch an expression in all threads.)

See [set remote hardware-watchpoint-limit], page 329.

5.1.3 Setting Catchpoints

You can use *catchpoints* to cause the debugger to stop for certain kinds of program events, such as C++ exceptions or the loading of a shared library. Use the **catch** command to set a catchpoint.

catch event

Stop when *event* occurs. The *event* can be any of the following:

```
throw [regex]
rethrow [regex]
catch [regex]
```

The throwing, re-throwing, or catching of a C++ exception.

If *regex* is given, then only exceptions whose type matches the regular expression will be caught.

The convenience variable `$_exception` is available at an exception-related catchpoint, on some systems. This holds the exception being thrown.

There are currently some limitations to C++ exception handling in GDB:

- The support for these commands is system-dependent. Currently, only systems using the ‘gnu-v3’ C++ ABI (see Section 22.7 [ABI], page 380) are supported.
- The regular expression feature and the `$_exception` convenience variable rely on the presence of some SDT probes in `libstdc++`. If these probes are not present, then these features cannot be used. These probes were first available in the GCC 4.8 release, but whether or not they are available in your GCC also depends on how it was built.
- The `$_exception` convenience variable is only valid at the instruction at which an exception-related catchpoint is set.
- When an exception-related catchpoint is hit, GDB stops at a location in the system library which implements runtime exception support for C++, usually `libstdc++`. You can use `up` (see Section 8.3 [Selection], page 119) to get to your code.
- If you call a function interactively, GDB normally returns control to you when the function has finished executing. If the call raises an exception, however, the call may bypass the mechanism that returns control to you and cause your program either to abort or to simply continue running until it hits a breakpoint, catches a signal that GDB is listening for, or exits. This is the case even if you set a catchpoint for the exception; catchpoints on exceptions are disabled within interactive calls. See Section 17.5 [Calling], page 283, for information on controlling this with `set unwind-on-terminating-exception`.
- You cannot raise an exception interactively.
- You cannot install an exception handler interactively.

exception [*name*]

An Ada exception being raised. If an exception name is specified at the end of the command (eg `catch exception Program_Error`), the debugger will stop only when this specific exception is raised. Otherwise, the debugger stops execution when any Ada exception is raised.

When inserting an exception catchpoint on a user-defined exception whose name is identical to one of the exceptions defined by the language, the fully qualified name must be used as the exception name. Otherwise, GDB will assume that it should stop on the pre-defined exception rather than the user-defined one. For instance, assuming an exception called `Constraint_Error` is defined in package `Pck`, then the command to use to catch such exceptions is `catch exception Pck.Constraint_Error`.

The convenience variable `$_ada_exception` holds the address of the exception being thrown. This can be useful when setting a condition for such a catchpoint.

exception **unhandled**

An exception that was raised but is not handled by the program. The convenience variable `$_ada_exception` is set as for `catch exception`.

handlers [*name*]

An Ada exception being handled. If an exception name is specified at the end of the command (eg `catch handlers Program_Error`), the debugger will stop only when this specific exception is handled. Otherwise, the debugger stops execution when any Ada exception is handled.

When inserting a handlers catchpoint on a user-defined exception whose name is identical to one of the exceptions defined by the language, the fully qualified name must be used as the exception name. Otherwise, GDB will assume that it should stop on the pre-defined exception rather than the user-defined one. For instance, assuming an exception called `Constraint_Error` is defined in package `Pck`, then the command to use to catch such exceptions handling is `catch handlers Pck.Constraint_Error`.

The convenience variable `$_ada_exception` is set as for `catch exception`.

assert A failed Ada assertion. Note that the convenience variable `$_ada_exception` is *not* set by this catchpoint.

exec A call to `exec`.

syscall

syscall [*name* | *number* | *group:groupname* | *g:groupname*] ...

A call to or return from a system call, a.k.a. *syscall*. A syscall is a mechanism for application programs to request a service from the

operating system (OS) or one of the OS system services. GDB can catch some or all of the syscalls issued by the debuggee, and show the related information for each syscall. If no argument is specified, calls to and returns from all system calls will be caught.

name can be any system call name that is valid for the underlying OS. Just what syscalls are valid depends on the OS. On GNU and Unix systems, you can find the full list of valid syscall names on `/usr/include/asm/unistd.h`.

Normally, GDB knows in advance which syscalls are valid for each OS, so you can use the GDB command-line completion facilities (see Section 3.3 [command completion], page 24) to list the available choices.

You may also specify the system call numerically. A syscall's number is the value passed to the OS's syscall dispatcher to identify the requested service. When you specify the syscall by its name, GDB uses its database of syscalls to convert the name into the corresponding numeric code, but using the number directly may be useful if GDB's database does not have the complete list of syscalls on your system (e.g., because GDB lags behind the OS upgrades).

You may specify a group of related syscalls to be caught at once using the **group:** syntax (**g:** is a shorter equivalent). For instance, on some platforms GDB allows you to catch all network related syscalls, by passing the argument **group:network** to **catch syscall**. Note that not all syscall groups are available in every system. You can use the command completion facilities (see Section 3.3 [command completion], page 24) to list the syscall groups available on your environment.

The example below illustrates how this command works if you don't provide arguments to it:

```
(gdb) catch syscall
Catchpoint 1 (syscall)
(gdb) r
Starting program: /tmp/catch-syscall

Catchpoint 1 (call to syscall 'close'), \
0xffffe424 in __kernel_vsyscall ()
(gdb) c
Continuing.

Catchpoint 1 (returned from syscall 'close'), \
0xffffe424 in __kernel_vsyscall ()
(gdb)
```

Here is an example of catching a system call by name:

```
(gdb) catch syscall chroot
Catchpoint 1 (syscall 'chroot' [61])
(gdb) r
Starting program: /tmp/catch-syscall

Catchpoint 1 (call to syscall 'chroot'), \
```

```

0xffffe424 in __kernel_vsyscall ()
(gdb) c
Continuing.

Catchpoint 1 (returned from syscall 'chroot'), \
0xffffe424 in __kernel_vsyscall ()
(gdb)

```

An example of specifying a system call numerically. In the case below, the syscall number has a corresponding entry in the XML file, so GDB finds its name and prints it:

```

(gdb) catch syscall 252
Catchpoint 1 (syscall(s) 'exit_group')
(gdb) r
Starting program: /tmp/catch-syscall

Catchpoint 1 (call to syscall 'exit_group'), \
0xffffe424 in __kernel_vsyscall ()
(gdb) c
Continuing.

Program exited normally.
(gdb)

```

Here is an example of catching a syscall group:

```

(gdb) catch syscall group:process
Catchpoint 1 (syscalls 'exit' [1] 'fork' [2] 'waitpid' [7]
'execve' [11] 'wait4' [114] 'clone' [120] 'vfork' [190]
'exit_group' [252] 'waitid' [284] 'unshare' [310])
(gdb) r
Starting program: /tmp/catch-syscall

Catchpoint 1 (call to syscall fork), 0x00007ffff7df4e27 in open64 ()
from /lib64/ld-linux-x86-64.so.2

(gdb) c
Continuing.

```

However, there can be situations when there is no corresponding name in XML file for that syscall number. In this case, GDB prints a warning message saying that it was not able to find the syscall name, but the catchpoint will be set anyway. See the example below:

```

(gdb) catch syscall 764
warning: The number '764' does not represent a known syscall.
Catchpoint 2 (syscall 764)
(gdb)

```

If you configure GDB using the '--without-expat' option, it will not be able to display syscall names. Also, if your architecture does not have an XML file describing its system calls, you will not be able to see the syscall names. It is important to notice that these two features are used for accessing the syscall name database. In either case, you will see a warning like this:

```

(gdb) catch syscall
warning: Could not open "syscalls/i386-linux.xml"
warning: Could not load the syscall XML file 'syscalls/i386-linux.xml'.

```

```
GDB will not be able to display syscall names.
Catchpoint 1 (syscall)
(gdb)
```

Of course, the file name will change depending on your architecture and system.

Still using the example above, you can also try to catch a syscall by its number. In this case, you would see something like:

```
(gdb) catch syscall 252
Catchpoint 1 (syscall(s) 252)
```

Again, in this case GDB would not be able to display syscall's names.

fork A call to **fork**.

vfork A call to **vfork**.

load [*regexp*]

unload [*regexp*]

The loading or unloading of a shared library. If *regexp* is given, then the catchpoint will stop only if the regular expression matches one of the affected libraries.

signal [*signal*... | 'all']

The delivery of a signal.

With no arguments, this catchpoint will catch any signal that is not used internally by GDB, specifically, all signals except 'SIGTRAP' and 'SIGINT'.

With the argument 'all', all signals, including those used by GDB, will be caught. This argument cannot be used with other signal names.

Otherwise, the arguments are a list of signal names as given to **handle** (see Section 5.4 [Signals], page 92). Only signals specified in this list will be caught.

One reason that **catch signal** can be more useful than **handle** is that you can attach commands and conditions to the catchpoint.

When a signal is caught by a catchpoint, the signal's **stop** and **print** settings, as specified by **handle**, are ignored. However, whether the signal is still delivered to the inferior depends on the **pass** setting; this can be changed in the catchpoint's commands.

tcatch event

Set a catchpoint that is enabled only for one stop. The catchpoint is automatically deleted after the first time the event is caught.

Use the **info break** command to list the current catchpoints.

5.1.4 Deleting Breakpoints

It is often necessary to eliminate a breakpoint, watchpoint, or catchpoint once it has done its job and you no longer want your program to stop there. This is called *deleting* the breakpoint. A breakpoint that has been deleted no longer exists; it is forgotten.

With the **clear** command you can delete breakpoints according to where they are in your program. With the **delete** command you can delete individual breakpoints, watchpoints, or catchpoints by specifying their breakpoint numbers.

It is not necessary to delete a breakpoint to proceed past it. GDB automatically ignores breakpoints on the first instruction to be executed when you continue execution without changing the execution address.

clear Delete any breakpoints at the next instruction to be executed in the selected stack frame (see Section 8.3 [Selecting a Frame], page 119). When the innermost frame is selected, this is a good way to delete a breakpoint where your program just stopped.

clear *locspec* Delete any breakpoint with a code location that corresponds to *locspec*. See Section 9.2 [Location Specifications], page 128, for the various forms of *locspec*. Which code locations correspond to *locspec* depends on the form used in the location specification *locspec*:

linenum

filename:linenum

-line *linenum*

-source *filename* **-line** *linenum*

If *locspec* specifies a line number, with or without a file name, the command deletes any breakpoint with a code location that is at or within the specified line *linenum* in files that match the specified *filename*. If *filename* is omitted, it defaults to the current source file.

***address** If *locspec* specifies an address, the command deletes any breakpoint with a code location that is at the given *address*.

function

-function *function*

If *locspec* specifies a function, the command deletes any breakpoint with a code location that is at the entry to any function whose name matches *function*.

Ambiguity in names of files and functions can be resolved as described in Section 9.2 [Location Specifications], page 128.

delete [**breakpoints**] [*list...*]

Delete the breakpoints, watchpoints, tracepoints, or catchpoints of the breakpoint list specified as argument. If no argument is specified, delete all breakpoints, watchpoints, tracepoints, and catchpoints (GDB asks confirmation, unless you have **set confirm off**). You can abbreviate this command as **d**.

5.1.5 Disabling Breakpoints

Rather than deleting a breakpoint, watchpoint, or catchpoint, you might prefer to *disable* it. This makes the breakpoint inoperative as if it had been deleted, but remembers the information on the breakpoint so that you can *enable* it again later.

You disable and enable breakpoints, watchpoints, tracepoints, and catchpoints with the **enable** and **disable** commands, optionally specifying one or more breakpoint numbers as arguments. Use **info break** to print a list of all breakpoints, watchpoints, tracepoints, and catchpoints if you do not know which numbers to use.

Disabling and enabling a breakpoint that has multiple locations affects all of its locations.

A breakpoint, watchpoint, or catchpoint can have any of several different states of enablement:

- Enabled. The breakpoint stops your program. A breakpoint set with the **break** command starts out in this state.
- Disabled. The breakpoint has no effect on your program.
- Enabled once. The breakpoint stops your program, but then becomes disabled.
- Enabled for a count. The breakpoint stops your program for the next N times, then becomes disabled.
- Enabled for deletion. The breakpoint stops your program, but immediately after it does so it is deleted permanently. A breakpoint set with the **tbreak** command starts out in this state.

You can use the following commands to enable or disable breakpoints, watchpoints, tracepoints, and catchpoints:

disable [**breakpoints**] [*list...*]

Disable the specified breakpoints—or all breakpoints, if none are listed. A disabled breakpoint has no effect but is not forgotten. All options such as ignore-counts, conditions and commands are remembered in case the breakpoint is enabled again later. You may abbreviate **disable** as **dis**.

enable [**breakpoints**] [*list...*]

Enable the specified breakpoints (or all defined breakpoints). They become effective once again in stopping your program.

enable [**breakpoints**] **once** *list...*

Enable the specified breakpoints temporarily. GDB disables any of these breakpoints immediately after stopping your program.

enable [**breakpoints**] **count** *count list...*

Enable the specified breakpoints temporarily. GDB records *count* with each of the specified breakpoints, and decrements a breakpoint's count when it is hit. When any count reaches 0, GDB disables that breakpoint. If a breakpoint has an ignore count (see Section 5.1.6 [Break Conditions], page 78), that will be decremented to 0 before *count* is affected.

enable [**breakpoints**] **delete** *list...*

Enable the specified breakpoints to work once, then die. GDB deletes any of these breakpoints as soon as your program stops there. Breakpoints set by the **tbreak** command start out in this state.

Except for a breakpoint set with **tbreak** (see Section 5.1.1 [Setting Breakpoints], page 60), breakpoints that you set are initially enabled; subsequently, they become disabled or enabled only when you use one of the commands above. (The command **until**

can set and delete a breakpoint of its own, but it does not change the state of your other breakpoints; see Section 5.2 [Continuing and Stepping], page 85.)

5.1.6 Break Conditions

The simplest sort of breakpoint breaks every time your program reaches a specified place. You can also specify a *condition* for a breakpoint. A condition is just a Boolean expression in your programming language (see Section 10.1 [Expressions], page 147). A breakpoint with a condition evaluates the expression each time your program reaches it, and your program stops only if the condition is *true*.

This is the converse of using assertions for program validation; in that situation, you want to stop when the assertion is violated—that is, when the condition is false. In C, if you want to test an assertion expressed by the condition *assert*, you should set the condition ‘*! assert*’ on the appropriate breakpoint.

Conditions are also accepted for watchpoints; you may not need them, since a watchpoint is inspecting the value of an expression anyhow—but it might be simpler, say, to just set a watchpoint on a variable name, and specify a condition that tests whether the new value is an interesting one.

Break conditions can have side effects, and may even call functions in your program. This can be useful, for example, to activate functions that log program progress, or to use your own print functions to format special data structures. The effects are completely predictable unless there is another enabled breakpoint at the same address. (In that case, GDB might see the other breakpoint first and stop your program without checking the condition of this one.) Note that breakpoint commands are usually more convenient and flexible than break conditions for the purpose of performing side effects when a breakpoint is reached (see Section 5.1.7 [Breakpoint Command Lists], page 79).

Breakpoint conditions can also be evaluated on the target’s side if the target supports it. Instead of evaluating the conditions locally, GDB encodes the expression into an agent expression (see Appendix F [Agent Expressions], page 871) suitable for execution on the target, independently of GDB. Global variables become raw memory locations, locals become stack accesses, and so forth.

In this case, GDB will only be notified of a breakpoint trigger when its condition evaluates to true. This mechanism may provide faster response times depending on the performance characteristics of the target since it does not need to keep GDB informed about every breakpoint trigger, even those with false conditions.

Break conditions can be specified when a breakpoint is set, by using ‘*if*’ in the arguments to the **break** command. See Section 5.1.1 [Setting Breakpoints], page 60. They can also be changed at any time with the **condition** command.

You can also use the *if* keyword with the **watch** command. The **catch** command does not recognize the *if* keyword; **condition** is the only way to impose a further condition on a catchpoint.

condition *bnum expression*

Specify *expression* as the break condition for breakpoint, watchpoint, or catchpoint number *bnum*. After you set a condition, breakpoint *bnum* stops your program only if the value of *expression* is true (nonzero, in C). When you use **condition**, GDB checks *expression* immediately for syntactic correctness,

and to determine whether symbols in it have referents in the context of your breakpoint. If *expression* uses symbols not referenced in the context of the breakpoint, GDB prints an error message:

```
No symbol "foo" in current context.
```

GDB does not actually evaluate *expression* at the time the **condition** command (or a command that sets a breakpoint with a condition, like **break if ...**) is given, however. See Section 10.1 [Expressions], page 147.

condition -force *bnum expression*

When the **-force** flag is used, define the condition even if *expression* is invalid at all the current locations of breakpoint *bnum*. This is similar to the **-force-condition** option of the **break** command.

condition *bnum*

Remove the condition from breakpoint number *bnum*. It becomes an ordinary unconditional breakpoint.

A special case of a breakpoint condition is to stop only when the breakpoint has been reached a certain number of times. This is so useful that there is a special way to do it, using the *ignore count* of the breakpoint. Every breakpoint has an ignore count, which is an integer. Most of the time, the ignore count is zero, and therefore has no effect. But if your program reaches a breakpoint whose ignore count is positive, then instead of stopping, it just decrements the ignore count by one and continues. As a result, if the ignore count value is *n*, the breakpoint does not stop the next *n* times your program reaches it.

ignore *bnum count*

Set the ignore count of breakpoint number *bnum* to *count*. The next *count* times the breakpoint is reached, your program's execution does not stop; other than to decrement the ignore count, GDB takes no action.

To make the breakpoint stop the next time it is reached, specify a count of zero.

When you use **continue** to resume execution of your program from a breakpoint, you can specify an ignore count directly as an argument to **continue**, rather than using **ignore**. See Section 5.2 [Continuing and Stepping], page 85.

If a breakpoint has a positive ignore count and a condition, the condition is not checked. Once the ignore count reaches zero, GDB resumes checking the condition.

You could achieve the effect of the ignore count with a condition such as `'$foo-- <= 0'` using a debugger convenience variable that is decremented each time. See Section 10.12 [Convenience Variables], page 173.

Ignore counts apply to breakpoints, watchpoints, tracepoints, and catchpoints.

5.1.7 Breakpoint Command Lists

You can give any breakpoint (or watchpoint or catchpoint) a series of commands to execute when your program stops due to that breakpoint. For example, you might want to print the values of certain expressions, or enable other breakpoints.

```

commands [list...]
... command-list ...

```

end Specify a list of commands for the given breakpoints. The commands themselves appear on the following lines. Type a line containing just **end** to terminate the commands.

To remove all commands from a breakpoint, type **commands** and follow it immediately with **end**; that is, give no commands.

With no argument, **commands** refers to the last breakpoint, watchpoint, or catchpoint set (not to the breakpoint most recently encountered). If the most recent breakpoints were set with a single command, then the **commands** will apply to all the breakpoints set by that command. This applies to breakpoints set by **rbreak**, and also applies when a single **break** command creates multiple breakpoints (see Section 10.2 [Ambiguous Expressions], page 148).

Pressing RET as a means of repeating the last GDB command is disabled within a *command-list*.

Inside a command list, you can use the command **disable** *\$_hit_bnum* to disable the encountered breakpoint.

If your breakpoint has several code locations, the command **disable** *\$_hit_bnum.\$_hit_locno* will disable the specific breakpoint code location encountered. If the breakpoint has only one location, this command will disable the encountered breakpoint.

You can use breakpoint commands to start your program up again. Simply use the **continue** command, or **step**, or any other command that resumes execution.

Any other commands in the command list, after a command that resumes execution, are ignored. This is because any time you resume execution (even with a simple **next** or **step**), you may encounter another breakpoint—which could have its own command list, leading to ambiguities about which list to execute.

If the first command you specify in a command list is **silent**, the usual message about stopping at a breakpoint is not printed. This may be desirable for breakpoints that are to print a specific message and then continue. If none of the remaining commands print anything, you see no sign that the breakpoint was reached. **silent** is meaningful only at the beginning of a breakpoint command list.

The commands **echo**, **output**, and **printf** allow you to print precisely controlled output, and are often useful in silent breakpoints. See Section 23.1.4 [Commands for Controlled Output], page 403.

For example, here is how you could use breakpoint commands to print the value of *x* at entry to *foo* whenever *x* is positive.

```

break foo if x>0
commands
silent
printf "x is %d\n",x
cont
end

```

One application for breakpoint commands is to compensate for one bug so you can test for another. Put a breakpoint just after the erroneous line of code, give it a condition

to detect the case in which something erroneous has been done, and give it commands to assign correct values to any variables that need them. End with the `continue` command so that your program does not stop, and start with the `silent` command so that no output is produced. Here is an example:

```
break 403
commands
silent
set x = y + 4
cont
end
```

5.1.8 Dynamic Printf

The dynamic printf command `dprintf` combines a breakpoint with formatted printing of your program's data to give you the effect of inserting `printf` calls into your program on-the-fly, without having to recompile it.

In its most basic form, the output goes to the GDB console. However, you can set the variable `dprintf-style` for alternate handling. For instance, you can ask to format the output by calling your program's `printf` function. This has the advantage that the characters go to the program's output device, so they can be recorded in redirects to files and so forth.

If you are doing remote debugging with a stub or agent, you can also ask to have the printf handled by the remote agent. In addition to ensuring that the output goes to the remote program's device along with any other output the program might produce, you can also ask that the `dprintf` remain active even after disconnecting from the remote target. Using the stub/agent is also more efficient, as it can do everything without needing to communicate with GDB.

dprintf *locspec*, *template*, *expression* [, *expression* ...]

Whenever execution reaches a code location that results from resolving *locspec*, print the values of one or more *expressions* under the control of the string *template*. To print several values, separate them with commas.

set dprintf-style *style*

Set the `dprintf` output to be handled in one of several different styles enumerated below. A change of style affects all existing dynamic printf's immediately. (If you need individual control over the print commands, simply define normal breakpoints with explicitly-supplied command lists.)

gdb Handle the output using the GDB `printf` command. When using this style, it is possible to use the `'%V'` format specifier (see [Format Specifier], page 404).

call Handle the output by calling a function in your program (normally `printf`). When using this style the supported format specifiers depend entirely on the function being called.

Most of GDB's format specifiers align with those supported by the `printf` function, however, GDB's `'%V'` format specifier extension is not supported by `printf`. When using `'call'` style `dprintf`, care should be taken to ensure that only format specifiers supported by

the output function are used, otherwise the results will be undefined.

agent Have the remote debugging agent (such as **gdbserver**) handle the output itself. This style is only available for agents that support running commands on the target. This style does not support the **%V** format specifier.

set dprintf-function *function*

Set the function to call if the dprintf style is **call**. By default its value is **printf**. You may set it to any expression that GDB can evaluate to a function, as per the **call** command.

set dprintf-channel *channel*

Set a “channel” for dprintf. If set to a non-empty value, GDB will evaluate it as an expression and pass the result as a first argument to the **dprintf-function**, in the manner of **fprintf** and similar functions. Otherwise, the dprintf format string will be the first argument, in the manner of **printf**.

As an example, if you wanted dprintf output to go to a logfile that is a standard I/O stream assigned to the variable **mylog**, you could do the following:

```
(gdb) set dprintf-style call
(gdb) set dprintf-function fprintf
(gdb) set dprintf-channel mylog
(gdb) dprintf 25,"at line 25, glob=%d\n",glob
Dprintf 1 at 0x123456: file main.c, line 25.
(gdb) info break
1      dprintf      keep y    0x00123456 in main at main.c:25
      call (void) fprintf (mylog,"at line 25, glob=%d\n",glob)
      continue

(gdb)
```

Note that the **info break** displays the dynamic printf commands as normal breakpoint commands; you can thus easily see the effect of the variable settings.

set disconnected-dprintf **on**

set disconnected-dprintf **off**

Choose whether dprintf commands should continue to run if GDB has disconnected from the target. This only applies if the **dprintf-style** is **agent**.

show disconnected-dprintf **off**

Show the current choice for disconnected dprintf.

GDB does not check the validity of function and channel, relying on you to supply values that are meaningful for the contexts in which they are being used. For instance, the function and channel may be the values of local variables, but if that is the case, then all enabled dynamic prints must be at locations within the scope of those locals. If evaluation fails, GDB will report an error.

5.1.9 How to save breakpoints to a file

To save breakpoint definitions to a file use the **save breakpoints** command.

save breakpoints [*filename*]

This command saves all current breakpoint definitions together with their commands and ignore counts, into a file *filename* suitable for use in a later debugging session. This includes all types of breakpoints (breakpoints, watchpoints, catchpoints, tracepoints). To read the saved breakpoint definitions, use the **source** command (see Section 23.1.3 [Command Files], page 401). Note that watchpoints with expressions involving local variables may fail to be recreated because it may not be possible to access the context where the watchpoint is valid anymore. Because the saved breakpoint definitions are simply a sequence of GDB commands that recreate the breakpoints, you can edit the file in your favorite editing program, and remove the breakpoint definitions you're not interested in, or that can no longer be recreated.

5.1.10 Static Probe Points

GDB supports *SDT* probes in the code. SDT stands for Statically Defined Tracing, and the probes are designed to have a tiny runtime code and data footprint, and no dynamic relocations.

Currently, the following types of probes are supported on ELF-compatible systems:

- **SystemTap** (<http://sourceware.org/systemtap/>) SDT probes¹. **SystemTap** probes are usable from assembly, C and C++ languages².
- **DTrace** (<http://oss.oracle.com/projects/DTrace>) USDt probes. **DTrace** probes are usable from C and C++ languages.

Some **SystemTap** probes have an associated semaphore variable; for instance, this happens automatically if you defined your probe using a **DTrace**-style *.d* file. If your probe has a semaphore, GDB will automatically enable it when you specify a breakpoint using the **'-probe-stap'** notation. But, if you put a breakpoint at a probe's location by some other method (e.g., **break file:line**), then GDB will not automatically set the semaphore. **DTrace** probes do not support semaphores.

You can examine the available static static probes using **info probes**, with optional arguments:

info probes [*type*] [*provider* [*name* [*objfile*]]]

If given, *type* is either **stap** for listing **SystemTap** probes or **dtrace** for listing **DTrace** probes. If omitted all probes are listed regardless of their types.

If given, *provider* is a regular expression used to match against provider names when selecting which probes to list. If omitted, probes by all probes from all providers are listed.

If given, *name* is a regular expression to match against probe names when selecting which probes to list. If omitted, probe names are not considered when deciding whether to display them.

¹ See <http://sourceware.org/systemtap/wiki/AddingUserSpaceProbingToApps> for more information on how to add **SystemTap** SDT probes in your applications.

² See <http://sourceware.org/systemtap/wiki/UserSpaceProbeImplementation> for a good reference on how the SDT probes are implemented.

If given, *objfile* is a regular expression used to select which object files (executable or shared libraries) to examine. If not given, all object files are considered.

info probes all

List the available static probes, from all types.

Some probe points can be enabled and/or disabled. The effect of enabling or disabling a probe depends on the type of probe being handled. Some **DTrace** probes can be enabled or disabled, but **SystemTap** probes cannot be disabled.

You can enable (or disable) one or more probes using the following commands, with optional arguments:

enable probes [*provider* [*name* [*objfile*]]]

If given, *provider* is a regular expression used to match against provider names when selecting which probes to enable. If omitted, all probes from all providers are enabled.

If given, *name* is a regular expression to match against probe names when selecting which probes to enable. If omitted, probe names are not considered when deciding whether to enable them.

If given, *objfile* is a regular expression used to select which object files (executable or shared libraries) to examine. If not given, all object files are considered.

disable probes [*provider* [*name* [*objfile*]]]

See the **enable probes** command above for a description of the optional arguments accepted by this command.

A probe may specify up to twelve arguments. These are available at the point at which the probe is defined—that is, when the current PC is at the probe’s location. The arguments are available using the convenience variables (see Section 10.12 [Convenience Vars], page 173) `$_probe_arg0`...`$_probe_arg11`. In **SystemTap** probes each probe argument is an integer of the appropriate size; types are not preserved. In **DTrace** probes types are preserved provided that they are recognized as such by GDB; otherwise the value of the probe argument will be a long integer. The convenience variable `$_probe_argc` holds the number of arguments at the current probe point.

These variables are always available, but attempts to access them at any location other than a probe point will cause GDB to give an error message.

5.1.11 “Cannot insert breakpoints”

If you request too many active hardware-assisted breakpoints and watchpoints, you will see this error message:

```
Stopped; cannot insert breakpoints.
```

```
You may have requested too many hardware breakpoints and watchpoints.
```

This message is printed when you attempt to resume the program, since only then GDB knows exactly how many hardware breakpoints and watchpoints it needs to insert.

When this message is printed, you need to disable or remove some of the hardware-assisted breakpoints and watchpoints, and then continue.

5.1.12 “Breakpoint address adjusted...”

Some processor architectures place constraints on the addresses at which breakpoints may be placed. For architectures thus constrained, GDB will attempt to adjust the breakpoint’s address to comply with the constraints dictated by the architecture.

One example of such an architecture is the Fujitsu FR-V. The FR-V is a VLIW architecture in which a number of RISC-like instructions may be bundled together for parallel execution. The FR-V architecture constrains the location of a breakpoint instruction within such a bundle to the instruction with the lowest address. GDB honors this constraint by adjusting a breakpoint’s address to the first in the bundle.

It is not uncommon for optimized code to have bundles which contain instructions from different source statements, thus it may happen that a breakpoint’s address will be adjusted from one source statement to another. Since this adjustment may significantly alter GDB’s breakpoint related behavior from what the user expects, a warning is printed when the breakpoint is first set and also when the breakpoint is hit.

A warning like the one below is printed when setting a breakpoint that’s been subject to address adjustment:

```
warning: Breakpoint address adjusted from 0x00010414 to 0x00010410.
```

Such warnings are printed both for user settable and GDB’s internal breakpoints. If you see one of these warnings, you should verify that a breakpoint set at the adjusted address will have the desired affect. If not, the breakpoint in question may be removed and other breakpoints may be set which will have the desired behavior. E.g., it may be sufficient to place the breakpoint at a later instruction. A conditional breakpoint may also be useful in some cases to prevent the breakpoint from triggering too often.

GDB will also issue a warning when stopping at one of these adjusted breakpoints:

```
warning: Breakpoint 1 address previously adjusted from 0x00010414
to 0x00010410.
```

When this warning is encountered, it may be too late to take remedial action except in cases where the breakpoint is hit earlier or more frequently than expected.

5.2 Continuing and Stepping

Continuing means resuming program execution until your program completes normally. In contrast, *stepping* means executing just one more “step” of your program, where “step” may mean either one line of source code, or one machine instruction (depending on what particular command you use). Either when continuing or when stepping, your program may stop even sooner, due to a breakpoint or a signal. (If it stops due to a signal, you may want to use `handle`, or use ‘`signal 0`’ to resume execution (see Section 5.4 [Signals], page 92), or you may step into the signal’s handler (see [stepping and signal handlers], page 94).)

```
continue [ignore-count]
c [ignore-count]
fg [ignore-count]
```

Resume program execution, at the address where your program last stopped; any breakpoints set at that address are bypassed. The optional argument *ignore-count* allows you to specify a further number of times to ignore a breakpoint at this location; its effect is like that of `ignore` (see Section 5.1.6 [Break Conditions], page 78).

The argument *ignore-count* is meaningful only when your program stopped due to a breakpoint. At other times, the argument to **continue** is ignored.

The synonyms **c** and **fg** (for *foreground*, as the debugged program is deemed to be the foreground program) are provided purely for convenience, and have exactly the same behavior as **continue**.

To resume execution at a different place, you can use **return** (see Section 17.4 [Returning from a Function], page 282) to go back to the calling function; or **jump** (see Section 17.2 [Continuing at a Different Address], page 280) to go to an arbitrary location in your program.

A typical technique for using stepping is to set a breakpoint (see Section 5.1 [Breakpoints; Watchpoints; and Catchpoints], page 59) at the beginning of the function or the section of your program where a problem is believed to lie, run your program until it stops at that breakpoint, and then step through the suspect area, examining the variables that are interesting, until you see the problem happen.

step Continue running your program until control reaches a different source line, then stop it and return control to GDB. This command is abbreviated **s**.

Warning: If you use the **step** command while control is within a function that was compiled without debugging information, execution proceeds until control reaches a function that does have debugging information. Likewise, it will not step into a function which is compiled without debugging information. To step through functions without debugging information, use the **stepi** command, described below.

The **step** command only stops at the first instruction of a source line. This prevents the multiple stops that could otherwise occur in **switch** statements, **for** loops, etc. **step** continues to stop if a function that has debugging information is called within the line. In other words, **step** *steps inside* any functions called within the line.

Also, the **step** command only enters a function if there is line number information for the function. Otherwise it acts like the **next** command. This avoids problems when using **cc -g1** on MIPS machines. Previously, **step** entered subroutines if there was any debugging information about the routine.

step count

Continue running as in **step**, but do so *count* times. If a breakpoint is reached, or a signal not related to stepping occurs before *count* steps, stepping stops right away.

next [count]

Continue to the next source line in the current (innermost) stack frame. This is similar to **step**, but function calls that appear within the line of code are executed without stopping. Execution stops when control reaches a different line of code at the original stack level that was executing when you gave the **next** command. This command is abbreviated **n**.

An argument *count* is a repeat count, as for **step**.

The **next** command only stops at the first instruction of a source line. This prevents multiple stops that could otherwise occur in **switch** statements, **for** loops, etc.

set step-mode

set step-mode on

The **set step-mode on** command causes the **step** command to stop at the first instruction of a function which contains no debug line information rather than stepping over it.

This is useful in cases where you may be interested in inspecting the machine instructions of a function which has no symbolic info and do not want GDB to automatically skip over this function.

set step-mode off

Causes the **step** command to step over any functions which contains no debug information. This is the default.

show step-mode

Show whether GDB will stop in or step over functions without source line debug information.

finish Continue running until just after function in the selected stack frame returns. Print the returned value (if any). This command can be abbreviated as **fin**.

Contrast this with the **return** command (see Section 17.4 [Returning from a Function], page 282).

set print finish [on|off]

show print finish

By default the **finish** command will show the value that is returned by the function. This can be disabled using **set print finish off**. When disabled, the value is still entered into the value history (see Section 10.11 [Value History], page 172), but not displayed.

until

u Continue running until a source line past the current line, in the current stack frame, is reached. This command is used to avoid single stepping through a loop more than once. It is like the **next** command, except that when **until** encounters a jump, it automatically continues execution until the program counter is greater than the address of the jump.

This means that when you reach the end of a loop after single stepping through it, **until** makes your program continue execution until it exits the loop. In contrast, a **next** command at the end of a loop simply steps back to the beginning of the loop, which forces you to step through the next iteration.

until always stops your program if it attempts to exit the current stack frame. **until** may produce somewhat counterintuitive results if the order of machine code does not match the order of the source lines. For example, in the following excerpt from a debugging session, the **f** (**frame**) command shows that execution is stopped at line 206; yet when we use **until**, we get to line 195:

```
(gdb) f
```

```
#0 main (argc=4, argv=0xf7fffae8) at m4.c:206
206      expand_input();
(gdb) until
195      for ( ; argc > 0; NEXTARG) {
```

This happened because, for execution efficiency, the compiler had generated code for the loop closure test at the end, rather than the start, of the loop—even though the test in a C `for`-loop is written before the body of the loop. The `until` command appeared to step back to the beginning of the loop when it advanced to this expression; however, it has not really gone to an earlier statement—not in terms of the actual machine code.

`until` with no argument works by means of single instruction stepping, and hence is slower than `until` with an argument.

`until locspec`

`u locspec` Continue running your program until either it reaches a code location that results from resolving *locspec*, or the current stack frame returns. *locspec* is any of the forms described in Section 9.2 [Location Specifications], page 128. This form of the command uses temporary breakpoints, and hence is quicker than `until` without an argument. The specified location is actually reached only if it is in the current frame. This implies that `until` can be used to skip over recursive function invocations. For instance in the code below, if the current location is line 96, issuing `until 99` will execute the program up to line 99 in the same invocation of `factorial`, i.e., after the inner invocations have returned.

```
94 int factorial (int value)
95 {
96     if (value > 1) {
97         value *= factorial (value - 1);
98     }
99     return (value);
100 }
```

`advance locspec`

Continue running your program until either it reaches a code location that results from resolving *locspec*, or the current stack frame returns. *locspec* is any of the forms described in Section 9.2 [Location Specifications], page 128. This command is similar to `until`, but `advance` will not skip over recursive function calls, and the target code location doesn't have to be in the same frame as the current one.

`stepi`

`stepi arg`

`si` Execute one machine instruction, then stop and return to the debugger.

It is often useful to do `'display/i $pc'` when stepping by machine instructions. This makes GDB automatically display the next instruction to be executed, each time your program stops. See Section 10.8 [Automatic Display], page 158.

An argument is a repeat count, as in `step`.

nexti
nexti arg
ni Execute one machine instruction, but if it is a function call, proceed until the function returns.

An argument is a repeat count, as in **next**.

By default, and if available, GDB makes use of target-assisted *range stepping*. In other words, whenever you use a stepping command (e.g., **step**, **next**), GDB tells the target to step the corresponding range of instruction addresses instead of issuing multiple single-steps. This speeds up line stepping, particularly for remote targets. Ideally, there should be no reason you would want to turn range stepping off. However, it's possible that a bug in the debug info, a bug in the remote stub (for remote targets), or even a bug in GDB could make line stepping behave incorrectly when target-assisted range stepping is enabled. You can use the following command to turn off range stepping if necessary:

set range-stepping
show range-stepping
 Control whether range stepping is enabled.
 If **on**, and the target supports it, GDB tells the target to step a range of addresses itself, instead of issuing multiple single-steps. If **off**, GDB always issues single-steps, even if range stepping is supported by the target. The default is **on**.

5.3 Skipping Over Functions and Files

The program you are debugging may contain some functions which are uninteresting to debug. The **skip** command lets you tell GDB to skip a function, all functions in a file or a particular function in a particular file when stepping.

For example, consider the following C function:

```
101     int func()
102     {
103         foo(boring());
104         bar(boring());
105     }
```

Suppose you wish to step into the functions **foo** and **bar**, but you are not interested in stepping through **boring**. If you run **step** at line 103, you'll enter **boring()**, but if you run **next**, you'll step over both **foo** and **boring**!

One solution is to **step** into **boring** and use the **finish** command to immediately exit it. But this can become tedious if **boring** is called from many places.

A more flexible solution is to execute **skip boring**. This instructs GDB never to step into **boring**. Now when you execute **step** at line 103, you'll step over **boring** and directly into **foo**.

Functions may be skipped by providing either a function name, linespec (see Section 9.2 [Location Specifications], page 128), regular expression that matches the function's name, file name or a **glob**-style pattern that matches the file name.

On Posix systems the form of the regular expression is "Extended Regular Expressions". See for example '**man 7 regex**' on GNU/Linux systems. On non-Posix systems the form of the regular expression is whatever is provided by the **regcomp** function of the underlying

system. See for example ‘man 7 glob’ on GNU/Linux systems for a description of glob-style patterns.

skip [*options*]

The basic form of the **skip** command takes zero or more options that specify what to skip. The *options* argument is any useful combination of the following:

-file *file*

Functions in *file* will be skipped over when stepping.

-gfile *file-glob-pattern*

Functions in files matching *file-glob-pattern* will be skipped over when stepping.

```
(gdb) skip -gfile utils/*.c
```

If *file-glob-pattern* is an absolute filename, for example, on Unix systems, if *file-glob-pattern* starts with /, then the glob will only match full filenames.

```
(gdb) skip -gfile /include/*.h
```

In this case `/include/foo.h` would match, but `/usr/include/foo.h` would not. In contrast

```
(gdb) skip -gfile include/*.h
```

This would match both `/include/foo.h` and `/usr/include/foo.h` as the glob pattern is not an absolute filename.

In addition to the standard glob special characters, GDB supports ‘**’. The ‘**’ pattern, called *globstar*, can be used to match against 0 or more filename components. For example, given the files `/include/a.h`, `/include/xxx/b.h`, `/include/yyy/zzz/c.h`, then all of these could be skipped using

```
(gdb) skip -gfile /include/**/*.h
```

The ‘**’ pattern must appear on its own within the pattern, it cannot be combined with other characters, so ‘`include/**/*.h`’ will provide the globstar behaviour, but ‘`include/x/**/*.h`’ will not as ‘`x**`’ combines the ‘**’ with the ‘x’ character. The ‘`x**`’ will be treated as ‘x’ followed by two ‘*’ characters, matching zero or more characters within that one directory component.

-function *linespec*

Functions named by *linespec* or the function containing the line named by *linespec* will be skipped over when stepping. See Section 9.2 [Location Specifications], page 128.

-rfunction *regex*

Functions whose name matches *regex* will be skipped over when stepping.

This form is useful for complex function names. For example, there is generally no need to step into C++ `std::string` constructors or destructors. Plus with C++ templates it can be hard to write out the full name of the function, and often it doesn’t matter what the

template arguments are. Specifying the function to be skipped as a regular expression makes this easier.

```
(gdb) skip -rfunction ^std::(allocator|basic_string)<.*>::~~?\1 *\()
```

If you want to skip every templated C++ constructor and destructor in the `std` namespace you can do:

```
(gdb) skip -rfunction ^std::([a-zA-z0-9_]+)<.*>::~~?\1 *\()
```

If no options are specified, the function you're currently debugging will be skipped.

`skip function` [*linespec*]

After running this command, the function named by *linespec* or the function containing the line named by *linespec* will be skipped over when stepping. See Section 9.2 [Location Specifications], page 128.

If you do not specify *linespec*, the function you're currently debugging will be skipped.

(If you have a function called `file` that you want to skip, use `skip function file`.)

`skip file` [*filename*]

After running this command, any function whose source lives in *filename* will be skipped over when stepping.

```
(gdb) skip file boring.c
File boring.c will be skipped when stepping.
```

If you do not specify *filename*, functions whose source lives in the file you're currently debugging will be skipped.

Skips can be listed, deleted, disabled, and enabled, much like breakpoints. These are the commands for managing your list of skips:

`info skip` [*range*]

Print details about the specified skip(s). If *range* is not specified, print a table with details about all functions and files marked for skipping. `info skip` prints the following information about each skip:

Identifier A number identifying this skip.

Enabled or Disabled

Enabled skips are marked with 'y'. Disabled skips are marked with 'n'.

Glob If the file name is a 'glob' pattern this is 'y'. Otherwise it is 'n'.

File The name or 'glob' pattern of the file to be skipped. If no file is specified this is '<none>'.

RE If the function name is a 'regular expression' this is 'y'. Otherwise it is 'n'.

Function The name or regular expression of the function to skip. If no function is specified this is '<none>'.

`skip delete [range]`

`delete skip [range]`

Delete the specified skip(s). If *range* is not specified, delete all skips.

`skip enable [range]`

`enable skip [range]`

Enable the specified skip(s). If *range* is not specified, enable all skips.

`skip disable [range]`

`disable skip [range]`

Disable the specified skip(s). If *range* is not specified, disable all skips.

`save skip filename`

Save all skips to the file *filename*. This command writes out the skips as a script that can be re-read into GDB using the `source` command.

`set debug skip [on|off]`

Set whether to print the debug output about skipping files and functions.

`show debug skip`

Show whether the debug output about skipping files and functions is printed.

5.4 Signals

A signal is an asynchronous event that can happen in a program. The operating system defines the possible kinds of signals, and gives each kind a name and a number. For example, in Unix `SIGINT` is the signal a program gets when you type an interrupt character (often *Ctrl-c*); `SIGSEGV` is the signal a program gets from referencing a place in memory far away from all the areas in use; `SIGALRM` occurs when the alarm clock timer goes off (which happens only if your program has requested an alarm).

Some signals, including `SIGALRM`, are a normal part of the functioning of your program. Others, such as `SIGSEGV`, indicate errors; these signals are *fatal* (they kill your program immediately) if the program has not specified in advance some other way to handle the signal. `SIGINT` does not indicate an error in your program, but it is normally fatal so it can carry out the purpose of the interrupt: to kill the program.

GDB has the ability to detect any occurrence of a signal in your program. You can tell GDB in advance what to do for each kind of signal, apart from `SIGKILL`, which has its usual effect regardless.

When specifying a signal by number, GDB translates the number to the target platform according to the corresponding signal name. For example, GDB always treats signal 1 as `SIGHUP`. So, when specifying ‘1’ as a signal, GDB will translate this to the target’s `SIGHUP`, whatever that might be.

Numbers may only be used for signals 1 through 15. GDB uses this mapping:

Number	Name
1	SIGHUP
2	SIGINT
3	SIGQUIT
4	SIGILL

```

5      SIGTRAP
6      SIGABRT
7      SIGEMT
8      SIGFPE
9      SIGKILL
10     SIGBUS
11     SIGSEGV
12     SIGSYS
13     SIGPIPE
14     SIGALRM
15     SIGTERM

```

Normally, GDB is set up to let the non-erroneous signals like **SIGALRM** be silently passed to your program (so as not to interfere with their role in the program's functioning) but to stop your program immediately whenever an error signal happens. You can change these settings with the **handle** command.

```

info signals
info handle

```

Print a table of all the kinds of signals and how GDB has been told to handle each one. You can use this to see the signal numbers of all the defined types of signals.

```

info signals sig

```

Similar, but print information only about the specified signal number.

info handle is an alias for **info signals**.

```

catch signal [signal... | 'all']

```

Set a catchpoint for the indicated signals. See Section 5.1.3 [Set Catchpoints], page 71, for details about this command.

```

handle signal [ signal ... ] [keywords...]

```

Change the way GDB handles each *signal*. Each *signal* can be the number of a signal or its name (with or without the 'SIG' at the beginning); a list of signal numbers of the form '*low-high*'; or the word '**all**', meaning all the known signals, except **SIGINT** and **SIGTRAP**, which are used by GDB. Optional argument *keywords*, described below, say what changes to make to all of the specified signals.

The keywords allowed by the **handle** command can be abbreviated. Their full names are:

nostop	GDB should not stop your program when this signal happens. It may still print a message telling you that the signal has come in.
stop	GDB should stop your program when this signal happens. This implies the print keyword as well.
print	GDB should print a message when this signal happens.
noprint	GDB should not mention the occurrence of the signal at all. This implies the nostop keyword as well.

pass

noignore GDB should allow your program to see this signal; your program can handle the signal, or else it may terminate if the signal is fatal and not handled. **pass** and **noignore** are synonyms.

nopass

ignore GDB should not allow your program to see this signal. **nopass** and **ignore** are synonyms.

When a signal stops your program, the signal is not visible to the program until you continue. Your program sees the signal then, if **pass** is in effect for the signal in question *at that time*. In other words, after GDB reports a signal, you can use the **handle** command with **pass** or **nopass** to control whether your program sees that signal when you continue.

The default is set to **nostop**, **noprint**, **pass** for non-erroneous signals such as **SIGALRM**, **SIGWINCH** and **SIGCHLD**, and to **stop**, **print**, **pass** for the erroneous signals.

You can also use the **signal** command to prevent your program from seeing a signal, or cause it to see a signal it normally would not see, or to give it any signal at any time. For example, if your program stopped due to some sort of memory reference error, you might store correct values into the erroneous variables and continue, hoping to see more execution; but your program would probably terminate immediately as a result of the fatal signal once it saw the signal. To prevent this, you can continue with ‘**signal 0**’. See Section 17.3 [Giving your Program a Signal], page 281.

GDB optimizes for stepping the mainline code. If a signal that has **handle nostop** and **handle pass** set arrives while a stepping command (e.g., **stepi**, **step**, **next**) is in progress, GDB lets the signal handler run and then resumes stepping the mainline code once the signal handler returns. In other words, GDB steps over the signal handler. This prevents signals that you’ve specified as not interesting (with **handle nostop**) from changing the focus of debugging unexpectedly. Note that the signal handler itself may still hit a breakpoint, stop for another signal that has **handle stop** in effect, or for any other event that normally results in stopping the stepping command sooner. Also note that GDB still informs you that the program received a signal if **handle print** is set.

If you set **handle pass** for a signal, and your program sets up a handler for it, then issuing a stepping command, such as **step** or **stepi**, when your program is stopped due to the signal will step *into* the signal handler (if the target supports that).

Likewise, if you use the **queue-signal** command to queue a signal to be delivered to the current thread when execution of the thread resumes (see Section 17.3 [Giving your Program a Signal], page 281), then a stepping command will step into the signal handler.

Here’s an example, using **stepi** to step to the first instruction of **SIGUSR1**’s handler:

```
(gdb) handle SIGUSR1
Signal      Stop      Print    Pass to program Description
SIGUSR1     Yes       Yes      Yes      User defined signal 1
(gdb) c
Continuing.

Program received signal SIGUSR1, User defined signal 1.
main () sigusr1.c:28
28      p = 0;
(gdb) si
sigusr1_handler () at sigusr1.c:9
```



```
9      {
```

The same, but using `queue-signal` instead of waiting for the program to receive the signal first:

```
(gdb) n
28      p = 0;
(gdb) queue-signal SIGUSR1
(gdb) si
sigusr1_handler () at sigusr1.c:9
9      {
(gdb)
```

On some targets, GDB can inspect extra signal information associated with the intercepted signal, before it is actually delivered to the program being debugged. This information is exported by the convenience variable `$_siginfo`, and consists of data that is passed by the kernel to the signal handler at the time of the receipt of a signal. The data type of the information itself is target dependent. You can see the data type using the `ptype $_siginfo` command. On Unix systems, it typically corresponds to the standard `siginfo_t` type, as defined in the `signal.h` system header.

Here's an example, on a GNU/Linux system, printing the stray referenced address that raised a segmentation fault.

```
(gdb) continue
Program received signal SIGSEGV, Segmentation fault.
0x0000000000400766 in main ()
69      *(int *)p = 0;
(gdb) ptype $_siginfo
type = struct {
    int si_signo;
    int si_errno;
    int si_code;
    union {
        int _pad[28];
        struct {...} _kill;
        struct {...} _timer;
        struct {...} _rt;
        struct {...} _sigchld;
        struct {...} _sigfault;
        struct {...} _sigpoll;
    } _sifields;
}
(gdb) ptype $_siginfo._sifields._sigfault
type = struct {
    void *si_addr;
}
(gdb) p $_siginfo._sifields._sigfault.si_addr
$1 = (void *) 0x7ffff7ff7000
```

Depending on target support, `$_siginfo` may also be writable.

5.5 Stopping and Starting Multi-thread Programs

GDB supports debugging programs with multiple threads (see Section 4.10 [Debugging Programs with Multiple Threads], page 48). There are two modes of controlling execution of your program within the debugger. In the default mode, referred to as *all-stop mode*, when any thread in your program stops (for example, at a breakpoint or while being stepped), all

other threads in the program are also stopped by GDB. On some targets, GDB also supports *non-stop mode*, in which other threads can continue to run freely while you examine the stopped thread in the debugger.

5.5.1 All-Stop Mode

In all-stop mode, whenever your program stops under GDB for any reason, *all* threads of execution stop, not just the current thread. This allows you to examine the overall state of the program, including switching between threads, without worrying that things may change underfoot.

Conversely, whenever you restart the program, *all* threads start executing. *This is true even when single-stepping* with commands like **step** or **next**.

In particular, GDB cannot single-step all threads in lockstep. Since thread scheduling is up to your debugging target's operating system (not controlled by GDB), other threads may execute more than one statement while the current thread completes a single step. Moreover, in general other threads stop in the middle of a statement, rather than at a clean statement boundary, when the program stops.

You might even find your program stopped in another thread after continuing or even single-stepping. This happens whenever some other thread runs into a breakpoint, a signal, or an exception before the first thread completes whatever you requested.

Whenever GDB stops your program, due to a breakpoint or a signal, it automatically selects the thread where that breakpoint or signal happened. GDB alerts you to the context switch with a message such as '[Switching to Thread n]' to identify the thread.

On some OSes, you can modify GDB's default behavior by locking the OS scheduler to allow only a single thread to run.

set scheduler-locking mode

Set the scheduler locking mode. It applies to normal execution, record mode, and replay mode. *mode* can be one of the following:

- | | |
|---------------|--|
| off | There is no locking and any thread may run at any time. |
| on | Only the current thread may run when the inferior is resumed. New threads created by the resumed thread are held stopped at their entry point, before they execute any instruction. |
| step | Behaves like on when stepping, and off otherwise. Threads other than the current never get a chance to run when you step, and they are completely free to run when you use commands like ' continue ', ' until ', or ' finish '.

This mode optimizes for single-stepping; it prevents other threads from preempting the current thread while you are stepping, so that the focus of debugging does not change unexpectedly. However, unless another thread hits a breakpoint during its timeslice, GDB does not change the current thread away from the thread that you are debugging. |
| replay | Behaves like on in replay mode, and off in either record mode or during normal execution. This is the default mode. |

show scheduler-locking

Display the current scheduler locking mode.

By default, when you issue one of the execution commands such as **continue**, **next** or **step**, GDB allows only threads of the current inferior to run. For example, if GDB is attached to two inferiors, each with two threads, the **continue** command resumes only the two threads of the current inferior. This is useful, for example, when you debug a program that forks and you want to hold the parent stopped (so that, for instance, it doesn't run to exit), while you debug the child. In other situations, you may not be interested in inspecting the current state of any of the processes GDB is attached to, and you may want to resume them all until some breakpoint is hit. In the latter case, you can instruct GDB to allow all threads of all the inferiors to run with the **set schedule-multiple** command.

set schedule-multiple

Set the mode for allowing threads of multiple processes to be resumed when an execution command is issued. When **on**, all threads of all processes are allowed to run. When **off**, only the threads of the current process are resumed. The default is **off**. The **scheduler-locking** mode takes precedence when set to **on**, or while you are stepping and set to **step**.

show schedule-multiple

Display the current mode for resuming the execution of threads of multiple processes.

5.5.2 Non-Stop Mode

By default, when a thread stops to report a debugging event, GDB stops all other threads as well. This is called *all-stop* mode.

For some multi-threaded targets, GDB supports an optional mode of operation in which you can examine stopped program threads in the debugger while other threads continue to execute freely. This minimizes intrusion when debugging live systems, such as programs where some threads have real-time constraints or must continue to respond to external events. This is referred to as *non-stop* mode.

In non-stop mode, when a thread stops to report a debugging event, *only* that thread is stopped; GDB does not stop other threads as well, in contrast to the all-stop mode behavior. Additionally, execution commands such as **continue** and **step** apply by default only to the current thread in non-stop mode, rather than all threads as in all-stop mode. This allows you to control threads explicitly in ways that are not possible in all-stop mode — for example, stepping one thread while allowing others to run freely, stepping one thread while holding all others stopped, or stepping several threads independently and simultaneously.

You can use these commands to manipulate the non-stop mode setting:

set non-stop on

Enable non-stop mode.

set non-stop off

Disable non-stop mode. Also known as enabling all-stop mode. This is the default.

show non-stop

Show the current non-stop enablement setting.

Note the **set non-stop** preference is only consulted when GDB starts or connects to the target program, and it is not possible to switch modes once debugging has started. Furthermore, since not all targets support non-stop mode, even when you have enabled non-stop mode, GDB may still fall back to all-stop operation by default.

In non-stop mode, all execution commands apply only to the current thread by default. That is, **continue** only continues one thread. To continue all threads, issue **continue -a** or **c -a**.

You can use GDB's background execution commands (see Section 5.5.3 [Background Execution], page 98) to run some threads in the background while you continue to examine or step others from GDB. The MI execution commands (see Section 27.15 [GDB/MI Program Execution], page 647) are always executed asynchronously in non-stop mode.

Suspending execution is done with the **interrupt** command when running in the background, or **Ctrl-c** during foreground execution. In all-stop mode, this stops the whole process; but in non-stop mode the interrupt applies only to the current thread. To stop the whole program, use **interrupt -a**.

Other execution commands do not currently support the **-a** option.

In non-stop mode, when a thread stops, GDB doesn't automatically make that thread current, as it does in all-stop mode. This is because the thread stop notifications are asynchronous with respect to GDB's command interpreter, and it would be confusing if GDB unexpectedly changed to a different thread just as you entered a command to operate on the previously current thread.

5.5.3 Background Execution

GDB's execution commands have two variants: the normal foreground (synchronous) behavior, and a background (asynchronous) behavior. In foreground execution, GDB waits for the program to report that some thread has stopped before prompting for another command. In background execution, GDB immediately gives a command prompt so that you can issue other commands while your program runs.

If the target doesn't support async mode, GDB issues an error message if you attempt to use the background execution commands.

To specify background execution, add a **&** to the command. For example, the background form of the **continue** command is **continue&**, or just **c&**. The execution commands that accept background execution are:

run	See Section 4.2 [Starting your Program], page 34.
attach	See Section 4.7 [Debugging an Already-running Process], page 41.
step	See Section 5.2 [Continuing and Stepping], page 85.
stepi	See Section 5.2 [Continuing and Stepping], page 85.
next	See Section 5.2 [Continuing and Stepping], page 85.
nexti	See Section 5.2 [Continuing and Stepping], page 85.
continue	See Section 5.2 [Continuing and Stepping], page 85.
finish	See Section 5.2 [Continuing and Stepping], page 85.

until See Section 5.2 [Continuing and Stepping], page 85.

Background execution is especially useful in conjunction with non-stop mode for debugging programs with multiple threads; see Section 5.5.2 [Non-Stop Mode], page 97. However, you can also use these commands in the normal all-stop mode with the restriction that you cannot issue another execution command until the previous one finishes. Examples of commands that are valid in all-stop mode while the program is running include **help** and **info break**.

You can interrupt your program while it is running in the background by using the **interrupt** command.

interrupt

interrupt -a

Suspend execution of the running program. In all-stop mode, **interrupt** stops the whole process, but in non-stop mode, it stops only the current thread. To stop the whole program in non-stop mode, use **interrupt -a**.

5.5.4 Thread-Specific Breakpoints

When your program has multiple threads (see Section 4.10 [Debugging Programs with Multiple Threads], page 48), you can choose whether to set breakpoints on all threads, or on a particular thread.

break locspec thread thread-id

break locspec thread thread-id if ...

locspec specifies a code location or locations in your program. See Section 9.2 [Location Specifications], page 128, for details.

Use the qualifier ‘**thread thread-id**’ with a breakpoint command to specify that you only want GDB to stop the program when a particular thread reaches this breakpoint. The *thread-id* specifier is one of the thread identifiers assigned by GDB, shown in the first column of the ‘**info threads**’ display.

If you do not specify ‘**thread thread-id**’ when you set a breakpoint, the breakpoint applies to *all* threads of your program.

You can use the **thread** qualifier on conditional breakpoints as well; in this case, place ‘**thread thread-id**’ before or after the breakpoint condition, like this:

```
(gdb) break frik.c:13 thread 28 if bartab > lim
```

Thread-specific breakpoints are automatically deleted when GDB detects the corresponding thread is no longer in the thread list. For example:

```
(gdb) c
```

```
Thread-specific breakpoint 3 deleted - thread 28 no longer in the thread list.
```

There are several ways for a thread to disappear, such as a regular thread exit, but also when you detach from the process with the **detach** command (see Section 4.7 [Debugging an Already-running Process], page 41), or if GDB loses the remote connection (see Chapter 20 [Remote Debugging], page 317), etc. Note that with some targets, GDB is only able to detect a thread has exited when the user explicitly asks for the thread list with the **info threads** command.

A breakpoint can't be both thread-specific and inferior-specific (see Section 4.9.1 [Inferior-Specific Breakpoints], page 47), or task-specific (see Section 15.4.10.7 [Ada Tasks], page 256); using more than one of the `thread`, `inferior`, or `task` keywords when creating a breakpoint will give an error.

5.5.5 Interrupted System Calls

There is an unfortunate side effect when using GDB to debug multi-threaded programs. If one thread stops for a breakpoint, or for some other reason, and another thread is blocked in a system call, then the system call may return prematurely. This is a consequence of the interaction between multiple threads and the signals that GDB uses to implement breakpoints and other events that stop execution.

To handle this problem, your program should check the return value of each system call and react appropriately. This is good programming style anyways.

For example, do not write code like this:

```
sleep (10);
```

The call to `sleep` will return early if a different thread stops at a breakpoint or for some other reason.

Instead, write this:

```
int unslept = 10;
while (unslept > 0)
    unslept = sleep (unslept);
```

A system call is allowed to return early, so the system is still conforming to its specification. But GDB does cause your multi-threaded program to behave differently than it would without GDB.

Also, GDB uses internal breakpoints in the thread library to monitor certain events such as thread creation and thread destruction. When such an event happens, a system call in another thread may return prematurely, even though your program does not appear to stop.

5.5.6 Observer Mode

If you want to build on non-stop mode and observe program behavior without any chance of disruption by GDB, you can set variables to disable all of the debugger's attempts to modify state, whether by writing memory, inserting breakpoints, etc. These operate at a low level, intercepting operations from all commands.

When all of these are set to `off`, then GDB is said to be *observer mode*. As a convenience, the variable `observer` can be set to disable these, plus enable non-stop mode.

Note that GDB will not prevent you from making nonsensical combinations of these settings. For instance, if you have enabled `may-insert-breakpoints` but disabled `may-write-memory`, then breakpoints that work by writing trap instructions into the code stream will still not be able to be placed.

```
set observer on
set observer off
```

When set to `on`, this disables all the permission variables below (except for `insert-fast-tracepoints`), plus enables non-stop debugging. Setting this to `off` switches back to normal debugging, though remaining in non-stop mode.

`show observer`

Show whether observer mode is on or off.

`set may-write-registers on`

`set may-write-registers off`

This controls whether GDB will attempt to alter the values of registers, such as with assignment expressions in `print`, or the `jump` command. It defaults to `on`.

`show may-write-registers`

Show the current permission to write registers.

`set may-write-memory on`

`set may-write-memory off`

This controls whether GDB will attempt to alter the contents of memory, such as with assignment expressions in `print`. It defaults to `on`.

`show may-write-memory`

Show the current permission to write memory.

`set may-insert-breakpoints on`

`set may-insert-breakpoints off`

This controls whether GDB will attempt to insert breakpoints. This affects all breakpoints, including internal breakpoints defined by GDB. It defaults to `on`.

`show may-insert-breakpoints`

Show the current permission to insert breakpoints.

`set may-insert-tracepoints on`

`set may-insert-tracepoints off`

This controls whether GDB will attempt to insert (regular) tracepoints at the beginning of a tracing experiment. It affects only non-fast tracepoints, fast tracepoints being under the control of `may-insert-fast-tracepoints`. It defaults to `on`.

`show may-insert-tracepoints`

Show the current permission to insert tracepoints.

`set may-insert-fast-tracepoints on`

`set may-insert-fast-tracepoints off`

This controls whether GDB will attempt to insert fast tracepoints at the beginning of a tracing experiment. It affects only fast tracepoints, regular (non-fast) tracepoints being under the control of `may-insert-tracepoints`. It defaults to `on`.

`show may-insert-fast-tracepoints`

Show the current permission to insert fast tracepoints.

`set may-interrupt on`

`set may-interrupt off`

This controls whether GDB will attempt to interrupt or stop program execution. When this variable is `off`, the `interrupt` command will have no effect, nor will `Ctrl-c`. It defaults to `on`.

`show may-interrupt`

Show the current permission to interrupt or stop the program.

6 Running programs backward

When you are debugging a program, it is not unusual to realize that you have gone too far, and some event of interest has already happened. If the target environment supports it, GDB can allow you to “rewind” the program by running it backward.

A target environment that supports reverse execution should be able to “undo” the changes in machine state that have taken place as the program was executing normally. Variables, registers etc. should revert to their previous values. Obviously this requires a great deal of sophistication on the part of the target environment; not all target environments can support reverse execution.

When a program is executed in reverse, the instructions that have most recently been executed are “un-executed”, in reverse order. The program counter runs backward, following the previous thread of execution in reverse. As each instruction is “un-executed”, the values of memory and/or registers that were changed by that instruction are reverted to their previous states. After executing a piece of source code in reverse, all side effects of that code should be “undone”, and all variables should be returned to their prior values¹.

On some platforms, GDB has built-in support for reverse execution, activated with the `record` or `record btrace` commands. See Chapter 7 [Process Record and Replay], page 105. Some remote targets, typically full system emulators, support reverse execution directly without requiring any special command.

If you are debugging in a target environment that supports reverse execution, GDB provides the following commands.

`reverse-continue` [*ignore-count*]
`rc` [*ignore-count*]

Beginning at the point where your program last stopped, start executing in reverse. Reverse execution will stop for breakpoints and synchronous exceptions (signals), just like normal execution. Behavior of asynchronous signals depends on the target environment.

`reverse-step` [*count*]

Run the program backward until control reaches the start of a different source line; then stop it, and return control to GDB.

Like the `step` command, `reverse-step` will only stop at the beginning of a source line. It “un-executes” the previously executed source line. If the previous source line included calls to debuggable functions, `reverse-step` will step (backward) into the called function, stopping at the beginning of the *last* statement in the called function (typically a return statement).

Also, as with the `step` command, if non-debuggable functions are called, `reverse-step` will run through them backward without stopping.

¹ Note that some side effects are easier to undo than others. For instance, memory and registers are relatively easy, but device I/O is hard. Some targets may be able undo things like device I/O, and some may not.

The contract between GDB and the reverse executing target requires only that the target do something reasonable when GDB tells it to execute backwards, and then report the results back to GDB. Whatever the target reports back to GDB, GDB will report back to the user. GDB assumes that the memory and registers that the target reports are in a consistent state, but GDB accepts whatever it is given.

reverse-stepi [*count*]

Reverse-execute one machine instruction. Note that the instruction to be reverse-executed is *not* the one pointed to by the program counter, but the instruction executed prior to that one. For instance, if the last instruction was a jump, **reverse-stepi** will take you back from the destination of the jump to the jump instruction itself.

reverse-next [*count*]

Run backward to the beginning of the previous line executed in the current (innermost) stack frame. If the line contains function calls, they will be “un-executed” without stopping. Starting from the first line of a function, **reverse-next** will take you back to the caller of that function, *before* the function was called, just as the normal **next** command would take you from the last line of a function back to its return to its caller².

reverse-nexti [*count*]

Like **nexti**, **reverse-nexti** executes a single instruction in reverse, except that called functions are “un-executed” atomically. That is, if the previously executed instruction was a return from another function, **reverse-nexti** will continue to execute in reverse until the call to that function (from the current stack frame) is reached.

reverse-finish

Just as the **finish** command takes you to the point where the current function returns, **reverse-finish** takes you to the point where it was called. Instead of ending up at the end of the current function invocation, you end up at the beginning.

set exec-direction

Set the direction of target execution.

set exec-direction reverse

GDB will perform all execution commands in reverse, until the **exec-direction** mode is changed to “forward”. Affected commands include **step**, **stepi**, **next**, **nexti**, **continue**, and **finish**. The **return** command cannot be used in reverse mode.

set exec-direction forward

GDB will perform all execution commands in the normal fashion. This is the default.

² Unless the code is too heavily optimized.

7 Recording Inferior's Execution and Replaying It

On some platforms, GDB provides a special *process record and replay* target that can record a log of the process execution, and replay it later with both forward and reverse execution commands.

When this target is in use, if the execution log includes the record for the next instruction, GDB will debug in *replay mode*. In the replay mode, the inferior does not really execute code instructions. Instead, all the events that normally happen during code execution are taken from the execution log. While code is not really executed in replay mode, the values of registers (including the program counter register) and the memory of the inferior are still changed as they normally would. Their contents are taken from the execution log.

If the record for the next instruction is not in the execution log, GDB will debug in *record mode*. In this mode, the inferior executes normally, and GDB records the execution log for future replay.

The process record and replay target supports reverse execution (see Chapter 6 [Reverse Execution], page 103), even if the platform on which the inferior runs does not. However, the reverse execution is limited in this case by the range of the instructions recorded in the execution log. In other words, reverse execution on platforms that don't support it directly can only be done in the replay mode.

When debugging in the reverse direction, GDB will work in replay mode as long as the execution log includes the record for the previous instruction; otherwise, it will work in record mode, if the platform supports reverse execution, or stop if not.

Currently, process record and replay is supported on ARM, Aarch64, LoongArch, Moxie, PowerPC, PowerPC64, S/390, RISC-V and x86 (i386/amd64) running GNU/Linux. Process record and replay can be used both when native debugging, and when remote debugging via `gdbserver`.

When recording an inferior, GDB may print auxiliary information during stepping commands and commands displaying the execution history.

For architecture environments that support process record and replay, GDB provides the following commands:

record method

This command starts the process record and replay target. The recording method can be specified as parameter. Without a parameter the command uses the **full** recording method. The following recording methods are available:

full Full record/replay recording using GDB's software record and replay implementation. This method allows replaying and reverse execution.

btrace format

Hardware-supported instruction recording, supported on Intel processors. This method does not record data. Further, the data is collected in a ring buffer so old data will be overwritten when the buffer is full. It allows limited reverse execution. Variables and

registers are not available during reverse execution. In remote debugging, recording continues on disconnect. Recorded data can be inspected after reconnecting. The recording may be stopped using **record stop**.

The recording format can be specified as parameter. Without a parameter the command chooses the recording format. The following recording formats are available:

- | | |
|------------|---|
| bts | Use the <i>Branch Trace Store</i> (BTS) recording format. In this format, the processor stores a from/to record for each executed branch in the btrace ring buffer. |
| pt | Use the <i>Intel Processor Trace</i> recording format. In this format, the processor stores the execution trace in a compressed form that is afterwards decoded by GDB.

The trace can be recorded with very low overhead. The compressed trace format also allows small trace buffers to already contain a big number of instructions compared to BTS.

Decoding the recorded execution trace, on the other hand, is more expensive than decoding BTS trace. This is mostly due to the increased number of instructions to process. You should increase the buffer-size with care. |

Not all recording formats may be available on all processors.

The process record and replay target can only debug a process that is already running. Therefore, you need first to start the process with the **run** or **start** commands, and then start the recording with the **record method** command.

Displaced stepping (see Appendix D [displaced stepping], page 777) will be automatically disabled when process record and replay target is started. That's because the process record and replay target doesn't support displaced stepping.

If the inferior is in the non-stop mode (see Section 5.5.2 [Non-Stop Mode], page 97) or in the asynchronous execution mode (see Section 5.5.3 [Background Execution], page 98), not all recording methods are available. The **full** recording method does not support these two modes.

record stop

Stop the process record and replay target. When process record and replay target stops, the entire execution log will be deleted and the inferior will either be terminated, or will remain in its final state.

When you stop the process record and replay target in record mode (at the end of the execution log), the inferior will be stopped at the next instruction that would have been recorded. In other words, if you record for a while and then stop recording, the inferior process will be left in the same state as if the recording never happened.

On the other hand, if the process record and replay target is stopped while in replay mode (that is, not at the end of the execution log, but at some earlier

point), the inferior process will become “live” at that earlier state, and it will then be possible to continue the usual “live” debugging of the process from that state.

When the inferior process exits, or GDB detaches from it, process record and replay target will automatically stop itself.

record goto

Go to a specific location in the execution log. There are several ways to specify the location to go to:

record goto begin

record goto start

Go to the beginning of the execution log.

record goto end

Go to the end of the execution log.

record goto *n*

Go to instruction number *n* in the execution log.

record save *filename*

Save the execution log to a file *filename*. Default filename is `gdb_record.process_id`, where *process.id* is the process ID of the inferior.

This command may not be available for all recording methods.

record restore *filename*

Restore the execution log from a file *filename*. File must have been created with **record save**.

set record full insn-number-max *limit*

set record full insn-number-max unlimited

Set the limit of instructions to be recorded for the **full** recording method. Default value is 200000.

If *limit* is a positive number, then GDB will start deleting instructions from the log once the number of the record instructions becomes greater than *limit*. For every new recorded instruction, GDB will delete the earliest recorded instruction to keep the number of recorded instructions at the limit. (Since deleting recorded instructions loses information, GDB lets you control what happens when the limit is reached, by means of the **stop-at-limit** option, described below.)

If *limit* is **unlimited** or zero, GDB will never delete recorded instructions from the execution log. The number of recorded instructions is limited only by the available memory.

show record full insn-number-max

Show the limit of instructions to be recorded with the **full** recording method.

set record full stop-at-limit

Control the behavior of the **full** recording method when the number of recorded instructions reaches the limit. If **ON** (the default), GDB will stop when the limit is reached for the first time and ask you whether you want to stop the inferior or

continue running it and recording the execution log. If you decide to continue recording, each new recorded instruction will cause the oldest one to be deleted. If this option is OFF, GDB will automatically delete the oldest record to make room for each new one, without asking.

show record full stop-at-limit

Show the current setting of **stop-at-limit**.

set record full memory-query

Control the behavior when GDB is unable to record memory changes caused by an instruction for the **full** recording method. If ON, GDB will query whether to stop the inferior in that case.

If this option is OFF (the default), GDB will automatically ignore the effect of such instructions on memory. Later, when GDB replays this execution log, it will mark the log of this instruction as not accessible, and it will not affect the replay results.

show record full memory-query

Show the current setting of **memory-query**.

The **btrace** record target does not trace data. As a convenience, when replaying, GDB reads read-only memory off the live program directly, assuming that the addresses of the read-only areas don't change. This for example makes it possible to disassemble code while replaying, but not to print variables. In some cases, being able to inspect variables might be useful. You can use the following command for that:

set record btrace replay-memory-access

Control the behavior of the **btrace** recording method when accessing memory during replay. If **read-only** (the default), GDB will only allow accesses to read-only memory. If **read-write**, GDB will allow accesses to read-only and to read-write memory. Beware that the accessed memory corresponds to the live target and not necessarily to the current replay position.

set record btrace cpu identifier

Set the processor to be used for enabling workarounds for processor errata when decoding the trace.

Processor errata are defects in processor operation, caused by its design or manufacture. They can cause a trace not to match the specification. This, in turn, may cause trace decode to fail. GDB can detect erroneous trace packets and correct them, thus avoiding the decoding failures. These corrections are known as *errata workarounds*, and are enabled based on the processor on which the trace was recorded.

By default, GDB attempts to detect the processor automatically, and apply the necessary workarounds for it. However, you may need to specify the processor if GDB does not yet support it. This command allows you to do that, and also allows to disable the workarounds.

The argument *identifier* identifies the CPU and is of the form: **vendor:processor identifier**. In addition, there are two special identifiers, **none** and **auto** (default).

The following vendor identifiers and corresponding processor identifiers are currently supported:

```
intel    family/model[/stepping]
```

On GNU/Linux systems, the processor *family*, *model*, and *stepping* can be obtained from `/proc/cpuinfo`.

If *identifier* is **auto**, enable errata workarounds for the processor on which the trace was recorded. If *identifier* is **none**, errata workarounds are disabled.

For example, when using an old GDB on a new system, decode may fail because GDB does not support the new processor. It often suffices to specify an older processor that GDB supports.

```
(gdb) info record
Active record target: record-btrace
Recording format: Intel Processor Trace.
Buffer size: 16kB.
Failed to configure the Intel Processor Trace decoder: unknown cpu.
(gdb) set record btrace cpu intel:6/158
(gdb) info record
Active record target: record-btrace
Recording format: Intel Processor Trace.
Buffer size: 16kB.
Recorded 84872 instructions in 3189 functions (0 gaps) for thread 1 (...).
```

```
show record btrace replay-memory-access
```

Show the current setting of **replay-memory-access**.

```
show record btrace cpu
```

Show the processor to be used for enabling trace decode errata workarounds.

```
set record btrace bts buffer-size size
```

```
set record btrace bts buffer-size unlimited
```

Set the requested ring buffer size for branch tracing in BTS format. Default is 64KB.

If *size* is a positive number, then GDB will try to allocate a buffer of at least *size* bytes for each new thread that uses the btrace recording method and the BTS format. The actually obtained buffer size may differ from the requested *size*. Use the **info record** command to see the actual buffer size for each thread that uses the btrace recording method and the BTS format.

If *limit* is **unlimited** or zero, GDB will try to allocate a buffer of 4MB.

Bigger buffers mean longer traces. On the other hand, GDB will also need longer to process the branch trace data before it can be used.

```
show record btrace bts buffer-size size
```

Show the current setting of the requested ring buffer size for branch tracing in BTS format.

```
set record btrace pt buffer-size size
```

```
set record btrace pt buffer-size unlimited
```

Set the requested ring buffer size for branch tracing in Intel Processor Trace format. Default is 16KB.

If *size* is a positive number, then GDB will try to allocate a buffer of at least *size* bytes for each new thread that uses the `btrace` recording method and the Intel Processor Trace format. The actually obtained buffer size may differ from the requested *size*. Use the `info record` command to see the actual buffer size for each thread.

If *limit* is `unlimited` or zero, GDB will try to allocate a buffer of 4MB.

Bigger buffers mean longer traces. On the other hand, GDB will also need longer to process the branch trace data before it can be used.

show record btrace pt buffer-size size

Show the current setting of the requested ring buffer size for branch tracing in Intel Processor Trace format.

set record btrace pt event-tracing

Enable or disable event tracing for branch tracing in Intel Processor Trace format. When enabled, events are recorded during execution as auxiliary information and will be printed during stepping commands and commands displaying the execution history. Changing this setting has no effect on an active recording. The default is off.

show record btrace pt event-tracing

Show the current setting of Intel Processor Trace event tracing.

info record

Show various statistics about the recording depending on the recording method:

full For the `full` recording method, it shows the state of process record and its in-memory execution log buffer, including:

- Whether in record mode or replay mode.
- Lowest recorded instruction number (counting from when the current execution log started recording instructions).
- Highest recorded instruction number.
- Current instruction about to be replayed (if in replay mode).
- Number of instructions contained in the execution log.
- Maximum number of instructions that may be contained in the execution log.

btrace For the `btrace` recording method, it shows:

- Recording format.
- Number of instructions that have been recorded.
- Number of blocks of sequential control-flow formed by the recorded instructions.
- Whether in record mode or replay mode.

For the `bts` recording format, it also shows:

- Size of the perf ring buffer.

For the `pt` recording format, it also shows:

- Size of the perf ring buffer.

record delete

When record target runs in replay mode (“in the past”), delete the subsequent execution log and begin to record a new execution log starting from the current address. This means you will abandon the previously recorded “future” and begin recording a new “future”.

record instruction-history

Disassembles instructions from the recorded execution log. By default, ten instructions are disassembled. This can be changed using the **set record instruction-history-size** command. Instructions are printed in execution order.

It can also print mixed source+disassembly if you specify the **/m** or **/s** modifier, and print the raw instructions in hex as well as in symbolic form by specifying the **/r** or **/b** modifier. The behavior of the **/m**, **/s**, **/r**, and **/b** modifiers are the same as for the *disassemble* command (see [*disassemble*], page 138).

The current position marker is printed for the instruction at the current program counter value. This instruction can appear multiple times in the trace and the current position marker will be printed every time. To omit the current position marker, specify the **/p** modifier.

To better align the printed instructions when the trace contains instructions from more than one function, the function name may be omitted by specifying the **/f** modifier.

Printing auxiliary information is enabled by default and can be omitted with the **/a** modifier.

Speculatively executed instructions are prefixed with ‘?’. This feature is not available for all recording formats.

There are several ways to specify what part of the execution log to disassemble:

record instruction-history *insn*

Disassembles ten instructions starting from instruction number *insn*.

record instruction-history *insn*, +/-*n*

Disassembles *n* instructions around instruction number *insn*. If *n* is preceded with **+**, disassembles *n* instructions after instruction number *insn*. If *n* is preceded with **-**, disassembles *n* instructions before instruction number *insn*.

record instruction-history

Disassembles ten more instructions after the last disassembly.

record instruction-history -

Disassembles ten more instructions before the last disassembly.

record instruction-history *begin*, *end*

Disassembles instructions beginning with instruction number *begin* until instruction number *end*. The instruction number *end* is included.

This command may not be available for all recording methods.

set record instruction-history-size *size*

set record instruction-history-size unlimited

Define how many instructions to disassemble in the **record instruction-history** command. The default value is 10. A *size* of **unlimited** means unlimited instructions.

show record instruction-history-size

Show how many instructions to disassemble in the **record instruction-history** command.

record function-call-history

Prints the execution history at function granularity. For each sequence of instructions that belong to the same function, it prints the name of that function, the source lines for this instruction sequence (if the **/l** modifier is specified), and the instructions numbers that form the sequence (if the **/i** modifier is specified). The function names are indented to reflect the call stack depth if the **/c** modifier is specified. Printing auxiliary information is enabled by default and can be omitted with the **/a** modifier. The **/l**, **/i**, **/a**, and **/c** modifiers can be given together.

```
(gdb) list 1, 10
1  void foo (void)
2  {
3  }
4
5  void bar (void)
6  {
7  ...
8  foo ();
9  ...
10 }
(gdb) record function-call-history /ilc
1 bar    inst 1,4    at foo.c:6,8
2 foo    inst 5,10   at foo.c:2,3
3 bar    inst 11,13  at foo.c:9,10
```

By default, ten functions are printed. This can be changed using the **set record function-call-history-size** command. Functions are printed in execution order. There are several ways to specify what to print:

record function-call-history *func*

Prints ten functions starting from function number *func*.

record function-call-history *func*, +/-*n*

Prints *n* functions around function number *func*. If *n* is preceded with **+**, prints *n* functions after function number *func*. If *n* is preceded with **-**, prints *n* functions before function number *func*.

record function-call-history

Prints ten more functions after the last ten-function print.

record function-call-history -

Prints ten more functions before the last ten-function print.

record function-call-history *begin, end*

Prints functions beginning with function number *begin* until function number *end*. The function number *end* is included.

This command may not be available for all recording methods.

set record function-call-history-size *size*

set record function-call-history-size **unlimited**

Define how many functions to print in the **record function-call-history** command. The default value is 10. A size of **unlimited** means unlimited functions.

show record function-call-history-size

Show how many functions to print in the **record function-call-history** command.

8 Examining the Stack

When your program has stopped, the first thing you need to know is where it stopped and how it got there.

Each time your program performs a function call, information about the call is generated. That information includes the location of the call in your program, the arguments of the call, and the local variables of the function being called. The information is saved in a block of data called a *stack frame*. The stack frames are allocated in a region of memory called the *call stack*.

When your program stops, the GDB commands for examining the stack allow you to see all of this information.

One of the stack frames is *selected* by GDB and many GDB commands refer implicitly to the selected frame. In particular, whenever you ask GDB for the value of a variable in your program, the value is found in the selected frame. There are special GDB commands to select whichever frame you are interested in. See Section 8.3 [Selecting a Frame], page 119.

When your program stops, GDB automatically selects the currently executing frame and describes it briefly, similar to the **frame** command (see Section 8.4 [Information about a Frame], page 121).

8.1 Stack Frames

The call stack is divided up into contiguous pieces called *stack frames*, or *frames* for short; each frame is the data associated with one call to one function. The frame contains the arguments given to the function, the function's local variables, and the address at which the function is executing.

When your program is started, the stack has only one frame, that of the function **main**. This is called the *initial* frame or the *outermost* frame. Each time a function is called, a new frame is made. Each time a function returns, the frame for that function invocation is eliminated. If a function is recursive, there can be many frames for the same function. The frame for the function in which execution is actually occurring is called the *innermost* frame. This is the most recently created of all the stack frames that still exist.

Inside your program, stack frames are identified by their addresses. A stack frame consists of many bytes, each of which has its own address; each kind of computer has a convention for choosing one byte whose address serves as the address of the frame. Usually this address is kept in a register called the *frame pointer register* (see Section 10.14 [Registers], page 181) while execution is going on in that frame.

GDB labels each existing stack frame with a *level*, a number that is zero for the innermost frame, one for the frame that called it, and so on upward. These level numbers give you a way of designating stack frames in GDB commands. The terms *frame number* and *frame level* can be used interchangeably to describe this number.

Some compilers provide a way to compile functions so that they operate without stack frames. (For example, the GCC option

```
'-fomit-frame-pointer'
```

generates functions without a frame.) This is occasionally done with heavily used library functions to save the frame setup time. GDB has limited facilities for dealing with

these function invocations. If the innermost function invocation has no stack frame, GDB nevertheless regards it as though it had a separate frame, which is numbered zero as usual, allowing correct tracing of the function call chain. However, GDB has no provision for frameless functions elsewhere in the stack.

8.2 Backtraces

A backtrace is a summary of how your program got where it is. It shows one line per frame, for many frames, starting with the currently executing frame (frame zero), followed by its caller (frame one), and on up the stack.

To print a backtrace of the entire stack, use the **backtrace** command, or its alias **bt**. This command will print one line per frame for frames in the stack. By default, all stack frames are printed. You can stop the backtrace at any time by typing the system interrupt character, normally **Ctrl-c**.

backtrace [*option*]... [*qualifier*]... [*count*]

bt [*option*]... [*qualifier*]... [*count*]

Print the backtrace of the entire stack.

The optional *count* can be one of the following:

n

n Print only the innermost *n* frames, where *n* is a positive number.

-n

-n Print only the outermost *n* frames, where *n* is a positive number.

Options:

-full Print the values of the local variables also. This can be combined with the optional *count* to limit the number of frames shown.

-no-filters

Do not run Python frame filters on this backtrace. See Section 23.3.2.10 [Frame Filter API], page 436, for more information. Additionally use [disable frame-filter all], page 124, to turn off all frame filters. This is only relevant when GDB has been configured with Python support.

-hide

A Python frame filter might decide to “elide” some frames. Normally such elided frames are still printed, but they are indented relative to the filtered frames that cause them to be elided. The **-hide** option causes elided frames to not be printed at all.

The **backtrace** command also supports a number of options that allow overriding relevant global print settings as set by **set backtrace** and **set print** subcommands:

-past-main [on|off]

Set whether backtraces should continue past **main**. Related setting: [set backtrace past-main], page 118.

-past-entry [on|off]

Set whether backtraces should continue past the entry point of a program. Related setting: [set backtrace past-entry], page 118.

-entry-values *no|only|preferred|if-needed|both|compact|default*
 Set printing of function arguments at function entry. Related setting: [set print entry-values], page 163.

-frame-arguments *all|scalars|none*
 Set printing of non-scalar frame arguments. Related setting: [set print frame-arguments], page 162.

-raw-frame-arguments *[on|off]*
 Set whether to print frame arguments in raw form. Related setting: [set print raw-frame-arguments], page 163.

-frame-info *auto|source-line|location|source-and-location|location-and-address|short-location*
 Set printing of frame information. Related setting: [set print frame-info], page 165.

The optional *qualifier* is maintained for backward compatibility. It can be one of the following:

full Equivalent to the **-full** option.

no-filters Equivalent to the **-no-filters** option.

hide Equivalent to the **-hide** option.

The names **where** and **info stack** (abbreviated **info s**) are additional aliases for **backtrace**.

In a multi-threaded program, GDB by default shows the backtrace only for the current thread. To display the backtrace for several or all of the threads, use the command **thread apply** (see Section 4.10 [Threads], page 48). For example, if you type **thread apply all backtrace**, GDB will display the backtrace for all the threads; this is handy when you debug a core dump of a multi-threaded program.

Each line in the backtrace shows the frame number and the function name. The program counter value is also shown—unless you use **set print address off**. The backtrace also shows the source file name and line number, as well as the arguments to the function. The program counter value is omitted if it is at the beginning of the code for that line number.

Here is an example of a backtrace. It was made with the command ‘**bt 3**’, so it shows the innermost three frames.

```
#0  m4_traceon (obs=0x24eb0, argc=1, argv=0x2b8c8)
    at builtin.c:993
#1  0x6e38 in expand_macro (sym=0x2b600, data=...) at macro.c:242
#2  0x6840 in expand_token (obs=0x0, t=177664, td=0xf7ffb08)
    at macro.c:71
(More stack frames follow...)
```

The display for frame zero does not begin with a program counter value, indicating that your program has stopped at the beginning of the code for line 993 of **builtin.c**.

The value of parameter **data** in frame 1 has been replaced by **...** By default, GDB prints the value of a parameter only if it is a scalar (integer, pointer, enumeration, etc). See command **set print frame-arguments** in Section 10.9 [Print Settings], page 159, for more

details on how to configure the way function parameter values are printed. The command `set print frame-info` (see Section 10.9 [Print Settings], page 159) controls what frame information is printed.

If your program was compiled with optimizations, some compilers will optimize away arguments passed to functions if those arguments are never used after the call. Such optimizations generate code that passes arguments through registers, but doesn't store those arguments in the stack frame. GDB has no way of displaying such arguments in stack frames other than the innermost one. Here's what such a backtrace might look like:

```
#0  m4_traceon (obs=0x24eb0, argc=1, argv=0x2b8c8)
    at builtin.c:993
#1  0x6e38 in expand_macro (sym=<optimized out>) at macro.c:242
#2  0x6840 in expand_token (obs=0x0, t=<optimized out>, td=0xf7fffb08)
    at macro.c:71
(More stack frames follow...)
```

The values of arguments that were not saved in their stack frames are shown as '`<optimized out>`'.

If you need to display the values of such optimized-out arguments, either deduce that from other variables whose values depend on the one you are interested in, or recompile without optimizations.

Most programs have a standard user entry point—a place where system libraries and startup code transition into user code. For C this is `main`¹. When GDB finds the entry function in a backtrace it will terminate the backtrace, to avoid tracing into highly system-specific (and generally uninteresting) code.

If you need to examine the startup code, or limit the number of levels in a backtrace, you can change this behavior:

```
set backtrace past-main
```

```
set backtrace past-main on
```

Backtraces will continue past the user entry point.

```
set backtrace past-main off
```

Backtraces will stop when they encounter the user entry point. This is the default.

```
show backtrace past-main
```

Display the current user entry point backtrace policy.

```
set backtrace past-entry
```

```
set backtrace past-entry on
```

Backtraces will continue past the internal entry point of an application. This entry point is encoded by the linker when the application is built, and is likely before the user entry point `main` (or equivalent) is called.

```
set backtrace past-entry off
```

Backtraces will stop when they encounter the internal entry point of an application. This is the default.

¹ Note that embedded programs (the so-called “free-standing” environment) are not required to have a `main` function as the entry point. They could even have multiple entry points.

show backtrace past-entry

Display the current internal entry point backtrace policy.

set backtrace limit *n*

set backtrace limit 0

set backtrace limit unlimited

Limit the backtrace to *n* levels. A value of **unlimited** or zero means unlimited levels.

show backtrace limit

Display the current limit on backtrace levels.

You can control how file names are displayed.

set filename-display

set filename-display relative

Display file names relative to the compilation directory. This is the default.

set filename-display basename

Display only basename of a filename.

set filename-display absolute

Display an absolute filename.

show filename-display

Show the current way to display filenames.

8.3 Selecting a Frame

Most commands for examining the stack and other data in your program work on whichever stack frame is selected at the moment. Here are the commands for selecting a stack frame; all of them finish by printing a brief description of the stack frame just selected.

frame [*frame-selection-spec*]

f [*frame-selection-spec*]

The **frame** command allows different stack frames to be selected. The *frame-selection-spec* can be any of the following:

num

level *num* Select frame level *num*. Recall that frame zero is the innermost (currently executing) frame, frame one is the frame that called the innermost one, and so on. The highest level frame is usually the one for **main**.

As this is the most common method of navigating the frame stack, the string **level** can be omitted. For example, the following two commands are equivalent:

(gdb) **frame 3**

(gdb) **frame level 3**

address *stack-address*

Select the frame with stack address *stack-address*. The *stack-address* for a frame can be seen in the output of **info frame**, for example:

(gdb) **info frame**

```
Stack level 1, frame at 0x7fffffffda30:
  rip = 0x40066d in b (amd64-entry-value.cc:59); saved rip 0x4004c5
  tail call frame, caller of frame at 0x7fffffffda30
  source language c++.
  Arglist at unknown address.
  Locals at unknown address, Previous frame's sp is 0x7fffffffda30
```

The *stack-address* for this frame is 0x7fffffffda30 as indicated by the line:

```
Stack level 1, frame at 0x7fffffffda30:
```

function *function-name*

Select the stack frame for function *function-name*. If there are multiple stack frames for function *function-name* then the inner most stack frame is selected.

view *stack-address* [*pc-addr*]

View a frame that is not part of GDB's backtrace. The frame viewed has stack address *stack-addr*, and optionally, a program counter address of *pc-addr*.

This is useful mainly if the chaining of stack frames has been damaged by a bug, making it impossible for GDB to assign numbers properly to all frames. In addition, this can be useful when your program has multiple stacks and switches between them.

When viewing a frame outside the current backtrace using **frame view** then you can always return to the original stack using one of the previous stack frame selection instructions, for example **frame level 0**.

- up** *n* Move *n* frames up the stack; *n* defaults to 1. For positive numbers *n*, this advances toward the outermost frame, to higher frame numbers, to frames that have existed longer.
- down** *n* Move *n* frames down the stack; *n* defaults to 1. For positive numbers *n*, this advances toward the innermost frame, to lower frame numbers, to frames that were created more recently. You may abbreviate **down** as **do**.

All of these commands end by printing two lines of output describing the frame. The first line shows the frame number, the function name, the arguments, and the source file and line number of execution in that frame. The second line shows the text of that source line.

For example:

```
(gdb) up
#1 0x22f0 in main (argc=1, argv=0xf7ffbf4, env=0xf7ffbf4)
    at env.c:10
10      read_input_file (argv[i]);
```

After such a printout, the **list** command with no arguments prints ten lines centered on the point of execution in the frame. You can also edit the program at the point of execution with your favorite editing program by typing **edit**. See Section 9.1 [Printing Source Lines], page 127, for details.

select-frame [*frame-selection-spec*]

The **select-frame** command is a variant of **frame** that does not display the new frame after selecting it. This command is intended primarily for use in GDB command scripts, where the output might be unnecessary and distracting. The *frame-selection-spec* is as for the **frame** command described in Section 8.3 [Selecting a Frame], page 119.

up-silently *n*

down-silently *n*

These two commands are variants of **up** and **down**, respectively; they differ in that they do their work silently, without causing display of the new frame. They are intended primarily for use in GDB command scripts, where the output might be unnecessary and distracting.

8.4 Information About a Frame

There are several other commands to print information about the selected stack frame.

frame

f When used without any argument, this command does not change which frame is selected, but prints a brief description of the currently selected stack frame. It can be abbreviated **f**. With an argument, this command is used to select a stack frame. See Section 8.3 [Selecting a Frame], page 119.

info frame

info f This command prints a verbose description of the selected stack frame, including:

- the address of the frame
- the address of the next frame down (called by this frame)
- the address of the next frame up (caller of this frame)
- the language in which the source code corresponding to this frame is written
- the address of the frame's arguments
- the address of the frame's local variables
- the program counter saved in it (the address of execution in the caller frame)
- which registers were saved in the frame

The verbose description is useful when something has gone wrong that has made the stack format fail to fit the usual conventions.

info frame [*frame-selection-spec*]

info f [*frame-selection-spec*]

Print a verbose description of the frame selected by *frame-selection-spec*. The *frame-selection-spec* is the same as for the **frame** command (see Section 8.3 [Selecting a Frame], page 119). The selected frame remains unchanged by this command.

info args [-q]

Print the arguments of the selected frame, each on a separate line.

The optional flag ‘-q’, which stands for ‘quiet’, disables printing header information and messages explaining why no argument have been printed.

info args [-q] [-t *type_regex*] [*regex*]

Like *info args*, but only print the arguments selected with the provided *regex*(s).

If *regex* is provided, print only the arguments whose names match the regular expression *regex*.

If *type_regex* is provided, print only the arguments whose types, as printed by the *what is* command, match the regular expression *type_regex*. If *type_regex* contains space(s), it should be enclosed in quote characters. If needed, use backslash to escape the meaning of special characters or quotes.

If both *regex* and *type_regex* are provided, an argument is printed only if its name matches *regex* and its type matches *type_regex*.

info locals [-q]

Print the local variables of the selected frame, each on a separate line. These are all variables (declared either static or automatic) accessible at the point of execution of the selected frame.

The optional flag ‘-q’, which stands for ‘quiet’, disables printing header information and messages explaining why no local variables have been printed.

info locals [-q] [-t *type_regex*] [*regex*]

Like *info locals*, but only print the local variables selected with the provided *regex*(s).

If *regex* is provided, print only the local variables whose names match the regular expression *regex*.

If *type_regex* is provided, print only the local variables whose types, as printed by the *what is* command, match the regular expression *type_regex*. If *type_regex* contains space(s), it should be enclosed in quote characters. If needed, use backslash to escape the meaning of special characters or quotes.

If both *regex* and *type_regex* are provided, a local variable is printed only if its name matches *regex* and its type matches *type_regex*.

The command *info locals -q -t type_regex* can usefully be combined with the commands *frame apply* and *thread apply*. For example, your program might use Resource Acquisition Is Initialization types (RAII) such as *lock_something_t*: each local variable of type *lock_something_t* automatically places a lock that is destroyed when the variable goes out of scope. You can then list all acquired locks in your program by doing

```
thread apply all -s frame apply all -s info locals -q -t lock_something_t
```

or the equivalent shorter form

```
tfaas i lo -q -t lock_something_t
```

8.5 Applying a Command to Several Frames.

frame apply [all | count | -count | level level...] [*option*]... *command*

The *frame apply* command allows you to apply the named *command* to one or more frames.

all	Specify all to apply <i>command</i> to all frames.
count	Use <i>count</i> to apply <i>command</i> to the innermost <i>count</i> frames, where <i>count</i> is a positive number.
-count	Use <i>-count</i> to apply <i>command</i> to the outermost <i>count</i> frames, where <i>count</i> is a positive number.
level	Use level to apply <i>command</i> to the set of frames identified by the <i>level</i> list. <i>level</i> is a frame level or a range of frame levels as <i>level1-level2</i> . The frame level is the number shown in the first field of the ‘ backtrace ’ command output. E.g., ‘2-4 6-8 3’ indicates to apply <i>command</i> for the frames at levels 2, 3, 4, 6, 7, 8, and then again on frame at level 3.

Note that the frames on which **frame apply** applies a command are also influenced by the **set backtrace** settings such as **set backtrace past-main** and **set backtrace limit N**. See Section 8.2 [Backtraces], page 116.

The **frame apply** command also supports a number of options that allow overriding relevant **set backtrace** settings:

-past-main [on off]	Whether backtraces should continue past main . Related setting: [set backtrace past-main] , page 118.
-past-entry [on off]	Whether backtraces should continue past the entry point of a program. Related setting: [set backtrace past-entry] , page 118.

By default, GDB displays some frame information before the output produced by *command*, and an error raised during the execution of a *command* will abort **frame apply**. The following options can be used to fine-tune these behaviors:

-c	The flag -c , which stands for ‘ continue ’, causes any errors in <i>command</i> to be displayed, and the execution of frame apply then continues.
-s	The flag -s , which stands for ‘ silent ’, causes any errors or empty output produced by a <i>command</i> to be silently ignored. That is, the execution continues, but the frame information and errors are not printed.
-q	The flag -q (‘ quiet ’) disables printing the frame information.

The following example shows how the flags **-c** and **-s** are working when applying the command **p j** to all frames, where variable **j** can only be successfully printed in the outermost **#1 main** frame.

```
(gdb) frame apply all p j
#0 some_function (i=5) at fun.c:4
No symbol "j" in current context.
(gdb) frame apply all -c p j
#0 some_function (i=5) at fun.c:4
No symbol "j" in current context.
#1 0x565555fb in main (argc=1, argv=0xffffd2c4) at fun.c:11
$1 = 5
(gdb) frame apply all -s p j
#1 0x565555fb in main (argc=1, argv=0xffffd2c4) at fun.c:11
$2 = 5
(gdb)
```

By default, ‘frame apply’, prints the frame location information before the command output:

```
(gdb) frame apply all p $sp
#0 some_function (i=5) at fun.c:4
$4 = (void *) 0xffffd1e0
#1 0x565555fb in main (argc=1, argv=0xffffd2c4) at fun.c:11
$5 = (void *) 0xffffd1f0
(gdb)
```

If the flag `-q` is given, no frame information is printed:

```
(gdb) frame apply all -q p $sp
$12 = (void *) 0xffffd1e0
$13 = (void *) 0xffffd1f0
(gdb)
```

faas command

Shortcut for `frame apply all -s command`. Applies *command* on all frames, ignoring errors and empty output.

It can for example be used to print a local variable or a function argument without knowing the frame where this variable or argument is, using:

```
(gdb) faas p some_local_var_i_do_not_remember_where_it_is
```

The `faas` command accepts the same options as the `frame apply` command. See Section 8.5 [frame apply], page 122.

Note that the command `tfaas command` applies *command* on all frames of all threads. See Section 4.10 [Threads], page 48.

8.6 Management of Frame Filters.

Frame filters are Python based utilities to manage and decorate the output of frames. See Section 23.3.2.10 [Frame Filter API], page 436, for further information.

Managing frame filters is performed by several commands available within GDB, detailed here.

info frame-filter

Print a list of installed frame filters from all dictionaries, showing their name, priority and enabled status.

disable frame-filter filter-dictionary filter-name

Disable a frame filter in the dictionary matching *filter-dictionary* and *filter-name*. The *filter-dictionary* may be `all`, `global`, `progspace`, or the name of

the object file where the frame filter dictionary resides. When **all** is specified, all frame filters across all dictionaries are disabled. The *filter-name* is the name of the frame filter and is used when **all** is not the option for *filter-dictionary*. A disabled frame-filter is not deleted, it may be enabled again later.

enable frame-filter *filter-dictionary filter-name*

Enable a frame filter in the dictionary matching *filter-dictionary* and *filter-name*. The *filter-dictionary* may be **all**, **global**, **progspace** or the name of the object file where the frame filter dictionary resides. When **all** is specified, all frame filters across all dictionaries are enabled. The *filter-name* is the name of the frame filter and is used when **all** is not the option for *filter-dictionary*.

Example:

```
(gdb) info frame-filter

global frame-filters:
  Priority  Enabled  Name
  1000     No     PrimaryFunctionFilter
  100      Yes     Reverse

progspace /build/test frame-filters:
  Priority  Enabled  Name
  100      Yes     ProgspaceFilter

objfile /build/test frame-filters:
  Priority  Enabled  Name
  999      Yes     BuildProgramFilter

(gdb) disable frame-filter /build/test BuildProgramFilter
(gdb) info frame-filter

global frame-filters:
  Priority  Enabled  Name
  1000     No     PrimaryFunctionFilter
  100      Yes     Reverse

progspace /build/test frame-filters:
  Priority  Enabled  Name
  100      Yes     ProgspaceFilter

objfile /build/test frame-filters:
  Priority  Enabled  Name
  999      No     BuildProgramFilter

(gdb) enable frame-filter global PrimaryFunctionFilter
(gdb) info frame-filter

global frame-filters:
  Priority  Enabled  Name
  1000     Yes     PrimaryFunctionFilter
  100      Yes     Reverse

progspace /build/test frame-filters:
  Priority  Enabled  Name
  100      Yes     ProgspaceFilter

objfile /build/test frame-filters:
```

Priority	Enabled	Name
999	No	BuildProgramFilter

`set frame-filter priority filter-dictionary filter-name priority`

Set the *priority* of a frame filter in the dictionary matching *filter-dictionary*, and the frame filter name matching *filter-name*. The *filter-dictionary* may be `global`, `progspace` or the name of the object file where the frame filter dictionary resides. The *priority* is an integer.

`show frame-filter priority filter-dictionary filter-name`

Show the *priority* of a frame filter in the dictionary matching *filter-dictionary*, and the frame filter name matching *filter-name*. The *filter-dictionary* may be `global`, `progspace` or the name of the object file where the frame filter dictionary resides.

Example:

```
(gdb) info frame-filter
```

```
global frame-filters:
```

Priority	Enabled	Name
1000	Yes	PrimaryFunctionFilter
100	Yes	Reverse

```
progspace /build/test frame-filters:
```

Priority	Enabled	Name
100	Yes	ProgspaceFilter

```
objfile /build/test frame-filters:
```

Priority	Enabled	Name
999	No	BuildProgramFilter

```
(gdb) set frame-filter priority global Reverse 50
```

```
(gdb) info frame-filter
```

```
global frame-filters:
```

Priority	Enabled	Name
1000	Yes	PrimaryFunctionFilter
50	Yes	Reverse

```
progspace /build/test frame-filters:
```

Priority	Enabled	Name
100	Yes	ProgspaceFilter

```
objfile /build/test frame-filters:
```

Priority	Enabled	Name
999	No	BuildProgramFilter

9 Examining Source Files

GDB can print parts of your program's source, since the debugging information recorded in the program tells GDB what source files were used to build it. When your program stops, GDB spontaneously prints the line where it stopped. Likewise, when you select a stack frame (see Section 8.3 [Selecting a Frame], page 119), GDB prints the line where execution in that frame has stopped. You can print other portions of source files by explicit command.

If you use GDB through its GNU Emacs interface, you may prefer to use Emacs facilities to view source; see Chapter 26 [Using GDB under GNU Emacs], page 607.

9.1 Printing Source Lines

To print lines from a source file, use the `list` command (abbreviated `l`). By default, ten lines are printed. There are several ways to specify what part of the file you want to print; see Section 9.2 [Location Specifications], page 128, for the full list.

Here are the forms of the `list` command most commonly used:

- `list linenum`
Print lines centered around line number *linenum* in the current source file.
- `list function`
Print lines centered around the beginning of function *function*.
- `list`
Print more lines. If the last lines printed were printed with a `list` command, this prints lines following the last lines printed; however, if the last line printed was a solitary line printed as part of displaying a stack frame (see Chapter 8 [Examining the Stack], page 115), this prints lines centered around that line. If no `list` command has been used and no solitary line was printed, it prints the lines around the function `main`.
- `list +`
Same as using with no arguments.
- `list -`
Print lines just before the lines last printed.
- `list .`
Print the lines surrounding the point of execution within the currently selected frame. If the inferior is not running, print lines around the start of the main function instead.

By default, GDB prints ten source lines with any of these forms of the `list` command. You can change this using `set listsize`:

- `set listsize count`
- `set listsize unlimited`
Make the `list` command display *count* source lines (unless the `list` argument explicitly specifies some other number). Setting *count* to `unlimited` or 0 means there's no limit.
- `show listsize`
Display the number of lines that `list` prints.

Repeating a `list` command with `RET` discards the argument, so it is equivalent to typing just `list`. This is more useful than listing the same lines again. An exception is made for

an argument of ‘-’; that argument is preserved in repetition so that each repetition moves up in the source file.

In general, the `list` command expects you to supply zero, one or two location specs. These location specs are interpreted to resolve to source code lines; there are several ways of writing them (see Section 9.2 [Location Specifications], page 128), but the effect is always to resolve to some source lines to display.

Here is a complete description of the possible arguments for `list`:

`list locspec`

Print lines centered around the line or lines of all the code locations that result from resolving *locspec*.

`list first,last`

Print lines from *first* to *last*. Both arguments are location specs. When a `list` command has two location specs, and the source file of the second location spec is omitted, this refers to the same source file as the first location spec. If either *first* or *last* resolve to more than one source line in the program, then the list command shows the list of resolved source lines and does not proceed with the source code listing.

`list ,last`

Print lines ending with *last*.

Likewise, if *last* resolves to more than one source line in the program, then the list command prints the list of resolved source lines and does not proceed with the source code listing.

`list first,`

Print lines starting with *first*.

`list +` Print lines just after the lines last printed.

`list -` Print lines just before the lines last printed.

`list` As described in the preceding table.

9.2 Location Specifications

Several GDB commands accept arguments that specify a location or locations of your program’s code. Many times locations are specified using a source line number, but they can also be specified by a function name, an address, a label, etc. The different forms of specifying a location that GDB recognizes are collectively known as forms of *location specification*, or *location spec*. This section documents the forms of specifying locations that GDB recognizes.

When you specify a location, GDB needs to find the place in your program, known as *code location*, that corresponds to the given location spec. We call this process of finding actual code locations corresponding to a location spec *location resolution*.

A concrete code location in your program is uniquely identifiable by a set of several attributes: its source line number, the name of its source file, the fully-qualified and proto-typed function in which it is defined, and an instruction address. Because each inferior has its own address space, the inferior number is also a necessary part of these attributes.

By contrast, location specs you type will many times omit some of these attributes. For example, it is customary to specify just the source line number to mean a line in the current source file, or specify just the basename of the file, omitting its directories. In other words, a location spec is usually incomplete, a kind of blueprint, and GDB needs to complete the missing attributes by using the implied defaults, and by considering the source code and the debug information available to it. This is what location resolution is about.

The resolution of an incomplete location spec can produce more than a single code location, if the spec doesn't allow distinguishing between them. Here are some examples of situations that result in a location spec matching multiple code locations in your program:

- The location spec specifies a function name, and there are several functions in the program which have that name. (To distinguish between them, you can specify a fully-qualified and prototyped function name, such as `A::func(int)` instead of just `func`.)
- The location spec specifies a source file name, and there are several source files in the program that share the same name, for example several files with the same base-name in different subdirectories. (To distinguish between them, specify enough leading directories with the file name.)
- For a C++ constructor, the GCC compiler generates several instances of the function body, used in different cases, but their source-level names are identical.
- For a C++ template function, a given line in the function can correspond to any number of instantiations.
- For an inlined function, a given source line can correspond to several actual code locations with that function's inlined code.

Resolution of a location spec can also fail to produce a complete code location, or even fail to produce any code location. Here are some examples of such situations:

- Some parts of the program lack detailed enough debug info, so the resolved code location lacks some attributes, like source file name and line number, leaving just the instruction address and perhaps also a function name. Such an incomplete code location is only usable in contexts that work with addresses and/or function names. Some commands can only work with complete code locations.
- The location spec specifies a function name, and there are no functions in the program by that name, or they only exist in a yet-unloaded shared library.
- The location spec specifies a source file name, and there are no source files in the program by that name, or they only exist in a yet-unloaded shared library.
- The location spec specifies both a source file name and a source line number, and even though there are source files in the program that match the file name, none of those files has the specified line number.

Locations may be specified using three different formats: linespec locations, explicit locations, or address locations. The following subsections describe these formats.

9.2.1 Linespec Locations

A *linespec* is a colon-separated list of source location parameters such as file name, function name, etc. Here are all the different ways of specifying a linespec:

linenum Specifies the line number *linenum* of the current source file.

-offset

+offset Specifies the line *offset* lines before or after the *current line*. For the **list** command, the current line is the last one printed; for the breakpoint commands, this is the line at which execution stopped in the currently selected *stack frame* (see Section 8.1 [Frames], page 115, for a description of stack frames.) When used as the second of the two linespecs in a **list** command, this specifies the line *offset* lines up or down from the first linespec.

filename:linenum

Specifies the line *linenum* in the source file *filename*. If *filename* is a relative file name, then it will match any source file name with the same trailing components. For example, if *filename* is 'gcc/expr.c', then it will match source file name of /build/trunk/gcc/expr.c, but not /build/trunk/libcpp/expr.c or /build/trunk/gcc/x-expr.c.

function Specifies the line that begins the body of the function *function*. For example, in C, this is the line with the open brace.

By default, in C++ and Ada, *function* is interpreted as specifying all functions named *function* in all scopes. For C++, this means in all namespaces and classes. For Ada, this means in all packages.

For example, assuming a program with C++ symbols named **A::B::func** and **B::func**, both commands **break func** and **break B::func** set a breakpoint on both symbols.

Commands that accept a linespec let you override this with the **-qualified** option. For example, **break -qualified func** sets a breakpoint on a free-function named **func** ignoring any C++ class methods and namespace functions called **func**.

See Section 9.2.2 [Explicit Locations], page 131.

function:label

Specifies the line where *label* appears in *function*.

filename:function

Specifies the line that begins the body of the function *function* in the file *filename*. You only need the file name with a function name to avoid ambiguity when there are identically named functions in different source files.

label Specifies the line at which the label named *label* appears in the function corresponding to the currently selected stack frame. If there is no current selected stack frame (for instance, if the inferior is not running), then GDB will not search for a label.

-pstack|-probe-stap [objfile:[provider:]]name

The GNU/Linux tool **SystemTap** provides a way for applications to embed static probes. See Section 5.1.10 [Static Probe Points], page 83, for more information on finding and using static probes. This form of linespec specifies the location of such a static probe.

If *objfile* is given, only probes coming from that shared library or executable matching *objfile* as a regular expression are considered. If *provider* is given,

then only probes from that provider are considered. If several probes match the spec, GDB will insert a breakpoint at each one of those probes.

9.2.2 Explicit Locations

Explicit locations allow the user to directly specify the source location's parameters using option-value pairs.

Explicit locations are useful when several functions, labels, or file names have the same name (base name for files) in the program's sources. In these cases, explicit locations point to the source line you meant more accurately and unambiguously. Also, using explicit locations might be faster in large programs.

For example, the linespec `'foo:bar'` may refer to a function `bar` defined in the file named `foo` or the label `bar` in a function named `foo`. GDB must search either the file system or the symbol table to know.

The list of valid explicit location options is summarized in the following table:

-source filename

The value specifies the source file name. To differentiate between files with the same base name, prepend as many directories as is necessary to uniquely identify the desired file, e.g., `foo/bar/baz.c`. Otherwise GDB will use the first file it finds with the given base name. This option requires the use of either `-function` or `-line`.

-function function

The value specifies the name of a function. Operations on function locations unmodified by other options (such as `-label` or `-line`) refer to the line that begins the body of the function. In C, for example, this is the line with the open brace.

By default, in C++ and Ada, *function* is interpreted as specifying all functions named *function* in all scopes. For C++, this means in all namespaces and classes. For Ada, this means in all packages.

For example, assuming a program with C++ symbols named `A::B::func` and `B::func`, both commands `break -function func` and `break -function B::func` set a breakpoint on both symbols.

You can use the `-qualified` flag to override this (see below).

-qualified

This flag makes GDB interpret a function name specified with `-function` as a complete fully-qualified name.

For example, assuming a C++ program with symbols named `A::B::func` and `B::func`, the `break -qualified -function B::func` command sets a breakpoint on `B::func`, only.

(Note: the `-qualified` option can precede a linespec as well (see Section 9.2.1 [Linespec Locations], page 129), so the particular example above could be simplified as `break -qualified B::func`.)

-label label

The value specifies the name of a label. When the function name is not specified, the label is searched in the function of the currently selected stack frame.

-line number

The value specifies a line offset for the location. The offset may either be absolute (**-line 3**) or relative (**-line +3**), depending on the command. When specified without any other options, the line offset is relative to the current line.

Explicit location options may be abbreviated by omitting any non-unique trailing characters from the option name, e.g., **break -s main.c -li 3**.

9.2.3 Address Locations

Address locations indicate a specific program address. They have the generalized form **address*.

For line-oriented commands, such as **list** and **edit**, this specifies a source line that contains *address*. For **break** and other breakpoint-oriented commands, this can be used to set breakpoints in parts of your program which do not have debugging information or source files.

Here *address* may be any expression valid in the current working language (see Chapter 15 [Languages], page 229) that specifies a code address. In addition, as a convenience, GDB extends the semantics of expressions used in locations to cover several situations that frequently occur during debugging. Here are the various forms of *address*:

expression

Any expression valid in the current working language.

funcaddr An address of a function or procedure derived from its name. In C, C++, Objective-C, Fortran, Pascal, Modula-2, minimal, and assembly, this is simply the function's name *function* (and actually a special case of a valid expression). In Ada, this is *function'Address* (although *&function* also works).

This form specifies the address of the function's first instruction, before the stack frame and arguments have been set up.

'filename'::funcaddr

Like **funcaddr** above, but also specifies the name of the source file explicitly. This is useful if the name of the function does not specify the function unambiguously, e.g., if there are several functions with identical names in different source files, see [variable name conflict], page 149. This may not be supported for all languages, see Section 10.1 [Expressions], page 147.

9.3 Editing Source Files

To edit the lines in a source file, use the **edit** command. The editing program of your choice is invoked with the current line set to the active line in the program. Alternatively, there are several ways to specify what part of the file you want to print if you want to see other parts of the program:

edit locspec

Edit the source file of the code location that results from resolving **locspec**. Editing starts at the source file and source line **locspec** resolves to. See Section 9.2 [Location Specifications], page 128, for all the possible forms of the *locspec* argument.

If `locspec` resolves to more than one source line in your program, then the command prints the list of resolved source lines and does not proceed with the editing.

Here are the forms of the `edit` command most commonly used:

`edit number`

Edit the current source file with *number* as the active line number.

`edit function`

Edit the file containing *function* at the beginning of its definition.

9.3.1 Choosing your Editor

You can customize GDB to use any editor you want¹. By default, it is `/bin/ex`, but you can change this by setting the environment variable `EDITOR` before using GDB. For example, to configure GDB to use the `vi` editor, you could use these commands with the `sh` shell:

```
EDITOR=/usr/bin/vi
export EDITOR
gdb ...
```

or in the `cs` shell,

```
setenv EDITOR /usr/bin/vi
gdb ...
```

9.4 Searching Source Files

There are two commands for searching through the current source file for a regular expression.

`forward-search regex`

`search regex`

The command ‘`forward-search regex`’ checks each line, starting with the one following the last line listed, for a match for *regex*. It lists the line that is found and saves it in the convenience variable `$_` (see Section 10.12 [Convenience Variables], page 173). You can use the synonym ‘`search regex`’ or abbreviate the command name as `fo`.

`reverse-search regex`

The command ‘`reverse-search regex`’ checks each line, starting with the one before the last line listed and going backward, for a match for *regex*. It lists the line that is found and saves it in the convenience variable `$_` (see Section 10.12 [Convenience Variables], page 173). You can abbreviate this command as `rev`.

9.5 Specifying Source Directories

Executable programs sometimes do not record the directories of the source files from which they were compiled, just the names. Even when they do, the directories could be moved between the compilation and your debugging session. GDB has a list of directories to search

¹ The only restriction is that your editor (say `ex`), recognizes the following command-line syntax:

```
ex +number file
```

The optional numeric value *+number* specifies the number of the line in the file where to start editing.

for source files; this is called the *source path*. Each time GDB wants a source file, it tries all the directories in the list, in the order they are present in the list, until it finds a file with the desired name.

For example, suppose an executable references the file `/usr/src/foo-1.0/lib/foo.c`, does not record a compilation directory, and the *source path* is `/mnt/cross`. GDB would look for the source file in the following locations:

1. `/usr/src/foo-1.0/lib/foo.c`
2. `/mnt/cross/usr/src/foo-1.0/lib/foo.c`
3. `/mnt/cross/foo.c`

If the source file is not present at any of the above locations then an error is printed. GDB does not look up the parts of the source file name, such as `/mnt/cross/src/foo-1.0/lib/foo.c`. Likewise, the subdirectories of the source path are not searched: if the source path is `/mnt/cross`, and the binary refers to `foo.c`, GDB would not find it under `/mnt/cross/usr/src/foo-1.0/lib`.

Plain file names, relative file names with leading directories, file names containing dots, etc. are all treated as described above, except that non-absolute file names are not looked up literally. If the *source path* is `/mnt/cross`, the source file is recorded as `../lib/foo.c`, and no compilation directory is recorded, then GDB will search in the following locations:

1. `/mnt/cross/../lib/foo.c`
2. `/mnt/cross/foo.c`

The *source path* will always include two special entries ‘`$cdir`’ and ‘`$cwd`’, these refer to the compilation directory (if one is recorded) and the current working directory respectively.

‘`$cdir`’ causes GDB to search within the compilation directory, if one is recorded in the debug information. If no compilation directory is recorded in the debug information then ‘`$cdir`’ is ignored.

‘`$cwd`’ is not the same as ‘`.`’—the former tracks the current working directory as it changes during your GDB session, while the latter is immediately expanded to the current directory at the time you add an entry to the source path.

If a compilation directory is recorded in the debug information, and GDB has not found the source file after the first search using *source path*, then GDB will combine the compilation directory and the filename, and then search for the source file again using the *source path*.

For example, if the executable records the source file as `/usr/src/foo-1.0/lib/foo.c`, the compilation directory is recorded as `/project/build`, and the *source path* is `/mnt/cross:$cdir:$cwd` while the current working directory of the GDB session is `/home/user`, then GDB will search for the source file in the following locations:

1. `/usr/src/foo-1.0/lib/foo.c`
2. `/mnt/cross/usr/src/foo-1.0/lib/foo.c`
3. `/project/build/usr/src/foo-1.0/lib/foo.c`
4. `/home/user/usr/src/foo-1.0/lib/foo.c`
5. `/mnt/cross/project/build/usr/src/foo-1.0/lib/foo.c`
6. `/project/build/project/build/usr/src/foo-1.0/lib/foo.c`
7. `/home/user/project/build/usr/src/foo-1.0/lib/foo.c`

8. `/mnt/cross/foo.c`
9. `/project/build/foo.c`
10. `/home/user/foo.c`

If the file name in the previous example had been recorded in the executable as a relative path rather than an absolute path, then the first look up would not have occurred, but all of the remaining steps would be similar.

When searching for source files on MS-DOS and MS-Windows, where absolute paths start with a drive letter (e.g. `C:/project/foo.c`), GDB will remove the drive letter from the file name before appending it to a search directory from *source path*; for instance if the executable references the source file `C:/project/foo.c` and *source path* is set to `D:/mnt/cross`, then GDB will search in the following locations for the source file:

1. `C:/project/foo.c`
2. `D:/mnt/cross/project/foo.c`
3. `D:/mnt/cross/foo.c`

Note that the executable search path is *not* used to locate the source files.

Whenever you reset or rearrange the source path, GDB clears out any information it has cached about where source files are found and where each line is in the file.

When you start GDB, its source path includes only `$cdir` and `$cwd`, in that order. To add other directories, use the `directory` command.

The search path is used to find both program source files and GDB script files (read using the `-command` option and `source` command).

In addition to the source path, GDB provides a set of commands that manage the list of source path substitution rules. A *substitution rule* specifies how to rewrite source directories stored in the program's debug information in case the sources were moved to a different directory between compilation and debugging. A rule is made of two strings, the first specifying what needs to be rewritten in the path, and the second specifying how it should be rewritten. In [set substitute-path], page 136, we name these two parts *from* and *to* respectively. GDB does a simple string replacement of *from* with *to* at the start of the directory part of the source file name, and uses that result instead of the original file name to look up the sources.

Using the previous example, suppose the `foo-1.0` tree has been moved from `/usr/src` to `/mnt/cross`, then you can tell GDB to replace `/usr/src` in all source path names with `/mnt/cross`. The first lookup will then be `/mnt/cross/foo-1.0/lib/foo.c` in place of the original location of `/usr/src/foo-1.0/lib/foo.c`. To define a source path substitution rule, use the `set substitute-path` command (see [set substitute-path], page 136).

To avoid unexpected substitution results, a rule is applied only if the *from* part of the directory name ends at a directory separator. For instance, a rule substituting `/usr/source` into `/mnt/cross` will be applied to `/usr/source/foo-1.0` but not to `/usr/sourceware/foo-2.0`. And because the substitution is applied only at the beginning of the directory name, this rule will not be applied to `/root/usr/source/baz.c` either.

In many cases, you can achieve the same result using the `directory` command. However, `set substitute-path` can be more efficient in the case where the sources are organized in a complex tree with multiple subdirectories. With the `directory` command, you need to

add each subdirectory of your project. If you moved the entire tree while preserving its internal organization, then `set substitute-path` allows you to direct the debugger to all the sources with one single command.

`set substitute-path` is also more than just a shortcut command. The source path is only used if the file at the original location no longer exists. On the other hand, `set substitute-path` modifies the debugger behavior to look at the rewritten location instead. So, if for any reason a source file that is not relevant to your executable is located at the original location, a substitution rule is the only method available to point GDB at the new location.

You can configure a default source path substitution rule by configuring GDB with the `--with-relocated-sources=dir` option. The *dir* should be the name of a directory under GDB's configured prefix (set with `--prefix` or `--exec-prefix`), and directory names in debug information under *dir* will be adjusted automatically if the installed GDB is moved to a new location. This is useful if GDB, libraries or executables with debug information and corresponding source code are being moved together.

`directory dirname ...`

`dir dirname ...`

Add directory *dirname* to the front of the source path. Several directory names may be given to this command, separated by `:` (`;` on MS-DOS and MS-Windows, where `:` usually appears as part of absolute file names) or white-space. You may specify a directory that is already in the source path; this moves it forward, so GDB searches it sooner.

The special strings `$cdir` (to refer to the compilation directory, if one is recorded), and `$cwd` (to refer to the current working directory) can also be included in the list of directories *dirname*. Though these will already be in the source path they will be moved forward in the list so GDB searches them sooner.

`directory`

Reset the source path to its default value (`$cdir:$cwd` on Unix systems). This requires confirmation.

`set directories path-list`

Set the source path to *path-list*. `$cdir:$cwd` are added if missing.

`show directories`

Print the source path: show which directories it contains.

`set substitute-path from to`

Define a source path substitution rule, and add it at the end of the current list of existing substitution rules. If a rule with the same *from* was already defined, then the old rule is also deleted.

For example, if the file `/foo/bar/baz.c` was moved to `/mnt/cross/baz.c`, then the command

```
(gdb) set substitute-path /foo/bar /mnt/cross
```

will tell GDB to replace `/foo/bar` with `/mnt/cross`, which will allow GDB to find the file `baz.c` even though it was moved.

In the case when more than one substitution rule have been defined, the rules are evaluated one by one in the order where they have been defined. The first one matching, if any, is selected to perform the substitution.

For instance, if we had entered the following commands:

```
(gdb) set substitute-path /usr/src/include /mnt/include
(gdb) set substitute-path /usr/src /mnt/src
```

GDB would then rewrite `/usr/src/include/defs.h` into `/mnt/include/defs.h` by using the first rule. However, it would use the second rule to rewrite `/usr/src/lib/foo.c` into `/mnt/src/lib/foo.c`.

unset substitute-path [path]

If a path is specified, search the current list of substitution rules for a rule that would rewrite that path. Delete that rule if found. A warning is emitted by the debugger if no rule could be found.

If no path is specified, then all substitution rules are deleted.

show substitute-path [path]

If a path is specified, then print the source path substitution rule which would rewrite that path, if any.

If no path is specified, then print all existing source path substitution rules.

If your source path is cluttered with directories that are no longer of interest, GDB may sometimes cause confusion by finding the wrong versions of source. You can correct the situation as follows:

1. Use **directory** with no argument to reset the source path to its default value.
2. Use **directory** with suitable arguments to reinstall the directories you want in the source path. You can add all the directories in one command.

9.6 Source and Machine Code

You can use the command **info line** to map source lines to program addresses (and vice versa), and the command **disassemble** to display a range of addresses as machine instructions. You can use the command **set disassemble-next-line** to set whether to disassemble next source line when execution stops. When run under GNU Emacs mode, the **info line** command causes the arrow to point to the line specified. Also, **info line** prints addresses in symbolic form as well as hex.

info line

info line locspec

Print the starting and ending addresses of the compiled code for the source lines of the code locations that result from resolving *locspec*. See Section 9.2 [Location Specifications], page 128, for the various forms of *locspec*.

With no *locspec*, information is printed about either:

- the last line listed (see Section 9.1 [List], page 127)
- the line after the line shown by the previous **info line** command (see [info line, repeated calls], page 138)
- the current source line

For example, we can use `info line` to discover the location of the object code for the first line of function `m4_changequote`:

```
(gdb) info line m4_changequote
Line 895 of "builtin.c" starts at pc 0x634c <m4_changequote> and \
ends at 0x6350 <m4_changequote+4>.
```

We can also inquire, using `*addr` as the form for *locs*pec, what source line covers a particular address *addr*:

```
(gdb) info line *0x63ff
Line 926 of "builtin.c" starts at pc 0x63e4 <m4_changequote+152> and \
ends at 0x6404 <m4_changequote+184>.
```

After `info line`, the default address for the `x` command is changed to the starting address of the line, so that `'x/i'` is sufficient to begin examining the machine code (see Section 10.6 [Examining Memory], page 154). Also, this address is saved as the value of the convenience variable `$_` (see Section 10.12 [Convenience Variables], page 173).

After `info line`, using `info line` again without specifying a location will display information about the next source line.

```
disassemble
disassemble /m
disassemble /s
disassemble /r
disassemble /b
```

This specialized command dumps a range of memory as machine instructions. It can also print mixed source+disassembly by specifying the `/m` or `/s` modifier and print the raw instructions in hex as well as in symbolic form by specifying the `/r` or `/b` modifier.

Only one of `/m` and `/s` can be used, attempting to use both flag will give an error.

Only one of `/r` and `/b` can be used, attempting to use both flag will give an error.

The default memory range is the function surrounding the program counter of the selected frame. A single argument to this command is a program counter value; GDB dumps the function surrounding this value. When two arguments are given, they should be separated by a comma, possibly surrounded by white-space. The arguments specify a range of addresses to dump, in one of two forms:

start,end

the addresses from *start* (inclusive) to *end* (exclusive)

start,+length

the addresses from *start* (inclusive) to *start+length* (exclusive).

When 2 arguments are specified, the name of the function is also printed (since there could be several functions in the given range).

The argument(s) can be any expression yielding a numeric value, such as `'0x32c4'`, `'&main+10'` or `'$pc - 8'`.

If the range of memory being disassembled contains current program counter, the instruction at that location is shown with a `=>` marker.

The following example shows the disassembly of a range of addresses of HP PA-RISC 2.0 code:

```
(gdb) disas 0x32c4, 0x32e4
Dump of assembler code from 0x32c4 to 0x32e4:
0x32c4 <main+204>:      addil 0,dp
0x32c8 <main+208>:      ldw 0x22c(sr0,r1),r26
0x32cc <main+212>:      ldil 0x3000,r31
0x32d0 <main+216>:      ble 0x3f8(sr4,r31)
0x32d4 <main+220>:      ldo 0(r31),rp
0x32d8 <main+224>:      addil -0x800,dp
0x32dc <main+228>:      ldo 0x588(r1),r26
0x32e0 <main+232>:      ldil 0x3000,r31
End of assembler dump.
```

The following two examples are for RISC-V, and demonstrates the difference between the `/r` and `/b` modifiers. First with `/b`, the bytes of the instruction are printed, in hex, in memory order:

```
(gdb) disassemble /b 0x00010150,0x0001015c
Dump of assembler code from 0x10150 to 0x1015c:
0x00010150 <call_me+4>:      22 dc                sw s0,56(sp)
0x00010152 <call_me+6>:      80 00                addi s0,sp,64
0x00010154 <call_me+8>:      23 26 a4 fe          sw a0,-20(s0)
0x00010158 <call_me+12>:     23 24 b4 fe          sw a1,-24(s0)
End of assembler dump.
```

In contrast, with `/r` the bytes of the instruction are displayed in the instruction order, for RISC-V this means that the bytes have been swapped to little-endian order:

```
(gdb) disassemble /r 0x00010150,0x0001015c
Dump of assembler code from 0x10150 to 0x1015c:
0x00010150 <call_me+4>:      dc22                sw s0,56(sp)
0x00010152 <call_me+6>:      0080                addi s0,sp,64
0x00010154 <call_me+8>:      fea42623          sw a0,-20(s0)
0x00010158 <call_me+12>:     feb42423          sw a1,-24(s0)
End of assembler dump.
```

Here is an example showing mixed source+assembly for Intel x86 with `/m` or `/s`, when the program is stopped just after function prologue in a non-optimized function with no inline code.

```
(gdb) disas /m main
Dump of assembler code for function main:
5      {
      0x08048330 <+0>:      push    %ebp
      0x08048331 <+1>:      mov     %esp,%ebp
      0x08048333 <+3>:      sub     $0x8,%esp
      0x08048336 <+6>:      and     $0xffffffff0,%esp
      0x08048339 <+9>:      sub     $0x10,%esp

6          printf ("Hello.\n");
=> 0x0804833c <+12>:      movl    $0x8048440, (%esp)
      0x08048343 <+19>:      call   0x8048284 <puts@plt>

7          return 0;
8      }
      0x08048348 <+24>:      mov     $0x0,%eax
      0x0804834d <+29>:      leave
      0x0804834e <+30>:      ret
```

End of assembler dump.

The `/m` option is deprecated as its output is not useful when there is either inlined code or re-ordered code. The `/s` option is the preferred choice. Here is an example for AMD x86-64 showing the difference between `/m` output and `/s` output. This example has one inline function defined in a header file, and the code is compiled with `'-O2'` optimization. Note how the `/m` output is missing the disassembly of several instructions that are present in the `/s` output.

foo.h:

```
int
foo (int a)
{
    if (a < 0)
        return a * 2;
    if (a == 0)
        return 1;
    return a + 10;
}
```

foo.c:

```
#include "foo.h"
volatile int x, y;
int
main ()
{
    x = foo (y);
    return 0;
}
```

(gdb) disas /m main

Dump of assembler code for function main:

```
5 {
    6  x = foo (y);
      0x0000000000400400 <+0>: mov     0x200c2e(%rip),%eax # 0x601034 <y>
      0x0000000000400417 <+23>: mov     %eax,0x200c13(%rip) # 0x601030 <x>

    7  return 0;
    8 }
      0x000000000040041d <+29>: xor     %eax,%eax
      0x000000000040041f <+31>: retq
      0x0000000000400420 <+32>: add     %eax,%eax
      0x0000000000400422 <+34>: jmp     0x400417 <main+23>
```

End of assembler dump.

(gdb) disas /s main

Dump of assembler code for function main:

```
foo.c:
5 {
    6  x = foo (y);
      0x0000000000400400 <+0>: mov     0x200c2e(%rip),%eax # 0x601034 <y>

foo.h:
4  if (a < 0)
      0x0000000000400406 <+6>: test    %eax,%eax
      0x0000000000400408 <+8>: js      0x400420 <main+32>

6  if (a == 0)
```

```

7     return 1;
8     return a + 10;
0x000000000040040a <+10>: lea     0xa(%rax),%edx
0x000000000040040d <+13>: test    %eax,%eax
0x000000000040040f <+15>: mov     $0x1,%eax
0x0000000000400414 <+20>: cmovne  %edx,%eax

foo.c:
6     x = foo (y);
0x0000000000400417 <+23>: mov     %eax,0x200c13(%rip) # 0x601030 <x>

7     return 0;
8 }
0x000000000040041d <+29>: xor     %eax,%eax
0x000000000040041f <+31>: retq

foo.h:
5     return a * 2;
0x0000000000400420 <+32>: add     %eax,%eax
0x0000000000400422 <+34>: jmp     0x400417 <main+23>

End of assembler dump.

```

Here is another example showing raw instructions in hex for AMD x86-64,

```

(gdb) disas /r 0x400281,+10
Dump of assembler code from 0x400281 to 0x40028b:
0x0000000000400281: 38 36  cmp     %dh,(%rsi)
0x0000000000400283: 2d 36 34 2e 73 sub     $0x732e3436,%eax
0x0000000000400288: 6f      outsl  %ds:(%rsi),(%dx)
0x0000000000400289: 2e 32 00      xor     %cs:(%rax),%al

End of assembler dump.

```

Note that the ‘disassemble’ command’s address arguments are specified using expressions in your programming language (see Section 10.1 [Expressions], page 147), not location specs (see Section 9.2 [Location Specifications], page 128). So, for example, if you want to disassemble function `bar` in file `foo.c`, you must type ‘disassemble ‘foo.c’::bar’ and not ‘disassemble foo.c:bar’.

Some architectures have more than one commonly-used set of instruction mnemonics or other syntax.

For programs that were dynamically linked and use shared libraries, instructions that call functions or branch to locations in the shared libraries might show a seemingly bogus location—it’s actually a location of the relocation table. On some architectures, GDB might be able to resolve these to actual function names.

set disassembler-options *option1*[,*option2*...]

This command controls the passing of target specific information to the disassembler. For a list of valid options, please refer to the `-M/--disassembler-options` section of the ‘objdump’ manual and/or the output of `objdump --help` (see Section “objdump” in *The GNU Binary Utilities*). The default value is the empty string.

If it is necessary to specify more than one disassembler option, then multiple options can be placed together into a comma separated list. Currently this command is only supported on targets ARC, ARM, MIPS, PowerPC and S/390.

show disassembler-options

Show the current setting of the disassembler options.

set disassembly-flavor *instruction-set*

Select the instruction set to use when disassembling the program via the `disassemble` or `x/i` commands.

Currently this command is only defined for the Intel x86 family. You can set *instruction-set* to either `intel` or `att`. The default is `att`, the AT&T flavor used by default by Unix assemblers for x86-based targets.

show disassembly-flavor

Show the current setting of the disassembly flavor.

set disassemble-next-line

show disassemble-next-line

Control whether or not GDB will disassemble the next source line or instruction when execution stops. If `ON`, GDB will display disassembly of the next source line when execution of the program being debugged stops. This is *in addition* to displaying the source line itself, which GDB always does if possible. If the next source line cannot be displayed for some reason (e.g., if GDB cannot find the source file, or there's no line info in the debug info), GDB will display disassembly of the next *instruction* instead of showing the next source line. If `AUTO`, GDB will display disassembly of next instruction only if the source line cannot be displayed. This setting causes GDB to display some feedback when you step through a function with no line info or whose source file is unavailable. The default is `OFF`, which means never display the disassembly of the next line or instruction.

9.7 Disable Reading Source Code

In some cases it can be desirable to prevent GDB from accessing source code files. One case where this might be desirable is if the source code files are located over a slow network connection.

The following command can be used to control whether GDB should access source code files or not:

set source open [on|off]

show source open

When this option is `on`, which is the default, GDB will access source code files when needed, for example to print source lines when GDB stops, or in response to the `list` command.

When this option is `off`, GDB will not access source code files.

10 Examining Data

The usual way to examine data in your program is with the **print** command (abbreviated **p**), or its synonym **inspect**. It evaluates and prints the value of an expression of the language your program is written in (see Chapter 15 [Using GDB with Different Languages], page 229). It may also print the expression using a Python-based pretty-printer (see Section 10.10 [Pretty Printing], page 170).

```
print [[options] --] expr
print [[options] --] /f expr
```

expr is an expression (in the source language). By default the value of *expr* is printed in a format appropriate to its data type; you can choose a different format by specifying */f*, where *f* is a letter specifying the format; see Section 10.5 [Output Formats], page 152.

The **print** command supports a number of options that allow overriding relevant global print settings as set by **set print** subcommands:

- address** [on|off]
Set printing of addresses. Related setting: [set print address], page 159.
- array** [on|off]
Pretty formatting of arrays. Related setting: [set print array], page 161.
- array-indexes** [on|off]
Set printing of array indexes. Related setting: [set print array-indexes], page 161.
- characters** *number-of-characters|elements|unlimited*
Set limit on string characters to print. The value **elements** causes the limit on array elements to print to be used. The value **unlimited** causes there to be no limit. Related setting: [set print characters], page 161.
- elements** *number-of-elements|unlimited*
Set limit on array elements and optionally string characters to print. See [set print characters], page 161, and the **-characters** option above for when this option applies to strings. The value **unlimited** causes there to be no limit. See [set print elements], page 162, for a related CLI command.
- max-depth** *depth|unlimited*
Set the threshold after which nested structures are replaced with ellipsis. Related setting: [set print max-depth], page 166.
- nibbles** [on|off]
Set whether to print binary values in groups of four bits, known as “nibbles”. See [set print nibbles], page 161.
- memory-tag-violations** [on|off]
Set printing of additional information about memory tag violations. See [set print memory-tag-violations], page 166.

- null-stop [on|off]**
Set printing of char arrays to stop at first null char. Related setting: [set print null-stop], page 167.
- object [on|off]**
Set printing C++ virtual function tables. Related setting: [set print object], page 169.
- pretty [on|off]**
Set pretty formatting of structures. Related setting: [set print pretty], page 167.
- raw-values [on|off]**
Set whether to print values in raw form, bypassing any pretty-printers for that value. Related setting: [set print raw-values], page 167.
- repeats *number-of-repeats*|unlimited**
Set threshold for repeated print elements. **unlimited** causes all elements to be individually printed. Related setting: [set print repeats], page 165.
- static-members [on|off]**
Set printing C++ static members. Related setting: [set print static-members], page 169.
- symbol [on|off]**
Set printing of symbol names when printing pointers. Related setting: [set print symbol], page 161.
- union [on|off]**
Set printing of unions interior to structures. Related setting: [set print union], page 168.
- vtbl [on|off]**
Set printing of C++ virtual function tables. Related setting: [set print vtbl], page 169.

Because the **print** command accepts arbitrary expressions which may look like options (including abbreviations), if you specify any command option, then you must use a double dash (--) to mark the end of option processing.

For example, this prints the value of the **-p** expression:

```
(gdb) print -p
```

While this repeats the last value in the value history (see below) with the **-pretty** option in effect:

```
(gdb) print -p --
```

Here is an example including both on option and an expression:

```
(gdb) print -pretty -- *myptr
$1 = {
  next = 0x0,
  flags = {
    sweet = 1,
    sour = 1
  },
  meat = 0x54 "Pork"
}
```

```
print [options]
print [options] /f
```

If you omit *expr*, GDB displays the last value again (from the *value history*; see Section 10.11 [Value History], page 172). This allows you to conveniently inspect the same value in an alternative format.

If the architecture supports memory tagging, the **print** command will display pointer/memory tag mismatches if what is being printed is a pointer or reference type. See Section 10.7 [Memory Tagging], page 157.

A more low-level way of examining data is with the **x** command. It examines data in memory at a specified address and prints it in a specified format. See Section 10.6 [Examining Memory], page 154.

If you are interested in information about types, or about how the fields of a struct or a class are declared, use the **ptype *expr*** command rather than **print**. See Chapter 16 [Examining the Symbol Table], page 265.

Another way of examining values of expressions and type information is through the Python extension command **explore** (available only if the GDB build is configured with **--with-python**). It offers an interactive way to start at the highest level (or, the most abstract level) of the data type of an expression (or, the data type itself) and explore all the way down to leaf scalar values/fields embedded in the higher level data types.

explore *arg*

arg is either an expression (in the source language), or a type visible in the current context of the program being debugged.

The working of the **explore** command can be illustrated with an example. If a data type **struct ComplexStruct** is defined in your C program as

```
struct SimpleStruct
{
    int i;
    double d;
};

struct ComplexStruct
{
    struct SimpleStruct *ss_p;
    int arr[10];
};
```

followed by variable declarations as

```
struct SimpleStruct ss = { 10, 1.11 };
struct ComplexStruct cs = { &ss, { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 } };
```

then, the value of the variable `cs` can be explored using the `explore` command as follows.

```
(gdb) explore cs
The value of 'cs' is a struct/class of type 'struct ComplexStruct' with
the following fields:
```

```
ss_p = <Enter 0 to explore this field of type 'struct SimpleStruct *'>
arr = <Enter 1 to explore this field of type 'int [10] '>
```

Enter the field number of choice:

Since the fields of `cs` are not scalar values, you are being prompted to choose the field you want to explore. Let's say you choose the field `ss_p` by entering 0. Then, since this field is a pointer, you will be asked if it is pointing to a single value. From the declaration of `cs` above, it is indeed pointing to a single value, hence you enter `y`. If you enter `n`, then you will be asked if it were pointing to an array of values, in which case this field will be explored as if it were an array.

```
'cs.ss_p' is a pointer to a value of type 'struct SimpleStruct'
Continue exploring it as a pointer to a single value [y/n]: y
The value of '*(cs.ss_p)' is a struct/class of type 'struct
SimpleStruct' with the following fields:
```

```
i = 10 .. (Value of type 'int')
d = 1.1100000000000001 .. (Value of type 'double')
```

Press enter to return to parent value:

If the field `arr` of `cs` was chosen for exploration by entering 1 earlier, then since it is an array, you will be prompted to enter the index of the element in the array that you want to explore.

```
'cs.arr' is an array of 'int'.
Enter the index of the element you want to explore in 'cs.arr': 5
```

```
'(cs.arr)[5]' is a scalar value of type 'int'.
```

```
(cs.arr)[5] = 4
```

Press enter to return to parent value:

In general, at any stage of exploration, you can go deeper towards the leaf values by responding to the prompts appropriately, or hit the return key to return to the enclosing data structure (the *higher* level data structure).

Similar to exploring values, you can use the `explore` command to explore types. Instead of specifying a value (which is typically a variable name or an expression valid in the current context of the program being debugged), you specify a type name. If you consider the same example as above, you can explore the type `struct ComplexStruct` by passing the argument `struct ComplexStruct` to the `explore` command.

```
(gdb) explore struct ComplexStruct
```

By responding to the prompts appropriately in the subsequent interactive session, you can explore the type `struct ComplexStruct` in a manner similar to how the value `cs` was explored in the above example.

The `explore` command also has two sub-commands, `explore value` and `explore type`. The former sub-command is a way to explicitly specify that value exploration of the argu-

ment is being invoked, while the latter is a way to explicitly specify that type exploration of the argument is being invoked.

explore value *expr*

This sub-command of **explore** explores the value of the expression *expr* (if *expr* is an expression valid in the current context of the program being debugged). The behavior of this command is identical to that of the behavior of the **explore** command being passed the argument *expr*.

explore type *arg*

This sub-command of **explore** explores the type of *arg* (if *arg* is a type visible in the current context of program being debugged), or the type of the value/expression *arg* (if *arg* is an expression valid in the current context of the program being debugged). If *arg* is a type, then the behavior of this command is identical to that of the **explore** command being passed the argument *arg*. If *arg* is an expression, then the behavior of this command will be identical to that of the **explore** command being passed the type of *arg* as the argument.

10.1 Expressions

print and many other GDB commands accept an expression and compute its value. Any kind of constant, variable or operator defined by the programming language you are using is valid in an expression in GDB. This includes conditional expressions, function calls, casts, and string constants. It also includes preprocessor macros, if you compiled your program to include this information; see Section 4.1 [Compilation], page 33.

GDB supports array constants in expressions input by the user. The syntax is *{element, element, ...}*. For example, you can use the command **print {1, 2, 3}** to create an array of three integers. If you pass an array to a function or assign it to a program variable, GDB copies the array to memory that is **malloced** in the target program.

Because C is so widespread, most of the expressions shown in examples in this manual are in C. See Chapter 15 [Using GDB with Different Languages], page 229, for information on how to use expressions in other languages.

In this section, we discuss operators that may be available in GDB expressions in addition to those of your programming language. However, they are not necessarily available in all working languages.

Casts are supported in all languages, not just in C, because it is so useful to cast a number into a pointer in order to examine a structure at that address in memory.

GDB supports these operators, in addition to those common to programming languages:

- @** ‘@’ is a binary operator for treating parts of memory as arrays. See Section 10.4 [Artificial Arrays], page 152, for more information.
- ::** ‘::’ allows you to specify a variable in terms of the file or function where it is defined. See Section 10.3 [Program Variables], page 149. This is currently not supported in Ada, Rust, Pascal, Modula-2 and D.

{*type*} *addr*

Refers to an object of type *type* stored at address *addr* in memory. The address *addr* may be any expression whose value is an integer or pointer (but parenthe-

ses are required around binary operators, just as in a cast). This construct is allowed regardless of what kind of data is normally supposed to reside at *addr*.

10.2 Ambiguous Expressions

Expressions can sometimes contain some ambiguous elements. For instance, some programming languages (notably Ada, C++ and Objective-C) permit a single function name to be defined several times, for application in different contexts. This is called *overloading*. Another example involving Ada is generics. A *generic package* is similar to C++ templates and is typically instantiated several times, resulting in the same function name being defined in different contexts.

In some cases and depending on the language, it is possible to adjust the expression to remove the ambiguity. For instance in C++, you can specify the signature of the function you want to break on, as in *break function(types)*. In Ada, using the fully qualified name of your function often makes the expression unambiguous as well.

When an ambiguity that needs to be resolved is detected, the debugger has the capability to display a menu of numbered choices for each possibility, and then waits for the selection with the prompt '>'. The first option is always '[0] cancel', and typing *0 RET* aborts the current command. If the command in which the expression was used allows more than one choice to be selected, the next option in the menu is '[1] all', and typing *1 RET* selects all possible choices.

For example, the following session excerpt shows an attempt to set a breakpoint at the overloaded symbol `String::after`. We choose three particular definitions of that function name:

```
(gdb) b String::after
[0] cancel
[1] all
[2] file:String.cc; line number:867
[3] file:String.cc; line number:860
[4] file:String.cc; line number:875
[5] file:String.cc; line number:853
[6] file:String.cc; line number:846
[7] file:String.cc; line number:735
> 2 4 6
Breakpoint 1 at 0xb26c: file String.cc, line 867.
Breakpoint 2 at 0xb344: file String.cc, line 875.
Breakpoint 3 at 0xafcc: file String.cc, line 846.
Multiple breakpoints were set.
Use the "delete" command to delete unwanted
breakpoints.
(gdb)
```

set multiple-symbols *mode*

This option allows you to adjust the debugger behavior when an expression is ambiguous.

By default, *mode* is set to *all*. If the command with which the expression is used allows more than one choice, then GDB automatically selects all possible choices. For instance, inserting a breakpoint on a function using an ambiguous name results in a breakpoint inserted on each possible match. However, if a unique choice must be made, then GDB uses the menu to help you disambiguate

the expression. For instance, printing the address of an overloaded function will result in the use of the menu.

When *mode* is set to **ask**, the debugger always uses the menu when an ambiguity is detected.

Finally, when *mode* is set to **cancel**, the debugger reports an error due to the ambiguity and the command is aborted.

`show multiple-symbols`

Show the current value of the `multiple-symbols` setting.

10.3 Program Variables

The most common kind of expression to use is the name of a variable in your program.

Variables in expressions are understood in the selected stack frame (see Section 8.3 [Selecting a Frame], page 119); they must be either:

- global (or file-static)

or

- visible according to the scope rules of the programming language from the point of execution in that frame

This means that in the function

```
foo (a)
    int a;
{
    bar (a);
    {
        int b = test ();
        bar (b);
    }
}
```

you can examine and use the variable **a** whenever your program is executing within the function `foo`, but you can only use or examine the variable **b** while your program is executing inside the block where **b** is declared.

There is an exception: you can refer to a variable or function whose scope is a single source file even if the current execution point is not in this file. But it is possible to have more than one such variable or function with the same name (in different source files). If that happens, referring to that name has unpredictable effects. If you wish, you can specify a static variable in a particular function or file by using the colon-colon (`::`) notation (this may not be supported for all languages, see Section 10.1 [Expressions], page 147):

```
file::variable
function::variable
```

Here *file* or *function* is the name of the context for the static *variable*. In the case of file names, you can use quotes to make sure GDB parses the file name as a single word—for example, to print a global value of **x** defined in `f2.c`:

```
(gdb) p 'f2.c'::x
```

The `::` notation is normally used for referring to static variables, since you typically disambiguate uses of local variables in functions by selecting the appropriate frame and

using the simple name of the variable. However, you may also use this notation to refer to local variables in frames enclosing the selected frame:

```
void
foo (int a)
{
    if (a < 10)
        bar (a);
    else
        process (a);    /* Stop here */
}

int
bar (int a)
{
    foo (a + 5);
}
```

For example, if there is a breakpoint at the commented line, here is what you might see when the program stops after executing the call `bar(0)`:

```
(gdb) p a
$1 = 10
(gdb) p bar::a
$2 = 5
(gdb) up 2
#2  0x080483d0 in foo (a=5) at foobar.c:12
(gdb) p a
$3 = 5
(gdb) p bar::a
$4 = 0
```

These uses of `::` are very rarely in conflict with the very similar use of the same notation in C++. When they are in conflict, the C++ meaning takes precedence; however, this can be overridden by quoting the file or function name with single quotes.

For example, suppose the program is stopped in a method of a class that has a field named `includefile`, and there is also an include file named `includefile` that defines a variable, `some_global`.

```
(gdb) p includefile
$1 = 23
(gdb) p includefile::some_global
A syntax error in expression, near ``'.
(gdb) p 'includefile'::some_global
$2 = 27
```

Warning: Occasionally, a local variable may appear to have the wrong value at certain points in a function—just after entry to a new scope, and just before exit.

You may see this problem when you are stepping by machine instructions. This is because, on most machines, it takes more than one instruction to set up a stack frame (including local variable definitions); if you are stepping by machine instructions, variables may appear to have the wrong values until the stack frame is completely built. On exit, it usually also takes more than one machine instruction to destroy a stack frame; after you begin stepping through that group of instructions, local variable definitions may be gone.

This may also happen when the compiler does significant optimizations. To be sure of always seeing accurate values, turn off all optimization when compiling.

Another possible effect of compiler optimizations is to optimize unused variables out of existence, or assign variables to registers (as opposed to memory addresses). Depending on the support for such cases offered by the debug info format used by the compiler, GDB might not be able to display values for such local variables. If that happens, GDB will print a message like this:

```
No symbol "foo" in current context.
```

To solve such problems, either recompile without optimizations, or use a different debug info format, if the compiler supports several such formats. See Section 4.1 [Compilation], page 33, for more information on choosing compiler options. See Section 15.4.1 [C and C++], page 233, for more information about debug info formats that are best suited to C++ programs.

If you ask to print an object whose contents are unknown to GDB, e.g., because its data type is not completely specified by the debug information, GDB will say ‘<incomplete type>’. See Chapter 16 [Symbols], page 265, for more about this.

If you try to examine or use the value of a (global) variable for which GDB has no type information, e.g., because the program includes no debug information, GDB displays an error message. See Chapter 16 [Symbols], page 265, for more about unknown types. If you cast the variable to its declared type, GDB gets the variable’s value using the cast-to type as the variable’s type. For example, in a C program:

```
(gdb) p var
'var' has unknown type; cast it to its declared type
(gdb) p (float) var
$1 = 3.14
```

If you append `@entry` string to a function parameter name you get its value at the time the function got called. If the value is not available an error message is printed. Entry values are available only with some compilers. Entry values are normally also printed at the function parameter list according to [set print entry-values], page 163.

```
Breakpoint 1, d (i=30) at gdb.base/entry-value.c:29
29  i++;
(gdb) next
30  e (i);
(gdb) print i
$1 = 31
(gdb) print i@entry
$2 = 30
```

Strings are identified as arrays of `char` values without specified signedness. Arrays of either `signed char` or `unsigned char` get printed as arrays of 1 byte sized integers. `-fsigned-char` or `-funsigned-char` GCC options have no effect as GDB defines literal string type `"char"` as `char` without a sign. For program code

```
char var0[] = "A";
signed char var1[] = "A";
```

You get during debugging

```
(gdb) print var0
$1 = "A"
(gdb) print var1
$2 = {65 'A', 0 '\0'}
```

10.4 Artificial Arrays

It is often useful to print out several successive objects of the same type in memory; a section of an array, or an array of dynamically determined size for which only a pointer exists in the program.

You can do this by referring to a contiguous span of memory as an *artificial array*, using the binary operator ‘@’. The left operand of ‘@’ should be the first element of the desired array and be an individual object. The right operand should be the desired length of the array. The result is an array value whose elements are all of the type of the left argument. The first element is actually the left argument; the second element comes from bytes of memory immediately following those that hold the first element, and so on. Here is an example. If a program says

```
int *array = (int *) malloc (len * sizeof (int));
```

you can print the contents of `array` with

```
p *array@len
```

The left operand of ‘@’ must reside in memory. Array values made with ‘@’ in this way behave just like other arrays in terms of subscripting, and are coerced to pointers when used in expressions. Artificial arrays most often appear in expressions via the value history (see Section 10.11 [Value History], page 172), after printing one out.

Another way to create an artificial array is to use a cast. This re-interprets a value as if it were an array. The value need not be in memory:

```
(gdb) p/x (short[2])0x12345678
$1 = {0x1234, 0x5678}
```

As a convenience, if you leave the array length out (as in ‘(type[])value’) GDB calculates the size to fill the value (as ‘sizeof(value)/sizeof(type)’):

```
(gdb) p/x (short[])0x12345678
$2 = {0x1234, 0x5678}
```

Sometimes the artificial array mechanism is not quite enough; in moderately complex data structures, the elements of interest may not actually be adjacent—for example, if you are interested in the values of pointers in an array. One useful work-around in this situation is to use a convenience variable (see Section 10.12 [Convenience Variables], page 173) as a counter in an expression that prints the first interesting value, and then repeat that expression via `RET`. For instance, suppose you have an array `dtab` of pointers to structures, and you are interested in the values of a field `fv` in each structure. Here is an example of what you might type:

```
set $i = 0
p dtab[$i++]->fv
RET
RET
...
```

10.5 Output Formats

By default, GDB prints a value according to its data type. Sometimes this is not what you want. For example, you might want to print a number in hex, or a pointer in decimal. Or you might want to view data in memory at a certain address as a character string or as an instruction. To do these things, specify an *output format* when you print a value.

The simplest use of output formats is to say how to print a value already computed. This is done by starting the arguments of the `print` command with a slash and a format letter. The format letters supported are:

- x** Print the binary representation of the value in hexadecimal.
- d** Print the binary representation of the value in decimal.
- u** Print the binary representation of the value as an decimal, as if it were unsigned.
- o** Print the binary representation of the value in octal.
- t** Print the binary representation of the value in binary. The letter ‘**t**’ stands for “two”.¹
- a** Print as an address, both absolute in hexadecimal and as an offset from the nearest preceding symbol. You can use this format used to discover where (in what function) an unknown address is located:

```
(gdb) p/a 0x54320
$3 = 0x54320 <_initialize_vx+396>
```

The command `info symbol 0x54320` yields similar results. See Chapter 16 [Symbols], page 265.

- c** Cast the value to an integer (unlike other formats, this does not just reinterpret the underlying bits) and print it as a character constant. This prints both the numerical value and its character representation. The character representation is replaced with the octal escape ‘`\nnn`’ for characters outside the 7-bit ASCII range.

Without this format, GDB displays `char`, `unsigned char`, and `signed char` data as character constants. Single-byte members of vectors are displayed as integer data.

- f** Regard the bits of the value as a floating point number and print using typical floating point syntax.
- s** Regard as a string, if possible. With this format, pointers to single-byte data are displayed as null-terminated strings and arrays of single-byte data are displayed as fixed-length strings. Other values are displayed in their natural types.

Without this format, GDB displays pointers to and arrays of `char`, `unsigned char`, and `signed char` as strings. Single-byte members of a vector are displayed as an integer array.

- z** Like ‘**x**’ formatting, the value is treated as an integer and printed as hexadecimal, but leading zeros are printed to pad the value to the size of the integer type.
- r** Print using the ‘**raw**’ formatting. By default, GDB will use a Python-based pretty-printer, if one is available (see Section 10.10 [Pretty Printing], page 170). This typically results in a higher-level display of the value’s contents. The ‘**r**’ format bypasses any Python pretty-printer which might exist.

¹ ‘**b**’ cannot be used because these format letters are also used with the `x` command, where ‘**b**’ stands for “byte”; see Section 10.6 [Examining Memory], page 154.

For example, to print the program counter in hex (see Section 10.14 [Registers], page 181), type

```
p/x $pc
```

Note that no space is required before the slash; this is because command names in GDB cannot contain a slash.

To reprint the last value in the value history with a different format, you can use the `print` command with just a format and no expression. For example, `'p/x'` reprints the last value in hex.

10.6 Examining Memory

You can use the command `x` (for “examine”) to examine memory in any of several formats, independently of your program’s data types.

```
x/nfu addr
```

```
x addr
```

`x` Use the `x` command to examine memory.

n, *f*, and *u* are all optional parameters that specify how much memory to display and how to format it; *addr* is an expression giving the address where you want to start displaying memory. If you use defaults for *nfu*, you need not type the slash `'/'`. Several commands set convenient defaults for *addr*.

n, the repeat count

The repeat count is a decimal integer; the default is 1. It specifies how much memory (counting by units *u*) to display. If a negative number is specified, memory is examined backward from *addr*.

f, the display format

The display format is one of the formats used by `print` (`'x'`, `'d'`, `'u'`, `'o'`, `'t'`, `'a'`, `'c'`, `'f'`, `'s'`), `'i'` (for machine instructions) and `'m'` (for displaying memory tags). The default is `'x'` (hexadecimal) initially. The default changes each time you use either `x` or `print`.

u, the unit size

The unit size is any of

`b` Bytes.

`h` Halfwords (two bytes).

`w` Words (four bytes). This is the initial default.

`g` Giant words (eight bytes).

Each time you specify a unit size with `x`, that size becomes the default unit the next time you use `x`. For the `'i'` format, the unit size is ignored and is normally not written. For the `'s'` format, the unit size defaults to `'b'`, unless it is explicitly given. Use `x /hs` to display 16-bit char strings and `x /ws` to display 32-bit strings. The next use of `x /s` will again display 8-bit strings. Note that the results depend on the programming language of the current compilation unit. If the language is C, the `'s'` modifier will use the UTF-16 encoding while

‘w’ will use UTF-32. The encoding is set by the programming language and cannot be altered.

addr, starting display address

addr is the address where you want GDB to begin displaying memory. The expression need not have a pointer value (though it may); it is always interpreted as an integer address of a byte of memory. See Section 10.1 [Expressions], page 147, for more information on expressions. The default for *addr* is usually just after the last address examined—but several other commands also set the default address: **info breakpoints** (to the address of the last breakpoint listed), **info line** (to the starting address of a line), and **print** (if you use it to display a value from memory).

For example, ‘x/3uh 0x54320’ is a request to display three halfwords (h) of memory, formatted as unsigned decimal integers (‘u’), starting at address 0x54320. ‘x/4xw \$sp’ prints the four words (‘w’) of memory above the stack pointer (here, ‘\$sp’; see Section 10.14 [Registers], page 181) in hexadecimal (‘x’).

You can also specify a negative repeat count to examine memory backward from the given address. For example, ‘x/-3uh 0x54320’ prints three halfwords (h) at 0x5431a, 0x5431c, and 0x5431e.

Since the letters indicating unit sizes are all distinct from the letters specifying output formats, you do not have to remember whether unit size or format comes first; either order works. The output specifications ‘4xw’ and ‘4wx’ mean exactly the same thing. (However, the count *n* must come first; ‘wx4’ does not work.)

Even though the unit size *u* is ignored for the formats ‘s’ and ‘i’, you might still want to use a count *n*; for example, ‘3i’ specifies that you want to see three machine instructions, including any operands. For convenience, especially when used with the **display** command, the ‘i’ format also prints branch delay slot instructions, if any, beyond the count specified, which immediately follow the last instruction that is within the count. The command **disassemble** gives an alternative way of inspecting machine instructions; see Section 9.6 [Source and Machine Code], page 137.

If a negative repeat count is specified for the formats ‘s’ or ‘i’, the command displays null-terminated strings or instructions before the given address as many as the absolute value of the given number. For the ‘i’ format, we use line number information in the debug info to accurately locate instruction boundaries while disassembling backward. If line info is not available, the command stops examining memory with an error message.

All the defaults for the arguments to **x** are designed to make it easy to continue scanning memory with minimal specifications each time you use **x**. For example, after you have inspected three machine instructions with ‘x/3i *addr*’, you can inspect the next seven with just ‘x/7’. If you use **RET** to repeat the **x** command, the repeat count *n* is used again; the other arguments default as for successive uses of **x**.

When examining machine instructions, the instruction at current program counter is shown with a => marker. For example:

```
(gdb) x/5i $pc-6
0x804837f <main+11>: mov    %esp,%ebp
0x8048381 <main+13>: push  %ecx
0x8048382 <main+14>: sub    $0x4,%esp
```

```
=> 0x8048385 <main+17>: movl    $0x8048460, (%esp)
    0x804838c <main+24>: call   0x80482d4 <puts@plt>
```

If the architecture supports memory tagging, the tags can be displayed by using ‘m’. See Section 10.7 [Memory Tagging], page 157.

The information will be displayed once per granule size (the amount of bytes a particular memory tag covers). For example, AArch64 has a granule size of 16 bytes, so it will display a tag every 16 bytes.

Due to the way GDB prints information with the `x` command (not aligned to a particular boundary), the tag information will refer to the initial address displayed on a particular line. If a memory tag boundary is crossed in the middle of a line displayed by the `x` command, it will be displayed on the next line.

The ‘m’ format doesn’t affect any other specified formats that were passed to the `x` command.

The addresses and contents printed by the `x` command are not saved in the value history because there is often too much of them and they would get in the way. Instead, GDB makes these values available for subsequent use in expressions as values of the convenience variables `$_` and `$__`. After an `x` command, the last address examined is available for use in expressions in the convenience variable `$_`. The contents of that address, as examined, are available in the convenience variable `$__`.

If the `x` command has a repeat count, the address and contents saved are from the last memory unit printed; this is not the same as the last address printed if several units were printed on the last line of output.

Most targets have an addressable memory unit size of 8 bits. This means that to each memory address are associated 8 bits of data. Some targets, however, have other addressable memory unit sizes. Within GDB and this document, the term *addressable memory unit* (or *memory unit* for short) is used when explicitly referring to a chunk of data of that size. The word *byte* is used to refer to a chunk of data of 8 bits, regardless of the addressable memory unit size of the target. For most systems, addressable memory unit is a synonym of byte.

When you are debugging a program running on a remote target machine (see Chapter 20 [Remote Debugging], page 317), you may wish to verify the program’s image in the remote machine’s memory against the executable file you downloaded to the target. Or, on any target, you may want to check whether the program has corrupted its own read-only sections. The `compare-sections` command is provided for such situations.

compare-sections [*section-name*|-r]

Compare the data of a loadable section *section-name* in the executable file of the program being debugged with the same section in the target machine’s memory, and report any mismatches. With no arguments, compares all loadable sections. With an argument of `-r`, compares all loadable read-only sections.

Note: for remote targets, this command can be accelerated if the target supports computing the CRC checksum of a block of memory (see [qCRC packet], page 812).

10.7 Memory Tagging

Memory tagging is a memory protection technology that uses a pair of tags to validate memory accesses through pointers. The tags are integer values usually comprised of a few bits, depending on the architecture.

There are two types of tags that are used in this setup: logical and allocation. A logical tag is stored in the pointers themselves, usually at the higher bits of the pointers. An allocation tag is the tag associated with particular ranges of memory in the physical address space, against which the logical tags from pointers are compared.

The pointer tag (logical tag) must match the memory tag (allocation tag) for the memory access to be valid. If the logical tag does not match the allocation tag, that will raise a memory violation.

Allocation tags cover multiple contiguous bytes of physical memory. This range of bytes is called a memory tag granule and is architecture-specific. For example, AArch64 has a tag granule of 16 bytes, meaning each allocation tag spans 16 bytes of memory.

If the underlying architecture supports memory tagging, like AArch64 MTE or SPARC ADI do, GDB can make use of it to validate pointers against memory allocation tags.

The `print` (see Chapter 10 [Data], page 143) and `x` (see Section 10.6 [Memory], page 154) commands will display tag information when appropriate, and a command prefix of `memory-tag` gives access to the various memory tagging commands.

The `memory-tag` commands are the following:

`memory-tag print-logical-tag pointer_expression`

Print the logical tag stored in *pointer_expression*.

`memory-tag with-logical-tag pointer_expression tag_bytes`

Print the pointer given by *pointer_expression*, augmented with a logical tag of *tag_bytes*.

`memory-tag print-allocation-tag address_expression`

Print the allocation tag associated with the memory address given by *address_expression*.

`memory-tag setatag starting_address length tag_bytes`

Set the allocation tag(s) for memory range [*starting_address*, *starting_address* + *length*) to *tag_bytes*.

`memory-tag check pointer_expression`

Check if the logical tag in the pointer given by *pointer_expression* matches the allocation tag for the memory referenced by the pointer.

This essentially emulates the hardware validation that is done when tagged memory is accessed through a pointer, but does not cause a memory fault as it would during hardware validation.

It can be used to inspect potential memory tagging violations in the running process, before any faults get triggered.

10.8 Automatic Display

If you find that you want to print the value of an expression frequently (to see how it changes), you might want to add it to the *automatic display list* so that GDB prints its value each time your program stops. Each expression added to the list is given a number to identify it; to remove an expression from the list, you specify that number. The automatic display looks like this:

```
2: foo = 38
3: bar[5] = (struct hack *) 0x3804
```

This display shows item numbers, expressions and their current values. As with displays you request manually using `x` or `print`, you can specify the output format you prefer; in fact, `display` decides whether to use `print` or `x` depending your format specification—it uses `x` if you specify either the ‘i’ or ‘s’ format, or a unit size; otherwise it uses `print`.

`display expr`

Add the expression `expr` to the list of expressions to display each time your program stops. See Section 10.1 [Expressions], page 147.

`display` does not repeat if you press `RET` again after using it.

`display/fmt expr`

For `fmt` specifying only a display format and not a size or count, add the expression `expr` to the auto-display list but arrange to display it each time in the specified format `fmt`. See Section 10.5 [Output Formats], page 152.

`display/fmt addr`

For `fmt` ‘i’ or ‘s’, or including a unit-size or a number of units, add the expression `addr` as a memory address to be examined each time your program stops. Examining means in effect doing ‘`x/fmt addr`’. See Section 10.6 [Examining Memory], page 154.

For example, ‘`display/i $pc`’ can be helpful, to see the machine instruction about to be executed each time execution stops (‘`$pc`’ is a common name for the program counter; see Section 10.14 [Registers], page 181).

`undisplay dnums...`

`delete display dnums...`

Remove items from the list of expressions to display. Specify the numbers of the displays that you want affected with the command argument `dnums`. It can be a single display number, one of the numbers shown in the first field of the ‘`info display`’ display; or it could be a range of display numbers, as in 2-4.

`undisplay` does not repeat if you press `RET` after using it. (Otherwise you would just get the error ‘No display number ...’.)

`disable display dnums...`

Disable the display of item numbers `dnums`. A disabled display item is not printed automatically, but is not forgotten. It may be enabled again later. Specify the numbers of the displays that you want affected with the command argument `dnums`. It can be a single display number, one of the numbers shown in the first field of the ‘`info display`’ display; or it could be a range of display numbers, as in 2-4.

enable display *dnums*...

Enable display of item numbers *dnums*. It becomes effective once again in auto display of its expression, until you specify otherwise. Specify the numbers of the displays that you want affected with the command argument *dnums*. It can be a single display number, one of the numbers shown in the first field of the ‘info display’ display; or it could be a range of display numbers, as in 2-4.

display Display the current values of the expressions on the list, just as is done when your program stops.

info display

Print the list of expressions previously set up to display automatically, each one with its item number, but without showing the values. This includes disabled expressions, which are marked as such. It also includes expressions which would not be displayed right now because they refer to automatic variables not currently available.

If a display expression refers to local variables, then it does not make sense outside the lexical context for which it was set up. Such an expression is disabled when execution enters a context where one of its variables is not defined. For example, if you give the command **display last_char** while inside a function with an argument **last_char**, GDB displays this argument while your program continues to stop inside that function. When it stops elsewhere—where there is no variable **last_char**—the display is disabled automatically. The next time your program stops where **last_char** is meaningful, you can enable the display expression once again.

10.9 Print Settings

GDB provides the following ways to control how arrays, structures, and symbols are printed. These settings are useful for debugging programs in any language:

set print address

set print address on

GDB prints memory addresses showing the location of stack traces, structure values, pointer values, breakpoints, and so forth, even when it also displays the contents of those addresses. The default is **on**. For example, this is what a stack frame display looks like with **set print address on**:

```
(gdb) f
#0  set_quotes (lq=0x34c78 "<<", rq=0x34c88 ">>")
    at input.c:530
530          if (lquote != def_lquote)
```

set print address off

Do not print addresses when displaying their contents. For example, this is the same stack frame displayed with **set print address off**:

```
(gdb) set print addr off
(gdb) f
#0  set_quotes (lq="<<", rq=">>") at input.c:530
530          if (lquote != def_lquote)
```

You can use ‘**set print address off**’ to eliminate all machine dependent displays from the GDB interface. For example, with **print address off**, you

should get the same text for backtraces on all machines—whether or not they involve pointer arguments.

show print address

Show whether or not addresses are to be printed.

When GDB prints a symbolic address, it normally prints the closest earlier symbol plus an offset. If that symbol does not uniquely identify the address (for example, it is a name whose scope is a single source file), you may need to clarify. One way to do this is with **info line**, for example ‘**info line *0x4537**’. Alternately, you can set GDB to print the source file and line number when it prints a symbolic address:

set print symbol-filename on

Tell GDB to print the source file name and line number of a symbol in the symbolic form of an address.

set print symbol-filename off

Do not print source file name and line number of a symbol. This is the default.

show print symbol-filename

Show whether or not GDB will print the source file name and line number of a symbol in the symbolic form of an address.

Another situation where it is helpful to show symbol filenames and line numbers is when disassembling code; GDB shows you the line number and source file that corresponds to each instruction.

Also, you may wish to see the symbolic form only if the address being printed is reasonably close to the closest earlier symbol:

set print max-symbolic-offset *max-offset*

set print max-symbolic-offset unlimited

Tell GDB to only display the symbolic form of an address if the offset between the closest earlier symbol and the address is less than *max-offset*. The default is **unlimited**, which tells GDB to always print the symbolic form of an address if any symbol precedes it. Zero is equivalent to **unlimited**.

show print max-symbolic-offset

Ask how large the maximum offset is that GDB prints in a symbolic address.

If you have a pointer and you are not sure where it points, try ‘**set print symbol-filename on**’. Then you can determine the name and source file location of the variable where it points, using ‘**p/a *pointer***’. This interprets the address in symbolic form. For example, here GDB shows that a variable **ptt** points at another variable **t**, defined in **hi2.c**:

```
(gdb) set print symbol-filename on
(gdb) p/a ptt
$4 = 0xe008 <t in hi2.c>
```

Warning: For pointers that point to a local variable, ‘**p/a**’ does not show the symbol name and filename of the referent, even with the appropriate **set print** options turned on.

You can also enable `‘/a’`-like formatting all the time using `‘set print symbol on’`:

`set print symbol on`

Tell GDB to print the symbol corresponding to an address, if one exists.

`set print symbol off`

Tell GDB not to print the symbol corresponding to an address. In this mode, GDB will still print the symbol corresponding to pointers to functions. This is the default.

`show print symbol`

Show whether GDB will display the symbol corresponding to an address.

Other settings control how different kinds of objects are printed:

`set print array`

`set print array on`

Pretty print arrays. This format is more convenient to read, but uses more space. The default is off.

`set print array off`

Return to compressed format for arrays.

`show print array`

Show whether compressed or pretty format is selected for displaying arrays.

`set print array-indexes`

`set print array-indexes on`

Print the index of each element when displaying arrays. May be more convenient to locate a given element in the array or quickly find the index of a given element in that printed array. The default is off.

`set print array-indexes off`

Stop printing element indexes when displaying arrays.

`show print array-indexes`

Show whether the index of each element is printed when displaying arrays.

`set print nibbles`

`set print nibbles on`

Print binary values in groups of four bits, known as *nibbles*, when using the print command of GDB with the option `‘/t’`. For example, this is what it looks like with `set print nibbles on`:

```
(gdb) print val_flags
$1 = 1230
(gdb) print/t val_flags
$2 = 0100 1100 1110
```

`set print nibbles off`

Don't printing binary values in groups. This is the default.

`show print nibbles`

Show whether to print binary values in groups of four bits.

```
set print characters number-of-characters
set print characters elements
set print characters unlimited
```

Set a limit on how many characters of a string GDB will print. If GDB is printing a large string, it stops printing after it has printed the number of characters set by the `set print characters` command. This equally applies to multi-byte and wide character strings, that is for strings whose character type is `wchar_t`, `char16_t`, or `char32_t` it is the number of actual characters rather than underlying bytes the encoding uses that this setting controls. Setting *number-of-characters* to `elements` means that the limit on the number of characters to print follows one for array elements; see [set print elements], page 162. Setting *number-of-characters* to `unlimited` means that the number of characters to print is unlimited. When GDB starts, this limit is set to `elements`.

```
show print characters
```

Display the number of characters of a large string that GDB will print.

```
set print elements number-of-elements
set print elements unlimited
```

Set a limit on how many elements of an array GDB will print. If GDB is printing a large array, it stops printing after it has printed the number of elements set by the `set print elements` command. By default this limit also applies to the display of strings; see [set print characters], page 161. When GDB starts, this limit is set to 200. Setting *number-of-elements* to `unlimited` or zero means that the number of elements to print is unlimited.

When printing very large arrays, whose size is greater than `max-value-size` (see [max-value-size], page 195), if the `print elements` is set such that the size of the elements being printed is less than or equal to `max-value-size`, then GDB will print the array (up to the `print elements` limit), and only `max-value-size` worth of data will be added into the value history (see Section 10.11 [Value History], page 172).

```
show print elements
```

Display the number of elements of a large array that GDB will print.

```
set print frame-arguments value
```

This command allows to control how the values of arguments are printed when the debugger prints a frame (see Section 8.1 [Frames], page 115). The possible values are:

- `all` The values of all arguments are printed.
- `scalars` Print the value of an argument only if it is a scalar. The value of more complex arguments such as arrays, structures, unions, etc, is replaced by `...`. This is the default. Here is an example where only scalar arguments are shown:

```
#1 0x08048361 in call_me (i=3, s=..., ss=0xbf8d508c, u=..., e=green)
    at frame-args.c:23
```

none None of the argument values are printed. Instead, the value of each argument is replaced by In this case, the example above now becomes:

```
#1 0x08048361 in call_me (i=..., s=..., ss=..., u=..., e=...)
   at frame-args.c:23
```

presence Only the presence of arguments is indicated by The ... are not printed for function without any arguments. None of the argument names and values are printed. In this case, the example above now becomes:

```
#1 0x08048361 in call_me (...) at frame-args.c:23
```

By default, only scalar arguments are printed. This command can be used to configure the debugger to print the value of all arguments, regardless of their type. However, it is often advantageous to not print the value of more complex parameters. For instance, it reduces the amount of information printed in each frame, making the backtrace more readable. Also, it improves performance when displaying Ada frames, because the computation of large arguments can sometimes be CPU-intensive, especially in large applications. Setting **print frame-arguments** to **scalars** (the default), **none** or **presence** avoids this computation, thus speeding up the display of each Ada frame.

show print frame-arguments

Show how the value of arguments should be displayed when printing a frame.

set print raw-frame-arguments on

Print frame arguments in raw, non pretty-printed, form.

set print raw-frame-arguments off

Print frame arguments in pretty-printed form, if there is a pretty-printer for the value (see Section 10.10 [Pretty Printing], page 170), otherwise print the value in raw form. This is the default.

show print raw-frame-arguments

Show whether to print frame arguments in raw form.

set print entry-values value

Set printing of frame argument values at function entry. In some cases GDB can determine the value of function argument which was passed by the function caller, even if the value was modified inside the called function and therefore is different. With optimized code, the current value could be unavailable, but the entry value may still be known.

The default value is **default** (see below for its description). Older GDB behaved as with the setting **no**. Compilers not supporting this feature will behave in the **default** setting the same way as with the **no** setting.

This functionality is currently supported only by DWARF 2 debugging format and the compiler has to produce 'DW_TAG_call_site' tags. With GCC, you need to specify **-O -g** during compilation, to get this information.

The *value* parameter can be one of the following:

no Print only actual parameter values, never print values from function entry point.

	<pre> #0 equal (val=5) #0 different (val=6) #0 lost (val=<optimized out>) #0 born (val=10) #0 invalid (val=<optimized out>) </pre>
only	Print only parameter values from function entry point. The actual parameter values are never printed. <pre> #0 equal (val@entry=5) #0 different (val@entry=5) #0 lost (val@entry=5) #0 born (val@entry=<optimized out>) #0 invalid (val@entry=<optimized out>) </pre>
preferred	Print only parameter values from function entry point. If value from function entry point is not known while the actual value is known, print the actual value for such parameter. <pre> #0 equal (val@entry=5) #0 different (val@entry=5) #0 lost (val@entry=5) #0 born (val=10) #0 invalid (val@entry=<optimized out>) </pre>
if-needed	Print actual parameter values. If actual parameter value is not known while value from function entry point is known, print the entry point value for such parameter. <pre> #0 equal (val=5) #0 different (val=6) #0 lost (val@entry=5) #0 born (val=10) #0 invalid (val=<optimized out>) </pre>
both	Always print both the actual parameter value and its value from function entry point, even if values of one or both are not available due to compiler optimizations. <pre> #0 equal (val=5, val@entry=5) #0 different (val=6, val@entry=5) #0 lost (val=<optimized out>, val@entry=5) #0 born (val=10, val@entry=<optimized out>) #0 invalid (val=<optimized out>, val@entry=<optimized out>) </pre>
compact	Print the actual parameter value if it is known and also its value from function entry point if it is known. If neither is known, print for the actual value <optimized out>. If not in MI mode (see Chapter 27 [GDB/MI], page 609) and if both values are known and identical, print the shortened <code>param=param@entry=VALUE</code> notation. <pre> #0 equal (val=val@entry=5) #0 different (val=6, val@entry=5) #0 lost (val@entry=5) #0 born (val=10) #0 invalid (val=<optimized out>) </pre>
default	Always print the actual parameter value. Print also its value from function entry point, but only if it is known. If not in MI mode (see

Chapter 27 [GDB/MI], page 609) and if both values are known and identical, print the shortened `param=param@entry=VALUE` notation.

```
#0 equal (val=val@entry=5)
#0 different (val=6, val@entry=5)
#0 lost (val=<optimized out>, val@entry=5)
#0 born (val=10)
#0 invalid (val=<optimized out>)
```

For analysis messages on possible failures of frame argument values at function entry resolution see [set debug entry-values], page 198.

`show print entry-values`

Show the method being used for printing of frame argument values at function entry.

`set print frame-info value`

This command allows to control the information printed when the debugger prints a frame. See Section 8.1 [Frames], page 115, Section 8.2 [Backtrace], page 116, for a general explanation about frames and frame information. Note that some other settings (such as `set print frame-arguments` and `set print address`) are also influencing if and how some frame information is displayed. In particular, the frame program counter is never printed if `set print address` is off.

The possible values for `set print frame-info` are:

`short-location`

Print the frame level, the program counter (if not at the beginning of the location source line), the function, the function arguments.

`location` Same as `short-location` but also print the source file and source line number.

`location-and-address`

Same as `location` but print the program counter even if located at the beginning of the location source line.

`source-line`

Print the program counter (if not at the beginning of the location source line), the line number and the source line.

`source-and-location`

Print what `location` and `source-line` are printing.

`auto` The information printed for a frame is decided automatically by the GDB command that prints a frame. For example, `frame` prints the information printed by `source-and-location` while `stepi` will switch between `source-line` and `source-and-location` depending on the program counter. The default value is `auto`.

`set print repeats number-of-repeats`

`set print repeats unlimited`

Set the threshold for suppressing display of repeated array elements. When the number of consecutive identical elements of an array exceeds the threshold,

GDB prints the string "<repeats *n* times>", where *n* is the number of identical repetitions, instead of displaying the identical elements themselves. Setting the threshold to `unlimited` or zero will cause all elements to be individually printed. The default threshold is 10.

`show print repeats`

Display the current threshold for printing repeated identical elements.

`set print max-depth depth`

`set print max-depth unlimited`

Set the threshold after which nested structures are replaced with ellipsis, this can make visualising deeply nested structures easier.

For example, given this C code

```
typedef struct s1 { int a; } s1;
typedef struct s2 { s1 b; } s2;
typedef struct s3 { s2 c; } s3;
typedef struct s4 { s3 d; } s4;
```

```
s4 var = { { { { 3 } } } };
```

The following table shows how different values of *depth* will effect how `var` is printed by GDB:

<i>depth</i> setting	Result of 'p var'
unlimited	\$1 = {d = {c = {b = {a = 3}}}}
0	\$1 = {...}
1	\$1 = {d = {...}}
2	\$1 = {d = {c = {...}}}
3	\$1 = {d = {c = {b = {...}}}}
4	\$1 = {d = {c = {b = {a = 3}}}}

To see the contents of structures that have been hidden the user can either increase the print max-depth, or they can print the elements of the structure that are visible, for example

```
(gdb) set print max-depth 2
(gdb) p var
$1 = {d = {c = {...}}}
(gdb) p var.d
$2 = {c = {b = {...}}}
(gdb) p var.d.c
$3 = {b = {a = 3}}
```

The pattern used to replace nested structures varies based on language, for most languages {...} is used, but Fortran uses (...).

`show print max-depth`

Display the current threshold after which nested structures are replaces with ellipsis.

`set print memory-tag-violations`

`set print memory-tag-violations on`

Cause GDB to display additional information about memory tag violations when printing pointers and addresses.

set print memory-tag-violations off

Stop printing memory tag violation information.

show print memory-tag-violations

Show whether memory tag violation information is displayed when printing pointers and addresses.

set print null-stop

Cause GDB to stop printing the characters of an array when the first NULL is encountered. This is useful when large arrays actually contain only short strings. The default is off.

show print null-stop

Show whether GDB stops printing an array on the first NULL character.

set print pretty on

Cause GDB to print structures in an indented format with one member per line, like this:

```
$1 = {
  next = 0x0,
  flags = {
    sweet = 1,
    sour = 1
  },
  meat = 0x54 "Pork"
}
```

set print pretty off

Cause GDB to print structures in a compact format, like this:

```
$1 = {next = 0x0, flags = {sweet = 1, sour = 1}, \
  meat = 0x54 "Pork"}
```

This is the default format.

show print pretty

Show which format GDB is using to print structures.

set print raw-values on

Print values in raw form, without applying the pretty printers for the value.

set print raw-values off

Print values in pretty-printed form, if there is a pretty-printer for the value (see Section 10.10 [Pretty Printing], page 170), otherwise print the value in raw form.

The default setting is “off”.

show print raw-values

Show whether to print values in raw form.

set print sevenbit-strings on

Print using only seven-bit characters; if this option is set, GDB displays any eight-bit characters (in strings or character values) using the notation `\nnn`. This setting is best if you are working in English (ASCII) and you use the high-order bit of characters as a marker or “meta” bit.

set print sevenbit-strings off

Print full eight-bit characters. This allows the use of more international character sets, and is the default.

show print sevenbit-strings

Show whether or not GDB is printing only seven-bit characters.

set print union on

Tell GDB to print unions which are contained in structures and other unions. This is the default setting.

set print union off

Tell GDB not to print unions which are contained in structures and other unions. GDB will print "{...}" instead.

show print union

Ask GDB whether or not it will print unions which are contained in structures and other unions.

For example, given the declarations

```
typedef enum {Tree, Bug} Species;
typedef enum {Big_tree, Acorn, Seedling} Tree_forms;
typedef enum {Caterpillar, Cocoon, Butterfly}
    Bug_forms;
```

```
struct thing {
    Species it;
    union {
        Tree_forms tree;
        Bug_forms bug;
    } form;
};
```

```
struct thing foo = {Tree, {Acorn}};
```

with **set print union on** in effect ‘p foo’ would print

```
$1 = {it = Tree, form = {tree = Acorn, bug = Cocoon}}
```

and with **set print union off** in effect it would print

```
$1 = {it = Tree, form = {...}}
```

set print union affects programs written in C-like languages and in Pascal.

These settings are of interest when debugging C++ programs:

set print demangle

set print demangle on

Print C++ names in their source form rather than in the encoded (“mangled”) form passed to the assembler and linker for type-safe linkage. The default is on.

show print demangle

Show whether C++ names are printed in mangled or demangled form.

set print asm-demangle

set print asm-demangle on

Print C++ names in their source form rather than their mangled form, even in assembler code printouts such as instruction disassemblies. The default is off.

`show print asm-demangle`

Show whether C++ names in assembly listings are printed in mangled or demangled form.

`set demangle-style style`

Choose among several encoding schemes used by different compilers to represent C++ names. If you omit *style*, you will see a list of possible formats. The default value is *auto*, which lets GDB choose a decoding style by inspecting your program.

`show demangle-style`

Display the encoding style currently in use for decoding C++ symbols.

`set print object`

`set print object on`

When displaying a pointer to an object, identify the *actual* (derived) type of the object rather than the *declared* type, using the virtual function table. Note that the virtual function table is required—this feature can only work for objects that have run-time type identification; a single virtual method in the object's declared type is sufficient. Note that this setting is also taken into account when working with variable objects via MI (see Chapter 27 [GDB/MI], page 609).

`set print object off`

Display only the declared type of objects, without reference to the virtual function table. This is the default setting.

`show print object`

Show whether actual, or declared, object types are displayed.

`set print static-members`

`set print static-members on`

Print static members when displaying a C++ object. The default is on.

`set print static-members off`

Do not print static members when displaying a C++ object.

`show print static-members`

Show whether C++ static members are printed or not.

`set print pascal_static-members`

`set print pascal_static-members on`

Print static members when displaying a Pascal object. The default is on.

`set print pascal_static-members off`

Do not print static members when displaying a Pascal object.

`show print pascal_static-members`

Show whether Pascal static members are printed or not.

`set print vtbl`

`set print vtbl on`

Pretty print C++ virtual function tables. The default is off. (The `vtbl` commands do not work on programs compiled with the HP ANSI C++ compiler (aCC).)

```
set print vtbl off
```

Do not pretty print C++ virtual function tables.

```
show print vtbl
```

Show whether C++ virtual function tables are pretty printed, or not.

10.10 Pretty Printing

GDB provides a mechanism to allow pretty-printing of values using Python code. It greatly simplifies the display of complex objects. This mechanism works for both MI and the CLI.

10.10.1 Pretty-Printer Introduction

When GDB prints a value, it first sees if there is a pretty-printer registered for the value. If there is then GDB invokes the pretty-printer to print the value. Otherwise the value is printed normally.

Pretty-printers are normally named. This makes them easy to manage. The ‘`info pretty-printer`’ command will list all the installed pretty-printers with their names. If a pretty-printer can handle multiple data types, then its *subprinters* are the printers for the individual data types. Each such subprinter has its own name. The format of the name is *printer-name;subprinter-name*.

Pretty-printers are installed by *registering* them with GDB. Typically they are automatically loaded and registered when the corresponding debug information is loaded, thus making them available without having to do anything special.

There are three places where a pretty-printer can be registered.

- Pretty-printers registered globally are available when debugging all inferiors.
- Pretty-printers registered with a program space are available only when debugging that program. See Section 23.3.2.26 [Progspace In Python], page 484, for more details on program spaces in Python.
- Pretty-printers registered with an objfile are loaded and unloaded with the corresponding objfile (e.g., shared library). See Section 23.3.2.27 [Objfiles In Python], page 487, for more details on objfiles in Python.

See Section 23.3.2.7 [Selecting Pretty-Printers], page 432, for further information on how pretty-printers are selected,

See Section 23.3.2.8 [Writing a Pretty-Printer], page 433, for implementing pretty printers for new types.

10.10.2 Pretty-Printer Example

Here is how a C++ `std::string` looks without a pretty-printer:

```
(gdb) print s
$1 = {
  static npos = 4294967295,
  _M_dataplus = {
    <std::allocator<char>> = {
      <__gnu_cxx::new_allocator<char>> = {
        <No data fields>}, <No data fields>
      },
    members of std::basic_string<char, std::char_traits<char>,

```

```

        std::allocator<char> >::_Alloc_hider:
    _M_p = 0x804a014 "abcd"
    }
}

```

With a pretty-printer for `std::string` only the contents are printed:

```

(gdb) print s
$2 = "abcd"

```

10.10.3 Pretty-Printer Commands

info pretty-printer [*object-regex*] [*name-regex*]

Print the list of installed pretty-printers. This includes disabled pretty-printers, which are marked as such.

object-regex is a regular expression matching the objects whose pretty-printers to list. Objects can be `global`, the program space's file (see Section 23.3.2.26 [Prospaces In Python], page 484), and the object files within that program space (see Section 23.3.2.27 [Objfiles In Python], page 487). See Section 23.3.2.7 [Selecting Pretty-Printers], page 432, for details on how GDB looks up a printer from these three objects.

name-regex is a regular expression matching the name of the printers to list.

disable pretty-printer [*object-regex*] [*name-regex*]

Disable pretty-printers matching *object-regex* and *name-regex*. A disabled pretty-printer is not forgotten, it may be enabled again later.

enable pretty-printer [*object-regex*] [*name-regex*]

Enable pretty-printers matching *object-regex* and *name-regex*.

Example:

Suppose we have three pretty-printers installed: one from `library1.so` named `foo` that prints objects of type `foo`, and another from `library2.so` named `bar` that prints two types of objects, `bar1` and `bar2`.

```

(gdb) info pretty-printer
library1.so:
  foo
library2.so:
  bar
    bar1
    bar2
(gdb) info pretty-printer library2
library2.so:
  bar
    bar1
    bar2
(gdb) disable pretty-printer library1
1 printer disabled
2 of 3 printers enabled
(gdb) info pretty-printer
library1.so:
  foo [disabled]
library2.so:
  bar
    bar1
    bar2

```

```

(gdb) disable pretty-printer library2 bar;bar1
1 printer disabled
1 of 3 printers enabled
(gdb) info pretty-printer library2
library2.so:
  bar
    bar1 [disabled]
    bar2
(gdb) disable pretty-printer library2 bar
1 printer disabled
0 of 3 printers enabled
(gdb) info pretty-printer
library1.so:
  foo [disabled]
library2.so:
  bar [disabled]
    bar1 [disabled]
    bar2

```

Note that for `bar` the entire printer can be disabled, as can each individual subprinter.

Printing values and frame arguments is done by default using the enabled pretty printers.

The print option `-raw-values` and GDB setting `set print raw-values` (see [set print raw-values], page 167) can be used to print values without applying the enabled pretty printers.

Similarly, the backtrace option `-raw-frame-arguments` and GDB setting `set print raw-frame-arguments` (see [set print raw-frame-arguments], page 163) can be used to ignore the enabled pretty printers when printing frame argument values.

10.11 Value History

Values printed by the `print` command are saved in the GDB *value history*. This allows you to refer to them in other expressions. Values are kept until the symbol table is re-read or discarded (for example with the `file` or `symbol-file` commands). When the symbol table changes, the value history is discarded, since the values may contain pointers back to the types defined in the symbol table.

The values printed are given *history numbers* by which you can refer to them. These are successive integers starting with one. `print` shows you the history number assigned to a value by printing `‘$num = ’` before the value; here *num* is the history number.

To refer to any previous value, use `‘$’` followed by the value’s history number. The way `print` labels its output is designed to remind you of this. Just `$` refers to the most recent value in the history, and `$$` refers to the value before that. `$$n` refers to the *n*th value from the end; `$$2` is the value just prior to `$$`, `$$1` is equivalent to `$$`, and `$$0` is equivalent to `$`.

For example, suppose you have just printed a pointer to a structure and want to see the contents of the structure. It suffices to type

```
p *$
```

If you have a chain of structures where the component `next` points to the next one, you can print the contents of the next one with this:

```
p *$.next
```

You can print successive links in the chain by repeating this command—which you can do by just typing **RET**.

Note that the history records values, not expressions. If the value of `x` is 4 and you type these commands:

```
print x
set x=5
```

then the value recorded in the value history by the **print** command remains 4 even though the value of `x` has changed.

show values

Print the last ten values in the value history, with their item numbers. This is like ‘**p \$\$\$**’ repeated ten times, except that **show values** does not change the history.

show values n

Print ten history values centered on history item number *n*.

show values +

Print ten history values just after the values last printed. If no more values are available, **show values +** produces no display.

Pressing **RET** to repeat **show values n** has exactly the same effect as ‘**show values +**’.

10.12 Convenience Variables

GDB provides *convenience variables* that you can use within GDB to hold on to a value and refer to it later. These variables exist entirely within GDB; they are not part of your program, and setting a convenience variable has no direct effect on further execution of your program. That is why you can use them freely.

Convenience variables are prefixed with ‘\$’. Any name preceded by ‘\$’ can be used for a convenience variable, unless it is one of the predefined machine-specific register names (see Section 10.14 [Registers], page 181). (Value history references, in contrast, are *numbers* preceded by ‘\$’. See Section 10.11 [Value History], page 172.)

You can save a value in a convenience variable with an assignment expression, just as you would set a variable in your program. For example:

```
set $foo = *object_ptr
```

would save in `$foo` the value contained in the object pointed to by `object_ptr`.

Using a convenience variable for the first time creates it, but its value is `void` until you assign a new value. You can alter the value with another assignment at any time.

Convenience variables have no fixed types. You can assign a convenience variable any type of value, including structures and arrays, even if that variable already has a value of a different type. The convenience variable, when used as an expression, has the type of its current value.

show convenience

Print a list of convenience variables used so far, and their values, as well as a list of the convenience functions. Abbreviated **show conv**.

init-if-undefined \$variable = expression

Set a convenience variable if it has not already been set. This is useful for user-defined commands that keep some state. It is similar, in concept, to using local static variables with initializers in C (except that convenience variables are global). It can also be used to allow users to override default values used in a command script.

If the variable is already defined then the expression is not evaluated so any side-effects do not occur.

One of the ways to use a convenience variable is as a counter to be incremented or a pointer to be advanced. For example, to print a field from successive elements of an array of structures:

```
set $i = 0
print bar[$i++]>contents
```

Repeat that command by typing **RET**.

Some convenience variables are created automatically by GDB and given values likely to be useful.

\$_ The variable **\$_** is automatically set by the **x** command to the last address examined (see Section 10.6 [Examining Memory], page 154). Other commands which provide a default address for **x** to examine also set **\$_** to that address; these commands include **info line** and **info breakpoint**. The type of **\$_** is **void (*)()** except when set by the **x** command, in which case it is a pointer to the type of **\$__**.

The **forward-search** and **reverse-search** commands (see Section 9.4 [Searching Source Files], page 133) set the **\$_** variable to the line number that is found as a result of the search. In that case the type of **\$_** becomes **int**.

\$__ The variable **\$__** is automatically set by the **x** command to the value found in the last address examined. Its type is chosen to match the format in which the data was printed.

\$_exitcode

When the program being debugged terminates normally, GDB automatically sets this variable to the exit code of the program, and resets **\$_exitsignal** to **void**.

\$_exitsignal

When the program being debugged dies due to an uncaught signal, GDB automatically sets this variable to that signal's number, and resets **\$_exitcode** to **void**.

To distinguish between whether the program being debugged has exited (i.e., **\$_exitcode** is not **void**) or signalled (i.e., **\$_exitsignal** is not **void**), the convenience function **\$_isvoid** can be used (see Section 10.13 [Convenience Functions], page 177). For example, considering the following source code:

```
#include <signal.h>

int
main (int argc, char *argv[])
```



```

{
    raise (SIGALRM);
    return 0;
}

```

A valid way of telling whether the program being debugged has exited or signalled would be:

```

(gdb) define has_exited_or_signalled
Type commands for definition of 'has_exited_or_signalled'.
End with a line saying just 'end'.
>if $_isvoid ($_exitsignal)
>echo The program has exited\n
>else
>echo The program has signalled\n
>end
>end
(gdb) run
Starting program:

Program terminated with signal SIGALRM, Alarm clock.
The program no longer exists.
(gdb) has_exited_or_signalled
The program has signalled

```

As can be seen, GDB correctly informs that the program being debugged has signalled, since it calls `raise` and raises a `SIGALRM` signal. If the program being debugged had not called `raise`, then GDB would report a normal exit:

```

(gdb) has_exited_or_signalled
The program has exited

```

`$_exception`

The variable `$_exception` is set to the exception object being thrown at an exception-related catchpoint. See Section 5.1.3 [Set Catchpoints], page 71.

`$_ada_exception`

The variable `$_ada_exception` is set to the address of the exception being caught or thrown at an Ada exception-related catchpoint. See Section 5.1.3 [Set Catchpoints], page 71.

`$_probe_argc`

`$_probe_arg0...$_probe_arg11`

Arguments to a static probe. See Section 5.1.10 [Static Probe Points], page 83.

`$_sdata`

The variable `$_sdata` contains extra collected static tracepoint data. See Section 13.1.6 [Tracepoint Action Lists], page 210. Note that `$_sdata` could be empty, if not inspecting a trace buffer, or if extra static tracepoint data has not been collected.

`$_siginfo`

The variable `$_siginfo` contains extra signal information (see [extra signal information], page 95). Note that `$_siginfo` could be empty, if the application has not yet received any signals. For example, it will be empty before you execute the `run` command.

`$_tlb`

The variable `$_tlb` is automatically set when debugging applications running on MS-Windows in native mode or connected to `gdbserver` that supports the

`qGetTIBAddr` request. See Section E.5 [General Query Packets], page 811. This variable contains the address of the thread information block.

`$_inferior`

The number of the current inferior. See Section 4.9 [Debugging Multiple Inferiors Connections and Programs], page 43.

`$_thread` The thread number of the current thread. See [thread numbers], page 48.

`$_gthread`

The global number of the current thread. See [global thread numbers], page 49.

`$_inferior_thread_count`

The number of live threads in the current inferior. See Section 4.10 [Threads], page 48.

`$_gdb_major`

`$_gdb_minor`

The major and minor version numbers of the running GDB. Development snapshots and pretest versions have their minor version incremented by one; thus, GDB pretest 9.11.90 will produce the value 12 for `$_gdb_minor`. These variables allow you to write scripts that work with different versions of GDB without errors caused by features unavailable in some of those versions.

`$_shell_exitcode`

`$_shell_exitsignal`

GDB commands such as `shell` and `|` are launching shell commands. When a launched command terminates, GDB automatically maintains the variables `$_shell_exitcode` and `$_shell_exitsignal` according to the exit status of the last launched command. These variables are set and used similarly to the variables `$_exitcode` and `$_exitsignal`.

`$_colorsupport`

Comma-separated list of *color space* names supported by terminal. Names could be any of `'monochrome'`, `'ansi_8color'`, `'aixterm_16color'`, `'xterm_256color'`, `'rgb_24bit'`. E.g., for plain linux terminal the value could be `'monochrome,ansi_8color'` and for terminal with truecolor support it could be `'monochrome,ansi_8color,aixterm_16color,xterm_256color,rgb_24bit'`. `'rgb_24bit'` availability is determined by the `COLORTERM` environment variable which may be `'truecolor'` or `'24bit'`. Other color spaces are determined by the "Co" termcap which in turn depends on the `TERM` environment variable.

`$_linker_namespace_count`

Number of active linker namespaces in the inferior (see Section 18.1 [Files], page 293). In systems with no support for linker namespaces or if the inferior hasn't started, this variable will always be set to `'0'`.

`$_linker_namespace`

The namespace which contains the current location in the inferior. This returns GDB's internal numbering for the namespace. In systems with no support for linker namespaces, this variable will always be set to `'0'`.

10.13 Convenience Functions

GDB also supplies some *convenience functions*. These have a syntax similar to convenience variables. A convenience function can be used in an expression just like an ordinary function; however, a convenience function is implemented internally to GDB.

These functions do not require GDB to be configured with `Python` support, which means that they are always available.

`$_isvoid (expr)`

Return one if the expression *expr* is `void`. Otherwise it returns zero.

A `void` expression is an expression where the type of the result is `void`. For example, you can examine a convenience variable (see Section 10.12 [Convenience Variables], page 173) to check whether it is `void`:

```
(gdb) print $_exitcode
$1 = void
(gdb) print $_isvoid ($_exitcode)
$2 = 1
(gdb) run
Starting program: ./a.out
[Inferior 1 (process 29572) exited normally]
(gdb) print $_exitcode
$3 = 0
(gdb) print $_isvoid ($_exitcode)
$4 = 0
```

In the example above, we used `$_isvoid` to check whether `$_exitcode` is `void` before and after the execution of the program being debugged. Before the execution there is no exit code to be examined, therefore `$_exitcode` is `void`. After the execution the program being debugged returned zero, therefore `$_exitcode` is zero, which means that it is not `void` anymore.

The `void` expression can also be a call of a function from the program being debugged. For example, given the following function:

```
void
foo (void)
{
}
```

The result of calling it inside GDB is `void`:

```
(gdb) print foo ()
$1 = void
(gdb) print $_isvoid (foo ())
$2 = 1
(gdb) set $v = foo ()
(gdb) print $v
$3 = void
(gdb) print $_isvoid ($v)
$4 = 1
```

`$_gdb_setting_str (setting)`

Return the value of the GDB *setting* as a string. *setting* is any setting that can be used in a `set` or `show` command (see Chapter 22 [Controlling GDB], page 371).

```
(gdb) show print frame-arguments
Printing of non-scalar frame arguments is "scalars".
```

```
(gdb) p $_gdb_setting_str("print frame-arguments")
$1 = "scalars"
(gdb) p $_gdb_setting_str("height")
$2 = "30"
(gdb)
```

`$_gdb_setting (setting)`

Return the value of the GDB *setting*. The type of the returned value depends on the setting.

The value type for boolean and auto boolean settings is `int`. The boolean values `off` and `on` are converted to the integer values 0 and 1. The value `auto` is converted to the value -1.

The value type for integer settings is either `unsigned int` or `int`, depending on the setting.

Some integer settings accept an `unlimited` value. Depending on the setting, the `set` command also accepts the value 0 or the value -1 as a synonym for `unlimited`. For example, `set height unlimited` is equivalent to `set height 0`.

Some other settings that accept the `unlimited` value use the value 0 to literally mean zero. For example, `set history size 0` indicates to not record any GDB commands in the command history. For such settings, -1 is the synonym for `unlimited`.

See the documentation of the corresponding `set` command for the numerical value equivalent to `unlimited`.

The `$_gdb_setting` function converts the `unlimited` value to a 0 or a -1 value according to what the `set` command uses.

```
(gdb) p $_gdb_setting_str("height")
$1 = "30"
(gdb) p $_gdb_setting("height")
$2 = 30
(gdb) set height unlimited
(gdb) p $_gdb_setting_str("height")
$3 = "unlimited"
(gdb) p $_gdb_setting("height")
$4 = 0
(gdb) p $_gdb_setting_str("history size")
$5 = "unlimited"
(gdb) p $_gdb_setting("history size")
$6 = -1
(gdb) p $_gdb_setting_str("disassemble-next-line")
$7 = "auto"
(gdb) p $_gdb_setting("disassemble-next-line")
$8 = -1
(gdb)
```

Other setting types (enum, filename, optional filename, string, string noescape) are returned as string values.

`$_gdb_maint_setting_str (setting)`

Like the `$_gdb_setting_str` function, but works with `maintenance set` variables.

\$_gdb_maint_setting (setting)

Like the `$_gdb_setting` function, but works with **maintenance set** variables.

\$_shell (command-string)

Invoke a shell to execute *command-string*. *command-string* must be a string. The shell runs on the host machine, the machine GDB is running on. Returns the command's exit status. On Unix systems, a command which exits with a zero exit status has succeeded, and non-zero exit status indicates failure. When a command terminates on a fatal signal whose number is *N*, GDB uses the value $128+N$ as the exit status, as is standard in Unix shells. Note that *N* is a host signal number, not a target signal number. If you're native debugging, they will be the same, but if cross debugging, the host vs target signal numbers may be completely unrelated. Please consult your host operating system's documentation for the mapping between host signal numbers and signal names. The shell to run is determined in the same way as for the `shell` command. See Section 2.3 [Shell Commands], page 20.

```
(gdb) print $_shell("true")
$1 = 0
(gdb) print $_shell("false")
$2 = 1
(gdb) p $_shell("echo hello")
hello
$3 = 0
(gdb) p $_shell("foobar")
bash: line 1: foobar: command not found
$4 = 127
```

This may also be useful in breakpoint conditions. For example:

```
(gdb) break function if $_shell("some command") == 0
```

In this scenario, you'll want to make sure that the shell command you run in the breakpoint condition takes the least amount of time possible. For example, avoid running a command that may block indefinitely, or that sleeps for a while before exiting. Prefer a command or script which analyzes some state and exits immediately. This is important because the debugged program stops for the breakpoint every time, and then GDB evaluates the breakpoint condition. If the condition is false, the program is re-resumed transparently, without informing you of the stop. A quick shell command thus avoids significantly slowing down the debugged program unnecessarily.

Note: unlike the `shell` command, the `$_shell` convenience function does not affect the `$_shell_exitcode` and `$_shell_exitsignal` convenience variables.

The following functions require GDB to be configured with Python support.

\$_memeq(buf1, buf2, length)

Returns one if the *length* bytes at the addresses given by *buf1* and *buf2* are equal. Otherwise it returns zero.

\$_regex(str, regex)

Returns one if the string *str* matches the regular expression *regex*. Otherwise it returns zero. The syntax of the regular expression is that specified by Python's regular expression support.

`$_streq(str1, str2)`

Returns one if the strings *str1* and *str2* are equal. Otherwise it returns zero.

`$_strlen(str)`

Returns the length of string *str*.

`$_caller_is(name[, number_of_frames])`

Returns one if the calling function's name is equal to *name*. Otherwise it returns zero.

If the optional argument *number_of_frames* is provided, it is the number of frames up in the stack to look. The default is 1.

Example:

```
(gdb) backtrace
#0  bottom_func ()
    at testsuite/gdb.python/py-caller-is.c:21
#1  0x0000000004005a0 in middle_func ()
    at testsuite/gdb.python/py-caller-is.c:27
#2  0x0000000004005ab in top_func ()
    at testsuite/gdb.python/py-caller-is.c:33
#3  0x0000000004005b6 in main ()
    at testsuite/gdb.python/py-caller-is.c:39
(gdb) print $_caller_is ("middle_func")
$1 = 1
(gdb) print $_caller_is ("top_func", 2)
$1 = 1
```

`$_caller_matches(regexp[, number_of_frames])`

Returns one if the calling function's name matches the regular expression *regexp*. Otherwise it returns zero.

If the optional argument *number_of_frames* is provided, it is the number of frames up in the stack to look. The default is 1.

`$_any_caller_is(name[, number_of_frames])`

Returns one if any calling function's name is equal to *name*. Otherwise it returns zero.

If the optional argument *number_of_frames* is provided, it is the number of frames up in the stack to look. The default is 1.

This function differs from `$_caller_is` in that this function checks all stack frames from the immediate caller to the frame specified by *number_of_frames*, whereas `$_caller_is` only checks the frame specified by *number_of_frames*.

`$_any_caller_matches(regexp[, number_of_frames])`

Returns one if any calling function's name matches the regular expression *regexp*. Otherwise it returns zero.

If the optional argument *number_of_frames* is provided, it is the number of frames up in the stack to look. The default is 1.

This function differs from `$_caller_matches` in that this function checks all stack frames from the immediate caller to the frame specified by *number_of_frames*, whereas `$_caller_matches` only checks the frame specified by *number_of_frames*.

`$_as_string(value)`

This convenience function is considered deprecated, and could be removed from future versions of GDB. Use the ‘%V’ format specifier instead (see [‘%V Format Specifier’], page 404).

Return the string representation of *value*.

This function is useful to obtain the textual label (enumerator) of an enumeration value. For example, assuming the variable *node* is of an enumerated type:

```
(gdb) printf "Visiting node of type %s\n", $_as_string(node)
Visiting node of type NODE_INTEGER
```

`$_cimag(value)`**`$_creal(value)`**

Return the imaginary (`$_cimag`) or real (`$_creal`) part of the complex number *value*.

The type of the imaginary or real part depends on the type of the complex number, e.g., using `$_cimag` on a float `complex` will return an imaginary part of type `float`.

GDB provides the ability to list and get help on convenience functions.

`help function`

Print a list of all convenience functions.

10.14 Registers

You can refer to machine register contents, in expressions, as variables with names starting with ‘\$’. The names of registers are different for each machine; use `info registers` to see the names used on your machine.

`info registers`

Print the names and values of all registers except floating-point and vector registers (in the selected stack frame).

`info all-registers`

Print the names and values of all registers, including floating-point and vector registers (in the selected stack frame).

`info registers reggroup ...`

Print the name and value of the registers in each of the specified *reggroups*. The *reggroup* can be any of those returned by `maint print reggroups` (see Appendix D [Maintenance Commands], page 777).

`info registers regname ...`

Print the *relativized* value of each specified register *regname*. As discussed in detail below, register values are normally relative to the selected stack frame. The *regname* may be any register name valid on the machine you are using, with or without the initial ‘\$’.

GDB has four “standard” register names that are available (in expressions) on most machines—whenever they do not conflict with an architecture’s canonical mnemonics for registers. The register names `$pc` and `$sp` are used for the program counter register and

the stack pointer. `$fp` is used for a register that contains a pointer to the current stack frame, and `$ps` is used for a register that contains the processor status. For example, you could print the program counter in hex with

```
p/x $pc
```

or print the instruction to be executed next with

```
x/i $pc
```

or add four to the stack pointer² with

```
set $sp += 4
```

Whenever possible, these four standard register names are available on your machine even though the machine has different canonical mnemonics, so long as there is no conflict. The `info registers` command shows the canonical names. For example, on the SPARC, `info registers` displays the processor status register as `$psr` but you can also refer to it as `$ps`; and on x86-based machines `$ps` is an alias for the EFLAGS register.

GDB always considers the contents of an ordinary register as an integer when the register is examined in this way. Some machines have special registers which can hold nothing but floating point; these registers are considered to have floating point values. There is no way to refer to the contents of an ordinary register as floating point value (although you can *print* it as a floating point value with `'print/f $regname'`).

Some registers have distinct “raw” and “virtual” data formats. This means that the data format in which the register contents are saved by the operating system is not the same one that your program normally sees. For example, the registers of the 68881 floating point coprocessor are always saved in “extended” (raw) format, but all C programs expect to work with “double” (virtual) format. In such cases, GDB normally works with the virtual format only (the format that makes sense for your program), but the `info registers` command prints the data in both formats.

Some machines have special registers whose contents can be interpreted in several different ways. For example, modern x86-based machines have SSE and MMX registers that can hold several values packed together in several different formats. GDB refers to such registers in `struct` notation:

```
(gdb) print $xmm1
$1 = {
  v4_float = {0, 3.43859137e-038, 1.54142831e-044, 1.821688e-044},
  v2_double = {9.92129282474342e-303, 2.7585945287983262e-313},
  v16_int8 = "\000\000\000\000\3706;\001\v\000\000\000\r\000\000",
  v8_int16 = {0, 0, 14072, 315, 11, 0, 13, 0},
  v4_int32 = {0, 20657912, 11, 13},
  v2_int64 = {88725056443645952, 55834574859},
  uint128 = 0x00000000d0000000b013b36f800000000
}
```

To set values of such registers, you need to tell GDB which view of the register you wish to change, as if you were assigning value to a `struct` member:

```
(gdb) set $xmm1.uint128 = 0x000000000000000000000000FFFFFFFF
```

² This is a way of removing one word from the stack, on machines where stacks grow downward in memory (most machines, nowadays). This assumes that the innermost stack frame is selected; setting `$sp` is not allowed when other stack frames are selected. To pop entire frames off the stack, regardless of machine architecture, use `return`; see Section 17.4 [Returning from a Function], page 282.

Normally, register values are relative to the selected stack frame (see Section 8.3 [Selecting a Frame], page 119). This means that you get the value that the register would contain if all stack frames farther in were exited and their saved registers restored. In order to see the true contents of hardware registers, you must select the innermost frame (with ‘`frame 0`’).

Usually ABIs reserve some registers as not needed to be saved by the callee (a.k.a.: “caller-saved”, “call-clobbered” or “volatile” registers). It may therefore not be possible for GDB to know the value a register had before the call (in other words, in the outer frame), if the register value has since been changed by the callee. GDB tries to deduce where the inner frame saved (“callee-saved”) registers, from the debug info, unwind info, or the machine code generated by your compiler. If some register is not saved, and GDB knows the register is “caller-saved” (via its own knowledge of the ABI, or because the debug/unwind info explicitly says the register’s value is undefined), GDB displays ‘<not saved>’ as the register’s value. With targets that GDB has no knowledge of the register saving convention, if a register was not saved by the callee, then its value and location in the outer frame are assumed to be the same of the inner frame. This is usually harmless, because if the register is call-clobbered, the caller either does not care what is in the register after the call, or has code to restore the value that it does care about. Note, however, that if you change such a register in the outer frame, you may also be affecting the inner frame. Also, the more “outer” the frame is you’re looking at, the more likely a call-clobbered register’s value is to be wrong, in the sense that it doesn’t actually represent the value the register had just before the call.

10.15 Floating Point Hardware

Depending on the configuration, GDB may be able to give you more information about the status of the floating point hardware.

info float

Display hardware-dependent information about the floating point unit. The exact contents and layout vary depending on the floating point chip. Currently, ‘`info float`’ is supported on the ARM and x86 machines.

10.16 Vector Unit

Depending on the configuration, GDB may be able to give you more information about the status of the vector unit.

info vector

Display information about the vector unit. The exact contents and layout vary depending on the hardware.

10.17 Operating System Auxiliary Information

GDB provides interfaces to useful OS facilities that can help you debug your program.

Some operating systems supply an *auxiliary vector* to programs at startup. This is akin to the arguments and environment that you specify for a program, but contains a system-dependent variety of binary values that tell system libraries important details about the

hardware, operating system, and process. Each value's purpose is identified by an integer tag; the meanings are well-known but system-specific. Depending on the configuration and operating system facilities, GDB may be able to show you this information. For remote targets, this functionality may further depend on the remote stub's support of the 'qXfer:auxv:read' packet, see [qXfer auxiliary vector read], page 834.

info auxv Display the auxiliary vector of the inferior, which can be either a live process or a core dump file. GDB prints each tag value numerically, and also shows names and text descriptions for recognized tags. Some values in the vector are numbers, some bit masks, and some pointers to strings or other data. GDB displays each value in the most appropriate form for a recognized tag, and in hexadecimal for an unrecognized tag.

On some targets, GDB can access operating system-specific information and show it to you. The types of information available will differ depending on the type of operating system running on the target. The mechanism used to fetch the data is described in Appendix H [Operating System Information], page 905. For remote targets, this functionality depends on the remote stub's support of the 'qXfer:osdata:read' packet, see [qXfer osdata read], page 837.

info os infotype

Display OS information of the requested type.

On GNU/Linux, the following values of *infotype* are valid:

- | | |
|----------------|---|
| cpus | Display the list of all CPUs/cores. For each CPU/core, GDB prints the available fields from /proc/cpuinfo. For each supported architecture different fields are available. Two common entries are processor which gives CPU number and bogomips; a system constant that is calculated during kernel initialization. |
| files | Display the list of open file descriptors on the target. For each file descriptor, GDB prints the identifier of the process owning the descriptor, the command of the owning process, the value of the descriptor, and the target of the descriptor. |
| modules | Display the list of all loaded kernel modules on the target. For each module, GDB prints the module name, the size of the module in bytes, the number of times the module is used, the dependencies of the module, the status of the module, and the address of the loaded module in memory. |
| msg | Display the list of all System V message queues on the target. For each message queue, GDB prints the message queue key, the message queue identifier, the access permissions, the current number of bytes on the queue, the current number of messages on the queue, the processes that last sent and received a message on the queue, the user and group of the owner and creator of the message queue, the times at which a message was last sent and received on the queue, and the time at which the message queue was last changed. |

processes

Display the list of processes on the target. For each process, GDB prints the process identifier, the name of the user, the command corresponding to the process, and the list of processor cores that the process is currently running on. (To understand what these properties mean, for this and the following info types, please consult the general GNU/Linux documentation.)

procggroups

Display the list of process groups on the target. For each process, GDB prints the identifier of the process group that it belongs to, the command corresponding to the process group leader, the process identifier, and the command line of the process. The list is sorted first by the process group identifier, then by the process identifier, so that processes belonging to the same process group are grouped together and the process group leader is listed first.

semaphores

Display the list of all System V semaphore sets on the target. For each semaphore set, GDB prints the semaphore set key, the semaphore set identifier, the access permissions, the number of semaphores in the set, the user and group of the owner and creator of the semaphore set, and the times at which the semaphore set was operated upon and changed.

shm

Display the list of all System V shared-memory regions on the target. For each shared-memory region, GDB prints the region key, the shared-memory identifier, the access permissions, the size of the region, the process that created the region, the process that last attached to or detached from the region, the current number of live attaches to the region, and the times at which the region was last attached to, detach from, and changed.

sockets

Display the list of Internet-domain sockets on the target. For each socket, GDB prints the address and port of the local and remote endpoints, the current state of the connection, the creator of the socket, the IP address family of the socket, and the type of the connection.

threads

Display the list of threads running on the target. For each thread, GDB prints the identifier of the process that the thread belongs to, the command of the process, the thread identifier, and the processor core that it is currently running on. The main thread of a process is not listed.

info os

If *infotype* is omitted, then list the possible values for *infotype* and the kind of OS information available for each *infotype*. If the target does not return a list of possible types, this command will report an error.

10.18 Memory Region Attributes

Memory region attributes allow you to describe special handling required by regions of your target's memory. GDB uses attributes to determine whether to allow certain types of memory accesses; whether to use specific width accesses; and whether to cache target memory. By default the description of memory regions is fetched from the target (if the current target supports this), but the user can override the fetched regions.

Defined memory regions can be individually enabled and disabled. When a memory region is disabled, GDB uses the default attributes when accessing memory in that region. Similarly, if no memory regions have been defined, GDB uses the default attributes when accessing all memory.

When a memory region is defined, it is given a number to identify it; to enable, disable, or remove a memory region, you specify that number.

mem lower upper attributes...

Define a memory region bounded by *lower* and *upper* with attributes *attributes...*, and add it to the list of regions monitored by GDB. Note that *upper* == 0 is a special case: it is treated as the target's maximum memory address. (0xffff on 16 bit targets, 0xffffffff on 32 bit targets, etc.)

mem auto Discard any user changes to the memory regions and use target-supplied regions, if available, or no regions if the target does not support.

delete mem nums...

Remove memory regions *nums...* from the list of regions monitored by GDB.

disable mem nums...

Disable monitoring of memory regions *nums...* A disabled memory region is not forgotten. It may be enabled again later.

enable mem nums...

Enable monitoring of memory regions *nums...*

info mem Print a table of all defined memory regions, with the following columns for each region:

Memory Region Number

Enabled or Disabled.

Enabled memory regions are marked with 'y'. Disabled memory regions are marked with 'n'.

Lo Address

The address defining the inclusive lower bound of the memory region.

Hi Address

The address defining the exclusive upper bound of the memory region.

Attributes The list of attributes set for this memory region.

10.18.1 Attributes

10.18.1.1 Memory Access Mode

The access mode attributes set whether GDB may make read or write accesses to a memory region.

While these attributes prevent GDB from performing invalid memory accesses, they do nothing to prevent the target system, I/O DMA, etc. from accessing memory.

ro	Memory is read only.
wo	Memory is write only.
rw	Memory is read/write. This is the default.

10.18.1.2 Memory Access Size

The access size attribute tells GDB to use specific sized accesses in the memory region. Often memory mapped device registers require specific sized accesses. If no access size attribute is specified, GDB may use accesses of any size.

8	Use 8 bit memory accesses.
16	Use 16 bit memory accesses.
32	Use 32 bit memory accesses.
64	Use 64 bit memory accesses.

10.18.1.3 Data Cache

The data cache attributes set whether GDB will cache target memory. While this generally improves performance by reducing debug protocol overhead, it can lead to incorrect results because GDB does not know about volatile variables or memory mapped device registers.

cache	Enable GDB to cache target memory.
nocache	Disable GDB from caching target memory. This is the default.

10.18.2 Memory Access Checking

GDB can be instructed to refuse accesses to memory that is not explicitly described. This can be useful if accessing such regions has undesired effects for a specific target, or to provide better error checking. The following commands control this behavior.

set mem inaccessible-by-default [on|off]

If **on** is specified, make GDB treat memory not explicitly described by the memory ranges as non-existent and refuse accesses to such memory. The checks are only performed if there's at least one memory range defined. If **off** is specified, make GDB treat the memory not explicitly described by the memory ranges as RAM. The default value is **on**.

show mem inaccessible-by-default

Show the current handling of accesses to unknown memory.

10.19 Copy Between Memory and a File

You can use the commands **dump**, **append**, and **restore** to copy data between target memory and a file. The **dump** and **append** commands write data to a file, and the **restore** command reads data from a file back into the inferior's memory. Files may be in binary, Motorola S-record, Intel hex, Tektronix Hex, or Verilog Hex format; however, GDB can only append to binary files, and cannot read from Verilog Hex files.

```
dump [format] memory filename start_addr end_addr
```

```
dump [format] value filename expr
```

Dump the contents of memory from *start_addr* to *end_addr*, or the value of *expr*, to *filename* in the given format.

The *format* parameter may be any one of:

binary	Raw binary form.
ihex	Intel hex format.
srec	Motorola S-record format.
tekhex	Tektronix Hex format.
verilog	Verilog Hex format.

GDB uses the same definitions of these formats as the GNU binary utilities, like 'objdump' and 'objcopy'. If *format* is omitted, GDB dumps the data in raw binary form.

```
append [binary] memory filename start_addr end_addr
```

```
append [binary] value filename expr
```

Append the contents of memory from *start_addr* to *end_addr*, or the value of *expr*, to the file *filename*, in raw binary form. (GDB can only append data to files in raw binary form.)

```
restore filename [binary] bias start end
```

Restore the contents of file *filename* into memory. The **restore** command can automatically recognize any known BFD file format, except for raw binary. To restore a raw binary file you must specify the optional keyword **binary** after the filename.

If *bias* is non-zero, its value will be added to the addresses contained in the file. Binary files always start at address zero, so they will be restored at address *bias*. Other bfd files have a built-in location; they will be restored at offset *bias* from that location.

If *start* and/or *end* are non-zero, then only data between file offset *start* and file offset *end* will be restored. These offsets are relative to the addresses in the file, before the *bias* argument is applied.

10.20 How to Produce a Core File from Your Program

A *core file* or *core dump* is a file that records the memory image of a running process and its process status (register values etc.). Its primary use is post-mortem debugging of a program that crashed while it ran outside a debugger. A program that crashes automatically

produces a core file, unless this feature is disabled by the user. See Section 18.1 [Files], page 293, for information on invoking GDB in the post-mortem debugging mode.

Occasionally, you may wish to produce a core file of the program you are debugging in order to preserve a snapshot of its state. GDB has a special command for that.

generate-core-file [*file*]

gcore [*file*]

Produce a core dump of the inferior process. The optional argument *file* specifies the file name where to put the core dump. If not specified, the file name defaults to **core.*pid***, where *pid* is the inferior process ID.

If supported by the filesystem where the core is written to, GDB generates a sparse core dump file.

Note that this command is implemented only for some systems (as of this writing, GNU/Linux, FreeBSD, Solaris, and S390).

On GNU/Linux, this command can take into account the value of the file **/proc/*pid*/coredump_filter** when generating the core dump (see [set use-coredump-filter], page 189), and by default honors the **VM_DONTDUMP** flag for mappings where it is present in the file **/proc/*pid*/smaps** (see [set dump-excluded-mappings], page 189).

set use-coredump-filter on

set use-coredump-filter off

Enable or disable the use of the file **/proc/*pid*/coredump_filter** when generating core dump files. This file is used by the Linux kernel to decide what types of memory mappings will be dumped or ignored when generating a core dump file. *pid* is the process ID of a currently running process.

To make use of this feature, you have to write in the **/proc/*pid*/coredump_filter** file a value, in hexadecimal, which is a bit mask representing the memory mapping types. If a bit is set in the bit mask, then the memory mappings of the corresponding types will be dumped; otherwise, they will be ignored. This configuration is inherited by child processes. For more information about the bits that can be set in the **/proc/*pid*/coredump_filter** file, please refer to the manpage of **core(5)**.

By default, this option is **on**. If this option is turned **off**, GDB does not read the **coredump_filter** file and instead uses the same default value as the Linux kernel in order to decide which pages will be dumped in the core dump file. This value is currently **0x33**, which means that bits 0 (anonymous private mappings), 1 (anonymous shared mappings), 4 (ELF headers) and 5 (private huge pages) are active. This will cause these memory mappings to be dumped automatically.

set dump-excluded-mappings on

set dump-excluded-mappings off

If **on** is specified, GDB will dump memory mappings marked with the **VM_DONTDUMP** flag. This flag is represented in the file **/proc/*pid*/smaps** with the acronym **dd**.

The default value is **off**.

10.21 Character Sets

If the program you are debugging uses a different character set to represent characters and strings than the one GDB uses itself, GDB can automatically translate between the character sets for you. The character set GDB uses we call the *host character set*; the one the inferior program uses we call the *target character set*.

For example, if you are running GDB on a GNU/Linux system, which uses the ISO Latin 1 character set, but you are using GDB's remote protocol (see Chapter 20 [Remote Debugging], page 317) to debug a program running on an IBM mainframe, which uses the EBCDIC character set, then the host character set is Latin-1, and the target character set is EBCDIC. If you give GDB the command `set target-charset EBCDIC-US`, then GDB translates between EBCDIC and Latin 1 as you print character or string values, or use character and string literals in expressions.

GDB has no way to automatically recognize which character set the inferior program uses; you must tell it, using the `set target-charset` command, described below.

Here are the commands for controlling GDB's character set support:

`set target-charset charset`

Set the current target character set to *charset*. To display the list of supported target character sets, type `set target-charset TABTAB`.

`set host-charset charset`

Set the current host character set to *charset*.

By default, GDB uses a host character set appropriate to the system it is running on; you can override that default using the `set host-charset` command. On some systems, GDB cannot automatically determine the appropriate host character set. In this case, GDB uses 'UTF-8'.

GDB can only use certain character sets as its host character set. If you type `set host-charset TABTAB`, GDB will list the host character sets it supports.

`set charset charset`

Set the current host and target character sets to *charset*. As above, if you type `set charset TABTAB`, GDB will list the names of the character sets that can be used for both host and target.

`show charset`

Show the names of the current host and target character sets.

`show host-charset`

Show the name of the current host character set.

`show target-charset`

Show the name of the current target character set.

`set target-wide-charset charset`

Set the current target's wide character set to *charset*. This is the character set used by the target's `wchar_t` type. To display the list of supported wide character sets, type `set target-wide-charset TABTAB`.

`show target-wide-charset`

Show the name of the current target's wide character set.

Here is an example of GDB's character set support in action. Assume that the following source code has been placed in the file `charset-test.c`:

```
#include <stdio.h>

char ascii_hello[]
    = {72, 101, 108, 108, 111, 44, 32, 119,
       111, 114, 108, 100, 33, 10, 0};
char ibm1047_hello[]
    = {200, 133, 147, 147, 150, 107, 64, 166,
       150, 153, 147, 132, 90, 37, 0};

main ()
{
    printf ("Hello, world!\n");
}
```

In this program, `ascii_hello` and `ibm1047_hello` are arrays containing the string 'Hello, world!' followed by a newline, encoded in the ASCII and IBM1047 character sets.

We compile the program, and invoke the debugger on it:

```
$ gcc -g charset-test.c -o charset-test
$ gdb -nw charset-test
GNU gdb 2001-12-19-cvs
Copyright 2001 Free Software Foundation, Inc.
...
(gdb)
```

We can use the `show charset` command to see what character sets GDB is currently using to interpret and display characters and strings:

```
(gdb) show charset
The current host and target character set is 'ISO-8859-1'.
(gdb)
```

For the sake of printing this manual, let's use ASCII as our initial character set:

```
(gdb) set charset ASCII
(gdb) show charset
The current host and target character set is 'ASCII'.
(gdb)
```

Let's assume that ASCII is indeed the correct character set for our host system — in other words, let's assume that if GDB prints characters using the ASCII character set, our terminal will display them properly. Since our current target character set is also ASCII, the contents of `ascii_hello` print legibly:

```
(gdb) print ascii_hello
$1 = 0x401698 "Hello, world!\n"
(gdb) print ascii_hello[0]
$2 = 72 'H'
(gdb)
```

GDB uses the target character set for character and string literals you use in expressions:

```
(gdb) print '+'
$3 = 43 '+'
(gdb)
```

The ASCII character set uses the number 43 to encode the '+' character.

GDB relies on the user to tell it which character set the target program uses. If we print `ibm1047_hello` while our target character set is still ASCII, we get jibberish:

```
(gdb) print ibm1047_hello
```

```
$4 = 0x4016a8 "\310\205\223\223\226k@\246\226\231\223\204Z%"
(gdb) print ibm1047_hello[0]
$5 = 200 '\310'
(gdb)
```

If we invoke the `set target-charset` followed by `TABTAB`, GDB tells us the character sets it supports:

```
(gdb) set target-charset
ASCII      EBCDIC-US  IBM1047     ISO-8859-1
(gdb) set target-charset
```

We can select `IBM1047` as our target character set, and examine the program's strings again. Now the ASCII string is wrong, but GDB translates the contents of `ibm1047_hello` from the target character set, `IBM1047`, to the host character set, `ASCII`, and they display correctly:

```
(gdb) set target-charset IBM1047
(gdb) show charset
The current host character set is 'ASCII'.
The current target character set is 'IBM1047'.
(gdb) print ascii_hello
$6 = 0x401698 "\110\145%?\054\040\167?\162%\144\041\012"
(gdb) print ascii_hello[0]
$7 = 72 '\110'
(gdb) print ibm1047_hello
$8 = 0x4016a8 "Hello, world!\n"
(gdb) print ibm1047_hello[0]
$9 = 200 'H'
(gdb)
```

As above, GDB uses the target character set for character and string literals you use in expressions:

```
(gdb) print '+'
$10 = 78 '+'
(gdb)
```

The `IBM1047` character set uses the number 78 to encode the `'+'` character.

10.22 Caching Data of Targets

GDB caches data exchanged between the debugger and a target. Each cache is associated with the address space of the inferior. See Section 4.9 [Inferiors Connections and Programs], page 43, about inferior and address space. Such caching generally improves performance in remote debugging (see Chapter 20 [Remote Debugging], page 317), because it reduces the overhead of the remote protocol by bundling memory reads and writes into large chunks. Unfortunately, simply caching everything would lead to incorrect results, since GDB does not necessarily know anything about volatile values, memory-mapped I/O addresses, etc. Furthermore, in non-stop mode (see Section 5.5.2 [Non-Stop Mode], page 97) memory can be changed *while* a `gdb` command is executing. Therefore, by default, GDB only caches data known to be on the stack³ or in the code segment. Other regions of memory can be explicitly marked as cacheable; see Section 10.18 [Memory Region Attributes], page 186.

³ In non-stop mode, it is moderately rare for a running thread to modify the stack of a stopped thread in a way that would interfere with a backtrace, and caching of stack reads provides a significant speed up of remote backtraces.

set remotecache on

set remotecache off

This option no longer does anything; it exists for compatibility with old scripts.

show remotecache

Show the current state of the obsolete remotecache flag.

set stack-cache on

set stack-cache off

Enable or disable caching of stack accesses. When **on**, use caching. By default, this option is **on**.

show stack-cache

Show the current state of data caching for memory accesses.

set code-cache on

set code-cache off

Enable or disable caching of code segment accesses. When **on**, use caching. By default, this option is **on**. This improves performance of disassembly in remote debugging.

show code-cache

Show the current state of target memory cache for code segment accesses.

info dcache [line]

Print the information about the performance of data cache of the current inferior's address space. The information displayed includes the dcache width and depth, and for each cache line, its number, address, and how many times it was referenced. This command is useful for debugging the data cache operation.

If a line number is specified, the contents of that line will be printed in hex.

set dcache size size

Set maximum number of entries in dcache (dcache depth above).

set dcache line-size line-size

Set number of bytes each dcache entry caches (dcache width above). Must be a power of 2.

show dcache size

Show maximum number of dcache entries. See Section 10.22 [Caching Target Data], page 192.

show dcache line-size

Show default size of dcache lines.

maint flush dcache

Flush the contents (if any) of the dcache. This maintainer command is useful when debugging the dcache implementation.

10.23 Search Memory

Memory can be searched for a particular sequence of bytes with the **find** command.

```
find [/sn] start_addr, +len, val1 [, val2, ...]
find [/sn] start_addr, end_addr, val1 [, val2, ...]
```

Search memory for the sequence of bytes specified by *val1*, *val2*, etc. The search begins at address *start_addr* and continues for either *len* bytes or through to *end_addr* inclusive.

s and *n* are optional parameters. They may be specified in either order, apart or together.

s, search query size

The size of each search query value.

b	bytes
h	halfwords (two bytes)
w	words (four bytes)
g	giant words (eight bytes)

All values are interpreted in the current language. This means, for example, that if the current source language is C/C++ then searching for the string “hello” includes the trailing ‘\0’. The null terminator can be removed from searching by using casts, e.g.: ‘{char[5]}“hello”’.

If the value size is not specified, it is taken from the value’s type in the current language. This is useful when one wants to specify the search pattern as a mixture of types. Note that this means, for example, that in the case of C-like languages a search for an untyped 0x42 will search for ‘(int) 0x42’ which is typically four bytes.

n, maximum number of finds

The maximum number of matches to print. The default is to print all finds.

You can use strings as search values. Quote them with double-quotes (“). The string value is copied into the search pattern byte by byte, regardless of the endianness of the target and the size specification.

The address of each match found is printed as well as a count of the number of matches found.

The address of the last value found is stored in convenience variable ‘\$_’. A count of the number of matches is stored in ‘\$numfound’.

For example, if stopped at the **printf** in this function:

```
void
hello ()
{
    static char hello[] = "hello-hello";
    static struct { char c; short s; int i; }
        __attribute__((packed)) mixed
        = { 'c', 0x1234, 0x87654321 };
    printf ("%s\n", hello);
}
```

you get during debugging:

```
(gdb) find &hello[0], +sizeof(hello), "hello"
0x804956d <hello.1620+6>
1 pattern found
```

```

(gdb) find &hello[0], +sizeof(hello), 'h', 'e', 'l', 'l', 'o'
0x8049567 <hello.1620>
0x804956d <hello.1620+6>
2 patterns found.
(gdb) find &hello[0], +sizeof(hello), {char[5]}"hello"
0x8049567 <hello.1620>
0x804956d <hello.1620+6>
2 patterns found.
(gdb) find /b1 &hello[0], +sizeof(hello), 'h', 0x65, 'l'
0x8049567 <hello.1620>
1 pattern found
(gdb) find &mixed, +sizeof(mixed), (char) 'c', (short) 0x1234, (int) 0x87654321
0x8049560 <mixed.1625>
1 pattern found
(gdb) print $numfound
$1 = 1
(gdb) print $_
$2 = (void *) 0x8049560

```

10.24 Value Sizes

Whenever GDB prints a value memory will be allocated within GDB to hold the contents of the value. It is possible in some languages with dynamic typing systems, that an invalid program may indicate a value that is incorrectly large, this in turn may cause GDB to try and allocate an overly large amount of memory.

set max-value-size bytes

set max-value-size unlimited

Set the maximum size of memory that GDB will allocate for the contents of a value to *bytes*, trying to display a value that requires more memory than that will result in an error.

Setting this variable does not effect values that have already been allocated within GDB, only future allocations.

There's a minimum size that **max-value-size** can be set to in order that GDB can still operate correctly, this minimum is currently 16 bytes.

The limit applies to the results of some subexpressions as well as to complete expressions. For example, an expression denoting a simple integer component, such as *x.y.z*, may fail if the size of *x.y* is dynamic and exceeds *bytes*. On the other hand, GDB is sometimes clever; the expression *A[i]*, where *A* is an array variable with non-constant size, will generally succeed regardless of the bounds on *A*, as long as the component size is less than *bytes*.

The default value of **max-value-size** is currently 64k.

show max-value-size

Show the maximum size of memory, in bytes, that GDB will allocate for the contents of a value.

11 Debugging Optimized Code

Almost all compilers support optimization. With optimization disabled, the compiler generates assembly code that corresponds directly to your source code, in a simplistic way. As the compiler applies more powerful optimizations, the generated assembly code diverges from your original source code. With help from debugging information generated by the compiler, GDB can map from the running program back to constructs from your original source.

GDB is more accurate with optimization disabled. If you can recompile without optimization, it is easier to follow the progress of your program during debugging. But, there are many cases where you may need to debug an optimized version.

When you debug a program compiled with ‘`-g -O`’, remember that the optimizer has rearranged your code; the debugger shows you what is really there. Do not be too surprised when the execution path does not exactly match your source file! An extreme example: if you define a variable, but never use it, GDB never sees that variable—because the compiler optimizes it out of existence.

Some things do not work as well with ‘`-g -O`’ as with just ‘`-g`’, particularly on machines with instruction scheduling. If in doubt, recompile with ‘`-g`’ alone, and if this fixes the problem, please report it to us as a bug (including a test case!). See Section 10.3 [Variables], page 149, for more information about debugging optimized code.

11.1 Inline Functions

Inlining is an optimization that inserts a copy of the function body directly at each call site, instead of jumping to a shared routine. GDB displays inlined functions just like non-inlined functions. They appear in backtraces. You can view their arguments and local variables, step into them with `step`, skip them with `next`, and escape from them with `finish`. You can check whether a function was inlined by using the `info frame` command.

For GDB to support inlined functions, the compiler must record information about inlining in the debug information — GCC using the DWARF 2 format does this, and several other compilers do also. GDB only supports inlined functions when using DWARF 2. Versions of GCC before 4.1 do not emit two required attributes (‘`DW_AT_call_file`’ and ‘`DW_AT_call_line`’); GDB does not display inlined function calls with earlier versions of GCC. It instead displays the arguments and local variables of inlined functions as local variables in the caller.

The body of an inlined function is directly included at its call site; unlike a non-inlined function, there are no instructions devoted to the call. GDB still pretends that the call site and the start of the inlined function are different instructions. Stepping to the call site shows the call site, and then stepping again shows the first line of the inlined function, even though no additional instructions are executed.

This makes source-level debugging much clearer; you can see both the context of the call and then the effect of the call. Only stepping by a single instruction using `stepi` or `nexti` does not do this; single instruction steps always show the inlined body.

There are some ways that GDB does not pretend that inlined function calls are the same as normal calls:

- Setting breakpoints at the call site of an inlined function may not work, because the call site does not contain any code. GDB may incorrectly move the breakpoint to the next line of the enclosing function, after the call. This limitation will be removed in a future version of GDB; until then, set a breakpoint on an earlier line or inside the inlined function instead.
- GDB cannot locate the return value of inlined calls after using the `finish` command. This is a limitation of compiler-generated debugging information; after `finish`, you can step to the next line and print a variable where your program stored the return value.

11.2 Tail Call Frames

Function B can call function C in its very last statement. In unoptimized compilation the call of C is immediately followed by return instruction at the end of B code. Optimizing compiler may replace the call and return in function B into one jump to function C instead. Such use of a jump instruction is called *tail call*.

During execution of function C, there will be no indication in the function call stack frames that it was tail-called from B. If function A regularly calls function B which tail-calls function C, then GDB will see A as the caller of C. However, in some cases GDB can determine that C was tail-called from B, and it will then create fictitious call frame for that, with the return address set up as if B called C normally.

This functionality is currently supported only by DWARF 2 debugging format and the compiler has to produce ‘DW_TAG_call_site’ tags. With GCC, you need to specify `-O -g` during compilation, to get this information.

`info frame` command (see Section 8.4 [Frame Info], page 121) will indicate the tail call frame kind by text `tail call frame` such as in this sample GDB output:

```
(gdb) x/i $pc - 2
0x40066b <b(int, double)+11>: jmp 0x400640 <c(int, double)>
(gdb) info frame
Stack level 1, frame at 0x7fffffffda30:
 rip = 0x40066d in b (amd64-entry-value.cc:59); saved rip 0x4004c5
 tail call frame, caller of frame at 0x7fffffffda30
 source language c++.
 Arglist at unknown address.
 Locals at unknown address, Previous frame's sp is 0x7fffffffda30
```

The detection of all the possible code path executions can find them ambiguous. There is no execution history stored (possible Chapter 6 [Reverse Execution], page 103, is never used for this purpose) and the last known caller could have reached the known callee by multiple different jump sequences. In such case GDB still tries to show at least all the unambiguous top tail callers and all the unambiguous bottom tail callees, if any.

set debug entry-values

When set to on, enables printing of analysis messages for both frame argument values at function entry and tail calls. It will show all the possible valid tail calls code paths it has considered. It will also print the intersection of them with the final unambiguous (possibly partial or even empty) code path result.

show debug entry-values

Show the current state of analysis messages printing for both frame argument values at function entry and tail calls.

The analysis messages for tail calls can for example show why the virtual tail call frame for function `c` has not been recognized (due to the indirect reference by variable `x`):

```
static void __attribute__((noinline, noclone)) c (void);
void (*x) (void) = c;
static void __attribute__((noinline, noclone)) a (void) { x++; }
static void __attribute__((noinline, noclone)) c (void) { a (); }
int main (void) { x (); return 0; }
```

Breakpoint 1, DW_OP_entry_value resolving cannot find
 DW_TAG_call_site 0x40039a in main
 a () at t.c:3
 3 static void __attribute__((noinline, noclone)) a (void) { x++; }
 (gdb) bt
 #0 a () at t.c:3
 #1 0x00000000040039a in main () at t.c:5

Another possibility is an ambiguous virtual tail call frames resolution:

```
int i;
static void __attribute__((noinline, noclone)) f (void) { i++; }
static void __attribute__((noinline, noclone)) e (void) { f (); }
static void __attribute__((noinline, noclone)) d (void) { f (); }
static void __attribute__((noinline, noclone)) c (void) { d (); }
static void __attribute__((noinline, noclone)) b (void)
{ if (i) c (); else e (); }
static void __attribute__((noinline, noclone)) a (void) { b (); }
int main (void) { a (); return 0; }
```

tailcall: initial: 0x4004d2(a) 0x4004ce(b) 0x4004b2(c) 0x4004a2(d)
 tailcall: compare: 0x4004d2(a) 0x4004cc(b) 0x400492(e)
 tailcall: reduced: 0x4004d2(a) |
 (gdb) bt
 #0 f () at t.c:2
 #1 0x0000000004004d2 in a () at t.c:8
 #2 0x000000000400395 in main () at t.c:9

Frames `#0` and `#2` are real, `#1` is a virtual tail call frame. The code can have possible execution paths `main->a->b->c->d->f` or `main->a->b->e->f`, GDB cannot find which one from the inferior state.

initial: state shows some random possible calling sequence GDB has found. It then finds another possible calling sequence - that one is prefixed by **compare:**. The non-ambiguous intersection of these two is printed as the **reduced:** calling sequence. That one could have many further **compare:** and **reduced:** statements as long as there remain any non-ambiguous sequence entries.

For the frame of function `b` in both cases there are different possible `$pc` values (`0x4004cc` or `0x4004ce`), therefore this frame is also ambiguous. The only non-ambiguous frame is the one for function `a`, therefore this one is displayed to the user while the ambiguous frames are omitted.

There can be also reasons why printing of frame argument values at function entry may fail:

```
int v;
```

```
static void __attribute__((noinline, noclone)) c (int i) { v++; }
static void __attribute__((noinline, noclone)) a (int i);
static void __attribute__((noinline, noclone)) b (int i) { a (i); }
static void __attribute__((noinline, noclone)) a (int i)
{ if (i) b (i - 1); else c (0); }
int main (void) { a (5); return 0; }
```

```
(gdb) bt
#0  c (i=i@entry=0) at t.c:2
#1  0x000000000400428 in a (DW_OP_entry_value resolving has found
function "a" at 0x400420 can call itself via tail calls
i=<optimized out>) at t.c:6
#2  0x00000000040036e in main () at t.c:7
```

GDB cannot find out from the inferior state if and how many times did function **a** call itself (via function **b**) as these calls would be tail calls. Such tail calls would modify the **i** variable, therefore GDB cannot be sure the value it knows would be right - GDB prints **<optimized out>** instead.

12 C Preprocessor Macros

Some languages, such as C and C++, provide a way to define and invoke “preprocessor macros” which expand into strings of tokens. GDB can evaluate expressions containing macro invocations, show the result of macro expansion, and show a macro’s definition, including where it was defined.

You may need to compile your program specially to provide GDB with information about preprocessor macros. Most compilers do not include macros in their debugging information, even when you compile with the `-g` flag. See Section 4.1 [Compilation], page 33.

A program may define a macro at one point, remove that definition later, and then provide a different definition after that. Thus, at different points in the program, a macro may have different definitions, or have no definition at all. If there is a current stack frame, GDB uses the macros in scope at that frame’s source code line. Otherwise, GDB uses the macros in scope at the current listing location; see Section 9.1 [List], page 127.

Whenever GDB evaluates an expression, it always expands any macro invocations present in the expression. GDB also provides the following commands for working with macros explicitly.

`macro expand expression`

`macro exp expression`

Show the results of expanding all preprocessor macro invocations in *expression*. Since GDB simply expands macros, but does not parse the result, *expression* need not be a valid expression; it can be any string of tokens.

`macro expand-once expression`

`macro exp1 expression`

(This command is not yet implemented.) Show the results of expanding those preprocessor macro invocations that appear explicitly in *expression*. Macro invocations appearing in that expansion are left unchanged. This command allows you to see the effect of a particular macro more clearly, without being confused by further expansions. Since GDB simply expands macros, but does not parse the result, *expression* need not be a valid expression; it can be any string of tokens.

`info macro [-a|-all] [--] macro`

Show the current definition or all definitions of the named *macro*, and describe the source location or compiler command-line where that definition was established. The optional double dash is to signify the end of argument processing and the beginning of *macro* for non C-like macros where the macro may begin with a hyphen.

`info macros locspec`

Show all macro definitions that are in effect at the source line of the code location that results from resolving *locspec*, and describe the source location or compiler command-line where those definitions were established.

`macro define macro replacement-list`

`macro define macro(arglist) replacement-list`

Introduce a definition for a preprocessor macro named *macro*, invocations of which are replaced by the tokens given in *replacement-list*. The first form of

this command defines an “object-like” macro, which takes no arguments; the second form defines a “function-like” macro, which takes the arguments given in *arglist*.

A definition introduced by this command is in scope in every expression evaluated in GDB, until it is removed with the **macro undef** command, described below. The definition overrides all definitions for *macro* present in the program being debugged, as well as any previous user-supplied definition.

macro undef *macro*

Remove any user-supplied definition for the macro named *macro*. This command only affects definitions provided with the **macro define** command, described above; it cannot remove definitions present in the program being debugged.

macro list

List all the macros defined using the **macro define** command.

Here is a transcript showing the above commands in action. First, we show our source files:

```
$ cat sample.c
#include <stdio.h>
#include "sample.h"

#define M 42
#define ADD(x) (M + x)

main ()
{
#define N 28
    printf ("Hello, world!\n");
#undef N
    printf ("We're so creative.\n");
#define N 1729
    printf ("Goodbye, world!\n");
}
$ cat sample.h
#define Q <
$
```

Now, we compile the program using the GNU C compiler, GCC. We pass the **-gdwarf-2**¹ and **-g3** flags to ensure the compiler includes information about preprocessor macros in the debugging information.

```
$ gcc -gdwarf-2 -g3 sample.c -o sample
$
```

Now, we start GDB on our sample program:

```
$ gdb -nw sample
GNU gdb 2002-05-06-cvs
Copyright 2002 Free Software Foundation, Inc.
GDB is free software, ...
(gdb)
```

¹ This is the minimum. Recent versions of GCC support **-gdwarf-3** and **-gdwarf-4**; we recommend always choosing the most recent version of DWARF.

We can expand macros and examine their definitions, even when the program is not running. GDB uses the current listing position to decide which macro definitions are in scope:

```
(gdb) list main
3
4     #define M 42
5     #define ADD(x) (M + x)
6
7     main ()
8     {
9     #define N 28
10    printf ("Hello, world!\n");
11    #undef N
12    printf ("We're so creative.\n");
(gdb) info macro ADD
Defined at /home/jimb/gdb/macros/play/sample.c:5
#define ADD(x) (M + x)
(gdb) info macro Q
Defined at /home/jimb/gdb/macros/play/sample.h:1
included at /home/jimb/gdb/macros/play/sample.c:2
#define Q <
(gdb) macro expand ADD(1)
expands to: (42 + 1)
(gdb) macro expand-once ADD(1)
expands to: once (M + 1)
(gdb)
```

In the example above, note that `macro expand-once` expands only the macro invocation explicit in the original text — the invocation of `ADD` — but does not expand the invocation of the macro `M`, which was introduced by `ADD`.

Once the program is running, GDB uses the macro definitions in force at the source line of the current stack frame:

```
(gdb) break main
Breakpoint 1 at 0x8048370: file sample.c, line 10.
(gdb) run
Starting program: /home/jimb/gdb/macros/play/sample

Breakpoint 1, main () at sample.c:10
10    printf ("Hello, world!\n");
(gdb)
```

At line 10, the definition of the macro `N` at line 9 is in force:

```
(gdb) info macro N
Defined at /home/jimb/gdb/macros/play/sample.c:9
#define N 28
(gdb) macro expand N Q M
expands to: 28 < 42
(gdb) print N Q M
$1 = 1
(gdb)
```

As we step over directives that remove `N`'s definition, and then give it a new definition, GDB finds the definition (or lack thereof) in force at each point:

```
(gdb) next
Hello, world!
12    printf ("We're so creative.\n");
(gdb) info macro N
```

```
The symbol 'N' has no definition as a C/C++ preprocessor macro
at /home/jimb/gdb/macros/play/sample.c:12
(gdb) next
We're so creative.
14      printf ("Goodbye, world!\n");
(gdb) info macro N
Defined at /home/jimb/gdb/macros/play/sample.c:13
#define N 1729
(gdb) macro expand N Q M
expands to: 1729 < 42
(gdb) print N Q M
$2 = 0
(gdb)
```

In addition to source files, macros can be defined on the compilation command line using the `-Dname=value` syntax. For macros defined in such a way, GDB displays the location of their definition as line zero of the source file submitted to the compiler.

```
(gdb) info macro __STDC__
Defined at /home/jimb/gdb/macros/play/sample.c:0
-D__STDC__=1
(gdb)
```

13 Tracepoints

In some applications, it is not feasible for the debugger to interrupt the program's execution long enough for the developer to learn anything helpful about its behavior. If the program's correctness depends on its real-time behavior, delays introduced by a debugger might cause the program to change its behavior drastically, or perhaps fail, even when the code itself is correct. It is useful to be able to observe the program's behavior without interrupting it.

Using GDB's `trace` and `collect` commands, you can specify locations in the program, called *tracepoints*, and arbitrary expressions to evaluate when those tracepoints are reached. Later, using the `tfind` command, you can examine the values those expressions had when the program hit the tracepoints. The expressions may also denote objects in memory—structures or arrays, for example—whose values GDB should record; while visiting a particular tracepoint, you may inspect those objects as if they were in memory at that moment. However, because GDB records these values without interacting with you, it can do so quickly and unobtrusively, hopefully not disturbing the program's behavior.

The tracepoint facility is currently available only for remote targets. See Chapter 19 [Targets], page 313. In addition, your remote target must know how to collect trace data. This functionality is implemented in the remote stub; however, none of the stubs distributed with GDB support tracepoints as of this writing. The format of the remote packets used to implement tracepoints are described in Section E.7 [Tracepoint Packets], page 840.

It is also possible to get trace data from a file, in a manner reminiscent of corefiles; you specify the filename, and use `tfind` to search through the file. See Section 13.4 [Trace Files], page 221, for more details.

This chapter describes the tracepoint commands and features.

13.1 Commands to Set Tracepoints

Before running such a *trace experiment*, an arbitrary number of tracepoints can be set. A tracepoint is actually a special type of breakpoint (see Section 5.1.1 [Set Breaks], page 60), so you can manipulate it using standard breakpoint commands. For instance, as with breakpoints, tracepoint numbers are successive integers starting from one, and many of the commands associated with tracepoints take the tracepoint number as their argument, to identify which tracepoint to work on.

For each tracepoint, you can specify, in advance, some arbitrary set of data that you want the target to collect in the trace buffer when it hits that tracepoint. The collected data can include registers, local variables, or global data. Later, you can use GDB commands to examine the values these data had at the time the tracepoint was hit.

Tracepoints do not support every breakpoint feature. Ignore counts on tracepoints have no effect, and tracepoints cannot run GDB commands when they are hit. Tracepoints may not be thread-specific either.

Some targets may support *fast tracepoints*, which are inserted in a different way (such as with a jump instead of a trap), that is faster but possibly restricted in where they may be installed.

Regular and fast tracepoints are dynamic tracing facilities, meaning that they can be used to insert tracepoints at (almost) any location in the target. Some targets may also support controlling *static tracepoints* from GDB. With static tracing, a set of instrumentation

points, also known as *markers*, are embedded in the target program, and can be activated or deactivated by name or address. These are usually placed at locations which facilitate investigating what the target is actually doing. GDB's support for static tracing includes being able to list instrumentation points, and attach them with GDB defined high level tracepoints that expose the whole range of convenience of GDB's tracepoints support. Namely, support for collecting registers values and values of global or local (to the instrumentation point) variables; tracepoint conditions and trace state variables. The act of installing a GDB static tracepoint on an instrumentation point, or marker, is referred to as *probing* a static tracepoint marker.

`gdbserver` supports tracepoints on some target systems. See Section 20.3 [Tracepoints support in `gdbserver`], page 322.

This section describes commands to set tracepoints and associated conditions and actions.

13.1.1 Create and Delete Tracepoints

`trace locspec`

The `trace` command is very similar to the `break` command. Its argument *locspec* can be any valid location specification. See Section 9.2 [Location Specifications], page 128. The `trace` command defines a tracepoint, which is a point in the target program where the debugger will briefly stop, collect some data, and then allow the program to continue. Setting a tracepoint or changing its actions takes effect immediately if the remote stub supports the '`InstallInTrace`' feature (see [install tracepoint in tracing], page 830). If remote stub doesn't support the '`InstallInTrace`' feature, all these changes don't take effect until the next `tstart` command, and once a trace experiment is running, further changes will not have any effect until the next trace experiment starts. In addition, GDB supports *pending tracepoints*—tracepoints whose address is not yet resolved. (This is similar to pending breakpoints.) Pending tracepoints are not downloaded to the target and not installed until they are resolved. The resolution of pending tracepoints requires GDB support—when debugging with the remote target, and GDB disconnects from the remote stub (see [disconnected tracing], page 214), pending tracepoints can not be resolved (and downloaded to the remote stub) while GDB is disconnected.

Here are some examples of using the `trace` command:

```
(gdb) trace foo.c:121      // a source file and line number

(gdb) trace +2             // 2 lines forward

(gdb) trace my_function    // first source line of function

(gdb) trace *my_function   // EXACT start address of function

(gdb) trace *0x2117c4      // an address
```

You can abbreviate `trace` as `tr`.

`trace locspec if cond`

Set a tracepoint with condition *cond*; evaluate the expression *cond* each time the tracepoint is reached, and collect data only if the value is nonzero—that is,

if *cond* evaluates as true. See Section 13.1.4 [Tracepoint Conditions], page 209, for more information on tracepoint conditions.

ftrace locspec [if cond]

The **ftrace** command sets a fast tracepoint. For targets that support them, fast tracepoints will use a more efficient but possibly less general technique to trigger data collection, such as a jump instruction instead of a trap, or some sort of hardware support. It may not be possible to create a fast tracepoint at the desired location, in which case the command will exit with an explanatory message.

GDB handles arguments to **ftrace** exactly as for **trace**.

On 32-bit x86-architecture systems, fast tracepoints normally need to be placed at an instruction that is 5 bytes or longer, but can be placed at 4-byte instructions if the low 64K of memory of the target program is available to install trampolines. Some Unix-type systems, such as GNU/Linux, exclude low addresses from the program's address space; but for instance with the Linux kernel it is possible to let GDB use this area by doing a **sysctl** command to set the **mmap_min_addr** kernel parameter, as in

```
sudo sysctl -w vm.mmap_min_addr=32768
```

which sets the low address to 32K, which leaves plenty of room for trampolines. The minimum address should be set to a page boundary.

strace [locspec | -m marker] [if cond]

The **strace** command sets a static tracepoint. For targets that support it, setting a static tracepoint probes a static instrumentation point, or marker, found at the code locations that result from resolving *locspec*. It may not be possible to set a static tracepoint at the desired code location, in which case the command will exit with an explanatory message.

GDB handles arguments to **strace** exactly as for **trace**, with the addition that the user can also specify **-m marker** instead of a location spec. This probes the marker identified by the *marker* string identifier. This identifier depends on the static tracepoint backend library your program is using. You can find all the marker identifiers in the 'ID' field of the **info static-tracepoint-markers** command output. See Section 13.1.8 [Listing Static Tracepoint Markers], page 213. For example, in the following small program using the UST tracing engine:

```
main ()
{
    trace_mark(ust, bar33, "str %s", "FOOBAZ");
}
```

the marker id is composed of joining the first two arguments to the **trace_mark** call with a slash, which translates to:

```
(gdb) info static-tracepoint-markers
Cnt Enb ID          Address          What
1   n  ust/bar33  0x0000000000400ddc in main at stexample.c:22
      Data: "str %s"
[etc...]
```

so you may probe the marker above with:

```
(gdb) strace -m ust/bar33
```

Static tracepoints accept an extra collect action — `collect $_sdata`. This collects arbitrary user data passed in the probe point call to the tracing library. In the UST example above, you'll see that the third argument to `trace_mark` is a printf-like format string. The user data is then the result of running that formatting string against the following arguments. Note that `info static-tracepoint-markers` command output lists that format string in the 'Data:' field.

You can inspect this data when analyzing the trace buffer, by printing the `$_sdata` variable like any other variable available to GDB. See Section 13.1.6 [Tracepoint Action Lists], page 210.

The convenience variable `$tpnum` records the tracepoint number of the most recently set tracepoint.

delete tracepoint [*num*]

Permanently delete one or more tracepoints. With no argument, the default is to delete all tracepoints. Note that the regular `delete` command can remove tracepoints also.

Examples:

```
(gdb) delete trace 1 2 3 // remove three tracepoints
```

```
(gdb) delete trace      // remove all tracepoints
```

You can abbreviate this command as `del tr`.

13.1.2 Enable and Disable Tracepoints

These commands are deprecated; they are equivalent to plain `disable` and `enable`.

disable tracepoint [*num*]

Disable tracepoint *num*, or all tracepoints if no argument *num* is given. A disabled tracepoint will have no effect during a trace experiment, but it is not forgotten. You can re-enable a disabled tracepoint using the `enable tracepoint` command. If the command is issued during a trace experiment and the debug target has support for disabling tracepoints during a trace experiment, then the change will be effective immediately. Otherwise, it will be applied to the next trace experiment.

enable tracepoint [*num*]

Enable tracepoint *num*, or all tracepoints. If this command is issued during a trace experiment and the debug target supports enabling tracepoints during a trace experiment, then the enabled tracepoints will become effective immediately. Otherwise, they will become effective the next time a trace experiment is run.

13.1.3 Tracepoint Passcounts

passcount [*n* [*num*]]

Set the *passcount* of a tracepoint. The passcount is a way to automatically stop a trace experiment. If a tracepoint's passcount is *n*, then the trace experiment will be automatically stopped on the *n*'th time that tracepoint is hit. If

the tracepoint number *num* is not specified, the **passcount** command sets the passcount of the most recently defined tracepoint. If no passcount is given, the trace experiment will run until stopped explicitly by the user.

Examples:

```
(gdb) passcount 5 2 // Stop on the 5th execution of
                  // tracepoint 2

(gdb) passcount 12 // Stop on the 12th execution of the
                  // most recently defined tracepoint.

(gdb) trace foo
(gdb) pass 3
(gdb) trace bar
(gdb) pass 2
(gdb) trace baz
(gdb) pass 1      // Stop tracing when foo has been
                  // executed 3 times OR when bar has
                  // been executed 2 times
                  // OR when baz has been executed 1 time.
```

13.1.4 Tracepoint Conditions

The simplest sort of tracepoint collects data every time your program reaches a specified place. You can also specify a *condition* for a tracepoint. A condition is just a Boolean expression in your programming language (see Section 10.1 [Expressions], page 147). A tracepoint with a condition evaluates the expression each time your program reaches it, and data collection happens only if the condition is true.

Tracepoint conditions can be specified when a tracepoint is set, by using ‘if’ in the arguments to the **trace** command. See Section 13.1.1 [Setting Tracepoints], page 206. They can also be set or changed at any time with the **condition** command, just as with breakpoints.

Unlike breakpoint conditions, GDB does not actually evaluate the conditional expression itself. Instead, GDB encodes the expression into an agent expression (see Appendix F [Agent Expressions], page 871) suitable for execution on the target, independently of GDB. Global variables become raw memory locations, locals become stack accesses, and so forth.

For instance, suppose you have a function that is usually called frequently, but should not be called after an error has occurred. You could use the following tracepoint command to collect data about calls of that function that happen while the error code is propagating through the program; an unconditional tracepoint could end up collecting thousands of useless trace frames that you would have to search through.

```
(gdb) trace normal_operation if errcode > 0
```

13.1.5 Trace State Variables

A *trace state variable* is a special type of variable that is created and managed by target-side code. The syntax is the same as that for GDB’s convenience variables (a string prefixed with “\$”), but they are stored on the target. They must be created explicitly, using a **tvariable** command. They are always 64-bit signed integers.

Trace state variables are remembered by GDB, and downloaded to the target along with tracepoint information when the trace experiment starts. There are no intrinsic limits on the number of trace state variables, beyond memory limitations of the target.

Although trace state variables are managed by the target, you can use them in print commands and expressions as if they were convenience variables; GDB will get the current value from the target while the trace experiment is running. Trace state variables share the same namespace as other “\$” variables, which means that you cannot have trace state variables with names like \$23 or \$pc, nor can you have a trace state variable and a convenience variable with the same name.

tvariable *\$name* [= *expression*]

The **tvariable** command creates a new trace state variable named *\$name*, and optionally gives it an initial value of *expression*. The *expression* is evaluated when this command is entered; the result will be converted to an integer if possible, otherwise GDB will report an error. A subsequent **tvariable** command specifying the same name does not create a variable, but instead assigns the supplied initial value to the existing variable of that name, overwriting any previous initial value. The default initial value is 0.

info tvariables

List all the trace state variables along with their initial values. Their current values may also be displayed, if the trace experiment is currently running.

delete tvariable [*\$name* ...]

Delete the given trace state variables, or all of them if no arguments are specified.

13.1.6 Tracepoint Action Lists

actions [*num*]

This command will prompt for a list of actions to be taken when the tracepoint is hit. If the tracepoint number *num* is not specified, this command sets the actions for the one that was most recently defined (so that you can define a tracepoint and then say **actions** without bothering about its number). You specify the actions themselves on the following lines, one action at a time, and terminate the actions list with a line containing just **end**. So far, the only defined actions are **collect**, **teval**, and **while-stepping**.

actions is actually equivalent to **commands** (see Section 5.1.7 [Breakpoint Command Lists], page 79), except that only the defined actions are allowed; any other GDB command is rejected.

To remove all actions from a tracepoint, type ‘**actions** *num*’ and follow it immediately with ‘**end**’.

```
(gdb) collect data // collect some data
```

```
(gdb) while-stepping 5 // single-step 5 times, collect data
```

```
(gdb) end // signals the end of actions.
```

In the following example, the action list begins with **collect** commands indicating the things to be collected when the tracepoint is hit. Then, in order to single-step and collect additional data following the tracepoint, a **while-stepping** command is used, followed by the list of things to be collected after each step in a sequence of single steps. The **while-stepping** command is terminated by

its own separate `end` command. Lastly, the action list is terminated by an `end` command.

```
(gdb) trace foo
(gdb) actions
Enter actions for tracepoint 1, one per line:
> collect bar,baz
> collect $regs
> while-stepping 12
  > collect $pc, arr[i]
  > end
end
```

`collect[/mods] expr1, expr2, ...`

Collect values of the given expressions when the tracepoint is hit. This command accepts a comma-separated list of any valid expressions. In addition to global, static, or local variables, the following special arguments are supported:

\$regs Collect all registers.

\$args Collect all function arguments.

\$locals Collect all local variables.

\$_ret Collect the return address. This is helpful if you want to see more of a backtrace.

Note: The return address location can not always be reliably determined up front, and the wrong address / registers may end up collected instead. On some architectures the reliability is higher for tracepoints at function entry, while on others it's the opposite. When this happens, backtracing will stop because the return address is found unavailable (unless another collect rule happened to match it).

\$_probe_argc Collects the number of arguments from the static probe at which the tracepoint is located. See Section 5.1.10 [Static Probe Points], page 83.

\$_probe_argn *n* is an integer between 0 and 11. Collects the *n*th argument from the static probe at which the tracepoint is located. See Section 5.1.10 [Static Probe Points], page 83.

\$_sdata Collect static tracepoint marker specific data. Only available for static tracepoints. See Section 13.1.6 [Tracepoint Action Lists], page 210. On the UST static tracepoints library backend, an instrumentation point resembles a `printf` function call. The tracing library is able to collect user specified data formatted to a character string using the format provided by the programmer that instrumented the program. Other backends have similar mechanisms. Here's an example of a UST marker call:

```
const char master_name[] = "$your_name";
trace_mark(channel1, marker1, "hello %s", master_name)
```

In this case, collecting `$_sdata` collects the string `'hello $yourname'`. When analyzing the trace buffer, you can inspect `'$_sdata'` like any other variable available to GDB.

You can give several consecutive `collect` commands, each one with a single argument, or one `collect` command with several arguments separated by commas; the effect is the same.

The optional *mods* changes the usual handling of the arguments. `s` requests that pointers to chars be handled as strings, in particular collecting the contents of the memory being pointed at, up to the first zero. The upper bound is by default the value of the `print characters` variable; if `s` is followed by a decimal number, that is the upper bound instead. So for instance `'collect/s25 mystr'` collects as many as 25 characters at `'mystr'`.

The command `info scope` (see Chapter 16 [Symbols], page 265) is particularly useful for figuring out what data to collect.

teval *expr1*, *expr2*, ...

Evaluate the given expressions when the tracepoint is hit. This command accepts a comma-separated list of expressions. The results are discarded, so this is mainly useful for assigning values to trace state variables (see Section 13.1.5 [Trace State Variables], page 209) without adding those values to the trace buffer, as would be the case if the `collect` action were used.

while-stepping *n*

Perform *n* single-step instruction traces after the tracepoint, collecting new data after each step. The `while-stepping` command is followed by the list of what to collect while stepping (followed by its own `end` command):

```
> while-stepping 12
> collect $regs, myglobal
> end
>
```

Note that `$pc` is not automatically collected by `while-stepping`; you need to explicitly collect that register if you need it. You may abbreviate `while-stepping` as `ws` or `stepping`.

set default-collect *expr1*, *expr2*, ...

This variable is a list of expressions to collect at each tracepoint hit. It is effectively an additional `collect` action prepended to every tracepoint action list. The expressions are parsed individually for each tracepoint, so for instance a variable named `xyz` may be interpreted as a global for one tracepoint, and a local for another, as appropriate to the tracepoint's location.

show default-collect

Show the list of expressions that are collected by default at each tracepoint hit.

13.1.7 Listing Tracepoints

info tracepoints [*num*...]

Display information about the tracepoint *num*. If you don't specify a tracepoint number, displays information about all the tracepoints defined so far. The

format is similar to that used for `info breakpoints`; in fact, `info tracepoints` is the same command, simply restricting itself to tracepoints.

A tracepoint's listing may include additional information specific to tracing:

- its passcount as given by the `passcount n` command
- the state about installed on target of each location

```
(gdb) info trace
Num      Type          Disp Enb Address      What
1        tracepoint    keep y  0x0804ab57 in foo() at main.cxx:7
        while-stepping 20
        collect globfoo, $regs
        end
        collect globfoo2
        end
        pass count 1200
2        tracepoint    keep y  <MULTIPLE>
        collect $eip
2.1      installed on target    y      0x0804859c in func4 at change-loc.h:35
2.2      installed on target    y      0xb7ffc480 in func4 at change-loc.h:35
2.3      installed on target    y      <PENDING> set_tracepoint
3        tracepoint    keep y  0x080485b1 in foo at change-loc.c:29
        not installed on target
(gdb)
```

This command can be abbreviated `info tp`.

13.1.8 Listing Static Tracepoint Markers

`info static-tracepoint-markers`

Display information about all static tracepoint markers defined in the program.

For each marker, the following columns are printed:

Count An incrementing counter, output to help readability. This is not a stable identifier.

ID The marker ID, as reported by the target.

Enabled or Disabled Probed markers are tagged with 'y'. 'n' identifies marks that are not enabled.

Address Where the marker is in your program, as a memory address.

What Where the marker is in the source for your program, as a file and line number. If the debug information included in the program does not allow GDB to locate the source of the marker, this column will be left blank.

In addition, the following information may be printed for each marker:

Data User data passed to the tracing library by the marker call. In the UST backend, this is the format string passed as argument to the marker call.

Static tracepoints probing the marker

The list of static tracepoints attached to the marker.

```
(gdb) info static-tracepoint-markers
Cnt ID      Enb Address      What
1   ust/bar2  y   0x0000000000400e1a in main at stexample.c:25
    Data: number1 %d number2 %d
    Probed by static tracepoints: #2
2   ust/bar33 n   0x0000000000400c87 in main at stexample.c:24
    Data: str %s
(gdb)
```

13.1.9 Starting and Stopping Trace Experiments

tstart This command starts the trace experiment, and begins collecting data. It has the side effect of discarding all the data collected in the trace buffer during the previous trace experiment. If any arguments are supplied, they are taken as a note and stored with the trace experiment's state. The notes may be arbitrary text, and are especially useful with disconnected tracing in a multi-user context; the notes can explain what the trace is doing, supply user contact information, and so forth.

tstop This command stops the trace experiment. If any arguments are supplied, they are recorded with the experiment as a note. This is useful if you are stopping a trace started by someone else, for instance if the trace is interfering with the system's behavior and needs to be stopped quickly.

Note: a trace experiment and data collection may stop automatically if any tracepoint's passcount is reached (see Section 13.1.3 [Tracepoint Passcounts], page 208), or if the trace buffer becomes full.

tstatus This command displays the status of the current trace data collection.

Here is an example of the commands we described so far:

```
(gdb) trace gdb.c.test
(gdb) actions
Enter actions for tracepoint #1, one per line.
> collect $regs,$locals,$args
> while-stepping 11
  > collect $regs
  > end
> end
(gdb) tstart
[time passes ...]
(gdb) tstop
```

You can choose to continue running the trace experiment even if GDB disconnects from the target, voluntarily or involuntarily. For commands such as **detach**, the debugger will ask what you want to do with the trace. But for unexpected terminations (GDB crash, network outage), it would be unfortunate to lose hard-won trace data, so the variable **disconnected-tracing** lets you decide whether the trace should continue running without GDB.


```
set disconnected-tracing on
```

```
set disconnected-tracing off
```

Choose whether a tracing run should continue to run if GDB has disconnected from the target. Note that `detach` or `quit` will ask you directly what to do about a running trace no matter what this variable's setting, so the variable is mainly useful for handling unexpected situations, such as loss of the network.

```
show disconnected-tracing
```

Show the current choice for disconnected tracing.

When you reconnect to the target, the trace experiment may or may not still be running; it might have filled the trace buffer in the meantime, or stopped for one of the other reasons. If it is running, it will continue after reconnection.

Upon reconnection, the target will upload information about the tracepoints in effect. GDB will then compare that information to the set of tracepoints currently defined, and attempt to match them up, allowing for the possibility that the numbers may have changed due to creation and deletion in the meantime. If one of the target's tracepoints does not match any in GDB, the debugger will create a new tracepoint, so that you have a number with which to specify that tracepoint. This matching-up process is necessarily heuristic, and it may result in useless tracepoints being created; you may simply delete them if they are of no use.

If your target agent supports a *circular trace buffer*, then you can run a trace experiment indefinitely without filling the trace buffer; when space runs out, the agent deletes already-collected trace frames, oldest first, until there is enough room to continue collecting. This is especially useful if your tracepoints are being hit too often, and your trace gets terminated prematurely because the buffer is full. To ask for a circular trace buffer, simply set `'circular-trace-buffer'` to on. You can set this at any time, including during tracing; if the agent can do it, it will change buffer handling on the fly, otherwise it will not take effect until the next run.

```
set circular-trace-buffer on
```

```
set circular-trace-buffer off
```

Choose whether a tracing run should use a linear or circular buffer for trace data. A linear buffer will not lose any trace data, but may fill up prematurely, while a circular buffer will discard old trace data, but it will have always room for the latest tracepoint hits.

```
show circular-trace-buffer
```

Show the current choice for the trace buffer. Note that this may not match the agent's current buffer handling, nor is it guaranteed to match the setting that might have been in effect during a past run, for instance if you are looking at frames from a trace file.

```
set trace-buffer-size n
```

```
set trace-buffer-size unlimited
```

Request that the target use a trace buffer of *n* bytes. Not all targets will honor the request; they may have a compiled-in size for the trace buffer, or some other limitation. Set to a value of `unlimited` or `-1` to let the target use whatever size it likes. This is also the default.

show trace-buffer-size

Show the current requested size for the trace buffer. Note that this will only match the actual size if the target supports size-setting, and was able to handle the requested size. For instance, if the target can only change buffer size between runs, this variable will not reflect the change until the next run starts. Use `tstatus` to get a report of the actual buffer size.

set trace-user text**show trace-user****set trace-notes text**

Set the trace run's notes.

show trace-notes

Show the trace run's notes.

set trace-stop-notes text

Set the trace run's stop notes. The handling of the note is as for `tstop` arguments; the `set` command is convenient way to fix a stop note that is mistaken or incomplete.

show trace-stop-notes

Show the trace run's stop notes.

13.1.10 Tracepoint Restrictions

There are a number of restrictions on the use of tracepoints. As described above, tracepoint data gathering occurs on the target without interaction from GDB. Thus the full capabilities of the debugger are not available during data gathering, and then at data examination time, you will be limited by only having what was collected. The following items describe some common problems, but it is not exhaustive, and you may run into additional difficulties not mentioned here.

- Tracepoint expressions are intended to gather objects (lvalues). Thus the full flexibility of GDB's expression evaluator is not available. You cannot call functions, cast objects to aggregate types, access convenience variables or modify values (except by assignment to trace state variables). Some language features may implicitly call functions (for instance Objective-C fields with accessors), and therefore cannot be collected either.
- Collection of local variables, either individually or in bulk with `$locals` or `$args`, during **while-stepping** may behave erratically. The stepping action may enter a new scope (for instance by stepping into a function), or the location of the variable may change (for instance it is loaded into a register). The tracepoint data recorded uses the location information for the variables that is correct for the tracepoint location. When the tracepoint is created, it is not possible, in general, to determine where the steps of a **while-stepping** sequence will advance the program—particularly if a conditional branch is stepped.
- Collection of an incompletely-initialized or partially-destroyed object may result in something that GDB cannot display, or displays in a misleading way.
- When GDB displays a pointer to character it automatically dereferences the pointer to also display characters of the string being pointed to. However, collecting the pointer during tracing does not automatically collect the string. You need to explicitly dereference the pointer and provide size information if you want to collect not only the

pointer, but the memory pointed to. For example, `*ptr@50` can be used to collect the 50 element array pointed to by `ptr`.

- It is not possible to collect a complete stack backtrace at a tracepoint. Instead, you may collect the registers and a few hundred bytes from the stack pointer with something like `*(unsigned char *)$esp@300` (adjust to use the name of the actual stack pointer register on your target architecture, and the amount of stack you wish to capture). Then the `backtrace` command will show a partial backtrace when using a trace frame. The number of stack frames that can be examined depends on the sizes of the frames in the collected stack. Note that if you ask for a block so large that it goes past the bottom of the stack, the target agent may report an error trying to read from an invalid address.
- If you do not collect registers at a tracepoint, GDB can infer that the value of `$pc` must be the same as the address of the tracepoint and use that when you are looking at a trace frame for that tracepoint. However, this cannot work if the tracepoint has multiple locations (for instance if it was set in a function that was inlined), or if it has a `while-stepping` loop. In those cases GDB will warn you that it can't infer `$pc`, and default it to zero.

13.2 Using the Collected Data

After the tracepoint experiment ends, you use GDB commands for examining the trace data. The basic idea is that each tracepoint collects a trace *snapshot* every time it is hit and another snapshot every time it single-steps. All these snapshots are consecutively numbered from zero and go into a buffer, and you can examine them later. The way you examine them is to *focus* on a specific trace snapshot. When the remote stub is focused on a trace snapshot, it will respond to all GDB requests for memory and registers by reading from the buffer which belongs to that snapshot, rather than from *real* memory or registers of the program being debugged. This means that **all** GDB commands (`print`, `info registers`, `backtrace`, etc.) will behave as if we were currently debugging the program state as it was when the tracepoint occurred. Any requests for data that are not in the buffer will fail.

13.2.1 `tfind n`

The basic command for selecting a trace snapshot from the buffer is `tfind n`, which finds trace snapshot number *n*, counting from zero. If no argument *n* is given, the next snapshot is selected.

Here are the various forms of using the `tfind` command.

`tfind start`

Find the first snapshot in the buffer. This is a synonym for `tfind 0` (since 0 is the number of the first snapshot).

`tfind none`

Stop debugging trace snapshots, resume *live* debugging.

`tfind end` Same as '`tfind none`'.

`tfind` No argument means find the next trace snapshot or find the first one if no trace snapshot is selected.

tfind - Find the previous trace snapshot before the current one. This permits retracing earlier steps.

tfind tracepoint *num*

Find the next snapshot associated with tracepoint *num*. Search proceeds forward from the last examined trace snapshot. If no argument *num* is given, it means find the next snapshot collected for the same tracepoint as the current snapshot.

tfind pc *addr*

Find the next snapshot associated with the value *addr* of the program counter. Search proceeds forward from the last examined trace snapshot. If no argument *addr* is given, it means find the next snapshot with the same value of PC as the current snapshot.

tfind outside *addr1*, *addr2*

Find the next snapshot whose PC is outside the given range of addresses (exclusive).

tfind range *addr1*, *addr2*

Find the next snapshot whose PC is between *addr1* and *addr2* (inclusive).

tfind line [*file*:]*n*

Find the next snapshot associated with the source line *n*. If the optional argument *file* is given, refer to line *n* in that source file. Search proceeds forward from the last examined trace snapshot. If no argument *n* is given, it means find the next line other than the one currently being examined; thus saying **tfind line** repeatedly can appear to have the same effect as stepping from line to line in a *live* debugging session.

The default arguments for the **tfind** commands are specifically designed to make it easy to scan through the trace buffer. For instance, **tfind** with no argument selects the next trace snapshot, and **tfind -** with no argument selects the previous trace snapshot. So, by giving one **tfind** command, and then simply hitting **RET** repeatedly you can examine all the trace snapshots in order. Or, by saying **tfind -** and then hitting **RET** repeatedly you can examine the snapshots in reverse order. The **tfind line** command with no argument selects the snapshot for the next source line executed. The **tfind pc** command with no argument selects the next snapshot with the same program counter (PC) as the current frame. The **tfind tracepoint** command with no argument selects the next trace snapshot collected by the same tracepoint as the current one.

In addition to letting you scan through the trace buffer manually, these commands make it easy to construct GDB scripts that scan through the trace buffer and print out whatever collected data you are interested in. Thus, if we want to examine the PC, FP, and SP registers from each trace frame in the buffer, we can say this:

```
(gdb) tfind start
(gdb) while ($trace_frame != -1)
> printf "Frame %d, PC = %08X, SP = %08X, FP = %08X\n", \
    $trace_frame, $pc, $sp, $fp
> tfind
> end
```

```

Frame 0, PC = 0020DC64, SP = 0030BF3C, FP = 0030BF44
Frame 1, PC = 0020DC6C, SP = 0030BF38, FP = 0030BF44
Frame 2, PC = 0020DC70, SP = 0030BF34, FP = 0030BF44
Frame 3, PC = 0020DC74, SP = 0030BF30, FP = 0030BF44
Frame 4, PC = 0020DC78, SP = 0030BF2C, FP = 0030BF44
Frame 5, PC = 0020DC7C, SP = 0030BF28, FP = 0030BF44
Frame 6, PC = 0020DC80, SP = 0030BF24, FP = 0030BF44
Frame 7, PC = 0020DC84, SP = 0030BF20, FP = 0030BF44
Frame 8, PC = 0020DC88, SP = 0030BF1C, FP = 0030BF44
Frame 9, PC = 0020DC8E, SP = 0030BF18, FP = 0030BF44
Frame 10, PC = 00203F6C, SP = 0030BE3C, FP = 0030BF14

```

Or, if we want to examine the variable `X` at each source line in the buffer:

```

(gdb) tfind start
(gdb) while ($trace_frame != -1)
> printf "Frame %d, X == %d\n", $trace_frame, X
> tfind line
> end

Frame 0, X = 1
Frame 7, X = 2
Frame 13, X = 255

```

13.2.2 tdump

This command takes no arguments. It prints all the data collected at the current trace snapshot.

```

(gdb) trace 444
(gdb) actions
Enter actions for tracepoint #2, one per line:
> collect $regs, $locals, $args, gdb_long_test
> end

(gdb) tstart

(gdb) tfind line 444
#0  gdb_test (p1=0x11, p2=0x22, p3=0x33, p4=0x44, p5=0x55, p6=0x66)
at gdb_test.c:444
444      printp( "%s: arguments = 0x%X 0x%X 0x%X 0x%X 0x%X 0x%X\n", )

(gdb) tdump
Data collected at tracepoint 2, trace frame 1:
d0      0xc4aa0085      -995491707
d1      0x18      24
d2      0x80      128
d3      0x33      51
d4      0x71aea3d      119204413
d5      0x22      34
d6      0xe0      224
d7      0x380035 3670069
a0      0x19e24a 1696330
a1      0x3000668      50333288
a2      0x100      256
a3      0x322000 3284992
a4      0x3000698      50333336
a5      0x1ad3cc 1758156
fp      0x30bf3c 0x30bf3c
sp      0x30bf34 0x30bf34

```

```

ps          0x0      0
pc          0x20b2c8 0x20b2c8
fpcontrol   0x0      0
fpstatus    0x0      0
fpiaddr     0x0      0
p = 0x20e5b4 "gdb-test"
p1 = (void *) 0x11
p2 = (void *) 0x22
p3 = (void *) 0x33
p4 = (void *) 0x44
p5 = (void *) 0x55
p6 = (void *) 0x66
gdb_long_test = 17 '\021'

(gdb)

```

`tdump` works by scanning the tracepoint's current collection actions and printing the value of each expression listed. So `tdump` can fail, if after a run, you change the tracepoint's actions to mention variables that were not collected during the run.

Also, for tracepoints with **while-stepping** loops, `tdump` uses the collected value of `$pc` to distinguish between trace frames that were collected at the tracepoint hit, and frames that were collected while stepping. This allows it to correctly choose whether to display the basic list of collections, or the collections from the body of the while-stepping loop. However, if `$pc` was not collected, then `tdump` will always attempt to dump using the basic collection list, and may fail if a while-stepping frame does not include all the same data that is collected at the tracepoint hit.

13.2.3 save tracepoints *filename*

This command saves all current tracepoint definitions together with their actions and pass-counts, into a file *filename* suitable for use in a later debugging session. To read the saved tracepoint definitions, use the `source` command (see Section 23.1.3 [Command Files], page 401). The `save-tracepoints` command is a deprecated alias for `save tracepoints`

13.3 Convenience Variables for Tracepoints

```

(int) $trace_frame
    The current trace snapshot (a.k.a. frame) number, or -1 if no snapshot is selected.

(int) $tracepoint
    The tracepoint for the current trace snapshot.

(int) $trace_line
    The line number for the current trace snapshot.

(char []) $trace_file
    The source file for the current trace snapshot.

(char []) $trace_func
    The name of the function containing $tracepoint.

```

Note: `$trace_file` is not suitable for use in `printf`, use `output` instead.

Here's a simple example of using these convenience variables for stepping through all the trace snapshots and printing some of their data. Note that these are not the same as trace state variables, which are managed by the target.

```
(gdb) tfind start

(gdb) while $trace.frame != -1
> output $trace_file
> printf ", line %d (tracepoint #%d)\n", $trace_line, $tracepoint
> tfind
> end
```

13.4 Using Trace Files

In some situations, the target running a trace experiment may no longer be available; perhaps it crashed, or the hardware was needed for a different activity. To handle these cases, you can arrange to dump the trace data into a file, and later use that file as a source of trace data, via the **target tfile** command.

```
tsave [ -r ] filename
tsave [-ctf] dirname
```

Save the trace data to *filename*. By default, this command assumes that *filename* refers to the host filesystem, so if necessary GDB will copy raw trace data up from the target and then save it. If the target supports it, you can also supply the optional argument **-r** (“remote”) to direct the target to save the data directly into *filename* in its own filesystem, which may be more efficient if the trace buffer is very large. (Note, however, that **target tfile** can only read from files accessible to the host.) By default, this command will save trace frame in tfile format. You can supply the optional argument **-ctf** to save data in CTF format. The *Common Trace Format* (CTF) is proposed as a trace format that can be shared by multiple debugging and tracing tools. Please go to ‘<http://www.elficios.com/ctf>’ to get more information.

```
target tfile filename
target ctf dirname
```

Use the file named *filename* or directory named *dirname* as a source of trace data. Commands that examine data work as they do with a live target, but it is not possible to run any new trace experiments. **tstatus** will report the state of the trace run at the moment the data was saved, as well as the current trace frame you are examining. Both *filename* and *dirname* must be on a filesystem accessible to the host.

The *filename* and *dirname* arguments supports escaping and quoting, see Section 3.4 [Filenames As Command Arguments], page 27.

```
(gdb) target ctf ctf.ctf
(gdb) tfind
Found trace frame 0, tracepoint 2
39          ++a; /* set tracepoint 1 here */
(gdb) tdump
Data collected at tracepoint 2, trace frame 0:
i = 0
a = 0
b = 1 '\001'
```

```
c = {"123", "456", "789", "123", "456", "789"}  
d = {{{a = 1, b = 2}, {a = 3, b = 4}}, {{a = 5, b = 6}, {a = 7, b = 8}}}  
(gdb) p b  
$1 = 1
```


14 Debugging Programs That Use Overlays

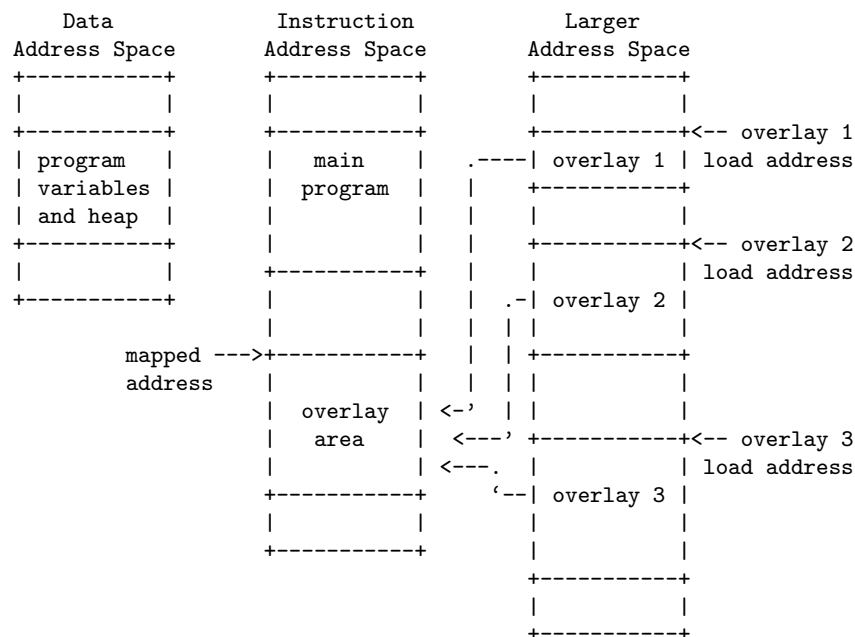
If your program is too large to fit completely in your target system's memory, you can sometimes use *overlays* to work around this problem. GDB provides some support for debugging programs that use overlays.

14.1 How Overlays Work

Suppose you have a computer whose instruction address space is only 64 kilobytes long, but which has much more memory which can be accessed by other means: special instructions, segment registers, or memory management hardware, for example. Suppose further that you want to adapt a program which is larger than 64 kilobytes to run on this system.

One solution is to identify modules of your program which are relatively independent, and need not call each other directly; call these modules *overlays*. Separate the overlays from the main program, and place their machine code in the larger memory. Place your main program in instruction memory, but leave at least enough space there to hold the largest overlay as well.

Now, to call a function located in an overlay, you must first copy that overlay's machine code from the large memory into the space set aside for it in the instruction memory, and then jump to its entry point there.



A code overlay

The diagram (see [A code overlay], page 223) shows a system with separate data and instruction address spaces. To map an overlay, the program copies its code from the larger address space to the instruction address space. Since the overlays shown here all use the same mapped address, only one may be mapped at a time. For a system with a single address space for data and instructions, the diagram would be similar, except that the program variables and heap would share an address space with the main program and the overlay area.

An overlay loaded into instruction memory and ready for use is called a *mapped* overlay; its *mapped address* is its address in the instruction memory. An overlay not present (or only partially present) in instruction memory is called *unmapped*; its *load address* is its address in the larger memory. The mapped address is also called the *virtual memory address*, or *VMA*; the load address is also called the *load memory address*, or *LMA*.

Unfortunately, overlays are not a completely transparent way to adapt a program to limited instruction memory. They introduce a new set of global constraints you must keep in mind as you design your program:

- Before calling or returning to a function in an overlay, your program must make sure that overlay is actually mapped. Otherwise, the call or return will transfer control to the right address, but in the wrong overlay, and your program will probably crash.
- If the process of mapping an overlay is expensive on your system, you will need to choose your overlays carefully to minimize their effect on your program's performance.
- The executable file you load onto your system must contain each overlay's instructions, appearing at the overlay's load address, not its mapped address. However, each overlay's instructions must be relocated and its symbols defined as if the overlay were at its mapped address. You can use GNU linker scripts to specify different load and relocation addresses for pieces of your program; see Section "Overlay Description" in *Using ld: the GNU linker*.
- The procedure for loading executable files onto your system must be able to load their contents into the larger address space as well as the instruction and data spaces.

The overlay system described above is rather simple, and could be improved in many ways:

- If your system has suitable bank switch registers or memory management hardware, you could use those facilities to make an overlay's load area contents simply appear at their mapped address in instruction space. This would probably be faster than copying the overlay to its mapped area in the usual way.
- If your overlays are small enough, you could set aside more than one overlay area, and have more than one overlay mapped at a time.
- You can use overlays to manage data, as well as instructions. In general, data overlays are even less transparent to your design than code overlays: whereas code overlays only require care when you call or return to functions, data overlays require care every time you access the data. Also, if you change the contents of a data overlay, you must copy its contents back out to its load address before you can copy a different data overlay into the same mapped area.

14.2 Overlay Commands

To use GDB's overlay support, each overlay in your program must correspond to a separate section of the executable file. The section's virtual memory address and load memory address must be the overlay's mapped and load addresses. Identifying overlays with sections allows GDB to determine the appropriate address of a function or variable, depending on whether the overlay is mapped or not.

GDB's overlay commands all start with the word **overlay**; you can abbreviate this as **ov** or **ovly**. The commands are:

overlay off

Disable GDB's overlay support. When overlay support is disabled, GDB assumes that all functions and variables are always present at their mapped addresses. By default, GDB's overlay support is disabled.

overlay manual

Enable *manual* overlay debugging. In this mode, GDB relies on you to tell it which overlays are mapped, and which are not, using the **overlay map-overlay** and **overlay unmap-overlay** commands described below.

overlay map-overlay overlay**overlay map overlay**

Tell GDB that *overlay* is now mapped; *overlay* must be the name of the object file section containing the overlay. When an overlay is mapped, GDB assumes it can find the overlay's functions and variables at their mapped addresses. GDB assumes that any other overlays whose mapped ranges overlap that of *overlay* are now unmapped.

overlay unmap-overlay overlay**overlay unmap overlay**

Tell GDB that *overlay* is no longer mapped; *overlay* must be the name of the object file section containing the overlay. When an overlay is unmapped, GDB assumes it can find the overlay's functions and variables at their load addresses.

overlay auto

Enable *automatic* overlay debugging. In this mode, GDB consults a data structure the overlay manager maintains in the inferior to see which overlays are mapped. For details, see Section 14.3 [Automatic Overlay Debugging], page 226.

overlay load-target**overlay load**

Re-read the overlay table from the inferior. Normally, GDB re-reads the table automatically each time the inferior stops, so this command should only be necessary if you have changed the overlay mapping yourself using GDB. This command is only useful when using automatic overlay debugging.

overlay list-overlays**overlay list**

Display a list of the overlays currently mapped, along with their mapped addresses, load addresses, and sizes.

Normally, when GDB prints a code address, it includes the name of the function the address falls in:

```
(gdb) print main
$3 = {int ()} 0x11a0 <main>
```

When overlay debugging is enabled, GDB recognizes code in unmapped overlays, and prints the names of unmapped functions with asterisks around them. For example, if *foo* is a function in an unmapped overlay, GDB prints it this way:

```
(gdb) overlay list
No sections are mapped.
(gdb) print foo
```

```
$5 = {int (int)} 0x100000 <foo*>
```

When `foo`'s overlay is mapped, GDB prints the function's name normally:

```
(gdb) overlay list
Section .ov.foo.text, loaded at 0x100000 - 0x100034,
        mapped at 0x1016 - 0x104a
(gdb) print foo
$6 = {int (int)} 0x1016 <foo>
```

When overlay debugging is enabled, GDB can find the correct address for functions and variables in an overlay, whether or not the overlay is mapped. This allows most GDB commands, like `break` and `disassemble`, to work normally, even on unmapped code. However, GDB's breakpoint support has some limitations:

- You can set breakpoints in functions in unmapped overlays, as long as GDB can write to the overlay at its load address.
- GDB can not set hardware or simulator-based breakpoints in unmapped overlays. However, if you set a breakpoint at the end of your overlay manager (and tell GDB which overlays are now mapped, if you are using manual overlay management), GDB will re-set its breakpoints properly.

14.3 Automatic Overlay Debugging

GDB can automatically track which overlays are mapped and which are not, given some simple co-operation from the overlay manager in the inferior. If you enable automatic overlay debugging with the `overlay auto` command (see Section 14.2 [Overlay Commands], page 224), GDB looks in the inferior's memory for certain variables describing the current state of the overlays.

Here are the variables your overlay manager must define to support GDB's automatic overlay debugging:

`_ovly_table`:

This variable must be an array of the following structures:

```
struct
{
    /* The overlay's mapped address. */
    unsigned long vma;

    /* The size of the overlay, in bytes. */
    unsigned long size;

    /* The overlay's load address. */
    unsigned long lma;

    /* Non-zero if the overlay is currently mapped;
       zero otherwise. */
    unsigned long mapped;
}
```

`_novlys`: This variable must be a four-byte signed integer, holding the total number of elements in `_ovly_table`.

To decide whether a particular overlay is mapped or not, GDB looks for an entry in `_ovly_table` whose `vma` and `lma` members equal the VMA and LMA of the overlay's section

in the executable file. When GDB finds a matching entry, it consults the entry's `mapped` member to determine whether the overlay is currently mapped.

In addition, your overlay manager may define a function called `_ovly_debug_event`. If this function is defined, GDB will silently set a breakpoint there. If the overlay manager then calls this function whenever it has changed the overlay table, this will enable GDB to accurately keep track of which overlays are in program memory, and update any breakpoints that may be set in overlays. This will allow breakpoints to work even if the overlays are kept in ROM or other non-writable memory while they are not being executed.

14.4 Overlay Sample Program

When linking a program which uses overlays, you must place the overlays at their load addresses, while relocating them to run at their mapped addresses. To do this, you must write a linker script (see Section “Overlay Description” in *Using ld: the GNU linker*). Unfortunately, since linker scripts are specific to a particular host system, target architecture, and target memory layout, this manual cannot provide portable sample code demonstrating GDB's overlay support.

However, the GDB source distribution does contain an overlaid program, with linker scripts for a few systems, as part of its test suite. The program consists of the following files from `gdb/testsuite/gdb.base`:

```
overlays.c      The main program file.
ovlymgr.c       A simple overlay manager, used by overlays.c.
foo.c
bar.c
baz.c
grbx.c         Overlay modules, loaded and used by overlays.c.
d10v.ld
m32r.ld       Linker scripts for linking the test program on the d10v-elf and m32r-elf
               targets.
```

You can build the test program using the `d10v-elf` GCC cross-compiler like this:

```
$ d10v-elf-gcc -g -c overlays.c
$ d10v-elf-gcc -g -c ovlymgr.c
$ d10v-elf-gcc -g -c foo.c
$ d10v-elf-gcc -g -c bar.c
$ d10v-elf-gcc -g -c baz.c
$ d10v-elf-gcc -g -c grbx.c
$ d10v-elf-gcc -g overlays.o ovlymgr.o foo.o bar.o \
    baz.o grbx.o -Wl,-Td10v.ld -o overlays
```

The build process is identical for any other architecture, except that you must substitute the appropriate compiler and linker script for the target system for `d10v-elf-gcc` and `d10v.ld`.

15 Using GDB with Different Languages

Although programming languages generally have common aspects, they are rarely expressed in the same manner. For instance, in ANSI C, dereferencing a pointer `p` is accomplished by `*p`, but in Modula-2, it is accomplished by `p^`. Values can also be represented (and displayed) differently. Hex numbers in C appear as `'0x1ae'`, while in Modula-2 they appear as `'1AEH'`.

Language-specific information is built into GDB for some languages, allowing you to express operations like the above in your program's native language, and allowing GDB to output values in a manner consistent with the syntax of your program's native language. The language you use to build expressions is called the *working language*.

15.1 Switching Between Source Languages

There are two ways to control the working language—either have GDB set it automatically, or select it manually yourself. You can use the `set language` command for either purpose. On startup, GDB defaults to setting the language automatically. The working language is used to determine how expressions you type are interpreted, how values are printed, etc.

In addition to the working language, every source file that GDB knows about has its own working language. For some object file formats, the compiler might indicate which language a particular source file is in. However, most of the time GDB infers the language from the name of the file. The language of a source file controls whether C++ names are demangled—this way `backtrace` can show each frame appropriately for its own language. There is no way to set the language of a source file from within GDB, but you can set the language associated with a filename extension. See Section 15.2 [Displaying the Language], page 231.

This is most commonly a problem when you use a program, such as `cfront` or `f2c`, that generates C but is written in another language. In that case, make the program use `#line` directives in its C output; that way GDB will know the correct language of the source code of the original program, and will display that source code, not the generated C code.

15.1.1 List of Filename Extensions and Languages

If a source file name ends in one of the following extensions, then GDB infers that its language is the one indicated.

<code>.ada</code>	
<code>.ads</code>	
<code>.adb</code>	
<code>.a</code>	Ada source file.
<code>.c</code>	C source file
<code>.C</code>	
<code>.cc</code>	
<code>.cp</code>	
<code>.cpp</code>	
<code>.cxx</code>	
<code>.c++</code>	C++ source file

<code>.d</code>	D source file
<code>.m</code>	Objective-C source file
<code>.f</code>	
<code>.F</code>	Fortran source file
<code>.mod</code>	Modula-2 source file
<code>.s</code>	
<code>.S</code>	Assembler source file. This actually behaves almost like C, but GDB does not skip over function prologues when stepping.

In addition, you may set the language associated with a filename extension. See Section 15.2 [Displaying the Language], page 231.

15.1.2 Setting the Working Language

If you allow GDB to set the language automatically, expressions are interpreted the same way in your debugging session and your program.

If you wish, you may set the language manually. To do this, issue the command ‘`set language lang`’, where *lang* is the name of a language, such as `c` or `modula-2`. For a list of the supported languages, type ‘`set language`’.

Setting the language manually prevents GDB from updating the working language automatically. This can lead to confusion if you try to debug a program when the working language is not the same as the source language, when an expression is acceptable to both languages—but means different things. For instance, if the current source file were written in C, and GDB was parsing Modula-2, a command such as:

```
print a = b + c
```

might not have the effect you intended. In C, this means to add `b` and `c` and place the result in `a`. The result printed would be the value of `a`. In Modula-2, this means to compare `a` to the result of `b+c`, yielding a `BOOLEAN` value.

15.1.3 Having GDB Infer the Source Language

To have GDB set the working language automatically, use ‘`set language local`’ or ‘`set language auto`’. GDB then infers the working language. That is, when your program stops in a frame (usually by encountering a breakpoint), GDB sets the working language to the language recorded for the function in that frame. If the language for a frame is unknown (that is, if the function or block corresponding to the frame was defined in a source file that does not have a recognized extension), the current working language is not changed, and GDB issues a warning.

This may not seem necessary for most programs, which are written entirely in one source language. However, program modules and libraries written in one source language can be used by a main program written in a different source language. Using ‘`set language auto`’ in this case frees you from having to set the working language manually.

The warning is enabled by default, but it can be controlled via a setting:

```
set warn-language-frame-mismatch [on|off]
```

Enable or disable the warning that is issued when the current language is set to a value that does not match the current frame.

show warn-language-frame-mismatch

Show whether the frame-mismatch warning will be issued.

15.2 Displaying the Language

The following commands help you find out which language is the working language, and also what language source files were written in.

show language

Display the current working language. This is the language you can use with commands such as **print** to build and compute expressions that may involve variables in your program.

info frame

Display the source language for this frame. This language becomes the working language if you use an identifier from this frame. See Section 8.4 [Information about a Frame], page 121, to identify the other information listed here.

info source

Display the source language of this source file. See Chapter 16 [Examining the Symbol Table], page 265, to identify the other information listed here.

In unusual circumstances, you may have source files with extensions not in the standard list. You can then set the extension associated with a language explicitly:

set extension-language *ext language*

Tell GDB that source files with extension *ext* are to be assumed as written in the source language *language*.

info extensions

List all the filename extensions and the associated languages.

15.3 Type and Range Checking

Some languages are designed to guard you against making seemingly common errors through a series of compile- and run-time checks. These include checking the type of arguments to functions and operators and making sure mathematical overflows are caught at run time. Checks such as these help to ensure a program's correctness once it has been compiled by eliminating type mismatches and providing active checks for range errors when your program is running.

By default GDB checks for these errors according to the rules of the current source language. Although GDB does not check the statements in your program, it can check expressions entered directly into GDB for evaluation via the **print** command, for example.

15.3.1 An Overview of Type Checking

Some languages, such as C and C++, are strongly typed, meaning that the arguments to operators and functions have to be of the correct type, otherwise an error occurs. These checks prevent type mismatch errors from ever causing any run-time problems. For example,

```
int klass::my_method(char *b) { return b ? 1 : 2; }
```

```
(gdb) print obj.my_method (0)
```

```

$1 = 2
but
(gdb) print obj.my_method (0x1234)
Cannot resolve method klass::my_method to any overloaded instance

```

The second example fails because in C++ the integer constant ‘0x1234’ is not type-compatible with the pointer parameter type.

For the expressions you use in GDB commands, you can tell GDB to not enforce strict type checking or to treat any mismatches as errors and abandon the expression; When type checking is disabled, GDB successfully evaluates expressions like the second example above.

Even if type checking is off, there may be other reasons related to type that prevent GDB from evaluating an expression. For instance, GDB does not know how to add an `int` and a `struct foo`. These particular type errors have nothing to do with the language in use and usually arise from expressions which make little sense to evaluate anyway.

GDB provides some additional commands for controlling type checking:

```

set check type on
set check type off
    Set strict type checking on or off. If any type mismatches occur in evaluating an
    expression while type checking is on, GDB prints a message and aborts evaluation
    of the expression.

show check type
    Show the current setting of type checking and whether GDB is enforcing strict
    type checking rules.

```

15.3.2 An Overview of Range Checking

In some languages (such as Modula-2), it is an error to exceed the bounds of a type; this is enforced with run-time checks. Such range checking is meant to ensure program correctness by making sure computations do not overflow, or indices on an array element access do not exceed the bounds of the array.

For expressions you use in GDB commands, you can tell GDB to treat range errors in one of three ways: ignore them, always treat them as errors and abandon the expression, or issue warnings but evaluate the expression anyway.

A range error can result from numerical overflow, from exceeding an array index bound, or when you type a constant that is not a member of any type. Some languages, however, do not treat overflows as an error. In many implementations of C, mathematical overflow causes the result to “wrap around” to lower values—for example, if m is the largest integer value, and s is the smallest, then

$$m + 1 \Rightarrow s$$

This, too, is specific to individual languages, and in some cases specific to individual compilers or machines. See Section 15.4 [Supported Languages], page 233, for further details on specific languages.

GDB provides some additional commands for controlling the range checker:

```

set check range auto
    Set range checking on or off based on the current working language. See
    Section 15.4 [Supported Languages], page 233, for the default settings for each
    language.

```